

INTERNATIONAL STANDARD

NORME INTERNATIONALE



**Field Device Integration (FDI) –
Part 2: FDI Client**

**Intégration des appareils de terrain (FDI) –
Partie 2: Client FDI**



THIS PUBLICATION IS COPYRIGHT PROTECTED

Copyright © 2015 IEC, Geneva, Switzerland

All rights reserved. Unless otherwise specified, no part of this publication may be reproduced or utilized in any form or by any means, electronic or mechanical, including photocopying and microfilm, without permission in writing from either IEC or IEC's member National Committee in the country of the requester. If you have any questions about IEC copyright or have an enquiry about obtaining additional rights to this publication, please contact the address below or your local IEC member National Committee for further information.

Droits de reproduction réservés. Sauf indication contraire, aucune partie de cette publication ne peut être reproduite ni utilisée sous quelque forme que ce soit et par aucun procédé, électronique ou mécanique, y compris la photocopie et les microfilms, sans l'accord écrit de l'IEC ou du Comité national de l'IEC du pays du demandeur. Si vous avez des questions sur le copyright de l'IEC ou si vous désirez obtenir des droits supplémentaires sur cette publication, utilisez les coordonnées ci-après ou contactez le Comité national de l'IEC de votre pays de résidence.

IEC Central Office
3, rue de Varembe
CH-1211 Geneva 20
Switzerland

Tel.: +41 22 919 02 11
Fax: +41 22 919 03 00
info@iec.ch
www.iec.ch

About the IEC

The International Electrotechnical Commission (IEC) is the leading global organization that prepares and publishes International Standards for all electrical, electronic and related technologies.

About IEC publications

The technical content of IEC publications is kept under constant review by the IEC. Please make sure that you have the latest edition, a corrigenda or an amendment might have been published.

IEC Catalogue - webstore.iec.ch/catalogue

The stand-alone application for consulting the entire bibliographical information on IEC International Standards, Technical Specifications, Technical Reports and other documents. Available for PC, Mac OS, Android Tablets and iPad.

IEC publications search - www.iec.ch/searchpub

The advanced search enables to find IEC publications by a variety of criteria (reference number, text, technical committee,...). It also gives information on projects, replaced and withdrawn publications.

IEC Just Published - webstore.iec.ch/justpublished

Stay up to date on all new IEC publications. Just Published details all new publications released. Available online and also once a month by email.

Electropedia - www.electropedia.org

The world's leading online dictionary of electronic and electrical terms containing more than 30 000 terms and definitions in English and French, with equivalent terms in 15 additional languages. Also known as the International Electrotechnical Vocabulary (IEV) online.

IEC Glossary - std.iec.ch/glossary

More than 60 000 electrotechnical terminology entries in English and French extracted from the Terms and Definitions clause of IEC publications issued since 2002. Some entries have been collected from earlier publications of IEC TC 37, 77, 86 and CISPR.

IEC Customer Service Centre - webstore.iec.ch/csc

If you wish to give us your feedback on this publication or need further assistance, please contact the Customer Service Centre: csc@iec.ch.

A propos de l'IEC

La Commission Electrotechnique Internationale (IEC) est la première organisation mondiale qui élabore et publie des Normes internationales pour tout ce qui a trait à l'électricité, à l'électronique et aux technologies apparentées.

A propos des publications IEC

Le contenu technique des publications IEC est constamment revu. Veuillez vous assurer que vous possédez l'édition la plus récente, un corrigendum ou amendement peut avoir été publié.

Catalogue IEC - webstore.iec.ch/catalogue

Application autonome pour consulter tous les renseignements bibliographiques sur les Normes internationales, Spécifications techniques, Rapports techniques et autres documents de l'IEC. Disponible pour PC, Mac OS, tablettes Android et iPad.

Recherche de publications IEC - www.iec.ch/searchpub

La recherche avancée permet de trouver des publications IEC en utilisant différents critères (numéro de référence, texte, comité d'études,...). Elle donne aussi des informations sur les projets et les publications remplacées ou retirées.

IEC Just Published - webstore.iec.ch/justpublished

Restez informé sur les nouvelles publications IEC. Just Published détaille les nouvelles publications parues. Disponible en ligne et aussi une fois par mois par email.

Electropedia - www.electropedia.org

Le premier dictionnaire en ligne de termes électroniques et électriques. Il contient plus de 30 000 termes et définitions en anglais et en français, ainsi que les termes équivalents dans 15 langues additionnelles. Egalement appelé Vocabulaire Electrotechnique International (IEV) en ligne.

Glossaire IEC - std.iec.ch/glossary

Plus de 60 000 entrées terminologiques électrotechniques, en anglais et en français, extraites des articles Termes et Définitions des publications IEC parues depuis 2002. Plus certaines entrées antérieures extraites des publications des CE 37, 77, 86 et CISPR de l'IEC.

Service Clients - webstore.iec.ch/csc

Si vous désirez nous donner des commentaires sur cette publication ou si vous avez des questions contactez-nous: csc@iec.ch.

INTERNATIONAL STANDARD

NORME INTERNATIONALE



Field Device Integration (FDI) – Part 2: FDI Client

Intégration des appareils de terrain (FDI) – Partie 2: Client FDI

INTERNATIONAL
ELECTROTECHNICAL
COMMISSION

COMMISSION
ELECTROTECHNIQUE
INTERNATIONALE

ICS 25.040.40; 35.100

ISBN 978-2-8322-2632-2

Warning! Make sure that you obtained this publication from an authorized distributor. Attention! Veuillez vous assurer que vous avez obtenu cette publication via un distributeur agréé.
--

CONTENTS

FOREWORD.....	10
INTRODUCTION.....	12
1 Scope.....	13
2 Normative references	13
3 Terms, definitions, abbreviated terms, acronyms and conventions	14
3.1 Terms and definitions.....	14
3.1.1 Terms used for Services	14
3.1.2 Terms used for Device Access Services	15
3.2 Abbreviated terms and acronyms	15
3.3 Conventions.....	15
4 Overview	16
5 FDI Client.....	17
5.1 Device Access Services	17
5.1.1 General	17
5.1.2 Device Model.....	17
5.1.3 Node model	19
5.1.4 Services	23
5.1.5 Base Property Services	27
5.1.6 Device Model Services	28
5.1.7 Locking Services	38
5.1.8 Direct Access Services	40
5.1.9 Data types	42
5.2 Hosting Services.....	47
5.2.1 General	47
5.2.2 Services	47
5.2.3 Parameter Type Definitions	53
6 UIP.....	54
6.1 UIP Services	54
6.1.1 Services	54
6.1.2 Parameter type definitions	58
6.2 UIP instantiation rules.....	59
6.3 UIP state machine.....	59
6.3.1 States	59
6.3.2 State transitions	60
6.4 UIP permissions.....	60
6.5 UIP downloads from FDI Server	61
7 Actions	62
7.1 General.....	62
7.2 Sequence diagram	63
7.3 FDI Action schema definition.....	66
8 User Interface Description (UID).....	67
8.1 Overview.....	67
8.2 UID execution	69
Annex A (normative) XML schema	72
A.1 General.....	72

A.2	AbortRequestT	72
A.3	AccessT	72
A.4	AcknowledgementRequestT	73
A.5	ActionListT	73
A.6	AbortingNotificationT	74
A.7	ActionRequestT	74
A.8	ActionResponseT	75
A.9	ActionT	76
A.10	AxisListT	76
A.11	AxisT	76
A.12	BitEnumerationItemListT	77
A.13	BitEnumerationItemT	78
A.14	ButtonListT	78
A.15	ChartT	79
A.16	ChartTypeT	79
A.17	ColorNameT	80
A.18	ColorT	81
A.19	ColorValueT	81
A.20	ColumnBreakT	82
A.21	DateTimeDataT	82
A.22	DelayMessageRequestT	82
A.23	DiagramLineT	83
A.24	EnumerationItemListT	84
A.25	EnumerationItemT	84
A.26	FormatSpecifierT	85
A.27	GraphT	85
A.28	GridT	86
A.29	HandlingT	86
A.30	ImageT	87
A.31	InfoRequestT	88
A.32	InputRequestT	88
A.33	InputResponseT	89
A.34	InputValueT	89
A.35	InputValueTypeT	90
A.36	LabelHelpT	90
A.37	LabelT	91
A.38	LineTypeT	91
A.39	MenuT	92
A.40	MenuReferenceT	93
A.41	MenuStyleT	94
A.42	NumericDataT	95
A.43	NumericTemplateT	95
A.44	OptionListT	96
A.45	OrientationT	96
A.46	ParameterInputRequestT	97
A.47	ParameterListT	97
A.48	ParameterT	98
A.49	PluginT	99
A.50	RangeListT	100

A.51	RangeT	100
A.52	ResponseT	101
A.53	RowBreakT	101
A.54	ScalingT	101
A.55	SelectionRequestT	101
A.56	SelectionResponseT	102
A.57	SeparatorT	102
A.58	SizeT	103
A.59	ParameterClassT	103
A.60	ActionClassT	105
A.61	SourceListT	106
A.62	SourceT	107
A.63	StringDataT	107
A.64	StringTemplateT	108
A.65	StringOptionListT	109
A.66	StringOptionT	109
A.67	StringT	109
A.68	TimeScaleT	110
A.69	UidLayoutInformation	110
A.70	UidRequestT	111
A.71	UidResponseT	111
A.72	UiElementSizeableT	112
A.73	UiElementT	112
A.74	UiTemplateT	113
A.75	VariantT	114
A.76	VariantOptionListT	115
A.77	VariantOptionT	115
A.78	VectorListT	116
A.79	VectorT	116
A.80	WaveformListT	117
A.81	WaveformT	117
A.82	WaveformTypeT	118
A.83	WaveformTypeHorizontalT	118
A.84	WaveformTypeVerticalT	118
A.85	WaveformTypeYTT	119
A.86	WaveformTypeXYT	120
A.87	WaveformKeyPointListT	121
A.88	WaveformVectorT	121
A.89	WaveformVectorElementListT	122
A.90	WaveformVectorElementT	122
Annex B (informative)	Action example	124
Annex C (informative)	Typical FDI Client use cases	133
C.1	General	133
C.2	Bulk operations	133
C.3	Progress bar support	133
Bibliography		135
Figure 1 – FDI architecture diagram		13

Figure 2 – Overall structure of a Device	18
Figure 3 – Structure of Blocks	18
Figure 4 – Device Model NodeClasses	19
Figure 5 – Example: Variable hierarchy representing a RECORD	22
Figure 6 – Variable hierarchy representing a VALUE_ARRAY of RECORDs	23
Figure 7 – UIP state machine	60
Figure 8 – FDI Action sequence diagram	64
Figure 9 – User Interface Descriptions	68
Figure 10 – User Interface Description sequence diagram	70
Figure B.1 – Action example (step 1)	127
Figure B.2 – Action example (step 2)	128
Figure B.3 – Action example (step 3)	129
Figure B.4 – Action example (step 4)	130
Figure B.5 – Action example (step 5)	131
Figure B.6 – Action example (step 6)	132
Figure C.1 – Progress bar support	134
Table 1 – BaseNodeClass Attributes	19
Table 2 – Object NodeClass Attributes	20
Table 3 – Variable NodeClass Attributes	20
Table 4 – Service Definition Table	24
Table 5 – StatusCode Bit Assignments	25
Table 6 – DataValue InfoBits	26
Table 7 – Service result codes	26
Table 8 – Operation level result codes	27
Table 9 – GetDeviceAccessInterfaceVersion Service parameters	28
Table 10 – GetOnlineAccessAvailability Service parameters	28
Table 11 – Browse Service parameters	29
Table 12 – CancelBrowse Service parameters	29
Table 13 – Read Service parameters	30
Table 14 – Read Service result codes	30
Table 15 – Read operation result codes	31
Table 16 – CancelRead Service parameters	31
Table 17 – Write Service parameters	32
Table 18 – Write operation result codes	33
Table 19 – CancelWrite Service parameters	33
Table 20 – CreateSubscription Service parameters	34
Table 21 – CreateSubscription Service result codes	34
Table 22 – Subscribe Service parameters	35
Table 23 – Subscribe operation result codes	36
Table 24 – Unsubscribe Service Parameters	37
Table 25 – Unsubscribe operation result codes	37
Table 26 – DeleteSubscription Service parameters	37

Table 27 – DataChangeCallback Service parameters	38
Table 28 – DataChangeCallback result codes	38
Table 29 – InitLock Service parameters	39
Table 30 – InitLock Service result codes	39
Table 31 – ExitLock Service parameters	40
Table 32 – ExitLock Service result codes	40
Table 33 – InitDirectAccess Service parameters	41
Table 34 – InitDirectAccess Service result codes	41
Table 35 – ExitDirectAccess Service parameters	41
Table 36 – ExitDirectAccess Service result codes	41
Table 37 – Transfer Service parameters	42
Table 38 – Transfer Service result codes	42
Table 39 – Base data types	43
Table 40 – Identifiers assigned to Attributes	43
Table 41 – NodeSpecifier	44
Table 42 – DataValue	44
Table 43 – InnerErrorInfo	45
Table 44 – LocalizedText Definition	45
Table 45 – LocaleId Examples	46
Table 46 – Range Data Type Structure	46
Table 47 – EUInformation Data Type Structure	47
Table 48 – EnumValueType Definition	47
Table 49 – GetClientTechnologyVersion Service parameters	48
Table 50 – OpenUserInterface Service parameters	48
Table 51 – LogAuditTrailMessage Service parameters	49
Table 52 – SaveUserSettings Service parameters	49
Table 53 – LoadUserSettings Service parameters	50
Table 54 – Trace Service parameters	50
Table 55 – ShowMessageBox Service parameters	51
Table 56 – ShowProgressBar Service parameters	51
Table 57 – UpdateShowProgressBar Service parameters	52
Table 58 – EndShowProgressBar Service parameters	52
Table 59 – StandardUIActionItemsChange Service parameters	53
Table 60 – SpecificUIActionItemsChange Service parameters	53
Table 61 – DefaultResult definition	53
Table 62 – ButtonSet definition	54
Table 63 – AcknStyle definition	54
Table 64 – Activate Service parameters	55
Table 65 – Deactivate Service parameters	55
Table 66 – SetSystemLabel Service parameters	56
Table 67 – SetTraceLevel Service parameters	56
Table 68 – GetStandardUIActionItems Service parameters	57
Table 69 – GetSpecificUIActionItems Service parameters	57

Table 70 – InvokeStandardUIAction Service parameters	57
Table 71 – InvokeSpecificUIAction Service parameters	58
Table 72 – TraceLevel definition	58
Table 73 – StandardUIAction definition	59
Table 74 – StandardUIActionItem definition	59
Table 75 – SpecificUIActionItem definition	59
Table 76 – UIP states	60
Table 77 – UIP state transitions	60
Table A.1 – Elements of AbortRequestT	72
Table A.2 – Enumerations of AccessT	73
Table A.3 – Elements of AcknowledgementRequestT	73
Table A.4 – Elements of ActionListT	73
Table A.5 – Elements of ActionRequestT	74
Table A.6 – Elements of ActionResponseT	75
Table A.7 – Elements of ActionT	76
Table A.8 – Elements of AxisListT	76
Table A.9 – Attributes of AxisT	77
Table A.10 – Elements of AxisT	77
Table A.11 – Elements of BitEnumerationItemListT	78
Table A.12 – Elements of BitEnumerationItemT	78
Table A.13 – Elements of ButtonListT	79
Table A.14 – Elements of ChartT	79
Table A.15 – Enumerations of ChartTypeT	80
Table A.16 – Enumerations of ColorNameT	81
Table A.17 – Enumerations of DateTimeDataT	82
Table A.18 – Elements of DelayMessageRequestT	83
Table A.19 – Attributes of DiagramLineT	83
Table A.20 – Elements of DiagramLineT	84
Table A.21 – Elements of EnumerationItemListT	84
Table A.22 – Elements of EnumerationItemT	85
Table A.23 – Elements of GraphT	86
Table A.24 – Elements of GridT	86
Table A.25 – Enumerations of HandlingT	87
Table A.26 – Attributes of ImageT	87
Table A.27 – Elements of ImageT	88
Table A.28 – Elements of InfoRequestT	88
Table A.29 – Elements of InputRequestT	89
Table A.30 – Elements of InputResponseT	89
Table A.31 – Elements of InputValueT	90
Table A.32 – Elements of InputValueTypeT	90
Table A.33 – Elements of LabelHelpT	91
Table A.34 – Elements of LabelT	91
Table A.35 – Enumerations of LineTypeT	92

Table A.36 – Attributes of MenuT	93
Table A.37 – Elements of MenuT	93
Table A.38 – Attributes of MenuReferenceT	94
Table A.39 – Elements of MenuReferenceT	94
Table A.40 – Enumerations of MenuStyleT	95
Table A.41 – Enumerations of NumericDataT	95
Table A.42 – Elements of NumericTemplateT	96
Table A.43 – Elements of OptionListT	96
Table A.44 – Enumerations of OrientationT	97
Table A.45 – Elements of ParameterInputRequestT	97
Table A.46 – Elements of ParameterListT	97
Table A.47 – Elements of ParameterT	99
Table A.48 – Elements of PluginT	100
Table A.49 – Elements of RangeListT	100
Table A.50 – Elements of RangeT	101
Table A.51 – Enumerations of ScalingT	101
Table A.52 – Elements of SelectionRequestT	102
Table A.53 – Elements of SelectionResponseT	102
Table A.54 – Enumerations of SizeT	103
Table A.55 – Enumerations of ParameterClassT	104
Table A.56 – Enumerations of ActionClassT	106
Table A.57 – Elements of SourceListT	106
Table A.58 – Elements of SourceT	107
Table A.59 – Enumerations of StringDataT	108
Table A.60 – Elements of StringTemplateT	108
Table A.61 – Elements of StringOptionListT	109
Table A.62 – Elements of StringOptionT	109
Table A.63 – Elements of StringT	110
Table A.64 – Enumerations of TimeScaleT	110
Table A.65 – Elements of UidLayoutInformation	111
Table A.66 – Elements of UidRequestT	111
Table A.67 – Elements of UidResponseT	112
Table A.68 – Attributes of UiElementSizeableT	112
Table A.69 – Elements of UiElementSizeableT	112
Table A.70 – Elements of UiElementT	113
Table A.71 – Elements of UiTemplateT	114
Table A.72 – Elements of VariantT	115
Table A.73 – Elements of VariantOptionListT	115
Table A.74 – Elements of VariantOptionT	116
Table A.75 – Elements of VectorListT	116
Table A.76 – Elements of VectorT	117
Table A.77 – Elements of WaveformListT	117
Table A.78 – Elements of WaveformT	118

Table A.79 – Elements of WaveformTypeHorizontalT	118
Table A.80 – Elements of WaveformTypeVerticalT	119
Table A.81 – Elements of WaveformTypeYTT	120
Table A.82 – Elements of WaveformTypeXYT	120
Table A.83 – Elements of WaveformKeyPointListT	121
Table A.84 – Attributes of WaveformVectorT	122
Table A.85 – Elements of WaveformVectorT	122
Table A.86 – Elements of WaveformVectorElementListT	122
Table A.87 – Elements of WaveformVectorElementT	123

INTERNATIONAL ELECTROTECHNICAL COMMISSION

FIELD DEVICE INTEGRATION (FDI) –**Part 2: FDI Client****FOREWORD**

- 1) The International Electrotechnical Commission (IEC) is a worldwide organization for standardization comprising all national electrotechnical committees (IEC National Committees). The object of IEC is to promote international co-operation on all questions concerning standardization in the electrical and electronic fields. To this end and in addition to other activities, IEC publishes International Standards, Technical Specifications, Technical Reports, Publicly Available Specifications (PAS) and Guides (hereafter referred to as "IEC Publication(s)"). Their preparation is entrusted to technical committees; any IEC National Committee interested in the subject dealt with may participate in this preparatory work. International, governmental and non-governmental organizations liaising with the IEC also participate in this preparation. IEC collaborates closely with the International Organization for Standardization (ISO) in accordance with conditions determined by agreement between the two organizations.
- 2) The formal decisions or agreements of IEC on technical matters express, as nearly as possible, an international consensus of opinion on the relevant subjects since each technical committee has representation from all interested IEC National Committees.
- 3) IEC Publications have the form of recommendations for international use and are accepted by IEC National Committees in that sense. While all reasonable efforts are made to ensure that the technical content of IEC Publications is accurate, IEC cannot be held responsible for the way in which they are used or for any misinterpretation by any end user.
- 4) In order to promote international uniformity, IEC National Committees undertake to apply IEC Publications transparently to the maximum extent possible in their national and regional publications. Any divergence between any IEC Publication and the corresponding national or regional publication shall be clearly indicated in the latter.
- 5) IEC itself does not provide any attestation of conformity. Independent certification bodies provide conformity assessment services and, in some areas, access to IEC marks of conformity. IEC is not responsible for any services carried out by independent certification bodies.
- 6) All users should ensure that they have the latest edition of this publication.
- 7) No liability shall attach to IEC or its directors, employees, servants or agents including individual experts and members of its technical committees and IEC National Committees for any personal injury, property damage or other damage of any nature whatsoever, whether direct or indirect, or for costs (including legal fees) and expenses arising out of the publication, use of, or reliance upon, this IEC Publication or any other IEC Publications.
- 8) Attention is drawn to the Normative references cited in this publication. Use of the referenced publications is indispensable for the correct application of this publication.

International Standard IEC 62769-2 has been prepared by subcommittee 65E: Devices and integration in enterprise systems, of IEC technical committee 65: Industrial-process measurement, control and automation.

The text of this standard is based on the following documents:

CDV	Report on voting
65E/345/CDV	65E/422/RVC

Full information on the voting for the approval of this standard can be found in the report on voting indicated in the above table.

This publication has been drafted in accordance with the ISO/IEC Directives, Part 2.

A list of all parts in the IEC 62769 series, published under the general title *Field Device Integration (FDI)*, can be found on the IEC website.

The committee has decided that the contents of this publication will remain unchanged until the stability date indicated on the IEC web site under "<http://webstore.iec.ch>" in the data related to the specific publication. At this date, the publication will be

- reconfirmed,
- withdrawn,
- replaced by a revised edition, or
- amended.

IMPORTANT – The 'colour inside' logo on the cover page of this publication indicates that it contains colours which are considered to be useful for the correct understanding of its contents. Users should therefore print this document using a colour printer.

INTRODUCTION

The International Electrotechnical Commission (IEC) draws attention to the fact that it is claimed that compliance with this document may involve the use of patents concerning

- a) Method for the supplying and installation of device-specific functionalities, see Patent Family DE10357276;
- b) Method and device for accessing a functional module of automation system, see Patent Family EP2182418;
- c) Methods and apparatus to reduce memory requirements for process control system software applications, see Patent Family US2013232186;
- d) extensible device object model, see Patent Family US12/893,680.

IEC takes no position concerning the evidence, validity and scope of this patent right.

The holders of these patent rights have assured the IEC that he/she is willing to negotiate licences either free of charge or under reasonable and non-discriminatory terms and conditions with applicants throughout the world. In this respect, the statement of the holder of this patent right is registered with IEC. Information may be obtained from:

- a) ABB Research Ltd
Claes Ryttoft
Affolterstrasse 4
Zurich, 8050
Switzerland
- b) Phoenix Contact GmbH & Co KG
Intellectual Property, Licenses & Standards
Flachsmarktstrasse 8, 32825 Blomberg
Germany
- c) Fisher Controls International LLC
John Dilger, Emerson Process Management LLLP
301 S. 1st Avenue, Marshalltown, Iowa 50158
USA
- d) Rockwell Automation Technologies, Inc.
1 Allen-Bradley Drive
Mayfield Heights, Ohio 44124
USA

Attention is drawn to the possibility that some of the elements of this document may be the subject of patent rights other than those identified above. IEC shall not be held responsible for identifying any or all such patent rights.

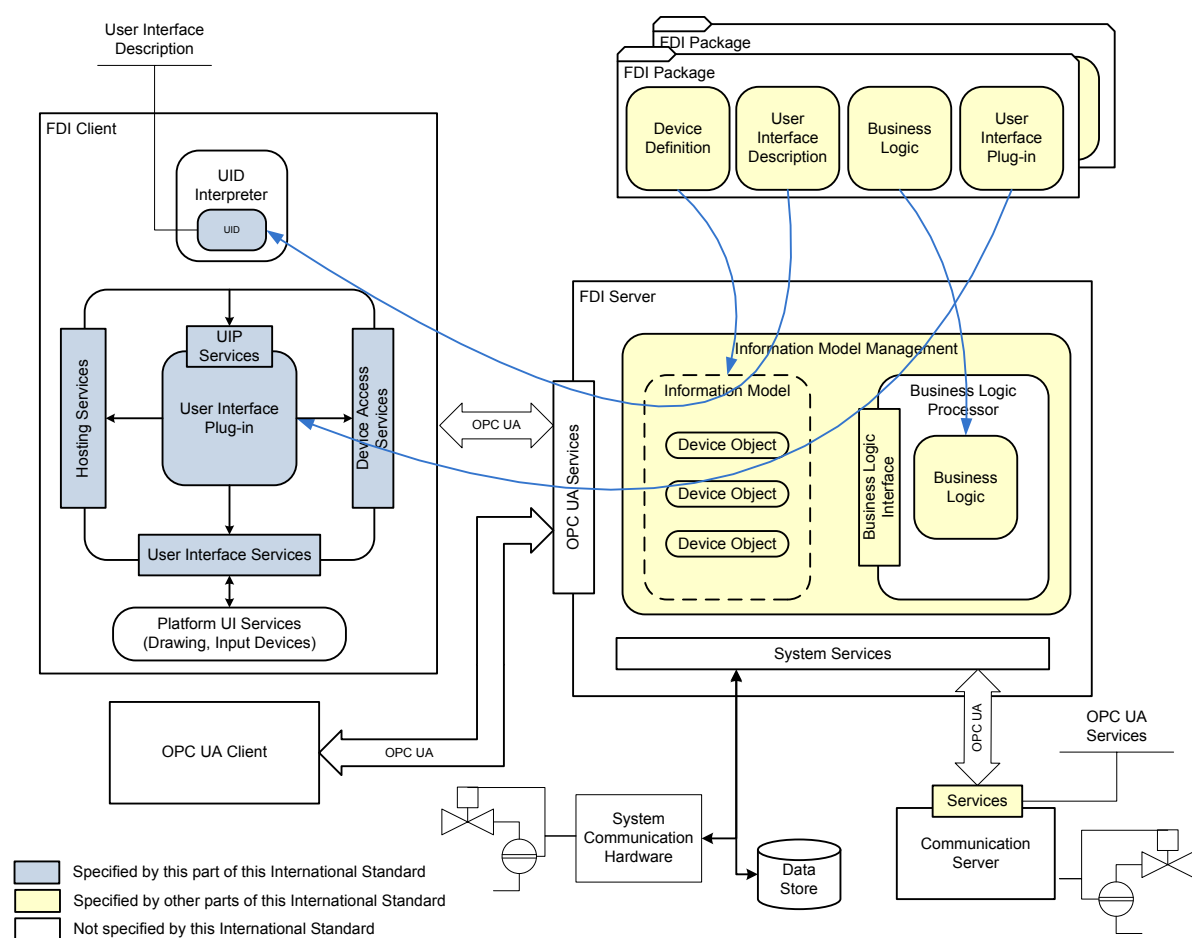
ISO (www.iso.org/patents) and IEC (<http://patents.iec.ch>) maintain on-line data bases of patents relevant to their standards. Users are encouraged to consult the data bases for the most up to date information concerning patents.

FIELD DEVICE INTEGRATION (FDI) –

Part 2: FDI Client

1 Scope

This part of IEC 62769 specifies the FDI Client. The overall FDI architecture is illustrated in Figure 1. The architectural components that are within the scope of this document have been highlighted in this figure.



IEC

Figure 1 – FDI architecture diagram

2 Normative references

The following documents, in whole or in part, are normatively referenced in this document and are indispensable for its application. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

IEC 62769-1, *Field Device Integration (FDI) – Part 1: Overview*

NOTE IEC 62769-1 is technically identical to FDI-2021

IEC 62769-3, *Field Device Integration (FDI) – Part 3: FDI Server*

NOTE IEC 62769-3 is technically identical to FDI-2023.

IEC 62769-4:2015, *Field Device Integration (FDI) – Part 4: FDI Packages*

NOTE IEC 62769-4 is technically identical to FDI-2024.

IEC 62769-5, *Field Device Integration (FDI) – Part 5: FDI Information Model*

NOTE IEC 62769-5 is technically identical to FDI-2025.

IEC 62769-6:2015, *Field Device Integration (FDI) – Part 6: FDI Technology Mapping*

NOTE IEC 62769-6 is technically identical to FDI-2026.

IEC 62541-3, *OPC Unified Architecture – Part 3: Address Space Model*

IEC 62541-4, *OPC Unified Architecture – Part 4: Services*

ISO 639, *Codes for the representation of names of languages*

ISO 3166, *Codes for the representation of names of countries and their subdivisions*

ISO/IEC 10918-1, *Information technology – Digital compression and coding of continuous-tone still images: Requirements and guidelines*

ISO/IEC 15948, *Information technology – Computer graphics and image processing – Portable Network Graphics (PNG): Functional specification*

IEEE 754, *IEEE Standard for Floating-Point Arithmetic*

IETF RFC 2083, *PNG (Portable Network Graphics) Specification Version 1.0*

IETF RFC 3066, *Tags for the Identification of Languages*

XML Schema-1, *XML Schema: Structures* (available at <http://www.w3.org/TR/xmlschema-1/>)

XML Schema-2, *XML Schema: Datatypes* (available at <http://www.w3.org/TR/xmlschema-2/>)

3 Terms, definitions, abbreviated terms, acronyms and conventions

3.1 Terms and definitions

For the purposes of this document, the terms and definitions given in IEC 62769-1 as well as the following apply.

3.1.1 Terms used for Services

3.1.1.1

Locking Services

set of Services through which access to a Device is controlled

3.1.1.2

Device Model Services

sub-set of the Device Access Services through which a UIP can access the information of a Device

3.1.1.3**Direct Access Services**

sub-set of the Device Access Services through which a UIP can directly access a Device

3.1.2 Terms used for Device Access Services**3.1.2.1****Attribute**

information element of a Node

Note 1 to entry: Some Attributes exist for all NodeClasses and some are specific to a given NodeClass.

Note 2 to entry: Supersedes the definition given in IEC 62769-1.

3.1.2.2**Device Model**

hierarchy of Nodes that represents an existing Device

3.1.2.3**Node**

element in the Device Model that can be addressed via the Device Access Services

Note 1 to entry: Supersedes the definition given in IEC 62769-1.

3.1.2.4**NodeClass**

either an Object or a Variable

Note 1 to entry: Supersedes the definition given in IEC 62769-1.

3.1.2.5**Object**

instance of the Object NodeClass

Note 1 to entry: Supersedes the definition given in IEC 62769-1.

3.1.2.6**Variable**

instance of the Variable NodeClass

Note 1 to entry: Supersedes the definition given in IEC 62769-1.

3.2 Abbreviated terms and acronyms

For the purposes of this document, the abbreviated terms and acronyms given in IEC 62769-1 and the following apply.

UTC	Coordinated Universal Time
XML	Extended mark-up language

3.3 Conventions

Conventions for service definitions are identical to IEC 62541-4.

Basic data types used are defined in IEC 62541-3.

"Parameter" is always an Information Model Parameter. "parameter" is the general use of the word. If ambiguous, additional context is given.

4 Overview

An FDI Package provides the necessary information for a Device Type to allow managing the Device within the system. It is provided by a device vendor and deployed in an FDI Server. It may contain two types of user interface components that are available to the FDI Client for display to a user. An FDI Package may contain only one type or both types. The two types are referred to as User Interface Plug-ins and User Interface Descriptions.

A User Interface Plug-in (UIP) is an executable element. A UIP is provided by an FDI Package and transferred to the FDI Client by the FDI Server. A UIP provides a set of UIP Services that the FDI Client uses to initialize and interact with the UIP.

NOTE 1 IEC 62769-6 defines application programming interfaces for the services described in this document.

User Interface Descriptions (UIDs) are defined using EDDL. A UID is provided to the FDI Client by the FDI Server. The FDI Client uses the UID Interpreter to interpret and execute the UID. A UID may make use of other UID and UIP components as subcomponents in order to provide a modular approach and make the best use of both descriptive and executable user interface elements.

NOTE 2 UIPs can make use of other UIPs but not of other UIDs.

The FDI Server makes UIDs and UIPs available to the FDI Client via the Information Model. The Information Model organizes the UIDs and UIPs by Device Type.

The FDI Client provides the execution environment for UIPs. The FDI Client loads the UIP from the FDI Server.

The FDI Client's UIP execution environment consists of the following sets of services that are made available to the UIP:

- Device Access Services
- Hosting Services
- User Interface Services
- Printing Services (if available in the hosting environment)

NOTE 3 It is implementation specific whether different UIPs get the same or different interface instances to access these services. The only requirement is that there is no side-effect if two UIPs use the same instance of an interface.

Similar to the FDI Client, each UIP shall also provide a set of services (UIP Services) by which the FDI Client activates, controls and shuts down UIPs (see 6.1).

The Device Access Services enable interaction between the UIP and the FDI Server-maintained Information Model. The FDI Client takes care of the interaction with the FDI Server freeing the UIP to focus on the application level only.

UIP access to the Information Model (IM) via the Device Access Services is restricted to the Device and its sub-devices.

The Hosting Services are provided by the FDI Client for use by the UIP. The Hosting Services include services related to the FDI Client allowing the UIP to acquire information about the environment.

The User Interface Services provide the means by which the UIP accesses the user interface services of the underlying operating system. These services provide access to the screen, keyboard, mouse, and other operating system resources. The User Interface Services are defined by the chosen implementation technology and therefore no additional definitions are included in this document (see IEC 62769-6). UIPs shall use the Hosting Services to display

message boxes or progress bars and shall never use comparable services provided by the underlying operating system.

There are no printing services provided by the Client. If a UIP needs to generate a printout it accesses the printing services of the underlying operating system. No additional definitions are included in this document.

The FDI Client uses the culture setting of the currently signed-in user for the execution environment of the UIP. It uses the culture when creating OPC UA Sessions and it sets the culture for each thread it creates. UIPs shall take care for culture setting in all threads that they create.

The FDI Client provides a UID Interpreter that is used to interpret and execute UIDs. The UID XML Schema is defined in this document (see Annex A).

Business Logic is executed in the FDI Server. Some Business Logic may be exposed to the FDI Client as Actions (see Clause 7) and can be triggered by FDI Clients.

5 FDI Client

5.1 Device Access Services

5.1.1 General

The Device Access Services provide access to both the online and offline information of a Device or its components as defined by the FDI Package, in particular for

- browsing the Device Model,
- reading / writing of data and subscribing to data changes,
- controlling access to the Device, and
- directly communicating with the Device.

The scope for the Device Access Services will be a Device, Block, or Communication Server to which the UIP is assigned. The FDI Client is expected to map the Device Access Services to OPC UA services provided by the FDI Server.

The main Services are the Device Model Services to view and access Parameters. The Locking Services are used to control simultaneous access to a Device. The Direct Access Services allow a UIP to communicate with the Device.

Whether and how the different services are mapped to real interfaces is defined in IEC 62769-6. IEC 62769-6 also specifies how interfaces are obtained.

5.1.2 Device Model

The Device Model defines the structure of all data that are available to a UIP. It is confined to a single device instance. The entities in the structure (Parameters, Images, and Documents) are built from FDI Package information. User interface elements, like Menus, Graphs, Waveforms, are not part of the UIP Device Model.

All Device elements are organized as a defined hierarchy. The root of the hierarchy can be either a Device or a Block subject to where (by which MENU) the UIP is referenced in the User Interface Description of the FDI Package (see IEC 62769-4). The Nodes in the hierarchy are either Objects or Variables, where the main difference is that Variables provide a Value.

Figure 2 illustrates the overall structure of a Device. Figure 3 shows the structure of a block in more detail. The elements that will really be available depend on the contents of the

respective FDI Package. Regular rectangles represent Object Nodes while the ones with rounded corners represent Variable Nodes. These NodeClasses are defined in 5.1.3. The top left Node is the root Node. The single hashed lines define the parent-child relationship in the hierarchy. As an example, the children of Device1 in Figure 2 are ParameterSet, ImageSet, Documentation, Blocks and SubDevices. The children of /SubDevices/Device_1b/ImageSet are Image_1 and Image_2.

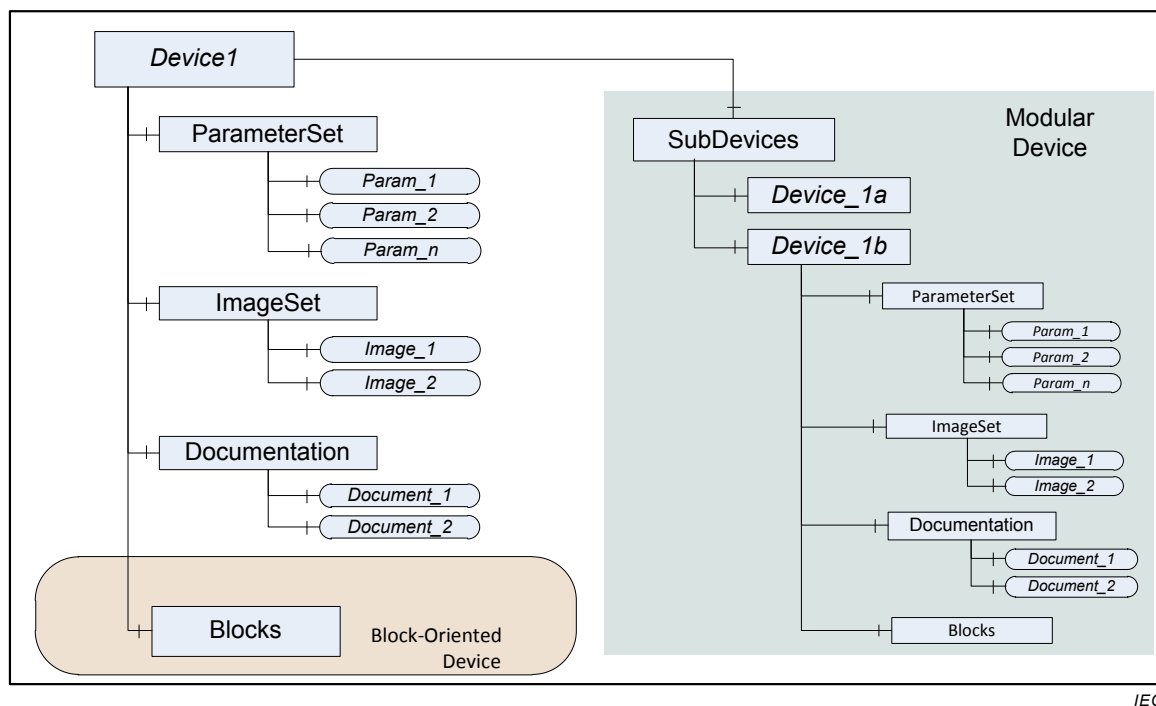


Figure 2 – Overall structure of a Device

Names that are in normal font are defined by the Device Access Services; names in italics are just place-holders for the real names as defined in the FDI Package.

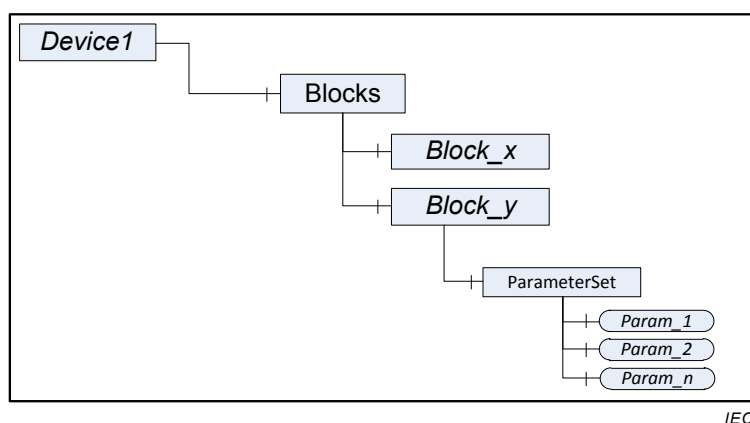


Figure 3 – Structure of Blocks

Each Node in the hierarchy is uniquely qualified with its pathname. This pathname is a concatenation of the individual Node Name Attributes. The separator is “/”.

EXAMPLE The following examples illustrate qualified pathnames:

```

/                               -- the root Node (here "Device1")

/ParameterSet/Param_2         -- a Variable Node

/SubDevices/Device_1b/ImageSet/Image_1  -- a picture

```

Certain Variables in the FDI Package may be tagged as “private” meaning they are non-browsable. Though they are non-browsable, they exist in the Device Model and can be addressed with their pathname.

5.1.3 Node model

5.1.3.1 General

The information of a Device is organised as a hierarchy of Nodes. Each Node is either an Object or a Variable. Both Object and Variable are derived from the BaseNodeClass, as illustrated in Figure 4.

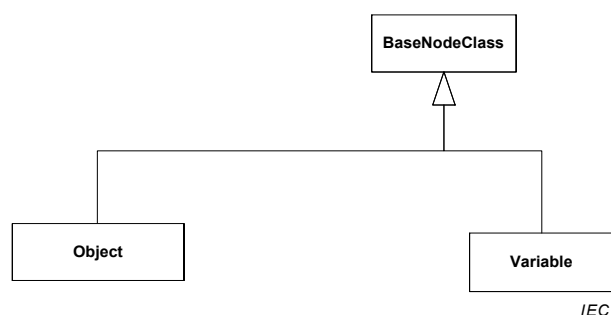


Figure 4 – Device Model NodeClasses

5.1.3.2 BaseNodeClass

This is the abstract parent NodeClass for Object and Variable Nodes. The BaseNodeClass Attributes are shown in Table 1. The Attributes of this NodeClass are available in both Object and Variable NodeClasses.

Table 1 – BaseNodeClass Attributes

Attribute	Datatype	Description
NodePath	String	Qualified path of the Node in the Device Model. This Attribute is returned by the Browse Service and cannot be read or written.
Name	String	Name of Node according to device specific documentation.
Label	LocalizedText	Human readable label of the Node.
Description (optional)	LocalizedText	Human readable help string describing the Node.

Additional Attributes will be available for derived NodeClasses. For example, a Variable will have Attributes that define the DataType and the AccessRights.

5.1.3.3 Object NodeClass

Table 2 contains the list of Attributes for Objects beyond those inherited from the Base NodeClass:

Table 2 – Object NodeClass Attributes

Attribute	Datatype	Description
LockedStatus	Boolean	This Attribute when “true” indicates that this Object is currently locked. Bad_AttributeInvalid defines that locking is not supported for this Object at all.

5.1.3.4 Variable NodeClass

5.1.3.4.1 General

Variables are used to represent Parameters, images and documents. When reading Variables that represent images or documents the system will provide them as a ByteString. For documents, the Name Attribute will consist of the filename including the extension that can be used to identify the document type. FDI supports “.pdf” and “.txt”. For the representation of images see 5.1.3.4.2.

Table 3 contains the list of Attributes for Variables beyond those inherited from the BaseNodeClass:

Table 3 – Variable NodeClass Attributes

Attribute	Datatype	Description
Value	Variant	The value of the Variable as returned from the device (i.e. without applying the ScalingFactor). The Variant is specified in 5.1.9.4.
DataType	UInt32	The DataType Attribute specifies the data type of the Value Attribute. One of the data types specified in 5.1.9. IEC 62769-6 specifies the assignment of unique identifiers to each type.
ValueRank	Int32	Indicates whether the Value Attribute is an array. It may have the following values: >1 (MoreDimensions) – the value is an array with the specified number of dimensions. 1 (OneDimension) – the value is an array with one dimension. 0 (OneOrMoreDimensions) – the value is an array with one or more dimensions. -1 (Scalar) – the value is not an array. -2 (Any) – the value can be a scalar or an array with any number of dimensions. -3 (ScalarOrOneDimension) – the value can be a scalar or a one dimensional array.
ArrayDimensions (optional)	UInt32[]	Specifies the length of each dimension for an array value. The Attribute is intended to describe the capability of the Variable, not the current size. The number of elements shall be equal to the value of the ValueRank Attribute. Shall be null if ValueRank <= 0. A value of 0 for an individual dimension indicates that the dimension has a Variable length. For example, if a Variable is defined by the following C array: Int32 myArray[346]; then this Variable's DataType would point to an Int32, the Variable's ValueRank has the value 1 and the ArrayDimensions is an array with one entry having the value 346.
AccessRights	Byte	The access rights for the Value. An enumeration with one of the following values: NONE_0 The Variable value cannot be accessed READ_1 The value of the Variable may be read WRITE_2 The value of the Variable may be written READORWRITE_3 The value of the Variable may be read or written
UserAccessRights	Byte	This Attribute specifies the access rights to the Value for the currently authenticated user. They may be less than the potential access rights. The same enumeration as for AccessRights is used.
ScalingFactor (optional)	Double	This Attribute specifies a suggested scaling factor. Note that the Value Attribute contains the raw value returned from the device. It is assumed, that the (raw) value is multiplied by this factor before being displayed.
EngineeringUnits (optional)	EUInformation	EngineeringUnits specifies the units for the value (e.g., °C, hertz, seconds). See 5.1.9.3.8 for the definition of the EUInformation data type.
Attributes for analog Variables (Variables that represent continuously-variable physical quantities (e.g., pressure, temperature)).		
EURange (optional)	Range[]	Defines one or more value ranges likely to be obtained in normal operation. They are intended for such use as automatically scaling a bar graph display. Sensor or instrument failure or deactivation can result in a returned item value that

Attribute	Datatype	Description
		is actually outside this range. UIP software shall be prepared to deal with this. See 5.1.9.3.7 for the definition of the Range data type. Ranges may change during operation , for example by changing the operation mode of an instrument. Like the Value itself, Ranges are always unscaled (i.e. without applying the ScalingFactor).
Attributes for discrete (enumerated) Variables (for data that may take on only a certain number of possible values (e.g., OPENING, OPEN, CLOSING, CLOSED)).		
CurrentLabel	String	Enumerated Variables expose the current numeric state in their Value Attribute. The CurrentLabel Attribute provides the name of the current enumeration value.
EnumValues	EnumValuesType[]	EnumValues is an array of {StateValue, Enumeration Name, and Help Information}. See 5.1.9.3.9 for the definition of this type. FDI Clients/UIPs may read this Attribute in advance and store it for lookup of name or help when they receive the numeric representation.
Attributes for bit-enumerated Variables (for data that represent a bit mask).		
OptionNames	String[]	Bit-enumerated Variables transmit a bit mask encoded in an unsigned integer of a length that is sufficient to represent all bits. The OptionNames Attribute provides a human-readable representation for each valid bit of the bit mask. The order of the bits of the bit mask points to a position of the array of Strings in the OptionNames Attribute, i.e., the first bit points to the first entry in the array, and so on. The array contains an empty String for each bit that has no specific meaning.

5.1.3.4.2 Representation of images

All images have the DataType and are transferred as a ByteString. FDI supports three image formats. To identify the format of the image provided in the ByteString, the initial bytes have to be parsed as outlined in the following table.

Image type	Description																		
GIF	Defines an image in GIF (Graphics Interchange Format). GIF is specified in http://www.w3.org/Graphics/GIF/spec-gif89a.txt . The first bytes of a GIF image are as follows: <table><tr><td>Byte</td><td>1</td><td>2</td><td>3</td></tr><tr><td>Hex</td><td>47</td><td>49</td><td>46</td></tr></table>	Byte	1	2	3	Hex	47	49	46										
Byte	1	2	3																
Hex	47	49	46																
JPG	Defines an image in JPG (Joint Photographic Experts Group File Interchange Format). JPG is defined in ISO/IEC 10918-1. The first bytes of a JPG image are as follows: <table><tr><td>Byte</td><td>1</td><td>2</td><td>3</td><td>4</td></tr><tr><td>Hex</td><td>FF</td><td>D8</td><td>FF</td><td>E0</td></tr></table>	Byte	1	2	3	4	Hex	FF	D8	FF	E0								
Byte	1	2	3	4															
Hex	FF	D8	FF	E0															
PNG	Defines an image in PNG (Portable Network Graphics format). PNG is defined in IETF RFC 2083 and ISO/IEC 15948. The first bytes of a PNG image are as follows: <table><tr><td>Byte</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td></tr><tr><td>Hex</td><td>89</td><td>50</td><td>4E</td><td>47</td><td>0D</td><td>0A</td><td>1A</td><td>0A</td></tr></table>	Byte	1	2	3	4	5	6	7	8	Hex	89	50	4E	47	0D	0A	1A	0A
Byte	1	2	3	4	5	6	7	8											
Hex	89	50	4E	47	0D	0A	1A	0A											

5.1.3.4.3 Representation of records

A Variable hierarchy is used to represent EDDL RECORD Parameters. The root Variable represents the record itself. It will have component Variables that represent the EDDL RECORD MEMBERS (the MEMBERS of an EDDL RECORD are defined in EDDL by means of a reference to an EDDL VARIABLE.).

An example of how a record is represented in the Device Model is shown in Figure 5.

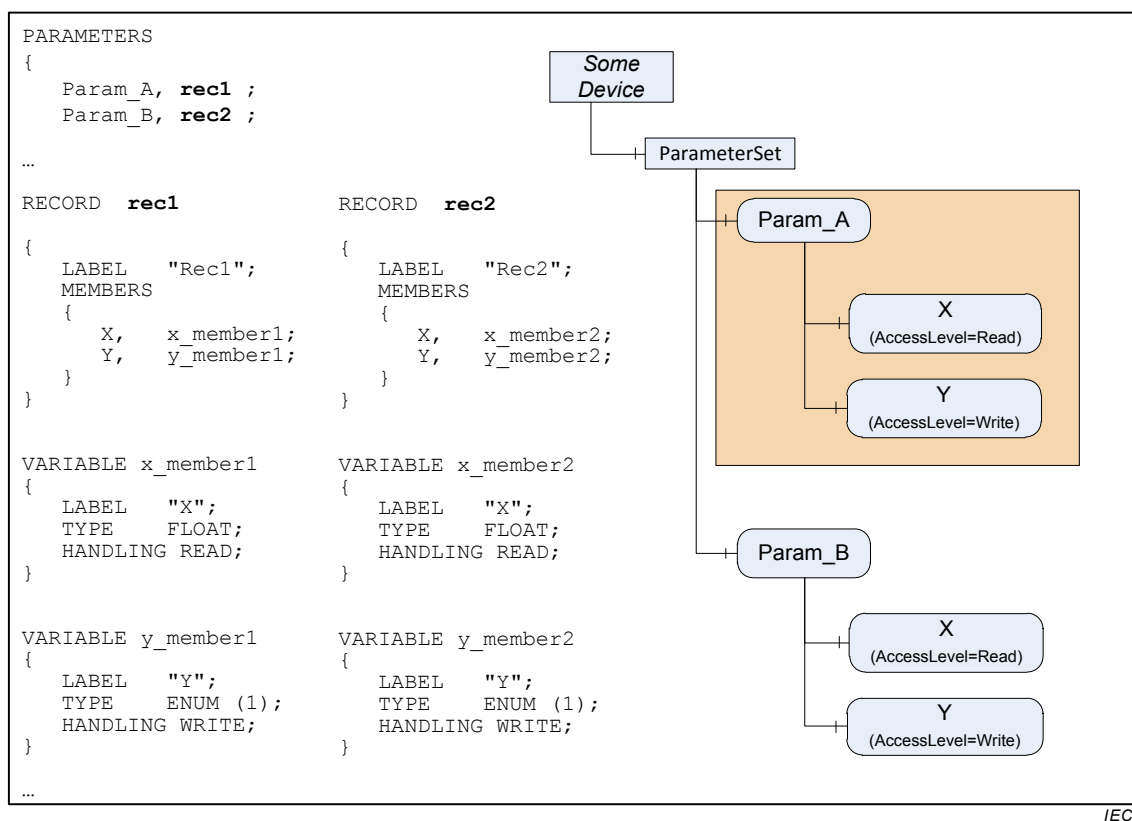


Figure 5 – Example: Variable hierarchy representing a RECORD

The Name and Label Attributes of the root Variable are set to the name of the RECORD and the LABEL Attribute, respectively. The DataType Attribute of the “root” Variable is Variant. The ValueRank Attribute is used to specify that the Value contains an array. The Value Attribute represents the values of all members in the order as defined for the RECORD. According to the example in Figure 5, the first variant will be a FLOAT and the second will be an ENUM.

For each component Variable that represents an EDDL RECORD MEMBER

- the Name Attribute is set to the identifier of the corresponding EDDL VARIABLE,
- the Label is the LABEL Attribute of the corresponding EDDL VARIABLE,
- the Description is the HELP Attribute of the corresponding EDDL VARIABLE, and
- the AccessRights Attribute is derived from the EDDL HANDLING Attribute.

Each member of the record can be accessed with its pathname as specified above. Browsing can also be used.

EXAMPLE Example of a pathname: /ParameterSet/Param_A/X.

5.1.3.4.4 Representation of arrays, and lists of members with simple data types

A single Variable will represent an EDDL VALUE_ARRAY or LIST item when the data type of the referenced array element has a simple data type.

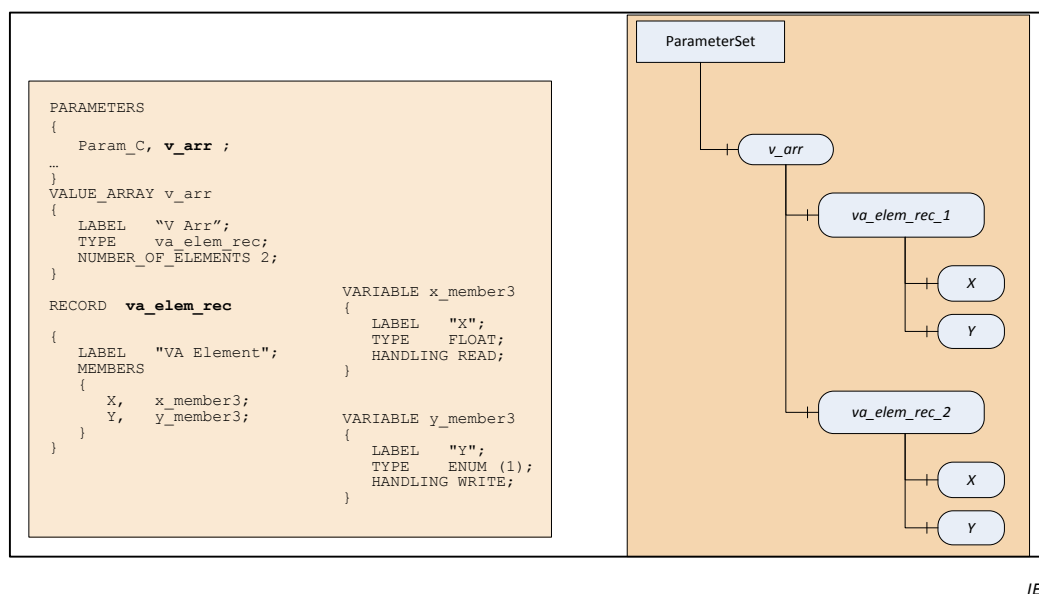
The DataType Attribute is set to one of the base data types (see 5.1.9.2).

The ValueRank Attribute is used to specify that the Value contains an array. In case of an EDDL VALUE_ARRAY the number of elements is exposed via the ArrayDimensions Attribute.

In the case of an EDDL LIST the number of elements is unspecified since the size can change dynamically.

5.1.3.4.5 Representation of arrays, and lists of RECORDs

Value arrays or lists of non-simple EDDL Parameters will be represented as an array of Variable hierarchies. Figure 6 shows the EDDL sample code of a VALUE_ARRAY of RECORDs and the corresponding Variable hierarchy.



IEC

Figure 6 – Variable hierarchy representing a VALUE_ARRAY of RECORDs

The Name and Label Attributes of the root Variable are set to the name of the ARRAY and the LABEL Attribute, respectively. The DataType Attribute of the “root” DataVariable is Variant. The ValueRank Attribute is used to specify that the Value contains an array. The Value Attribute represents all VALUE_ARRAY entries. The first Variant corresponds to the first array entry and so on. Each Variant in turn contains an array. This may either be an array of simple types or an array of Variants. A RECORD is always represented as an array of Variant.

The VALUE_ARRAY element, which is in fact a RECORD, is represented as a component Variable hierarchy. The Name and Label Attributes of each root Variable representing a RECORD are set to the name of the RECORD and the LABEL Attribute, respectively. The array index (`_1`, `_2`) is appended to allow unique identification. Note that the index always begins with ‘1’.

The RECORD MEMBERS are also represented as component Variables as specified in 5.1.3.4.3.

Members of each record can be accessed with a pathname. Browsing can also be used.

EXAMPLE Example of a pathname: `/ParameterSet/v_arr/va_elem_rec_2/Y`.

5.1.4 Services

5.1.4.1 General

All Services specified in this part of IEC 62769 rely on the conventions defined in 5.1.4.2. They are specified in an abstract manner with request and response parameters in a single table.

Programmatic access to the services may be synchronous or asynchronous (see IEC 62769-6). However, asynchronicity exists to maintain responsiveness of the User Interface. Service execution shall always be assumed to be sequential.

5.1.4.2 Conventions for service definitions

The service specifications use tables to describe service parameters, as shown in Table 4. Parameters are organised in this table into request parameters and response parameters.

Table 4 – Service Definition Table

Name	Type	Description
Request		Defines the request parameters of the service
simple Parameter Name		Description of this parameter
constructed Parameter Name	structure	Description of the constructed parameter
component Parameter Name		Description of the component parameter
Response		Defines the response parameters of the service

The Name, Type and Description columns contain the name, data type and description of each parameter. All parameters are mandatory, although some may be unused under certain circumstances. The description column specifies the value to be supplied when a parameter is unused. Parameter names always begin with a lower case character. This allows differentiating if name and type are the same, for example, name = “nodeId”, type = “Nodeld”.

Two types of parameters are defined in these tables, simple and constructed. Simple parameters have a simple data type, such as Boolean or String.

Constructed parameters are composed of two or more component parameters, which can be simple or constructed. Component parameter names are indented below the constructed parameter name.

The data types used in these tables may be base types or service-specific types. Base data types are listed in 5.1.9. Data types that are service-specific are defined in the parameter table of the service.

5.1.4.3 Auditing

Auditing is a requirement in many systems. It provides a means for tracking activities that occur as part of normal operation of the system. It also provides a means for tracking abnormal behaviour. It is also a requirement from a security standpoint.

When an audit trail is maintained by the system, audit trail records for the Write Service invoked by the UIP will be implicitly created by the system.

In addition, UIPs have means for providing additional audit context information for things that the system cannot know:

- They can call the LogAuditTrailMessage service (see 5.2.2.5). This shall be executed in particular when the DirectAccess Services are used. The UIP shall include sufficient information in the message for precise description of the activity performed.
- They can provide a context text when using the Locking Services (see 5.1.7) or the Direct Access Services (see 5.1.8).

5.1.4.4 Services and operations

Several of the Device Model Services (e.g. Read and Write) allow specifying an array of elements that shall be processed. The processing of each individual element is called an

“operation”. This is an important differentiation for error handling as a service execution is considered successful even if individual operations fail. The following list explains the differences between the service and operation result codes.

- Service result code

If a service succeeds, the result code is “Good” and the response parameters are valid. If it fails, the service result code is any of the “Bad_...” service failure codes defined in 5.1.4.6. In such a case an InnerErrorInfo (see 5.1.9.3.4) may be provided as well. Programmatic access to service failure codes is specific to the technology and may be based on exceptions (see IEC 62769-6).

- Operation result code

The result code for each operation is returned as part of the service-specific response parameters (see 5.1.4.7 for the list of available operation result codes).

Operations can return an InnerErrorInfo with each result code other than “Good” if returnInnerErrorInfo=”true” in the service request.

5.1.4.5 StatusCode

The StatusCode used for service results or operational results reports the outcome of an operation. It is a 32-bit unsigned integer. The top 16 bits represent the numeric value of the code that shall be used for detecting specific errors or conditions. The bottom 16 bits are bit flags that contain additional information but do not affect the meaning of the StatusCode.

UIPs shall always check the StatusCode associated with a result before using it. Results that have an uncertain/warning status associated with them shall be used with care since these results might not be valid in all situations. Results with a bad/failed status shall never be used.

The exact bit assignments are shown in Table 5. IEC 62769-6 provides functions to help in evaluation of the StatusCode.

Table 5 – StatusCode Bit Assignments

Field	Bit Range	Description															
Severity	30 .. 31	Indicates whether the StatusCode represents a good, bad or uncertain condition. These bits have the following meanings:															
		<table><tr><th>Severity</th><th>Bits</th><th>Description</th></tr><tr><td>Good Success</td><td>00</td><td>The operation was successful; results may be used.</td></tr><tr><td>Uncertain Warning</td><td>01</td><td>The operation was partially successful; results might not be suitable for some purposes.</td></tr><tr><td>Bad Failure</td><td>10</td><td>The operation failed and any associated results cannot be used.</td></tr><tr><td>Reserved</td><td>11</td><td>Reserved for future use. Should also be treated as "Bad".</td></tr></table>	Severity	Bits	Description	Good Success	00	The operation was successful; results may be used.	Uncertain Warning	01	The operation was partially successful; results might not be suitable for some purposes.	Bad Failure	10	The operation failed and any associated results cannot be used.	Reserved	11	Reserved for future use. Should also be treated as "Bad".
		Severity	Bits	Description													
		Good Success	00	The operation was successful; results may be used.													
		Uncertain Warning	01	The operation was partially successful; results might not be suitable for some purposes.													
		Bad Failure	10	The operation failed and any associated results cannot be used.													
Reserved	11	Reserved for future use. Should also be treated as "Bad".															
Reserved	29 .. 28	Reserved for future use. Shall always be zero.															
SubCode	16 .. 27	The code is a numeric value assigned to represent different conditions. Each code has a symbolic name and a numeric value. All descriptions in this specification refer to the symbolic name. IEC 62769-6 maps the symbolic names onto a numeric value.															
Reserved	12 .. 15	Reserved for future use. Shall always be zero.															
InfoType	10 .. 11	The type of information contained in the info bits. These bits have the following meanings:															
		<table><tr><th>InfoType</th><th>Bits</th><th>Description</th></tr><tr><td>NotUsed</td><td>00</td><td>The info bits are not used and shall be set to zero.</td></tr><tr><td>DataValue</td><td>01</td><td>The StatusCode and its info bits are associated with a data value returned from the FDI Server.</td></tr><tr><td>Reserved</td><td>1X</td><td>Reserved for future use. The info bits shall be ignored.</td></tr></table>	InfoType	Bits	Description	NotUsed	00	The info bits are not used and shall be set to zero.	DataValue	01	The StatusCode and its info bits are associated with a data value returned from the FDI Server.	Reserved	1X	Reserved for future use. The info bits shall be ignored.			
		InfoType	Bits	Description													
		NotUsed	00	The info bits are not used and shall be set to zero.													
		DataValue	01	The StatusCode and its info bits are associated with a data value returned from the FDI Server.													
Reserved	1X	Reserved for future use. The info bits shall be ignored.															
InfoBits	0 .. 9	Additional information bits that depend on the Info Type field.															

Table 6 describes the structure of the InfoBits when the Info Type is set to DataValue (01).

Table 6 – DataValue InfoBits

Info Type	Bit Range	Description		
LimitBits	8 .. 9	The limit bits associated with the data value. The limits bits have the following meanings:		
		Limit	Bits	Description
		None	00	The value is free to change.
		Low	01	The value is at the lower limit for the data source.
		High	10	The value is at the higher limit for the data source.
		Constant	11	The value is constant and cannot change.
Overflow	7	If this bit is set, not every detected change has been returned since the FDI Server's queue buffer for the subscribed Variable reached its limit and had to purge out data.		
Reserved	0 .. 6	Reserved for future use. Shall always be zero.		

5.1.4.6 Service failure codes

Table 7 defines result codes for the services. The column Service in Table 7 lists which code may be returned by which service. Result codes that are specific to an individual service are also specified with the service.

Table 7 – Service result codes

Exception name code	Description	Service
Bad_AlreadyLocked	The Node is already locked by another FDI Client.	InitLock
Bad_LockRequired	The passed Node is not yet locked.	Write, DirectAccess
Bad_MaxAgeInvalid	The max age parameter is invalid.	Read
Bad_NodeInvalid	The identifier does not refer to a valid Node in the Device Model. This result code is used both as service- and as operation-level result code.	Browse, InitLock, ExitLock
Bad_NothingToDo	There was nothing to do because the caller passed an empty list of operations.	Read , Write, Subscribe, Unsubscribe
Bad_NotSupported	The Service is not supported for the specified Node or for the Device/Block in general.	Locking, DirectAccess
Bad_RequestCancelled	The request was cancelled by Client / UIP.	All
Bad_SubscriptionIdInvalid	The subscription id is not valid.	Subscribe, Unsubscribe, DeleteSubscription
Bad_Timeout	The operation timed out.	All
Bad_TooManySubscriptions	The FDI Server has reached its maximum number of subscriptions.	CreateSubscription
Bad_InvalidState	The specified Node is in a state that does not permit this operation.	Services for Locking
Bad_TooManyOperations	The request could not be processed because it specified too many operations.	Read, Write, Subscribe
Bad_UnexpectedError	An unexpected error occurred.	All

5.1.4.7 Operational status codes

Table 8 defines the status codes for all operation level results (for services that have individual operations such as Read, Write or Subscribe). Each service defines the applicable subset and instead of a description will contain references to this table.

NOTE The value in Table 8 is referring to the Parameter value, not to the communication quality.

Table 8 – Operation level result codes

Operation Level Result Code	Description
Good	The operation completed successfully.
Good_LocalOverride	The value has been overridden.
Good_PostActionFailed	The value of a Variable was successfully read or written but one of the post actions failed.
Good_Edited	The Value has been modified in the EditContext and is not yet in the Device.
Good_DependentValueChanged	The Value of a dependent Variable was changed but not yet applied.
Uncertain	The operation completed however its outputs may not be usable.
Uncertain_NoCommunicationLastValue	Communication to the data source has failed. The Variable value is the last value that had a good quality.
Uncertain_LastUsableValue	Whatever was updating this value will no longer be doing so.
Uncertain_SubstituteValue	The value is an operational value that was manually overwritten.
Uncertain_InitialValue	The value is an initial value for a Variable that normally receives its value from another Variable.
Uncertain_SensorNotAccurate	The value is at one of the sensor limits.
Uncertain_EngineeringUnitsExceeded	The value is outside of the range of values defined for this parameter.
Uncertain_SubNormal	The value is derived from multiple sources and has less than the required number of good sources.
Uncertain_DominantValueChanged	A change of a dominant value – e.g. an engineering unit – made a dependent value invalid.
Bad	The operation failed.
Bad_AttributeInvalid	The attributeId is not supported for the specified Node.
Bad_ConfigurationError	There is a problem with the configuration that affects the usefulness of the value.
Bad_DeviceFailure	There has been a failure in the device/data source that generates the value.
Bad_IndexRangeInvalid	The indexRange parameter has an invalid syntax.
Bad_NodeInvalid	The identifier does not refer to a valid Node in the Device Model. This result code is used both as service- and as operation-level result code.
Bad_NotConnected	The Variable should receive its value from another Variable, but has never been configured to do so.
Bad_NotReadable	The access level does not allow reading or subscribing to the Node.
Bad_NotWritable	The access level does not allow writing to the Node.
Bad_OutOfRange	The value was out of range.
Bad_OutOfService	The source of the data is not operational.
Bad_SensorFailure	There has been a failure in the sensor from which the value is derived by the device/data source.
Bad_TypeMismatch	The value supplied is not of the same type as the Variable's value.
Bad_UIPHandleInvalid	The handle does not refer to a subscribed Node Attribute.
Bad_UserAccessDenied	User does not have permission to perform the requested operation.
Bad_WaitingForInitialData	Waiting for the FDI Server to obtain values from the underlying data source. After subscribing to Variables, it may take some time before values are delivered. In such cases an initial update may be sent with this status prior to the Notification with the first valid value.

5.1.5 Base Property Services

5.1.5.1 Overview

The Base Property Services provide access to basic Device Access properties.

5.1.5.2 Get DeviceAccess interface version

5.1.5.2.1 Description

This service returns the version of the interface that the UIP is using.

5.1.5.2.2 Parameters

Table 9 defines the parameters for the service.

Table 9 – GetDeviceAccessInterfaceVersion Service parameters

Name	Type	Description
Request		
Response		
version	String	FDI Technology Version of the DeviceAccess interface. The format of the value is xx.yy.zz as defined in IEC 62769-4.

5.1.5.2.3 Service results

There are no service results other than the common codes specified in 5.1.4.6.

5.1.5.3 GetOnlineAccessAvailability**5.1.5.3.1 Description**

This service returns a hint as to whether online access is available in principle. This is general information. It does not clarify whether online access to a specific device is possible.

5.1.5.3.2 Parameters

Table 10 defines the parameters for the service.

Table 10 – GetOnlineAccessAvailability Service parameters

Name	Type	Description
Request		
Response		
onlineAccess	Boolean	This value when “false” specifies that online access is not available. “true” indicates basic availability.

5.1.5.3.3 Service results

There are no service results other than the common codes specified in 5.1.4.6.

5.1.6 Device Model Services**5.1.6.1 Overview**

The Device Model Services include:

- Browse
- Read
- Write
- Subscription
 - CreateSubscription
 - Subscribe
 - Unsubscribe
 - DeleteSubscription

The services provide access to the offline representation and the online representation of the Device. The online Nodes for a Device are always present. However, access to data that require communication may be rejected with proper status codes such as Bad_NotConnected.

NOTE The online model does not contain SubDevices. If the UIP uses a path that includes SubDevices to access an online Node, this path will be evaluated in the offline hierarchy.

Cancel services are available for Browse, Read, and Write. The exact definition and mechanics depend on the technology used for the implementation of these services.

5.1.6.2 Browse

5.1.6.2.1 Description

Browses a single level in the hierarchy in the Device Model and returns Attributes for all children of the specified Node.

5.1.6.2.2 Parameters

Table 11 defines the parameters for the service.

Table 11 – Browse Service parameters

Name	Type	Description
Request		
nodeToBrowse	NodeSpecifier	The identifier of the Node to be browsed. See 5.1.9.3.2 for the definition of the NodeSpecifier type.
Response		
browseResult[]	structure	List of Nodes that are on the next level down in the hierarchy. For each Node the following Attributes will be returned.
nodePath	String	See BaseNodeClass in 5.1.3.2.
name	String	See BaseNodeClass in 5.1.3.2.
label	LocalizedText	See BaseNodeClass in 5.1.3.2.

5.1.6.2.3 Service results

No specific Service result codes are defined for Browse. Common StatusCodes are defined in Table 7.

5.1.6.3 CancelBrowse

5.1.6.3.1 Description

Calling CancelBrowse indicates that the UIP has no further interest in the results of this service. Execution will be stopped when possible.

Cancel is a suggestion to the system. Due to asynchronous execution, the service may already be fully or partially completed.

5.1.6.3.2 Parameters

Table 12 defines the parameters for the service.

Table 12 – CancelBrowse Service parameters

Name	Type	Description
Request		
serviceId	<technology dependent>	The identifier of the service to cancel.
Response		

5.1.6.3.3 Service results

No specific Service result codes are defined for CancelBrowse. Successfully cancelled Browse requests shall respond with Bad_RequestCancelled. Common StatusCodes are defined in Table 7.

5.1.6.4 Read

5.1.6.4.1 Description

This service is used to read Attributes of Object or Variable Nodes. UIPs that need to monitor Variable Attributes for changes shall use the Subscription services instead of Read.

5.1.6.4.2 Parameters

Table 13 defines the parameters for the service.

Table 13 – Read Service parameters

Name	Type	Description
Request		
returnInnerErrorInfo	Boolean	A value of "true" requests error information from calls to an underlying system to be returned when available. "false" defines that this information shall not be returned.
attributesToRead[]	Structure	The Attributes to read.
node	NodeSpecifier	The identifier of the Node that contains the Attribute to read. See 5.1.9.3.2 for the definition of the NodeSpecifier type.
attributeId	AttributeId	Numeric identifier of the Attribute to Read. See 5.1.9.3.3.
indexRange	NumericRange	This parameter is used to identify a single element of an array, or a single range of indexes for arrays. If a range of elements is specified, the values are returned as a composite. The first element is identified by index 0 (zero). This parameter is ignored if the specified Attribute is not an array or a structure. However, if the specified Attribute is an array or a structure, and this parameter is null (Null or empty String), then all elements are to be included in the range. See 5.1.9.3.6 for a detailed definition.
maxAge	UInt32	Maximum age of the value to be read in milliseconds. If the FDI Server has a cached value no older than maxAge, it will return the cached value rather than requesting a new value from the device. If maxAge is set to 0, the FDI Server shall read a new value from the data source. Values greater than $2^{31} - 1$ (0x7fff ffff) are invalid for maxAge.
Response		
readResult []	DataValue	The StatusCode, Value and timestamps for each Node Attribute that was read. The order of this list matches the order of the attributesToRead request parameter. The DataValue is defined in 5.1.9.3.3.
innerErrorInfos []	InnerErrorInfo	List of error information from calls to an underlying system. See 5.1.9.3.4. Matches the size and order of the attributesToRead request parameter. This list is empty if inner error information was not requested or if no information was encountered in the processing of the request.

5.1.6.4.3 Service results

Table 14 defines values for the Service result code. Other common StatusCodes are defined in Table 7.

Table 14 – Read Service result codes

Result code	Description
Bad_MaxAgeInvalid	The max age parameter is invalid.

5.1.6.4.4 Operation result codes

Table 15 defines values for the operation status code contained in the DataValue of each readResult element. All operational status codes with their description are in Table 8.

Table 15 – Read operation result codes

Result code
Good
Good_LocalOverride
Good_PostActionFailed
Good_DependentValueChanged
Uncertain
Uncertain_NoCommunicationLastValue
Uncertain_LastUsableValue
Uncertain_SubstituteValue
Uncertain_InitialValue
Uncertain_SensorNotAccurate
Uncertain_EngineeringUnitsExceeded
Uncertain_SubNormal
Uncertain_DominantValueChanged
Bad
Bad_UserAccessDenied
Bad_ConfigurationError
Bad_NotConnected
Bad_DeviceFailure
Bad_SensorFailure
Bad_OutOfRange
Bad_OutOfService
Bad_NodeInvalid
Bad_AttributeInvalid
Bad_IndexRangeInvalid
Bad_NotReadable

5.1.6.5 CancelRead

5.1.6.5.1 Description

Calling CancelRead indicates that the UIP has no further interest in the results of this service. Execution will be stopped when possible.

Cancel is a suggestion to the system. Due to asynchronous execution, the service may already be fully or partially completed.

5.1.6.5.2 Parameters

Table 16 defines the parameters for the service.

Table 16 – CancelRead Service parameters

Name	Type	Description
Request		
serviceId	<technology dependent>	The identifier of the service to cancel.
Response		

5.1.6.5.3 Service results

No specific Service result codes are defined for CancelRead. Successfully cancelled Read requests shall respond with Bad_RequestCancelled. Common StatusCodes are defined in Table 7.

5.1.6.6 Write

5.1.6.6.1 Description

This service is used to write values to one or more Variables. For array values, this service allows writing the entire array, writing individual elements or writing ranges of elements.

Explicit locking is required (see 5.1.7). If the Node is not locked, the request will be rejected with Bad_LockRequired.

The service response is not returned until the write operation has been executed by the system. Rollback is the responsibility of the FDI Client/UIP.

The values shall have the correct data type.

NOTE No automatic data type translation takes place (see Bad_TypeMismatch operation result code).

5.1.6.6.2 Parameters

Table 17 defines the parameters for the service.

Table 17 – Write Service parameters

Name	Type	Description
Request		
returnInnerErrorInfo	Boolean	A value of "true" requests error information from calls to an underlying system to be returned when available. "false" defines that this information shall not be returned.
variablesToWrite[]	structure	List of Variables to write to. No assumptions should be made about the order of processing this list. If the order matters, separate service requests shall be used.
node	NodeSpecifier	The identifier of the Node that contains the Attribute to write. See 5.1.9.3.2 for the definition of the NodeSpecifier type.
indexRange	NumericRange	See 5.1.9.3.6 for a detailed definition.
value	Variant	Value to write.
Response		
writeResult[]	UInt32	Status codes for operation results as defined in Table 18. The order of this list matches the order of the variablesToWrite request parameter.
innerErrorInfos []	InnerErrorInfo	List of error information from calls to an underlying system. See 5.1.9.3.4. Matches the size and order of the variablesToWrite request parameter. This list is empty if inner error information was not requested or if no information was encountered in processing of the request.

5.1.6.6.3 Service results

There are no service results other than the common codes specified in 5.1.4.6.

5.1.6.6.4 Operation result codes

Table 18 defines values for the operation status code contained in the writeResult elements. All operational status codes with their description are in Table 8.

Table 18 – Write operation result codes

Result code
Good
Good_PostActionFailed
Bad
Bad_UserAccessDenied
Bad_NodeInvalid
Bad_IndexRangeInvalid
Bad_TypeMismatch
Bad_OutOfRange
Bad_NotWritable

5.1.6.7 CancelWrite

5.1.6.7.1 Description

Calling CancelWrite indicates that the UIP has no further interest in the results of this service. Execution will be stopped when possible.

Cancel is a suggestion to the system. Due to asynchronous execution, the service may already be fully or partially completed.

5.1.6.7.2 Parameters

Table 19 defines the parameters for the service.

Table 19 – CancelWrite Service parameters

Name	Type	Description
Request		
serviceId	<technology dependent>	The identifier of the service to cancel.
Response		

5.1.6.7.3 Service results

No specific Service result codes are defined for CancelWrite. Successfully cancelled Write requests shall respond with Bad_RequestCancelled. Common StatusCodes are defined in Table 7.

5.1.6.8 Subscriptions

5.1.6.8.1 Subscription mechanism

Subscriptions allow the UIP to receive unsolicited callbacks from the FDI Client when the subscribed Node Attributes change. Subscriptions are a more efficient way to get periodic updates of data than by issuing repeated calls to the Read service, i.e., polling. The UIP creates a subscription by calling the CreateSubscription service (see 5.1.6.8.2). When the CreateSubscription service is called the UIP shall supply a callback parameter with the UIP-specific DataChangeCallback service (see 5.1.6.8.6). After receiving the subscription identifier in the CreateSubscription response the UIP shall add Node Attributes of interest to the Subscription by calling the Subscribe service.

Once the initial values have been reported, the DataChangeCallback service is only called when Node Attributes change and only the Node Attributes that have changed are reported to the callback. The UIP controls the maximum frequency at which the callback should be invoked by specifying a rate in milliseconds.

After Node Attributes have been subscribed, their values may not be immediately available and may become available at different times. As such, the initial callback may not include all of the subscribed Node Attributes. Furthermore, it is possible that the initial update for a subscribed Node Attribute will return a Bad or Uncertain StatusCode.

Furthermore, the time span between when a change happens and when the callback is invoked depends on where the information to be changed is maintained. Some values are maintained by the system, which allows immediate notification; others are located in the physical device so that communication is required to access it.

5.1.6.8.2 CreateSubscription Service

5.1.6.8.2.1 Description

This service is used to create a subscription.

5.1.6.8.2.2 Parameters

Table 20 defines the parameters for the service.

Table 20 – CreateSubscription Service parameters

Name	Type	Description
Request		
requestedUpdateRate	UInt32	The fastest rate, in milliseconds, at which the UIP requests to be called back with data changes, specified by the minimum milliseconds to elapse between updates. Regardless of the requested rate, a callback only occurs if data has changed. A rate of "0" indicates that the caller wants to be notified of changes as soon as possible. The service will return the fastest possible rate as revisedUpdateRate.
dataChangeCallback	DataChangeCallback	Callback for sending data change updates to the UIP. See 5.1.6.8.6. The DataChangeCallback is a service implemented and provided by the UIP.
Response		
revisedUpdateRate	UInt32	The actual rate that the FDI Server will use, expressed as the minimum milliseconds to elapse between updates (if data has changed since the previous update).
subscriptionId	SubscriptionId	The callee-assigned identifier for the subscription. The SubscriptionId type is technology dependent.

5.1.6.8.2.3 Service results

Table 21 defines values for the service result. Common results are defined in Table 7.

Table 21 – CreateSubscription Service result codes

Result code	Description
Bad_TooManySubscriptions	The FDI Server has reached its maximum number of subscriptions.

5.1.6.8.3 Subscribe Service

5.1.6.8.3.1 Description

This service is used to add one or more Node Attributes to an existing subscription.

Subscribing is permitted for all Node Attributes.

5.1.6.8.3.2 Parameters

Table 22 defines the parameters for the service.

Table 22 – Subscribe Service parameters

Name	Type	Description
Request		
subscriptionId	SubscriptionId	The identifier for an existing subscription that was returned by the CreateSubscription service.
returnInnerErrorInfo	Boolean	A value of "true" requests error information from calls to an underlying system to be passed in DataChangeCallbacks, when available. "false" defines that this information shall not be returned.
monitoredItemsToAdd[]	structure	The Attributes to add to the subscription.
node	NodeSpecifier	The identifier of the Node that contains the Attribute to subscribe. See 5.1.9.3.2 for the definition of the NodeSpecifier type.
attributeId	AttributeId	Numeric identifier of the Attribute to subscribe. See 5.1.9.3.3.
indexRange	NumericRange	This parameter is used to identify a single element of an array, or a single range of indexes for arrays. If a range of elements is specified, the values are returned as a composite. The first element is identified by index 0 (zero). This parameter is ignored if the specified Attribute is not an array or a structure. However, if the specified Attribute is an array or a structure, and this parameter is null, then all elements are to be included in the range. See 5.1.9.3.6 for a detailed definition.
samplingInterval	Int32	The interval that defines the fastest rate at which the Attribute should be accessed and evaluated. This interval is defined in milliseconds. The value 0 indicates that the FDI Server should use the fastest practical rate. The value -1 indicates that the default sampling interval defined by the UpdateRate of the Subscription is used. See 5.1.6.8.3.3 for further details on the sampling interval.
uiPHandle	UInt32	A handle (an identifier) provided by the UIP for the subscribed Node Attribute. This handle will be passed together with the data in the DataChangeCallback service so that the UIP can easily associate each changed value with the subscribed Node Attribute. The uiPHandle is likely an index into a table somewhere. It does not have to be unique (there could be multiple subscribed items pointing to the same table entry).
Response		
subscribeResult []	structure	List of results for the subscribed Attributes. The size and order of the list matches the size and order of the attributesToSubscribe request parameter.
statusCode	UInt32	Status code for the respective Attribute to subscribe as defined in Table 23.
monitoredItemId	UInt32	FDI Server-assigned id for the subscribed Attribute. This id is unique within the Subscription and shall be used when calling Unsubscribe. This parameter is present only if the statusCode indicates that the Attribute was successfully subscribed.
revisedSamplingInterval	Int32	The actual sampling interval that will be used. This value is based on a number of factors, including the capabilities of the underlying system.

Subscribing will succeed even if the current user is not authorized to access the Node Attribute. If this is the case, the initial callback will return the operation result "Bad_UserAccessDenied". Once this denial is cleared away (for instance, after a more powerful user identity is provided) the UIP will receive data changes for this Node Attribute.

It is possible to subscribe to any Attribute – not just the Value. While it may not make sense for all Attributes, monitoring of some Attributes provides additional possibilities. Some examples include:

- monitoring the LockedStatus to determine when the lock by some other client is removed;
- monitoring the CurrentLabel of Enumerations to receive a displayable name rather than a numeric value.

5.1.6.8.3.3 Sampling interval

Each subscribed item is assigned a sampling interval that is either inherited from the updateRate of the Subscription or that is defined specifically to override that rate. The sampling interval indicates the fastest rate at which the value should be sampled in the device for data changes.

The assigned sampling interval defines a “best effort” cyclic rate that is used to sample the item from its source. “Best effort” in this context means that the system does its best to sample at this rate. Sampling at rates faster than this rate is acceptable, but not necessary to meet the needs of the UIP. How the system deals with the sampling rate and how often it actually polls its data source internally is a system implementation detail. However, the time between values returned to the UIP shall be greater than or equal to the sampling interval.

The FDI Client may also specify 0 for the sampling interval, which indicates that the system should use the fastest practical rate. It is expected that systems will support only a limited set of sampling intervals to optimize their operation. If the exact interval requested by the UIP is not supported, then the most appropriate interval as determined by the system will be assigned and returned to the UIP.

Data may be collected based on a sampling model or generated based on an exception-based model. The fastest supported sampling interval may be equal to 0, which indicates that the data item is exception-based rather than being sampled at some period. When it is exception-based the underlying system does not require sampling.

In many cases, the system has no knowledge of the data update logic. In this case, even though the system samples at the negotiated rate, the data might be updated by the underlying system at a much slower rate. In this case, changes can only be detected at this slower rate.

UIPs should also be aware that the sampling by the system and the update cycle of the device are usually not synchronized, which may lead to additional delays.

5.1.6.8.3.4 Service results

There are no service results other than the common codes specified in 5.1.4.6.

5.1.6.8.3.5 Operation result codes

Table 23 defines values for the operation statusCode contained in the results. All operational status codes with their description are in Table 8.

Table 23 – Subscribe operation result codes

Result code
Good
Bad
Bad_NodeInvalid
Bad_AttributeInvalid
Bad_NotReadable
Bad_UserAccessDenied

5.1.6.8.4 Unsubscribe Service

5.1.6.8.4.1 Description

This service is used to unsubscribe to one or more Node Attributes.

5.1.6.8.4.2 Parameters

Table 24 defines the parameters for the service.

Table 24 – Unsubscribe Service Parameters

Name	Type	Description
Request		
subscriptionId	SubscriptionId	The identifier for an existing subscription that was returned by the CreateSubscription service.
monitoredItemIds[]	UInt32	Identifiers for subscribed Attributes that were returned by the Subscribe service.
Response		
unsubscribeResult[]	UInt32	Status codes for operation results as defined in Table 25. The order of this list matches the order of the monitoredItemIds request parameter.

5.1.6.8.4.3 Service results

There are no service results other than the common codes specified in 5.1.4.6.

5.1.6.8.4.4 Operation result codes

Table 25 defines values for the operation status code contained in unsubscribeResult. All operational status codes with their description are in Table 8.

Table 25 – Unsubscribe operation result codes

Result code
Good
Bad
Bad_UIPHandleInvalid

5.1.6.8.5 DeleteSubscription Service

5.1.6.8.5.1 Description

This service is used to delete a subscription. Due to the asynchronous nature of the callbacks, the UIP may receive additional callbacks for a subscription after the subscription is deleted

5.1.6.8.5.2 Parameters

Table 26 defines the parameters for the service.

Table 26 – DeleteSubscription Service parameters

Name	Type	Description
Request		
subscriptionId	SubscriptionId	The identifier for an existing subscription that was returned by the CreateSubscription service.
Response		

5.1.6.8.5.3 Service results

There are no service results other than the common codes specified in 5.1.4.6.

5.1.6.8.6 DataChangeCallback Service

5.1.6.8.6.1 Description

This service is used for sending data change updates to the UIP. This service is implemented and provided by the UIP when calling the CreateSubscription service.

Due to the asynchronous nature of the callbacks, the UIP may receive additional callbacks for a subscription after the subscription is deleted.

5.1.6.8.6.2 Parameters

Table 27 defines the parameters for the service.

Table 27 – DataChangeCallback Service parameters

Name	Type	Description
Request		
subscriptionId	SubscriptionId	Identifier that was returned by the CreateSubscription service
dataChangeData[]	structure	Data that has changed. No specific order of array elements is ensured.
uiPHandle	UInt32	The handle provided by the UIP in the Subscribe service
value	DataValue	The StatusCode, Value and timestamps of the subscribed Node Attribute. The DataValue is defined in 5.1.9.3.3.
innerErrorInfos []	InnerErrorInfo	List of error information from calls to an underlying system. See 5.1.9.3.4. Matches the size and order of the dataChangeData parameter. This list is empty if inner error information was not requested or if no information was encountered in the processing of the request.
Response		

5.1.6.8.6.3 Operation result codes

Table 28 defines values for the operation status code contained in the DataChangeData structure of each values element. In addition, all Read operation status codes apply also (see Table 15). All operational status codes with their description are in Table 8.

Table 28 – DataChangeCallback result codes

Result code
Bad_WaitingForInitialData

5.1.7 Locking Services

5.1.7.1 Overview

Locking is the means to avoid concurrent modifications to a Device or a Block and its Parameters. UIPs shall use the locking services before making changes (for example, write operations and Direct Access Services).

The lock always applies to both the online and the offline version.

When locking a Modular Device, the lock applies to the complete device (including all modules). Equally, when locking a Block Device, the lock applies to the complete Device (including all Blocks).

If no lock is applied to the top-level Device (for Modular Device or for Block Device), the Sub-Devices or Blocks, respectively, can be locked independently.

While locked, requests from other FDI Clients to write to Parameters, or to perform Direct Access will be rejected.

The lock is removed when ExitLock is called.

5.1.7.2 InitLock service

5.1.7.2.1 Description

InitLock reserves the specified Device or Block. During a lock other FDI Clients will not be able to write to the Parameters of this element. A lock for an element that is already locked by another FDI Client will be rejected. A UIP can subscribe to the LockedStatus Attribute in order to be informed when the lock is removed by the other Client.

FDI Clients shall allow “nested” InitLock calls from the same UIP. The FDI Client expects the same number of ExitLock calls before it actually removes the lock.

FDI Clients are responsible for preventing simultaneous locks to a Device or Block by independent components that this Client hosts. Such components include the Client itself, independent UIPs or the UID Interpreter.

5.1.7.2.2 Parameters

Table 29 defines the parameters for the service.

Table 29 – InitLock Service parameters

Name	Type	Description
Request		
node	NodeSpecifier	The identifier of the Node (representing a device or block) to be locked. See 5.1.9.3.2 for the definition of the NodeSpecifier type.
context	String	Used to provide context information about the current activity going on in the UIP. This will be used to enhance the Audit Trail.
Response		

5.1.7.2.3 Service results

Table 30 defines values for the Service result code. Other common StatusCodes are defined in Table 7.

Table 30 – InitLock Service result codes

Result code	Description
Bad_NotSupported	The Node does not support locking.
Bad_AlreadyLocked	The Node is already locked by another FDI Client or another independent component within the FDI Client.

5.1.7.3 ExitLock service

5.1.7.3.1 Description

ExitLock removes the lock.

5.1.7.3.2 Parameters

Table 31 defines the parameters for the service.

Table 31 – ExitLock Service parameters

Name	Type	Description
Request		
node	NodeSpecifier	The identifier of the Node (representing a device or block) to be unlocked. See 5.1.9.3.2 for the definition of the NodeSpecifier type.
Response		

5.1.7.3.3 Service results

Table 32 defines values for the Service result code. Other common StatusCodes are defined in Table 7.

Table 32 – ExitLock Service result codes

Result code	Description
Bad_InvalidState	The Node is not locked.

5.1.8 Direct Access Services

5.1.8.1 Overview

Direct Access Services provide direct communication with a Device. This can be used for operations that cannot or at least not easily be performed through the Device Model Services. Use cases include the transmission of large data buckets from or to the Device, for example, historical data or firmware. The Direct Access Services should not influence the structure and the data integrity of the Device Model. Support of Direct Access Services is mandatory for FDI Hosts. However, Host Systems shall provide means to disable/enable Direct Access Services on demand. Commissioning engineers or plant operators can then disallow the use of Direct Access in specific scenarios. This can be temporary or even permanent and might apply to the complete or specific parts of the plant. Therefore, UIPs shall not depend on the availability of Direct Access for the initial setup of a Device (e.g. needed for commissioning).

The following behaviour applies when using Direct Access.

- Only one FDI Client/UIP can use DirectAccess at a given time. While in Direct Access mode, other Clients will not be able to access this Device (not even for Reading).
- The Device shall have been locked prior to entering this mode.
- If Variable Attributes may have changed due to DirectAccess, the UIP shall set the InvalidateCache in the EndDirectAccess argument to “true”.

The Direct Access Services include:

- InitDirectAccess
- Transfer
- ExitDirectAccess

Guideline:

These services are not to be used as alternative to Information Model access. Instead, DirectAccess is used for the transfer of data that are not reflected in the Information Model.

5.1.8.2 InitDirectAccess

5.1.8.2.1 Description

This service initializes the Device for the use of DirectAccess.

5.1.8.2.2 Parameters

Table 33 defines the parameters for the service.

Table 33 – InitDirectAccess Service parameters

Name	Type	Description
Request		
context	String	Used to provide context information about the current activity going on in the UIP. This will be used to enhance the Audit Trail.
Response		

5.1.8.2.3 Service results

Table 34 defines values for the Service result code. Other common StatusCodes are defined in Table 7.

Table 34 – InitDirectAccess Service result codes

Result code	Description
Bad_NotSupported	DirectAccess is (currently) not supported.
Bad_LockRequired	The Node has not been locked.
Bad_InvalidState	DirectAccess is already initialized.

5.1.8.3 ExitDirectAccess

5.1.8.3.1 Description

This service ends the use of DirectAccess.

5.1.8.3.2 Parameters

Table 35 defines the parameters for the service.

Table 35 – ExitDirectAccess Service parameters

Name	Type	Description
Request		
invalidateCache	Boolean	If "true", any cached values for Device Parameters will be invalidated. This means that these Parameters will be re-read from the Device the next time they are used again.
Response		

5.1.8.3.3 Service results

Table 36 defines values for the Service result code. Other common StatusCodes are defined in Table 7.

Table 36 – ExitDirectAccess Service result codes

Result code	Description
Bad_InvalidState	The device is not in DirectAccess mode.

5.1.8.4 Transfer

5.1.8.4.1 Description

This service is used to transfer data to and from the Device. The format of send or receive data is protocol specific.

Direct Access by its nature does not allow automatic generation of Audit Trail information that complies with the various regulations. Therefore, UIPs shall invoke the LogAuditTrailMessage Service (see 5.2.2.5) to provide explicit information about what is being transferred.

For other service calls that affect the Device indirectly via the Device Model (Write), the service parameters provide sufficient information.

5.1.8.4.2 Parameters

Table 37 defines the parameters for the service.

Table 37 – Transfer Service parameters

Name	Type	Description
Request		
sendData	String	XML document based on the TransferSendDataT as specified in the communication profile-specific XML schema.
Response		
receiveData	String	XML document based on the TransferResultDataT as specified in the communication profile-specific XML schema.

5.1.8.4.3 Service results

Table 38 defines values for the Service result code. Other common StatusCodes are defined in Table 7.

Table 38 – Transfer Service result codes

Result code	Description
Bad_InvalidState	The device is not in DirectAccess mode.

5.1.9 Data types

5.1.9.1 General

Subclause 5.1.9 specifies the data types used for Service parameters, Variable values, and the values of other Node Attributes. They may occur either scalar or as an array.

5.1.9.2 Base data types

Table 39 lists base data types, i.e., types that are typically supported native by programming languages.

Table 39 – Base data types

DataType	Description
Boolean	Defines a value that is either "true" or "false".
String	Represents text as a series of Unicode characters. The actual string representation depends on the technology mapping. See IEC 62769-6.
ByteString	A value that is a sequence of Byte values preceded by a 32-Bit Length.
UtcTime	A DateTime used to define Coordinated Universal Time (UTC) values. All time values are UTC values. FDI Clients shall provide any conversions between UTC and local time. UtcTime is a 64-bit signed integer that represents the number of 100 nanosecond intervals since January 1, 1601. It corresponds to the Windows FILETIME.
Int8	A signed integer between -128 and 127 inclusive.
Int16	A signed integer between -32 768 and 32 767 inclusive.
Int32	A signed integer between -2 147 483 648 and 2 147 483 647 inclusive.
Int64	A signed integer between -9 223 372 036 854 775 808 and 9 223 372 036 854 775 807 inclusive.
Byte	A value in the range of 0 to 255.
UInt16	An unsigned integer between 0 and 65 535 inclusive.
UInt32	An unsigned integer between 0 and 4 294 967 295 inclusive.
UInt64	An unsigned integer between 0 and 18 446 744 073 709 551 615 inclusive.
Float	Defines a value that shall be according to the IEEE 754 single precision data type definition.
Double	Defines a value that shall be according to the IEEE 754 double precision data type definition.
Duration	Same representation as Double.

5.1.9.3 Special types

5.1.9.3.1 AttributeIds

AttributeIds are represented as UInt32. Table 40 lists the Attributes and their identifiers.

Table 40 – Identifiers assigned to Attributes

Attribute	Identifier
Name	10
Label	11
Description	12
	...
LockedStatus	30
	...
Value	100
DataType	101
ValueRank	102
ArrayDimensions	103
AccessRights	105
UserAccessRights	106
ScalingFactor	107
	...
EURange	111
EngineeringUnits	112
	...
EnumValues	120
CurrentLabel	121
OptionNames	122

5.1.9.3.2 NodeSpecifier

Each Node in the Device Model (see 5.1.2) is uniquely addressable with its path name qualified by an online/offline specifier. When online is specified, the service shall operate in

the online version of the Device Model. When offline is specified, it shall operate in the offline model.

The path name follows the hierarchy of the device model as illustrated in 5.1.2, It is a concatenation of the individual names, separated by slash ('/') characters. See 5.1.2 for example path names. All parameters are identified via “/ParameterSet/<ParamName>”, all images via “/ImageSet/<ImageName>”, and so on.

Subclauses 5.1.3.4.3 and 5.1.3.4.5 specify the representation of parameters that hold record values or arrays of record values. A pathname for record elements is built by extending the path to the Parameter. See examples in the referenced subclauses. The components of this parameter are defined in Table 42.

Table 41 – NodeSpecifier

Name	Type	Description
NodeSpecifier	structure	Specifies a Node in the device model.
nodePath	String	The path description that enables finding a Node in the Information Model.
useOnline	Boolean	If “true” the path addresses a Node in the online model; with “false” a Node in the offline model is referenced.

An empty path name identifies the Root.

5.1.9.3.3 DataValue

This type describes the value of a Node Attribute in the response of a Read and in the DataChangeCallback. The components of this parameter are defined in Table 42.

Table 42 – DataValue

Name	Type	Description
DataValue	structure	The value and associated information.
value	Variant	The Node Attribute value. For the definition of Variant see 5.1.9.4.
statusCode	UInt32	The StatusCode that defines the ability to access/provide the value. The StatusCode type is defined in 5.1.4.5.
sourceTimestamp	UtcTime	The source timestamp for the value.
serverTimestamp	UtcTime	The server timestamp for the value.

The statusCode is used to indicate the conditions under which a Node Attribute value was generated, and thereby can be used as an indicator of the usability of the value.

It is required to check the StatusCode (as a minimum the Severity) of all results before accessing and using the value.

The sourceTimestamp reflects the timestamp that was applied by the data source. The sourceTimestamp is only returned with the Value Attribute of Variable Nodes. For all other Attributes the returned sourceTimestamp is set to null.

In the case of a bad or uncertain status sourceTimestamp is used to reflect the time that the source recognized the non-good status or the time the FDI Server last tried to recover from the bad or uncertain status.

The serverTimestamp is used to reflect the time that the FDI Server received a Variable value or knew it to be accurate. In the case of a bad or uncertain status, serverTimestamp is used to reflect the time that the FDI Server received the status or that the FDI Server last tried to recover from the bad or uncertain status.

The serverTimestamp is updated each time a new value is received.

5.1.9.3.4 InnerErrorInfo

The Services described in 5.1 return StatusCodes on the service level and on the operation level. These StatusCodes are protocol-independent and device-independent. In cases where a StatusCode results from a call to the underlying system (e.g., communication system, device) the status of the underlying system can be reported via InnerErrorInfo.

The components of this parameter are defined in Table 43.

Table 43 – InnerErrorInfo

Name	Type	Description
InnerErrorInfo	structure	Communication- or Device-specific information.
symbolicId	String	This string shall be used to identify the result of some internal operation. The maximum length of this string is 32 characters. Systems wishing to return a numeric return code should convert the return code into a string and use this string as symbolicId (e.g., "0xC0040007" or "-4").
errorText	LocalizedText	A textual representation of the symbolic id. The maximum length of this text string is 256 characters

5.1.9.3.5 LocalizedText

This data type defines a structure containing a String in a locale-specific translation specified in the identifier for the locale. Its elements are defined in Table 44.

Table 44 – LocalizedText Definition

Name	Type	Description
LocalizedText	structure	—
text	String	The localized text.
locale	String	The identifier for the locale (e.g. "en-US").

The element locale shall be a string composed of a language component and a country/region component according to RFC 3066. The <country/region> component is always preceded by a hyphen. The format of the LocaleId string is shown below:

<language>[-<country/region>], where
 <language> shall be a two-letter code for a language according to ISO 639,
 <country/region> shall be a two-letter code for the country/region according to ISO 3166.

The rules for constructing LocaleIds shall be according to RFC 3066 and shall be restricted as follows:

- This specification permits only zero or one <country/region> component to follow the <language> component.
- This specification also permits the "-CHS" and "-CHT" three-letter <country/region> codes for "Simplified" and "Traditional" Chinese locales.
- This specification also allows the use of other <country/region> codes as deemed necessary by the FDI Client or the FDI Server.

Table 45 shows examples of locale ids.

Table 45 – LocaleId Examples

Locale	OPC UA LocaleId
English	en
English (US)	en-US
German	de
German (Germany)	de-DE
German (Austrian)	de-AT

An empty or NULL string indicates that the locale is unknown.

5.1.9.3.6 Numeric Range

Numeric Range is represented as a String. The syntax for the String contains one of the following two constructs. The first construct is the String representation of an individual integer. For example, “6” is valid, but “6,0” and “3,2” are not. The minimum and maximum values that can be expressed are defined by the use of this parameter and not by this parameter type definition. The second construct is a range represented by two integers separated by the colon (“:”) character. The first integer shall always have a lower value than the second. For example, “5:7” is valid, while “7:5” and “5:5” are not. No other characters, including white-space characters, are permitted.

All indexes start with 0. The maximum index is one less than the length of the array.

When reading with a numeric range outside the bounds of the array, the FDI Server shall return a partial result if some elements exist within the range. The FDI Server shall return a `Bad_OutOfRange` if no elements exist within the range.

When writing a value the numeric range shall be within the array.

A numeric range can also be used to specify substrings for ByteString and String values.

5.1.9.3.7 Range

This structure defines the range structure needed for the `EURange` Attribute. It is defined in Table 46.

Table 46 – Range Data Type Structure

Name	Type	Description
Range	structure	“low” and “high” can contain any data type that is appropriate for the data type of the Variable Value Attribute.
low	Variant	Lowest value in the range.
high	Variant	Highest value in the range.

5.1.9.3.8 EUInformation

This structure contains information about the `EngineeringUnits`. Its elements are defined in Table 47.

Table 47 – EUInformation Data Type Structure

Name	Type	Description
EUInformation	structure	—
unitId	UInt32	Identifier for programmatic evaluation. 0 is used if a unitId is not available.
displayName	LocalizedText	The displayName of the engineering unit is typically the abbreviation of the engineering unit, e.g. "h" for hour or "m/s" for meter per second.
description	LocalizedText	Contains the full name of the engineering unit such as hour or meter per second. An empty text field indicates that no description is available.

5.1.9.3.9 EnumValueType

This Structured DataType is used to represent a human-readable representation of an Enumeration. Its elements are described in Table 48. When this type is used in an array representing human-readable representations of an enumeration, each value of an IntegerRepresentation will be unique in that array.

Table 48 – EnumValueType Definition

Name	Type	Description
EnumValueType	structure	—
value	Int64	The Integer representation of an Enumeration.
displayName	LocalizedText	A human-readable representation of the integer representation of the Enumeration.
description	LocalizedText	A localized description of the enumeration value. This field can contain an empty String if no description is available.

5.1.9.4 Variant

A Variant is a union of all data types. Variants can also contain arrays of any of these types.

Variants can be empty. An empty Variant is described as having a Null value. A Null value in a Variant is not the same as a Null String.

Variants can contain arrays of Variants but they cannot directly contain another Variant.

5.2 Hosting Services

5.2.1 General

The Hosting Services are provided by the FDI Client for use by the UIP. The Hosting Services include services related to the FDI Client environment allowing the UIP to acquire information about the environment. The Hosting Services also include services to launch other UIPs as well as showing feedback.

5.2.2 Services

5.2.2.1 General

Programmatic access to the services may be synchronous or asynchronous.

5.2.2.2 GetClientTechnology Version

5.2.2.2.1 Description

This service returns the technology version of the FDI Client that hosts the UIP.

5.2.2.2.2 Parameters

Table 49 defines the parameters for the service.

Table 49 – GetClientTechnologyVersion Service parameters

Name	Type	Description
Request		
Response		
version	String	FDI Technology Version that is supported by the FDI Client. The format of the value is xx.yy.zz as defined in IEC 62769-4.

5.2.2.2.3 Service results

This service always succeeds.

5.2.2.3 OpenUserInterface Service

5.2.2.3.1 Description

This service is used by a UIP to request the FDI Client to open another UIP. The UIP is opened in either a modal or a non-modal window as follows:

- UIPs are always invoked in a modal window, if the calling UIP runs in a modal window or if their style is dialog.
- UIPs are invoked non-modal if their style = window and the calling UIP runs in non-modal mode also.

If the UIP is already opened, this service shall bring the respective window into the foreground.

5.2.2.3.2 Parameters

Table 50 defines the parameters for the service.

Table 50 – OpenUserInterface Service parameters

Name	Type	Description
Request		
uiplD	String	Identification of the UIP to be opened. This string is a UUID that defines a Node in the Information Model. The value of this UUID is defined in the FDI Package corresponding to the UIP to be opened (see IEC 62769-4).
Response		
Errors		

5.2.2.4 CloseUserInterface Service

5.2.2.4.1 Description

This service is used by a UIP to close itself. The UIP shall finish all pending or running functions that are still open.

UIPs call this Service only on user request (Ok, Close or Cancel).

5.2.2.4.2 Parameters

There are no specific parameters for the service.

5.2.2.5 LogAuditTrailMessage Service

5.2.2.5.1 Description

This service is used by a UIP to log an audit trail message.

5.2.2.5.2 Parameters

Table 51 defines the parameters for the service.

Table 51 – LogAuditTrailMessage Service parameters

Name	Type	Description
Request		
message	String	The message to be logged in the audit trail.
Response		None.

5.2.2.6 SaveUserSettings Service

5.2.2.6.1 Description

This service is used by a UIP to instruct the FDI Client to save the supplied user settings. The settings are stored per UIP type in a persistent store under the control of the FDI Client. All previously saved user settings of this UIP type are replaced. The FDI Client shall not alter the user settings requested of the storage.

Typically, user settings include layout information or other user preferences. They are not designed to be used for instance-specific data.

There is no support for partial modifications of the user settings. A UIP has to load all settings, update them and save the complete set.

The structure and content of the user setting strings is UIP type specific. For example, a UIP could use name-value pairs. Versioning issues of this data are under the responsibility of the UIP. Different UIPs of the same type can overwrite each other's settings.

Data structure is completely under control of the UIP. Therefore also versioning issues with respect to different UIP versions are under the responsibility of the UIP.

5.2.2.6.2 Parameters

Table 52 defines the parameters for the service.

Table 52 – SaveUserSettings Service parameters

Name	Type	Description
Request		
userSetting[]	String	List of user settings. The content of the strings is UIP-type specific.
Response		

5.2.2.7 LoadUserSettings Service

5.2.2.7.1 Description

This service is used by a UIP to instruct the FDI Client to retrieve the previously saved user settings.

5.2.2.7.2 Parameters

Table 53 defines the parameters for the service.

Table 53 – LoadUserSettings Service parameters

Name	Type	Description
Request		
Response		
userSetting[]	String	List of user settings. The content of the strings is UIP-specific.

5.2.2.8 Trace Service

5.2.2.8.1 Description

This service shall be used by the UIP to provide the FDI Client with information about internal events in the UIP. The trace messages are typically used for trouble shooting. The UIP shall call this service according to the trace level settings specified in the UIP Service SetTraceLevel (see 6.1.1.4).

5.2.2.8.2 Parameters

Table 54 defines the parameters for the service.

Table 54 – Trace Service parameters

Name	Type	Description
Request		
eventType	TraceLevel	Severity of the trace message. One of the values that specifies the severity of the trace data (see 6.1.2).
classification	String	UIP-specific classification of the trace message.
message	String	Trace message. The trace message language shall be English. Embedded text might be localized.
Response		

5.2.2.9 ShowMessageBox Service

5.2.2.9.1 Description

This service opens a message box and waits until the user presses one of the given buttons. An open message box shall block any activity for the related device instance. It is recommended that simultaneous interactions with other Device instances are not affected.

5.2.2.9.2 Parameters

Table 55 defines the parameters for the service.

Table 55 – ShowMessageBox Service parameters

Name	Type	Description
Request		
acknStyle	AcknStyle	See Table 63.
message	String	Message to be shown to the user.
caption	String	Title bar caption to display.
buttonSet	ButtonSet	See Table 62.
defaultResult	Integer	Id of the default button; see Table 61.
Response		
buttonSelected	Integer	Id of the selected button; see Table 61.

5.2.2.10 ShowProgressBar Service

5.2.2.10.1 Description

This service opens a progress bar. The progress bar remains on the user interface after the service returns. The information shown can be updated with the UpdateShowProgressBar service (see 5.2.2.11). The progress bar can be closed with the EndShowProgressBar service (see 5.2.2.12).

Only one progress bar per UIP can be shown at a time.

5.2.2.10.2 Parameters

Table 56 defines the parameters for the service.

Table 56 – ShowProgressBar Service parameters

Name	Type	Description
Request		
message	String	Message to be shown to the user.
callback	CancelCallback	Service called by the Client to inform the UIP that the user has cancelled the operation. See 0. The CancelCallback service is implemented and provided by the UIP.
Response		

5.2.2.11 UpdateShowProgressBar Service

5.2.2.11.1 Description

This service updates an already open progress bar.

5.2.2.11.2 Parameters

Table 57 defines the parameters for the service.

NOTE A call of this service is rejected without a preceeding ShowProgressBar service call.

Table 57 – UpdateShowProgressBar Service parameters

Name	Type	Description
Request		
message	String	Updated message to be shown to the user
percentage	Integer	Updated percentage of progress to be shown to the user
Response		

5.2.2.12 EndShowProgressBar Service**5.2.2.12.1 Description**

This service closes an open progress bar. The service will wait until the user has pressed a button or will immediately return with the result if the user has previously pressed the button.

5.2.2.12.2 Parameters

Table 58 defines the parameters for the service.

Table 58 – EndShowProgressBar Service parameters

Name	Type	Description
Request		
message	String	Updated message to be shown to the user
percentage	Integer	Updated percentage of progress to be shown to the user
Response		

5.2.2.13 CancelCallback Service**5.2.2.13.1 Description**

With this service the Client informs the UIP that the User requested to cancel the operation. This service is implemented and provided by the UIP when calling the ShowProgressBar service. The UIP is responsible for closing the ProgressBar.

Due to the asynchronous nature of callbacks, the CancelCallback might be called even after the UIP has called the EndShowProgressBar service.

5.2.2.13.2 Parameters

There are no specific parameters for the service.

5.2.2.14 StandardUIActionItemsChangeCallback Service**5.2.2.14.1 Description**

This service is used by the UIP to notify the FDI Client about the change in the standard UI action items' state (enabled/disabled). See 6.1.2.2 for standard UI action items.

5.2.2.14.2 Parameters

Table 59 defines the parameters for the service.

Table 59 – StandardUIActionItemsChange Service parameters

Name	Type	Description
Request		
Response		
StandardUIActionItems[]	StandardUIActionItem	Updated list of Standard Actions provided by the UIP.

5.2.2.15 SpecificUIActionItemsChangeCallback Service

5.2.2.15.1 Description

This service is used by the UIP to notify FDI Client about the change in the UI action items that are specific to this UIP. The UIP shall use this callback whenever there is addition or removal of action items or whenever there is a change in the state of an action item (i.e. enabled/disabled). See 6.1.2.4 for specific UI actions.

5.2.2.15.2 Parameters

Table 60 defines the parameters for the service.

Table 60 – SpecificUIActionItemsChange Service parameters

Name	Type	Description
Request		
Response		
SpecificUIActionItems[]	SpecificUIActionItem	Updated list of UI action Items specific to the UIP.

5.2.3 Parameter Type Definitions

5.2.3.1 DefaultResult Definition

The components of this parameter are defined in Table 61.

Table 61 – DefaultResult definition

Name	Type	Description
DefaultResult	Integer	Definition of a button on a message box. Used to specify the default button and the button selected by the user. This value is an enumeration with one of the following values: BUTTONNONE_0 BUTTONOK_1 BUTTONCANCEL_2 BUTTONYES_3 BUTTONNO_4

5.2.3.2 ButtonSet

The components of this parameter are defined in Table 62.

Table 62 – ButtonSet definition

Name	Type	Description
ButtonSet	Integer	Definition of the buttons on a message box or progress bar shown to the user. This value is an enumeration with one of the following values: BUTTONSETOK_0 Only ok button is shown BUTTONSETOKCANCEL_1 Ok and cancel buttons are shown BUTTONSETYESNOCANCEL_2 Yes, no, and cancel buttons are shown BUTTONSETYESNO_3 Yes and no buttons are shown

5.2.3.3 AcknStyle

The components of this parameter are defined in Table 63.

Table 63 – AcknStyle definition

Name	Type	Description
AcknStyle	Integer	The style of the message box or progress bar shown to the user. For example, this parameter may influence the icon of the message box. This value is an enumeration with one of the following values: ACKNSTYLEINFO_0 Information style ACKNSTYLEWARNING_1 Warning style ACKNSTYLEERROR_2 Error style

6 UIP

6.1 UIP Services

6.1.1 Services

6.1.1.1 Activate Service

6.1.1.1.1 Description

This service is used by the FDI Client to initialize a UIP (see 6.1.2). The FDI Client shall call this service after the creation of an instance of a UIP. The following rules define the window type:

- A modal window is used, if the UIP style = dialog, or the invoking parent (UID or UIP) is modal.
- A non-modal window is used if the UIP style = window and the invoking parent runs in non-modal mode also.

6.1.1.1.2 Parameters

Table 64 defines the parameters for the service.

Table 64 – Activate Service parameters

Name	Type	Description
Request		
hostingInterface	Interface	FDI Client hosting the interface to be used by the UIP (see 5.2).
deviceAccessInterface	Interface	FDI Client device access interface to be used by the UIP (see 5.1).
context	UInteger	Specifies in which context the UIP is activated: 0_ONLINE 1_OFFLINE The Client shall derive the context from the FunctionalGroup where this UIP has been retrieved from, i.e., whether the FunctionalGroup is part of the Offline device representation or the online device representation. If a UIP is invoked from another UIP with the OpenUserInterface or OpenModalUserInterface service, the context is inherited.
localeSetting	<technology dependent>	The locale description consists of at least a language identifier, display information and country. The format of the property value is technology dependent.
Response		

The localeSetting shall not depend on the operating system settings. The operating system settings can be used by default but shall be alterable by the user.

The language identifier shall indicate the language for localized textual information.

The display information shall be used to control colors (e.g. background color, text color for different states) and the format of specific elements (e.g. for numbers and dates).

Parameter country shall be used to apply country-specific regulations such as allowed engineering units. To do this, the UIP will modify the EngineeringUnit Attribute of correlated Variable Nodes to match the specified country. The FDI Server is then responsible to reformat the values accordingly.

6.1.1.2 Deactivate Service

6.1.1.2.1 Description

This service is used by the FDI Client to deactivate a UIP. The UIP shall release all references to other components and finish all pending or running functions including UIPs that it opened that are still open.

The UIP may indicate that it is not ready to be deactivated. This shall be used if cancelling pending or running functions would have severe side-effects.

6.1.1.2.2 Parameters

Table 65 defines the parameters for the service.

Table 65 – Deactivate Service parameters

Name	Type	Description
Request		
Response		
deactivateCancelled	Boolean	Indication whether the UIP refused the Deactivate request. deactivateCancelled = "true" denotes that the UIP refused the Deactivate request. deactivateCancelled = "false" denotes that the UIP accepted the Deactivate request.

6.1.1.3 SetSystemLabel Service

6.1.1.3.1 Description

This service is used by the FDI Client to specify a human identifier of the UIP instance in the context of the FDI Client.

The label shall be used for user interface elements displayed and managed by the UIP (e.g. a message box) and will assure that the user can uniquely identify to which Device the user interface element is directed. The UIP can extend this label with specific information when appropriate.

The FDI Client shall call this service before the Activate service (see 6.1.1.1). If it has not been called, the UIP shall use the text portion of the Label Attribute of the Device by default (see 5.1.3.2)

6.1.1.3.2 Parameters

Table 66 defines the parameters for the service.

Table 66 – SetSystemLabel Service parameters

Name	Type	Description
Request		
systemLabel	String	Human readable label that identifies the UIP within the FDI Client.
Response		None.

6.1.1.4 SetTraceLevel Service

6.1.1.4.1 Description

This service is used by the FDI Client to notify the UIP of the types of Trace messages that should be logged via the Trace service (see 5.2.2.8). Multiple types can be set (for instance: Critical, Error, and Warning).

6.1.1.4.2 Parameters

Table 67 defines the parameters for the service.

Table 67 – SetTraceLevel Service parameters

Name	Type	Description
Request		
traceLevel	TraceLevel	Trace level that shall control the type of information (see Table 72).
Response		

6.1.1.5 GetStandardUIActionItems Service

6.1.1.5.1 Description

This service is used by the FDI Client to get the available standard UI actions in the UIP (for instance: Close, Apply, Help). The FDI Client provides consistent representation of these standard UI actions, in the FDI Client UI area.

6.1.1.5.2 Parameters

Table 68 defines the parameters for the service.

Table 68 – GetStandardUIActionItems Service parameters

Name	Type	Description
Request		
Response		
StandardUIActionItems[]	StandardUIActionItem	List of Standard Actions provided by the UIP.

6.1.1.6 GetSpecificUIActionItems Service

6.1.1.6.1 Description

This service is used by the FDI Client to get the available UI actions that are specific for the UIP. The FDI Client provides consistent representation of these UI actions, in the FDI Client UI area.

6.1.1.6.2 Parameters

Table 69 defines the parameters for the service.

Table 69 – GetSpecificUIActionItems Service parameters

Name	Type	Description
Request		
Response		
SpecificUIActionItems[]	SpecificUIActionItem	List of UI actions specific for the UIP.

6.1.1.7 InvokeStandardUIAction Service

6.1.1.7.1 Description

This service is used by the FDI Client to notify the UIP to perform a standard action (for instance: Close, Apply, Help). The FDI Client passes the type of standard action to be performed in this service.

6.1.1.7.2 Parameters

Table 70 defines the parameters for the service.

Table 70 – InvokeStandardUIAction Service parameters

Name	Type	Description
Request		
actionId	StandardUIAction	One of the values defined in 6.1.2.2.
Response		

6.1.1.8 InvokeSpecificUIAction Service

6.1.1.8.1 Description

This service is used by the FDI Client to notify the UIP to perform a UI action that is specific to this UIP. The FDI Client passes the identifier of the action to be performed in this service.

6.1.1.8.2 Parameters

Table 71 defines the parameters for the service.

Table 71 – InvokeSpecificUIAction Service parameters

Name	Type	Description
Request		
actionId	UInteger	This is the identifier of the specific UI action to be performed.
Response		

6.1.2 Parameter type definitions

6.1.2.1 TraceLevel

TraceLevel is defined in Table 72.

Table 72 – TraceLevel definition

Name	Type	Description
TraceLevel	UInteger	Severity level that controls the type of information that is passed to the Trace service (see 5.2.2.8). This value is a bit enumeration with one of the following values: NONE_0 Completely switch off tracing CRITICAL_1 Fatal error or application crash ERROR_2 Recoverable error WARNING_4 Noncritical error INFO_8 Informational message VERBOSE_16 Debugging trace

6.1.2.2 StandardUIAction

Standard actions are defined to allow consistency in appearance and processing. They will be requested by the hosting Client. These actions are not expected to be mapped 1:1 to user interface buttons. Rather, the Client will display OK, CANCEL, and APPLY.

Clicking OK, CANCEL or APPLY by the User causes the following actions to be called.

- OK will cause a call to the Apply action followed by a call to the Close action.
- APPLY will cause a call to the Apply action.
- CANCEL will cause a call to the Close action. If changes are still outstanding, the UIP shall open a dialog asking the user to confirm.

The StandardUIAction enumeration is defined in Table 73.

Table 73 – StandardUIAction definition

Name	Type	Description
StandardUIAction	UInteger	Identifier for the standard UI action item with one of the following values: APPLY_0 Apply Standard UI Action - With this action the Client requests that parameter changes in the user interface will be applied to the source. CLOSE_1 Close Standard UI Action - With this action the Client requests that the UIP be closed. If changes to the parameters have not been applied, the UIP shall open a dialog asking the user to confirm. HELP_2 Help Standard UI Action - Requests to display online help describing the UIP functionality (a help document). It is no substitute for context-sensitive help which will typically be provided via tooltips.

6.1.2.3 StandardUIActionItem

The components of this parameter are defined in Table 74.

Table 74 – StandardUIActionItem definition

Name	Type	Description
StandardUIActionItem	Structure	
actionId	StandardUIAction	Identifier for the standard UI action item.
enabled	Boolean	Indicates the state of the action item. "true" indicates the item is enabled. "false" indicates the item is disabled.

6.1.2.4 SpecificUIActionItem

The components of this parameter are defined in Table 75.

Table 75 – SpecificUIActionItem definition

Name	Type	Description
SpecificUIActionItem	structure	
actionId	UInteger	Identifier for the action item that is specific to the UIP.
descriptor	String	A human readable string which provides information about the action.
enabled	Boolean	Indicates the state of the Action Item. "true" indicates the item is enabled. "false" indicates the item is disabled.
label	String	Label of the action item.

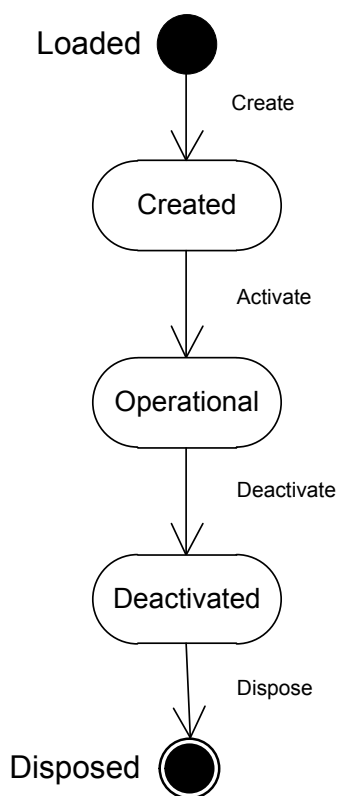
6.2 UIP instantiation rules

For each device instance there may be a maximum of two instances of the same UIP type, one for the offline version and one for the online version.

6.3 UIP state machine

6.3.1 States

Figure 7 shows the UIP state machine. If the UIP Services (see 6.1) trigger state changes, they are mentioned on the state machine edges.



IEC

NOTE Create and Dispose are technology dependent services described in IEC 62769-6. Create uses the StartElementName Property (see IEC 62769-5) of the UIP to create the appropriate UIP.

Figure 7 – UIP state machine

The UIP states are specified in Table 76.

Table 76 – UIP states

State	Description
Created	The initial state after the UIP instance is created by the FDI Client.
Operational	The normal execution state.
Deactivated	The final state before a UIP is disposed.

6.3.2 State transitions

The UIP state transitions are defined in Table 77.

Table 77 – UIP state transitions

Source state	Event	Destination state
Loaded	The FDI Client created the UIP instance.	Created
Created	Activate service has been called on UIP instance.	Operational
Operational	Deactivate service has been successfully called on UIP instance.	Deactivated
Deactivated	The FDI Client disposed the UIP instance.	Disposed

6.4 UIP permissions

A UIP is granted the following permissions with respect to operating system resources

- File system access (read/write) (mandatory).

- Printer access (mandatory – if a printer is available in the hosting environment).
- Network access (optional).
- ActiveX control usage (optional).

NOTE 1 The installation of ActiveX Controls is outside the scope of this standard.

Mandatory means that an FDI Client shall grant the permissions as they are granted by the operating system. This does not mean that a UIP has unrestricted access to the complete file system. The permissions for accessing the file system and printers are defined by the system configuration.

NOTE 2 The setting of permissions for operation system resources in a system configuration is outside the scope of this standard.

Optional means that an FDI Client can restrict the permissions compared to the permissions given by the system configuration. The means by which an FDI Client can restrict the network access and/or ActiveX control usage is specified in IEC 62769-6.

UIPs shall never block the Client, for instance by issuing synchronous calls to potentially block operating system resources. Worker threads are a common means for avoiding blocking situations.

6.5 UIP downloads from FDI Server

The following scenarios require an FDI Client to retrieve a UIP.

- The UID specifies a UIP in its XML element Plugin (see Annex A).
- The UIP opens another UIP with the OpenUserInterface service (see 5.2.2.3).

In all scenarios the FDI Client gets a UipId as identification for the UIP. A UipId is a UUID (universally unique identifier). A UIP may have several UIP Variants that differ in their platform and runtime support (see IEC 62769-4:2015, 5.3.3.2.2).

NOTE A UipId does not contain version information. Resolving a UipId to the appropriate version of the UIP is done by the FDI Server based on FDI Package information.

The same UipId may be resolved to different UIP versions for different Devices. At different points in time even for the same Device the same UipId may be resolved differently if a UIP update happened in the FDI Server in between.

With the UipId the FDI Client can retrieve the following information about all corresponding UIP Variants (see IEC 62769-5) from the FDI Server:

- RuntimeId and PlatformId
- UIPVariantVersion
- FDITechnologyVersion

For this the FDI Client calls the OPC UA service TranslateBrowsePathToNodeIds with the following list of relative names:

```
"UIPSet/<UipId>"
"UIPSet/<UipId >/RuntimeId"
"UIPSet/<UipId >/PlatformId"
"UIPSet/<UipId >/FDITechnologyVersion"
"UIPSet/<UipId >/Style"
"UIPSet/<UipId >/StartElementName"
"UIPSet/<UipId >/UIPVariantVersion"
```

The FDI Server returns arrays of NodeIds for each relative name. The number of entries in each array matches the number of UIP Variants for the UiPId. Next the FDI Client can read the property values.

If multiple UIP Variants are available, the FDI Client chooses the most appropriate UIP Variant based on FDITechnologyVersion, RuntimeId, and PlatformId.

The FDI Client may maintain a cache of UIP Variants already downloaded from the FDI Server. If UiPId, RuntimeId, PlatformId, FDITechnologyVersion and UIPVariantVersion of the chosen UIP Variant match with a UIP Variant in its UIP Variant cache, this UIP Variant can be used and no download from the FDI Server is required.

If no cache is maintained or no matching UIP Variant is found in the cache, the FDI Client shall read the FDITechnologyVersion (see IEC 62769-5) of the chosen UIP Variant. The parameter FDITechnologyVersion helps the FDI Client to determine whether it can support the execution of the UIP. The FDI Client shall execute a UIP of the same major version independently from the minor version or revision.

Finally the FDI Client can download the selected UIP from the FDI Server by reading the value of the UIP Variant.

An FDI Client may implement another sequence than the one described here. For example, it may check the FDITechnologyVersion first to determine whether it can run the UIP at all. The FDITechnologyVersion is the same for all UIP Variants of a specific version of a UIP.

The FDI Client may implement optimization strategies. For example, it may cache UIP versions (in addition to UIP Variant versions). If the version of the UIP is identical to the cached UIP version, there is no need to read the UIP Variant versions because they are required to be the same for an identical UIP version (see IEC 62769-4).

7 Actions

7.1 General

The EDD contained in the FDI Package as defined in IEC 62769-4 may have EDD methods. Some EDD methods may be exposed to the FDI Client as Actions and can be triggered by FDI Clients.

NOTE These Actions have no relationship to the EDD ACTION construct.

Actions are executed in the FDI Server. The Action state machine is defined in IEC 62769-3.

An FDI Client can invoke an Action by calling the OPC UA InvokeAction method (see IEC 62769-5). State changes of the corresponding Action state machine are sent to the FDI Client via the subscription mechanism (see IEC 62541-4). Additional data that may come with a state change are transferred as an XML document.

An Action may involve user interaction. The result of a user interaction is sent to the FDI Server with the RespondAction service (see IEC 62769-5). Data that are sent with this service are transferred as an XML document.

An Action can be aborted either by the Client or by the Server.

If the Server aborts an Action, it first sends an AbortingNotification. The Client shall reply with an AbortNotificationConfirmation but still continue processing ActionRequests. Once the AbortNotification was received, the Client shall disable the CancelButton. When the abort process is finished, the Server will send an AbortRequest to the Client. The Client replies with an AbortResponse and closes the ActionWindow.

An FDI Client may abort an Action as follows:

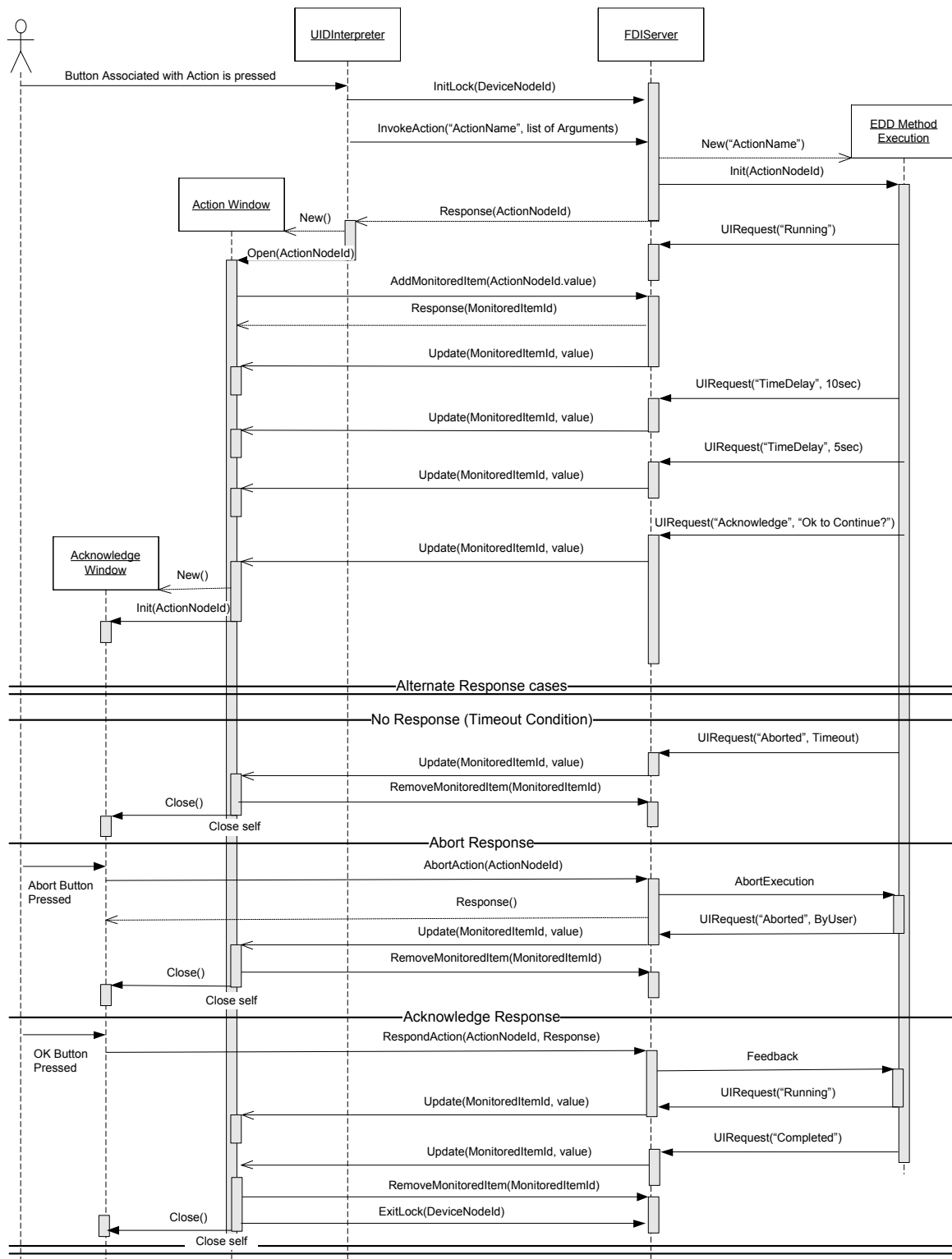
- 1) Call the AbortAction service (see IEC 62769-5)
- 2) Reply to an existing ActionRequest by sending a valid ActionResponse.
- 3) Continue processing of ActionRequests.
- 4) The Server sends an AbortingNotification once it has started the abort process. The Client shall reply with an AbortNotificationConfirmation but still continue processing ActionRequests. Once the AbortNotification was received, the Client shall disable the CancelButton.
- 5) The Action is aborted after an AbortingRequest is received from the Server. The Client shall reply with an AbortResponse and closes the ActionWindow.

Annex B contains an example of an EDD method being executed by an FDI Server and the resulting interaction with an FDI Client. It includes state diagrams and exchanged messages (XML snippets).

7.2 Sequence diagram

The sequence diagram shown in Figure 8 shows the client/server interaction of an Action call. This diagram assumes as a pre-condition that the Action can be started from a user interface shown by the FDI Client or by a UIP.

NOTE 1 The diagram shows UIDInterpreter, ActionWindow and AcknowledgeWindow as subsystems of the FDI Client. It also shows EDDMethodExecution as a subsystem of the FDI Server. This is for explanatory reasons only.



IEC

Figure 8 – FDI Action sequence diagram

The Action is initiated from the user interface. If not already locked, InitLock has to be called first. The FDI Client then calls the OPC UA InvokeAction method (see IEC 62769-5). The name of the Action to be invoked, as well as any Action arguments needed, is given as OPC UA method arguments.

NOTE 2 This description is identical whether the Action is initiated from a UIP or a UID. The only difference is that a UIP does not invoke the Action in the FDI Server directly but uses corresponding services in the DeviceAccess interface. The FDI Client then calls the Action on the FDI Server on behalf of the UIP.

NOTE 3 The interaction between FDI Client and FDI Server like the subscription to the ActionNodeId and the rendering of user interfaces (if requested during Action execution) is always under the responsibility of the FDI Client regardless of whether the Action has been initiated by a UID or UIP. Therefore a UIP need not be aware of the XML documents (as defined by the XML schema in Annex A) that are exchanged between FDI Client and FDI Server.

The FDI Server responds to the InvokeAction call by creating an Action execution state machine and responds to the FDI Client by returning an ActionNodeId that represents the state machine in the Information Model.

The FDI Server is responsible for running the state machine (as defined in IEC 62769-3) and updating its state in the Information Model using the provided ActionNodeId. The ActionNodeId is also used by the FDI Client to establish subscriptions with the FDI Server in order to monitor the Action execution. As the Action execution proceeds, the FDI Server updates the state machine and the corresponding Node in the Information Model. If there are any existing subscriptions for the ActionNodeId the FDI Server will process these changes and advise the FDI Client of the changes.

During the Action execution, operations such as notifications, read and write (also to other Nodes than the Node with the ActionNodeId) are not blocked.

The FDI Server begins an Action by first setting the state to Running and then starts to execute the EDD method. The FDI Client uses the OPC UA AddMonitoredItem service (see IEC 62541-4) to establish a subscription using the provided ActionNodeId. The FDI Server responds to the newly created subscription by advising the FDI Client of the current state of the Action state machine. In this sequence diagram the subscription was established before the Action execution reaches any notable state such as the TimeDelay state. Therefore the first notice to the FDI Client would indicate "Running". If the Action had reached the TimeDelay state before the subscription was established the first notice to the FDI Client would indicate "TimeDelay" since this would be the current state.

The FDI Client shall not maintain an Action state machine.

NOTE 4 The information sent to the FDI Client combines state information and the last UI request. Therefore there is no race condition with respect to when the FDI Client adds the state Node as a monitored item. Even if the Action state machine has gone through several state transitions, the FDI Client need not know this. The standard OPC UA behaviour is that the current information is sent when a Node is added as monitored item.

When the FDI Server enters the TimeDelay state the ActionNodeId is updated at the beginning of the delay and at the end of the delay. There may also be intermediate updates regarding the length of the delay. The actual frequency of the intermediate updates is determined by the design of the FDI Server, typically every few seconds. All updates of the ActionNodeId will be submitted to the FDI Client via the subscription mechanism.

EXAMPLE 1 In Figure 8 the FDI Server has entered a time delay of 10 seconds and sends updates every 5 seconds.

When the FDI Server enters the WaitingForFeedback state the Information Model is updated including the user prompt. The state machine pauses the execution of the Action until a response from the user is provided or a timeout expires. The state machine change results in the FDI Client being provided a notification. The FDI Client detecting that the new state is WaitingForFeedback opens a message dialog presenting the prompt provided in the state information as well as a means to provide feedback.

EXAMPLE 2 In Figure 8 the FDI Client shows the buttons to "OK" the method or "Abort" the method.

The sequence diagram shows three alternate response cases.

Response case “Timeout”: If the user fails to respond to the prompt before the server timeout expires the FDI Server will abort the Action execution. The state in the Information Model is set to Aborted with an indication that the cause was a timeout condition. The Action execution terminates at this point but the FDI Server maintains the state Node and the final state until the subscriptions are terminated either by the FDI Client removing the monitored item or a general timeout of the FDI Client session with the FDI Server.

Response case “Abort”: If the user responds to the FDI Server with an abort response the method execution is aborted. The state in the Information Model is set to Aborted with an indication that the cause was a user action. The FDI Server maintains the state Node and the final state until the subscriptions are terminated either by the FDI Client removing the monitored item or a general timeout of the FDI Client session with the FDI Server. The state Node is also preserved if the Completed or Aborted state of the Action state machine is reached before the FDI Client established a subscription and added the state Node as a monitored item.

Response case “Positive feedback”: If the user responds to the FDI Server with positive feedback the Action execution sets the state in the Information Model back to Running and continues the normal execution of the Action.

EXAMPLE 3 In Figure 8 no further state changes occur before the end of the Action and therefore the next state change is to the final Completed state.

The state machine terminates after the final state is set. The FDI Server maintains the state Node and the final state until the subscriptions are terminated either by the FDI Client removing the monitored item or a general timeout of the FDI Client session with the FDI Server.

NOTE 5 No separate KeepAlive method is needed. The supervision of the session tells the FDI Server that the FDI Client is alive.

After the Action execution ended and the monitored item has been removed the lock can be removed.

7.3 FDI Action schema definition

The XML schema for the XML sent between the FDI Client and FDI Server is defined in Annex A.

NOTE The XML schema in Annex A also includes the definitions for UIDs.

ActionRequest is the root element of the XML documents that are exchanged from the FDI Server to the FDI Client during Action execution. These XML documents are sent to the FDI Client with the Update service (see IEC 62541-4). The mandatory part of these documents is the ActionState that specifies the current state of the corresponding Action state machine (see IEC 62769-3). Additional data is specified if the FDI Client has to show a user interface on behalf of the Action. The user interface style as well as content is specified in this additional data.

ActionResponse is the root element of the XML documents that are exchanged from FDI Client to FDI Server. These documents are used when the user provides feedback for a UI request. The Action state machine (as defined in IEC 62769-3) shall be in state WaitingForFeedback or WaitingForFeedbackA. The FDI Server sets the state back to Running or Aborting after receiving the user feedback. These XML documents are sent with the RespondAction service (as defined in IEC 62769-5).

Arguments for Actions are also specified by XML documents. The arguments are specified with the ListOfActionArguments type as name-value pairs.

8 User Interface Description (UID)

8.1 Overview

As defined in IEC 62769-5, top-level FunctionalGroup in the Information Model may contain a UIDescription (UID) Node or a set of UIPlugin (UIP) Nodes. The Value Attribute of a UID Node is a string, and the UID XML Schema defines the contents of that string. The XML schema is defined in Annex A.

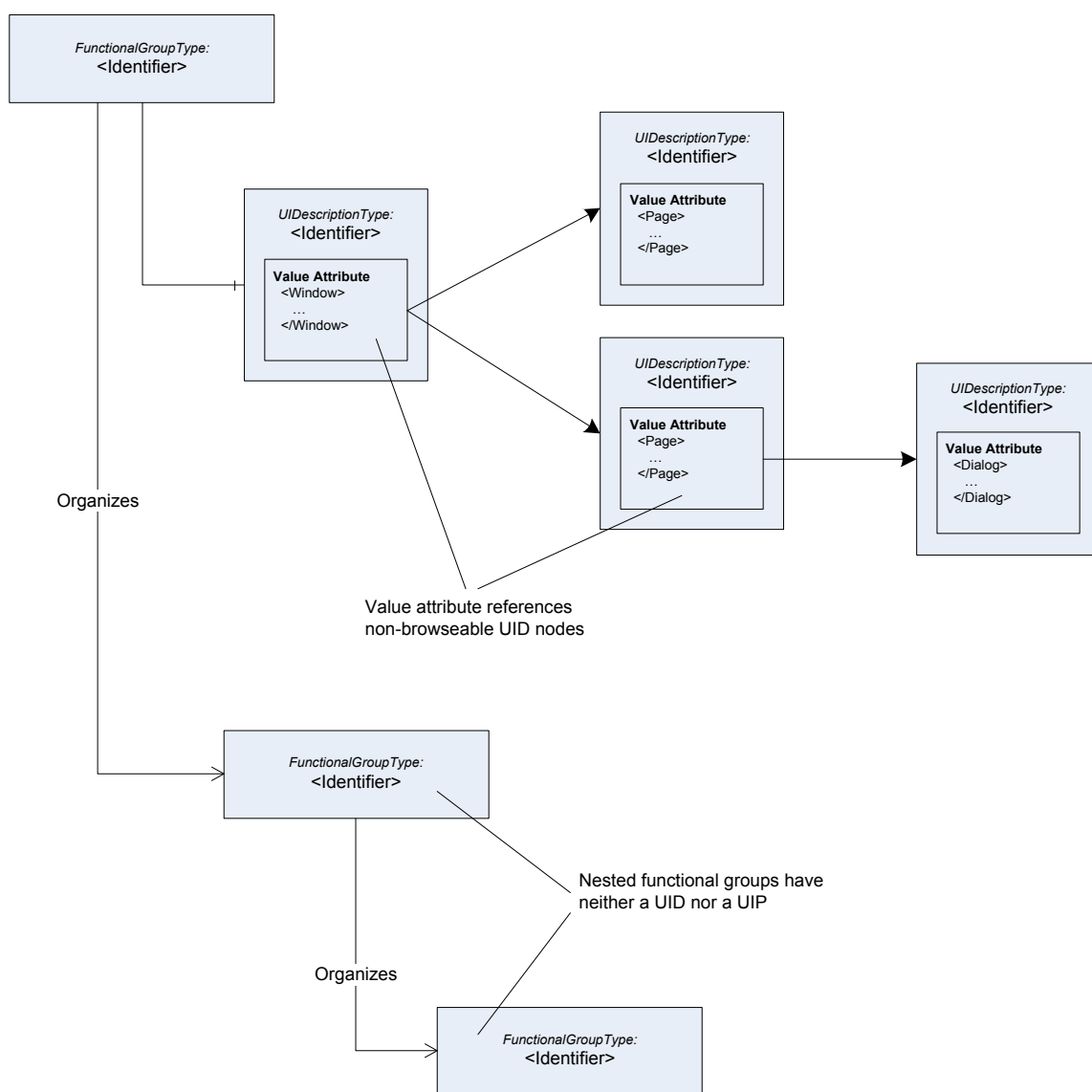
NOTE 1 The XML schema in Annex A also includes the definitions for FDI Actions.

In addition to the UID Nodes exposed in the Information Model via FunctionalGroups, there may be non-browseable UID Nodes. As shown in Figure 9, non-browseable UID Nodes are linked to a parent UID Node via the NodePath attribute. The parent UID Node may be either a browseable or non-browseable Node.

The root element of the Value Attribute of a FunctionalGroup's UID Node shall be a Window, Dialog, Menu, or Table element. The root element of the Value Attribute of a non-browseable UID Node shall be a Window, Dialog, Page, Menu, or Table element.

The Value Attribute of a FunctionalGroup's UID Node shall contain enough information to allow the FDI Client to render the visible parts of the view. The FDI Server may omit those parts of the view that are not visible. In place of the omitted information the FDI Server shall provide a reference (via the NodePath attribute) to a non-browseable UID Node that contains the missing information. The FDI Client may use the specified NodePath to obtain the missing information from the FDI Server as it deems necessary.

NOTE 2 An example of a non-visible part of a window would be the second page of a tabbed dialog. The label of the second page is visible, but the content of the second page is not.



IEC

Figure 9 – User Interface Descriptions

The XML returned to the FDI Client from an FDI Server contains layout elements and content elements. Layout elements define the visual organization, positioning, and structure of the user interface. Content elements are the basic building blocks of the user interface.

NOTE 3 The XML only includes "valid" elements. New XML documents will be returned if validity changes.

The layout elements are

- ColumnBreak
- Dialog
- EditDisplay
- Group
- Menu
- Page
- RowBreak
- Table

- Window

The content elements are

- Action
- Chart
- Graph
- Grid
- Image
- Parameter
- Plugin
- Text

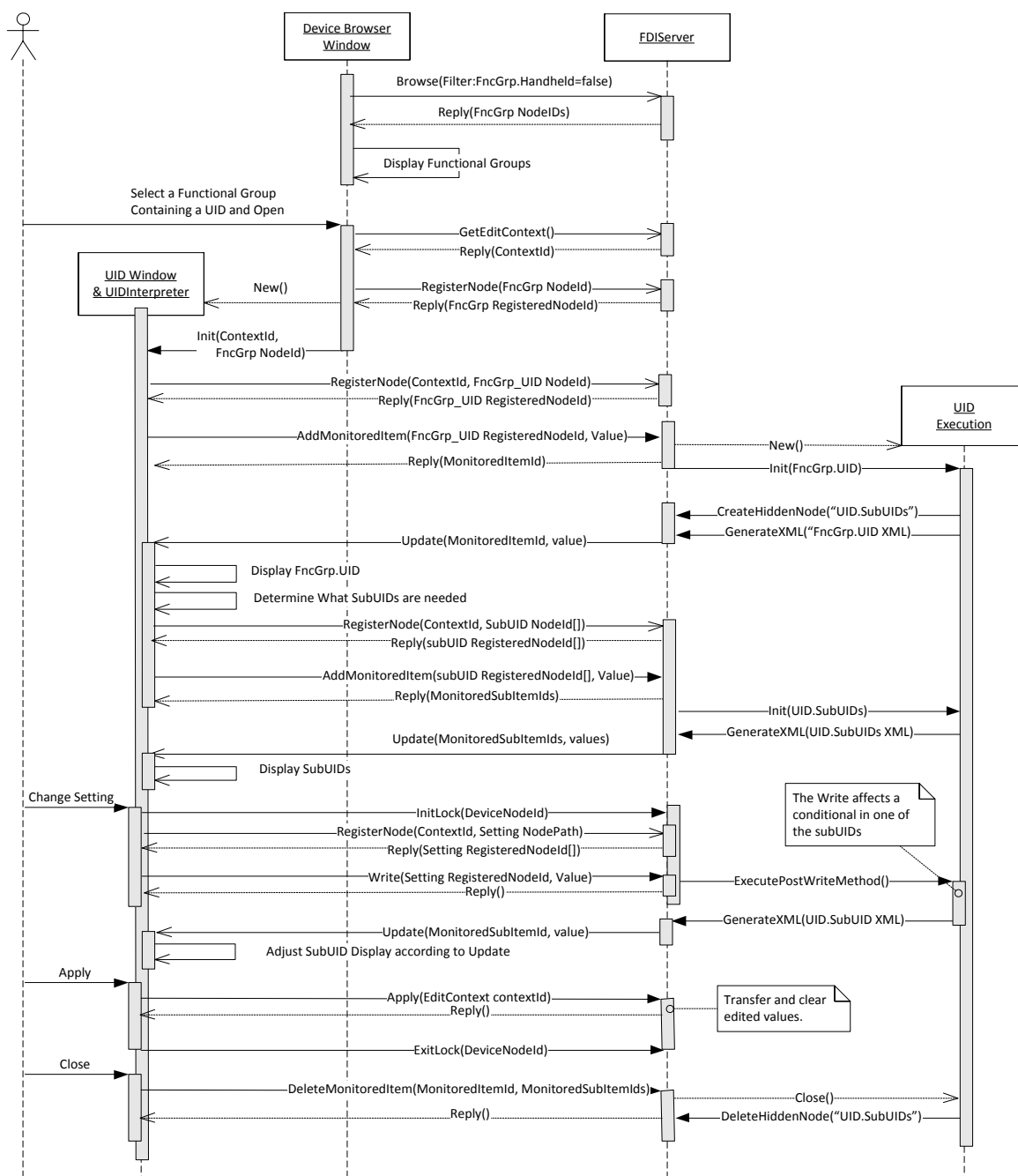
The algorithm for rendering these elements onto a computer screen is specified in IEC 61804-4.

NOTE 4 The definition of UIDs is heavily influenced by IEC 61804-3 and IEC 61804-4.

8.2 UID execution

Figure 10 illustrates an example of the sequence of steps used by an FDI Client to call up and execute a UIDescription (UID). This example assumes as a pre-condition that the FDI Client has established a session with the FDI Server, the user has navigated to a Device using some form of Information Model browsing or lookup, and the FDI Client is presenting the user with a list of FunctionalGroups. The example includes a sub UID that is referenced in the XML of the UID and that contains conditional content. The example illustrates the modification of a Parameter used by the sub UID to calculate the conditional content.

NOTE 1 The diagram shows UIDWindow, UIDInterpreter and Device Browser Window as subsystems of the FDI Client. It also shows UIDExecution as subsystem of the FDI Server. This is for explanatory reasons only.



IEC

Figure 10 – User Interface Description sequence diagram

The sequence in Figure 10 begins with the user selecting one of the FunctionalGroups to open. The Device Browser Window initiates the opening of the UID associated with the selected FunctionalGroup by acquiring an EditContext and creating a new UID window and UID Interpreter that will be used to present the UID to the user. The newly created UID window is initialized by providing it with the EditContext and the NodeId of the selected FunctionalGroup in the EditContext. To retrieve this NodeId the RegisterNodesByRelativePath Method is called passing the BrowsePath of the FunctionalGroup and the EditContext as input.

Locking is required for the Write Service.

NOTE 2 The example therefore includes calls to lock (InitLock) and unlock (ExitLock) the Device.

The UID Window begins by establishing a subscription to the value of the UID contained in the referenced FunctionalGroup using the OPC UA AddMonitoredItem service in order to obtain the content (i.e., the XML representation) of the UID. The FDI Server responds to the subscription request by creating a UID Execution machine and initializing it by passing an internal identifier of the selected UID. The UID Execution machine interprets the UID definition, creates the referenced non-browseable sub UID Nodes in the Information Model and generates the XML representation of the top level UID. The generated XML representation is maintained in the Information Model to support the UID Window's subscribe. The NodePaths of the sub UIDs are included in the XML of the parent UIDs. Using the RegisterNodesByRelativePath Method the NodeSpecifiers are translated into NodeIds. The UID Window can use those NodeIds to create subsequent subscriptions. The FDI Server provides the content of the top level UID that was subscribed to by the UID Window.

The UID Window interprets the XML content of the updated UID value provided by the subscription and renders the user interface. It also collects the references to the sub UIDs and issues a second OPC UA AddMonitoredItem service request to include the sub UIDs in the subscription. The FDI Server responds to the AddMonitoredItem by initializing the sub UIDs Nodes resulting in the generation of XML representations of the sub UID content. The FDI Server updates the UID Window with the generated value of each of the sub UIDs included in the OPC UA AddMonitoredItem request.

NOTE 3 The example uses multiple sub UIDs (but is unspecific about how many). For example, AddMonitoredItem(UID.SubUIDs.value) adds the Nodes of all SubUIDs to the subscription.

The UID Window responds to the subscription updates by rendering the sub UIDs in the user interface. In addition to the sub UIDs, UIDs also contain NodePaths referencing parameters and actions. Using the RegisterNodesByRelativePath those NodePaths are translated to NodeIds and the UID Window uses the AddMonitoredItem service to subscribe to the values of the parameters. The FDI Server provides the values to the UID Window. The UID is now fully displayed and ready.

When the user starts making a change to a value the UID Window requests a lock using the InitLock Method.

There are different strategies when to acquire a lock. At the latest it needs to be acquired before any pre-edit actions are executed or the value is written to the FDI server.

If a pre-edit action is defined on the changed parameter the UID Window calls that action.

When the user finished changing the value the value is written to the EditContext using the Write service. If a post-edit action is defined on the changed parameter the UID Window calls that action.

The UID Execution determines that the value change results in the change of a conditional contained in one of the sub UIDs. The XML value of the sub UID that is affected is regenerated and the Information Model is updated. The FDI Server reacts to the Information Model update by notifying the UID Window of the change.

The UID Window processes the value change of the sub UID and makes the necessary adjustments to the user interface to reflect the change.

The user completes the example sequence by clicking on an apply button that instructs the UID Window to shut down. The UID Window calls the Apply Method for its EditContext and the changes are applied to the device. Any pre- and post-write actions are executed. The UID Window removes the subscriptions by calling the OPC UA DeleteMonitoredItem service call.

The FDI Server processes the OPC UA DeleteMonitoredItem call by removing the items from the subscription. Afterwards the UID Window calls the Discard Method on the EditContext and the FDI server removes the non-browseable sub UID Nodes and closes the UID Execution.

Annex A (normative)

XML schema

A.1 General

The following is the XML schema definitions for UIDs as well as Actions. The types are listed in alphabetical order. The namespace xs: in the elements refers to the W3C XML Schema.

A.2 AbortRequestT

This type specifies a request sent from an FDI Server to an FDI Client when an Action is aborted. The FDI Client may inform the user about the reason the Action is aborting. Upon acknowledgement from the user, the FDI Client sends an ActionResponse back to the FDI Server.

The XML schema for an AbortRequestT type is:

```
<xs:complexType name="AbortRequestT">
  <xs:sequence>
    <xs:element name="Message" type="xs:string"/>
  </xs:sequence>
</xs:complexType>
```

The elements of an AbortRequestT type are described in Table A.1.

Table A.1 – Elements of AbortRequestT

Element	Description
Message	This required element specifies the reason the action has been aborted.

A.3 AccessT

This type specifies whether the access shall be ONLINE or OFFLINE.

The XML schema for an AccessT enumeration type is:

```
<xs:simpleType name="AccessT">
  <xs:restriction base="xs:string">
    <xs:enumeration value="ONLINE"/>
    <xs:enumeration value="OFFLINE"/>
  </xs:restriction>
</xs:simpleType>
```

The enumeration values of an AccessT enumeration type are described in Table A.2.

Table A.2 – Enumerations of AccessT

Enumeration	Description
ONLINE	The access shall be done ONLINE.
OFFLINE	The access shall be done OFFLINE.

A.4 AcknowledgementRequestT

This type specifies a request sent from an FDI Server to an FDI Client when the user needs to acknowledge a condition or state within an Action. Upon acknowledgement from the user, the FDI Client sends an AcknowledgementResponse back to the FDI Server. An AcknowledgementRequest shall only be used to acknowledge normal operating conditions. An AbortRequest shall be used to acknowledge a condition that leads to the Action being aborted.

The XML schema for an AcknowledgementRequestT type is:

```
<xs:complexType name="AcknowledgementRequestT">
  <xs:sequence>
    <xs:element name="Message" type="xs:string"/>
  </xs:sequence>
</xs:complexType>
```

The elements of an AcknowledgementRequestT type are described in Table A.3.

Table A.3 – Elements of AcknowledgementRequestT

Element	Description
Message	This required element specifies the condition the user is being asked to acknowledge in the form of a message to the user.

A.5 ActionListT

This type specifies a list of Action elements.

The XML schema for an ActionListT type is:

```
<xs:complexType name="ActionListT">
  <xs:sequence maxOccurs="unbounded">
    <xs:element name="Action" type="clnt:ActionT"/>
  </xs:sequence>
</xs:complexType>
```

The elements of an ActionListT type are described in Table A.4.

Table A.4 – Elements of ActionListT

Element	Description
Action	An element of the list.

A.6 AbortingNotificationT

This type is used in the ActionRequest element to notify the client that the action has been aborted on the server.

The XML schema for an AbortingNotificationT type is:

```
<xs:complexType name="AbortingNotificationT"/>
```

A.7 ActionRequestT

This type specifies an action request from the FDI server.

The XML schema for an ActionRequestT type is:

```
<xs:complexType name="ActionRequestT">
  <xs:sequence>
    <xs:element name="EditContext" type="xs:string"/>
    <xs:choice>
      <xs:element name="AbortingNotification" type="clnt:AbortingNotificationT"/>
      <xs:element name="AcknowledgementRequest"
        type="clnt:AcknowledgementRequestT"/>
      <xs:element name="AbortRequest" type="clnt:AbortRequestT"/>
      <xs:element name="UIDRequest" type="clnt:UidRequestT"/>
      <xs:element name="SelectionRequest" type="clnt:SelectionRequestT"/>
      <xs:element name="InputRequest" type="clnt:InputRequestT"/>
      <xs:element name="ParameterInputRequest"
        type="clnt:ParameterInputRequestT"/>
      <xs:element name="InfoRequest" type="clnt:InfoRequestT"/>
      <xs:element name="DelayMessageRequest" type="clnt:DelayMessageRequestT"/>
    </xs:choice>
  </xs:sequence>
</xs:complexType>
```

The elements of an ActionRequestT type are described in Table A.5.

Table A.5 – Elements of ActionRequestT

Element	Description
EditContext	This element specifies the EditContext to be used when editing Variables for this Action.
AbortingNotification	This element specifies that the action has been aborted on the server. To ensure correct abort processing, the User Interface shall not allow the cancellation of further action requests.
AcknowledgementRequest	This optional element specifies a request from an FDI Server to an FDI Client for the user to acknowledge a condition. The FDI Client will not respond to the FDI Server until the user acknowledged the condition.
AbortRequest	This optional element specifies a request from an FDI Server to an FDI Client to notify the user the Action is aborting.
UIDRequest	This optional element specifies a request from an FDI Server to an FDI Client to display complex user interface.
SelectionRequest	This optional element specifies a request from an FDI Server to an FDI Client for the user to make a selection from a list of possible alternatives
InputRequest	This optional element specifies a request from an FDI Server to an FDI Client for the user to edit data.
ParameterInputRequest	This optional element specifies a request from an FDI Server to an FDI Client for the user to edit a parameter.

Element	Description
InfoRequest	This optional element specifies a request from an FDI Server to an FDI Client for a message to be displayed to the user. The message does not require user acknowledgement and the FDI Client will respond immediately after displaying the message.
DelayMessageRequest	This type specifies a request sent from an FDI Server to an FDI Client when a delay is requested within an Action. The FDI Client informs the user of the reason for and duration of the delay. The FDI Client times the delay and sends a DelayMessageResponse back to the FDI Server after the time has elapsed.

A.8 ActionResponseT

This type specifies an action response to the FDI Server.

The XML schema for an ActionResponseT type is:

```
<xs:complexType name="ActionResponseT">
  <xs:choice>
    <xs:element name="AbortingNotificationConfirmation" type="clnt:ResponseT"/>
    <xs:element name="AcknowledgementResponse" type="clnt:ResponseT"/>
    <xs:element name="AbortResponse" type="clnt:ResponseT"/>
    <xs:element name="UidResponse" type="clnt:UidResponseT"/>
    <xs:element name="SelectionResponse" type="clnt:SelectionResponseT"/>
    <xs:element name="InputResponse" type="clnt:InputResponseT"/>
    <xs:element name="ParameterInputResponse" type="clnt:ResponseT"/>
    <xs:element name="InfoResponse" type="clnt:ResponseT"/>
    <xs:element name="DelayMessageResponse" type="clnt:ResponseT"/>
  </xs:choice>
</xs:complexType>
```

The elements of an ActionResponseT type are described in Table A.6.

Table A.6 – Elements of ActionResponseT

Element	Description
AbortingNotificationConfirmation	This optional element specifies the response of an FDI Client to an AbortingNotification from an FDI Server.
AcknowledgementResponse	This optional element specifies the response of an FDI Client to an AcknowledgementRequest from an FDI Server.
AbortResponse	This optional element specifies the response of an FDI Client to an AbortRequest from an FDI Server.
UidResponse	This optional element specifies the response of an FDI Client to an UIDRequest from an FDI Server.
SelectionResponse	This optional element specifies the response of an FDI Client to a SelectionRequest from an FDI Server.
InputResponse	This optional element specifies the response of an FDI Client to an InputRequest from an FDI Server.
ParameterInputResponse	This optional element specifies the response of an FDI Client to an ParameterInputRequest from an FDI Server.
InfoResponse	This optional element specifies the response of an FDI Client to an InfoRequest from an FDI Server.
DelayMessageResponse	This optional element specifies the response of an FDI Client to a DelayMessageRequest from an FDI Server.

A.9 ActionT

This type specifies an Action, which is a sequence of steps that requires collaboration between an FDI Client and an FDI Server.

The XML schema for an ActionT type is:

```
<xs:complexType name="ActionT">
  <xs:complexContent>
    <xs:extension base="clnt:UiElementT">
      <xs:sequence>
        <xs:element name="Name" type="xs:string"/>
        <xs:element name="Access" type="clnt:AccessT" minOccurs="0"/>
        <xs:element name="Class" type="clnt:ActionClassT" minOccurs="0"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

The elements of an ActionT type are described in Table A.7.

Table A.7 – Elements of ActionT

Element	Description
Name	This required element specifies the name of the Action, which may be passed to the InvokeAction method of a UID Node in an FDI Server.
Access	This optional element specifies whether the action shall be used ONLINE or OFFLINE.
Class	This optional element specifies the EDD CLASS attribute of the action.

A.10 AxisListT

This type specifies list of named axis elements of a graph or a chart.

The XML schema for an AxisListT type is:

```
<xs:complexType name="AxisListT">
  <xs:sequence minOccurs="0" maxOccurs="unbounded">
    <xs:element name="Axis" type="clnt:AxisT"/>
  </xs:sequence>
</xs:complexType>
```

The elements of a AxisListT type are described in Table A.8.

Table A.8 – Elements of AxisListT

Element	Description
Axis	An element of the AxisListT.

A.11 AxisT

This type specifies an axis of a graph or a chart.

The XML schema for an AxisT type is:

```
<xs:complexType name="AxisT">
  <xs:complexContent>
    <xs:extension base="clnt:LabelHelpT">
      <xs:sequence>
        <xs:element name="MaximumValue" type="clnt:VariantT" minOccurs="0"/>
        <xs:element name="MinimumValue" type="clnt:VariantT" minOccurs="0"/>
        <xs:element name="DisplayedRange" minOccurs="0">
          <xs:complexType>
            <xs:attribute name="NodePathViewMinimum" type="xs:string"
              use="required"/>
            <xs:attribute name="NodePathViewMaximum" type="xs:string"
              use="required"/>
          </xs:complexType>
        </xs:element>
        <xs:element name="Scaling" type="clnt:ScalingT" default="Linear"
          minOccurs="0"/>
        <xs:element name="Unit" type="xs:string" minOccurs="0"/>
      </xs:sequence>
      <xs:attribute name="Name" type="xs:string" use="required"/>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

The attributes of an AxisT type are described in Table A.9.

Table A.9 – Attributes of AxisT

Attribute	Description
Name	Unique name of axis in the context of Chart/Graph that this axis is present.

The elements of an AxisT type are described in Table A.10.

Table A.10 – Elements of AxisT

Element	Description
MaximumValue	This required element specifies the largest value that lies within the bounds of the axis.
MinimumValue	This required element specifies the smallest value that lies within the bounds of the axis.
DisplayedRange	Sn1p1t Sn1p1t
Scaling	This optional element specifies how the values should be scaled. The default is Linear.
Unit	This optional element specifies the engineering unit of the axis.

A.12 BitEnumerationItemListT

This type specifies the content of a bit enumeration

The XML schema for a BitEnumerationItemListT type is:

```
<xs:complexType name="BitEnumerationItemListT">
  <xs:sequence maxOccurs="unbounded">
```

```

    <xs:element name="BitEnumerationItem" type="clnt:BitEnumerationItemT"/>
  </xs:sequence>
</xs:complexType>

```

The elements of a BitEnumerationItemListT type are described in Table A.11.

Table A.11 – Elements of BitEnumerationItemListT

Element	Description
BitEnumerationItem	An element of the bit enumeration.

A.13 BitEnumerationItemT

This type specifies the meaning of a single bit of a bitmapped value.

The XML schema for a BitEnumerationItemT type is:

```

<xs:complexType name="BitEnumerationItemT">
  <xs:complexContent>
    <xs:extension base="clnt:LabelHelpT">
      <xs:sequence>
        <xs:element name="Value" type="xs:unsignedLong"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>

```

The elements of a BitEnumerationItemT type are described in Table A.12.

Table A.12 – Elements of BitEnumerationItemT

Element	Description
Value	This required element specifies the bit as a bit mask, i.e., 0x1 specifies the least significant bit, 0x2 specifies the second least significant bit, 0x4 specifies the third most significant bit, and so on.

A.14 ButtonListT

This type specifies a list of Button elements. The UID response contains the 0-based index of the selected button.

The XML schema for a ButtonListT type is:

```

<xs:complexType name="ButtonListT">
  <xs:sequence maxOccurs="unbounded">
    <xs:element name="Button" type="clnt:LabelT"/>
  </xs:sequence>
</xs:complexType>

```

The elements of a ButtonListT type are described in Table A.13.

Table A.13 – Elements of ButtonListT

Element	Description
Button	An element of the button list.

A.15 ChartT

This type specifies a chart that graphically displays data from a device. The data is read from the device periodically and continuously. As new data arrives, it is displayed.

The XML schema for a ChartT type is:

```
<xs:complexType name="ChartT">
  <xs:complexContent>
    <xs:extension base="clnt:UiElementSizeableT">
      <xs:sequence>
        <xs:element name="Length" type="xs:nonNegativeInteger" default="600000"
          minOccurs="0"/>
        <xs:element name="Type" type="clnt:ChartTypeT" default="Strip"
          minOccurs="0"/>
        <xs:element name="CycleTime" type="xs:nonNegativeInteger" default="1000"
          minOccurs="0"/>
        <xs:element name="AxisList" type="clnt:AxisListT">
          <xs:key name="AxisKey">
            <xs:selector xpath="clnt:Axis"/>
            <xs:field xpath="@Name"/>
          </xs:key>
        </xs:element>
        <xs:element name="SourceList" type="clnt:SourceListT"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

The elements of a ChartT type are described in Table A.14.

Table A.14 – Elements of ChartT

Element	Description
Length	This optional element specifies the length of time, in milliseconds, that is displayed by the chart. The number of samples displayed by a chart can be calculated by dividing the Length by the CycleTime. The default is 600 000.
Type	This optional element specifies the type of the chart. The default is Strip.
CycleTime	This optional element specifies the rate, in milliseconds, at which data on the chart is updated. The default is 1 000.
AxisList	This required element specifies the vertical axis data displayed by the chart.
SourceList	This required element specifies the data displayed by the chart.

A.16 ChartTypeT

This type specifies the general appearance and behavior of a chart.

The XML schema for a ChartTypeT enumeration type is:

```
<xs:simpleType name="ChartTypeT">
  <xs:restriction base="xs:string">
    <xs:enumeration value="Gauge"/>
    <xs:enumeration value="HorizontalBar"/>
    <xs:enumeration value="Scope"/>
    <xs:enumeration value="Strip"/>
    <xs:enumeration value="Sweep"/>
    <xs:enumeration value="VerticalBar"/>
  </xs:restriction>
</xs:simpleType>
```

The enumeration values of a ChartTypeT enumeration type are described in Table A.15.

Table A.15 – Enumerations of ChartTypeT

Enumeration	Description
Gauge	A single source value is displayed as a gauge, which is a graphical representation similar to the fuel gauge in an automobile.
HorizontalBar	The source values are displayed as horizontal bars.
Scope	The source values are displayed as a curve moving from left to right. When the source values reach the far right of the display area, the display area is erased and the new source values are displayed beginning at the far left.
Strip	The source values are displayed as a curve moving from left to right. When the source values reach the far right of the display area, the display area is scrolled with the source values on the far left being removed and the new source values being displayed on the right.
Sweep	The source values are displayed as a curve moving from left to right. When the source values reach the far right of the display area, the new source values are displayed beginning at the far left, but unlike Scope only the portion of the display area needed to display the new source values is erased.
VerticalBar	The source values are displayed as vertical bars.

A.17 ColorNameT

This type specifies the name of a predefined color.

The XML schema for a ColorNameT enumeration type is:

```
<xs:simpleType name="ColorNameT">
  <xs:restriction base="xs:string">
    <xs:enumeration value="Agua"/>
    <xs:enumeration value="Black"/>
    <xs:enumeration value="Blue"/>
    <xs:enumeration value="Fuchsia"/>
    <xs:enumeration value="Gray"/>
    <xs:enumeration value="Green"/>
    <xs:enumeration value="Lime"/>
    <xs:enumeration value="Maroon"/>
    <xs:enumeration value="Navy"/>
    <xs:enumeration value="Olive"/>
    <xs:enumeration value="Purple"/>
    <xs:enumeration value="Red"/>
    <xs:enumeration value="Silver"/>
    <xs:enumeration value="Teal"/>
  </xs:restriction>
</xs:simpleType>
```

```

    <xs:enumeration value="White"/>
    <xs:enumeration value="Yellow"/>
  </xs:restriction>
</xs:simpleType>

```

The enumeration values of a ColorNameT enumeration type are described in Table A.16.

Table A.16 – Enumerations of ColorNameT

Enumeration	Description
Agua	RGB value #00FFFF.
Black	RGB value #000000.
Blue	RGB value #0000FF.
Fuchsia	RGB value #FF00FF.
Gray	RGB value #808080.
Green	RGB value #008000.
Lime	RGB value #00FF00.
Maroon	RGB value #800000.
Navy	RGB value #000080.
Olive	RGB value #808000.
Purple	RGB value #800080.
Red	RGB value #FF0000.
Silver	RGB value #C0C0C0.
Teal	RGB value #008080.
White	RGB value #FFFFFF.
Yellow	RGB value #FFFF00.

A.18 ColorT

This type specifies a color, either a name or an RGB value.

The XML schema for a ColorT type is:

```

<xs:simpleType name="ColorT">
  <xs:union memberTypes="clnt:ColorNameT clnt:ColorValueT"/>
</xs:simpleType>

```

A.19 ColorValueT

This type specifies an RGB value consisting of a hash character (#) followed by three or six hexadecimal digits. If three hexadecimal digits are specified, each character is repeated, for example, #F0F is equivalent to #FF00FF.

The XML schema for a ColorValueT type is:

```

<xs:simpleType name="ColorValueT">
  <xs:restriction base="xs:string">
    <xs:pattern value="#"[0-9a-fA-F]{3}"/>
    <xs:pattern value="#"[0-9a-fA-F]{6}"/>
  </xs:restriction>
</xs:simpleType>

```

A.20 ColumnBreakT

This type specifies a column break.

The XML schema for a ColumnBreakT type is:

```
<xs:complexType name="ColumnBreakT"/>
```

A.21 DateTimeDataT

This type specifies the data type of a date/time value.

The XML schema for a DateTimeDataT enumeration type is:

```
<xs:simpleType name="DateTimeDataT">
  <xs:restriction base="xs:string">
    <xs:enumeration value="Date"/>
    <xs:enumeration value="Time"/>
    <xs:enumeration value="DateTime"/>
    <xs:enumeration value="Duration"/>
    <xs:enumeration value="TimeValue"/>
  </xs:restriction>
</xs:simpleType>
```

The enumeration values of a DateTimeDataT enumeration type are described in Table A.17.

Table A.17 – Enumerations of DateTimeDataT

Enumeration	Description
Date	A value that represents a date without a time.
Time	A value that represents a time without a date.
DateTime	A value that represents a date and time.
Duration	A value that represents a length of time.
TimeValue	A value that represents a time of the day.

A.22 DelayMessageRequestT

This type specifies a request sent from an FDI Server to an FDI Client when a delay occurs within an Action. The FDI Client informs the user of the reason for the delay and expected duration of the delay. The FDI Client then sends an ActionResponse back to the FDI Server. The FDI Server may send DelayMessageRequests frequently with updated information regarding the length of the delay. When the delay is finished, the FDI Server sends a DelayMessageRequest with the SecondsToWait element set to zero.

The XML schema for a DelayMessageRequestT type is:

```
<xs:complexType name="DelayMessageRequestT">
  <xs:sequence>
    <xs:element name="Message" type="xs:string"/>
    <xs:element name="SecondsToWait" type="xs:unsignedLong"/>
  </xs:sequence>
</xs:complexType>
```

The elements of a DelayMessageRequestT type are described in Table A.18.

Table A.18 – Elements of DelayMessageRequestT

Element	Description
Message	This required element specifies the message to be displayed to the user.
SecondsToWait	This required element specifies the number of seconds the delay is expected to last.

A.23 DiagramLineT

This type specifies an abstract source

The XML schema for a DiagramLineT type is:

```
<xs:complexType name="DiagramLineT" abstract="true">
  <xs:complexContent>
    <xs:extension base="clnt:UiElementT">
      <xs:sequence>
        <xs:element name="Emphasis" type="xs:boolean" default="false"
          minOccurs="0"/>
        <xs:element name="LineColor" type="clnt:ColorT" minOccurs="0"/>
        <xs:element name="LineType" type="clnt:LineTypeT" minOccurs="0"/>
        <xs:element name="VerticalAxis" minOccurs="0">
          <xs:complexType>
            <xs:attribute name="AxisRef" type="xs:string"/>
          </xs:complexType>
        </xs:element>
        <xs:element name="InitActionList" type="clnt:ActionListT" minOccurs="0"/>
        <xs:element name="RefreshActionList" type="clnt:ActionListT"
          minOccurs="0"/>
        <xs:element name="ExitActionList" type="clnt:ActionListT" minOccurs="0"/>
      </xs:sequence>
      <xs:attribute name="Name" type="xs:string" use="required"/>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

The attributes of a DiagramLineT type are described in Table A.19.

Table A.19 – Attributes of DiagramLineT

Attribute	Description
Name	Unique name of DiagramlineT in the context of Chart/Graph that this DiagramlineT is present.

The elements of a DiagramLineT type are described in Table A.20.

Table A.20 – Elements of DiagramLineT

Element	Description
Emphasis	This optional element specifies whether or not this data should be graphically emphasized. The default is false.
LineColor	This optional element specifies the color in which to display the data. The default is determined by the FDI Client.
LineType	This optional element specifies the type of line to be displayed. Every data with the same LineType should be displayed in the same fashion (line pattern, line thickness, etc).
VerticalAxis	This optional element specifies appearance of the vertical axis. Sn1p1t
InitActionList	This optional element specifies the Actions to be executed before the data is displayed.
RefreshActionList	This optional element specifies the Actions to be executed after the data is read from the device but before it is displayed.
ExitActionList	This optional element specifies the Actions to be executed when the graph or chart containing the data is closed.

A.24 EnumerationItemListT

This type specifies the content of an enumeration.

The XML schema for an EnumerationItemListT type is:

```
<xs:complexType name="EnumerationItemListT">
  <xs:sequence maxOccurs="unbounded">
    <xs:element name="EnumerationItem" type="clnt:EnumerationItemT"/>
  </xs:sequence>
</xs:complexType>
```

The elements of an EnumerationItemListT type are described in Table A.21.

Table A.21 – Elements of EnumerationItemListT

Element	Description
EnumerationItem	An element of the enumeration.

A.25 EnumerationItemT

This type specifies one of the possible values of an enumerated value.

The XML schema for an EnumerationItemT type is:

```
<xs:complexType name="EnumerationItemT">
  <xs:complexContent>
    <xs:extension base="clnt:LabelHelpT">
      <xs:sequence>
        <xs:element name="Value" type="xs:unsignedLong"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

```

    </xs:complexContent>
</xs:complexType>

```

The elements of an EnumerationItemT type are described in Table A.22.

Table A.22 – Elements of EnumerationItemT

Element	Description
Value	This required element specifies the unique identifier of the list entry in the context of the parent list element.

A.26 FormatSpecifierT

This type specifies the allowed format specifier of parameters (ANSI C compatible).

The XML schema for a FormatSpecifierT type is:

```

<xs:simpleType name="FormatSpecifierT">
  <xs:restriction base="xs:string">
    <xs:pattern value="%?[#-0+ ]*\d*(\.\d*)?[hlL]?[dioxXucsfeEgGpn%]" />
  </xs:restriction>
</xs:simpleType>

```

A.27 GraphT

This type specifies a graph that graphically displays a finite data set from a device.

The XML schema for a GraphT type is:

```

<xs:complexType name="GraphT">
  <xs:complexContent>
    <xs:extension base="clnt:UiElementSizeableT">
      <xs:sequence>
        <xs:element name="CycleTime" type="xs:nonNegativeInteger" default="0"
          minOccurs="0"/>
        <xs:element name="AxisList" type="clnt:AxisListT">
          <xs:key name="GraphAxisKey">
            <xs:selector xpath="clnt:Axis"/>
            <xs:field xpath="@Name"/>
          </xs:key>
        </xs:element>
        <xs:element name="HorizontalAxis" minOccurs="0">
          <xs:complexType>
            <xs:attribute name="AxisRef" type="xs:string"/>
          </xs:complexType>
        </xs:element>
        <xs:element name="WaveformList" type="clnt:WaveformListT"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>

```

The elements of a GraphT type are described in Table A.23.

Table A.23 – Elements of GraphT

Element	Description
CycleTime	This optional element specifies the rate, in milliseconds, at which data set is re-read from the device and re-displayed. If the CycleTime is 0, the graph is never refreshed. The default is 0.
AxisList	This required element contains all the axes referred from graph or waveform.
HorizontalAxis	Sn1p1t
WaveformList	This required element specifies the waveforms displayed on the graph.

A.28 GridT

This type specifies a grid that displays data in a table-like grid.

The XML schema for a GridT type is:

```
<xs:complexType name="GridT">
  <xs:complexContent>
    <xs:extension base="clnt:UiElementSizeableT">
      <xs:sequence>
        <xs:element name="Handling" type="clnt:HandlingT" default="rw"
          minOccurs="0"/>
        <xs:element name="Orientation" type="clnt:OrientationT"
          default="Vertical" minOccurs="0"/>
        <xs:element name="VectorList" type="clnt:VectorListT"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

The elements of a GridT type are described in Table A.24.

Table A.24 – Elements of GridT

Element	Description
Handling	This optional element specifies whether or not the data within the grid may be modified. The default is ReadWrite.
Orientation	This optional element specifies whether the vectors are displayed horizontally or vertically. The default is Vertical.
VectorList	This required element specifies the data displayed within the grid.

A.29 HandlingT

This type specifies how an item may be accessed (ro = read only, wo = write only, rw = read and write).

The XML schema for a HandlingT enumeration type is:

```
<xs:simpleType name="HandlingT">
  <xs:restriction base="xs:string">
    <xs:enumeration value="rw"/>
  </xs:restriction>
</xs:simpleType>
```



```

    <xs:enumeration value="wo"/>
    <xs:enumeration value="ro"/>
  </xs:restriction>
</xs:simpleType>

```

The enumeration values of a HandlingT enumeration type are described in Table A.25.

Table A.25 – Enumerations of HandlingT

Enumeration	Description
rw	The item may only be read.
wo	The item may only be written.
ro	The item may be read or written.

A.30 ImageT

This type specifies a visual entity such as a picture or illustration.

The XML schema for an ImageT type is:

```

<xs:complexType name="ImageT">
  <xs:complexContent>
    <xs:extension base="clnt:UiElementT">
      <xs:sequence>
        <xs:element name="Inline" type="xs:boolean" default="false"
          minOccurs="0"/>
        <xs:element name="AlignLeft" type="xs:boolean" default="false"
          minOccurs="0"/>
        <xs:element name="AlignRight" type="xs:boolean" default="false"
          minOccurs="0"/>
        <xs:element name="Link" minOccurs="0">
          <xs:complexType>
            <xs:choice>
              <xs:element name="Menu" type="clnt:MenuReferenceT"/>
              <xs:element name="Plugin" type="clnt:PluginT"/>
              <xs:element name="Action" type="clnt:ActionT"/>
            </xs:choice>
          </xs:complexType>
        </xs:element>
      </xs:sequence>
      <xs:attribute name="NodePath" type="xs:string" use="required"/>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>

```

The attributes of an ImageT type are described in Table A.26.

Table A.26 – Attributes of ImageT

Attribute	Description
NodePath	This is a mandatory attribute which gives the qualified path of the Node in the Device Model.

The elements of an ImageT type are described in Table A.27.

Table A.27 – Elements of ImageT

Element	Description
Inline	If the image is in a window, dialog, page or group and the inline qualifier is not specified, the image should span with the width of the MENU; otherwise, the image should be displayed inline with other layoutitems of the itemlist.
AlignLeft	If the image is in a window, dialog, page or group and the inline qualifier is not specified, the image should span with the width of the MENU; otherwise, the image should be displayed inline with other layoutitems of the itemlist.
AlignRight	If the image is in a window, dialog, page or group and the inline qualifier is not specified, the image should span with the width of the MENU; otherwise, the image should be displayed inline with other layoutitems of the itemlist.
Link	This optional element specifies a link to Window, Dialog, Menu, Table, or Action related to the image.

A.31 InfoRequestT

This type specifies a request sent from an FDI Server to an FDI Client when a message needs to be displayed to the user needs during an Action. Unlike an AcknowledgementRequest, when receiving an InfoRequest the FDI Client does not wait for the user to acknowledge the message. Once the message has been displayed, the FDI Client immediately sends an InfoResponse back to the FDI Server.

The XML schema for an InfoRequestT type is:

```
<xs:complexType name="InfoRequestT">
  <xs:sequence>
    <xs:element name="Message" type="xs:string"/>
  </xs:sequence>
</xs:complexType>
```

The elements of an InfoRequestT type are described in Table A.28.

Table A.28 – Elements of InfoRequestT

Element	Description
Message	This required element specifies the message to be displayed to the user.

A.32 InputRequestT

This type specifies a request sent from an FDI Server to an FDI Client when the user needs to modify a value during an Action. Once the user has finished modifying the value, the FDI Client sends an InputResponse back to the FDI Server.

The XML schema for an InputRequestT type is:

```
<xs:complexType name="InputRequestT">
  <xs:sequence>
    <xs:element name="Prompt" type="xs:string"/>
    <xs:element name="InputValue" type="clnt:InputValueT"/>
  </xs:sequence>
</xs:complexType>
```

```

    </xs:sequence>
</xs:complexType>

```

The elements of an InputRequestT type are described in Table A.29.

Table A.29 – Elements of InputRequestT

Element	Description
Prompt	This required element specifies the prompt to be displayed to the user.
InputValue	This required element specifies the value to be edited by the user.

A.33 InputResponseT

The response sent by an FDI Client to an FDI Server as a result of processing an InputRequest.

The XML schema for an InputResponseT type is:

```

<xs:complexType name="InputResponseT">
  <xs:sequence>
    <xs:element name="InputValue" type="clnt:VariantT"/>
  </xs:sequence>
</xs:complexType>

```

The elements of an InputResponseT type are described in Table A.30.

Table A.30 – Elements of InputResponseT

Element	Description
InputValue	This required element specifies the value the user specified.

A.34 InputValueT

This type specifies the value to be modified by the user during an InputRequest.

The XML schema for an InputValueT type is:

```

<xs:complexType name="InputValueT">
  <xs:complexContent>
    <xs:extension base="clnt:LabelHelpT">
      <xs:sequence>
        <xs:element name="InitialValue" type="clnt:VariantT"/>
        <xs:element name="Type" type="clnt:InputValueType" minOccurs="0"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>

```

The elements of an InputValueType type are described in Table A.31.

Table A.31 – Elements of InputValueType

Element	Description
InitialValue	This required element specifies the initial value shown to the user.
Type	This optional element specifies the type of the value.

A.35 InputValueType

This type specifies the type of the value modified by the user during an InputRequest.

The XML schema for an InputValueType type is:

```
<xs:complexType name="InputValueType">
  <xs:choice>
    <xs:element name="NumericType" type="clnt:NumericDataT"/>
    <xs:element name="StringType" type="clnt:StringT"/>
    <xs:element name="Enumeration" type="clnt:EnumerationItemListT">
      <xs:unique name="InputValueUniqueEnumValue">
        <xs:selector xpath="clnt:EnumerationItem/clnt:Value"/>
        <xs:field xpath="."/>
      </xs:unique>
    </xs:element>
    <xs:element name="BitEnumeration" type="clnt:BitEnumerationItemListT">
      <xs:unique name="InputValueUniqueBitMask">
        <xs:selector xpath="clnt:BitEnumerationItem/clnt:Value"/>
        <xs:field xpath="."/>
      </xs:unique>
    </xs:element>
    <xs:element name="DateTimeType" type="clnt:DateTimeDataT"/>
  </xs:choice>
</xs:complexType>
```

The elements of an InputValueType type are described in Table A.32.

Table A.32 – Elements of InputValueType

Element	Description
NumericType	This optional element specifies an integer or floating point value.
StringType	This optional element specifies a string value.
Enumeration	This element specifies an enumeration. Each enumeration item defines a description and help text for a parameter value.
BitEnumeration	This element specifies an enumeration. Each enumeration item defines a description and help text for a bitmask of the parameter value.
DateTimeType	This element specifies a date or time value.

A.36 LabelHelpT

This type specifies an extended layout object with an additional localized help text.

The XML schema for a LabelHelpT type is:

```

<xs:complexType name="LabelHelpT" abstract="true">
  <xs:complexContent>
    <xs:extension base="clnt:LabelT">
      <xs:sequence>
        <xs:element name="Help" type="xs:string" minOccurs="0"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>

```

The elements of a LabelHelpT type are described in Table A.33.

Table A.33 – Elements of LabelHelpT

Element	Description
Help	This optional element specifies a localized help text of the element.

A.37 LabelT

This type specifies the layout object with a localized description.

The XML schema for a LabelT type is:

```

<xs:complexType name="LabelT">
  <xs:sequence>
    <xs:element name="Label" type="xs:string"/>
  </xs:sequence>
</xs:complexType>

```

The elements of a LabelT type are described in Table A.34.

Table A.34 – Elements of LabelT

Element	Description
Label	This required element specifies the localized name of the element.

A.38 LineTypeT

This type specifies the type of line on a chart or a graph. All lines of the same type shall have the same visual appearance, for example all Data0 lines will appear the same and all Data1 lines will appear the same, but the two sets of lines shall be visually distinct.

The XML schema for a LineTypeT enumeration type is:

```

<xs:simpleType name="LineTypeT">
  <xs:restriction base="xs:string">
    <xs:enumeration value="Data"/>
    <xs:enumeration value="Data0"/>
    <xs:enumeration value="Data1"/>
    <xs:enumeration value="Data2"/>
    <xs:enumeration value="Data3"/>
    <xs:enumeration value="Data4"/>
    <xs:enumeration value="Data5"/>
    <xs:enumeration value="Data6"/>
  </xs:restriction>
</xs:simpleType>

```

```

<xs:enumeration value="Data7"/>
<xs:enumeration value="Data8"/>
<xs:enumeration value="Data9"/>
<xs:enumeration value="LowLimit"/>
<xs:enumeration value="LowLowLimit"/>
<xs:enumeration value="HighLimit"/>
<xs:enumeration value="HighHighLimit"/>
<xs:enumeration value="Transparent"/>
</xs:restriction>
</xs:simpleType>

```

The enumeration values of a LineTypeT enumeration type are described in Table A.35.

Table A.35 – Enumerations of LineTypeT

Enumeration	Description
Data	The line represents data.
Data0	The line represents data.
Data1	The line represents data.
Data2	The line represents data.
Data3	The line represents data.
Data4	The line represents data.
Data5	The line represents data.
Data6	The line represents data.
Data7	The line represents data.
Data8	The line represents data.
Data9	The line represents data.
LowLimit	The line represents a low limit.
LowLowLimit	The line represents a low low limit.
HighLimit	The line represent a high limit.
HighHighLimit	The line represents a high high limit.
Transparent	Represents a transparent line type.

A.39 MenuT

This type specifies the layout of a Window, Dialog, Page, Group, Menu, or Table.

The XML schema for a MenuT type is:

```

<xs:complexType name="MenuT">
  <xs:complexContent>
    <xs:extension base="clnt:UiElementT">
      <xs:sequence>
        <xs:element name="InitActionList" type="clnt:ActionListT" minOccurs="0"/>
        <xs:element name="PreEditActionList" type="clnt:ActionListT"
          minOccurs="0"/>
        <xs:element name="PostEditActionList" type="clnt:ActionListT"
          minOccurs="0"/>
        <xs:element name="ExitActionList" type="clnt:ActionListT" minOccurs="0"/>
        <xs:element name="Access" type="clnt:AccessT" minOccurs="0"/>
        <xs:element name="ItemList">
          <xs:complexType>

```

```

<xs:choice minOccurs="0" maxOccurs="unbounded">
  <xs:element name="Menu" type="clnt:MenuReferenceT"/>
  <xs:element name="Chart" type="clnt:ChartT"/>
  <xs:element name="Graph" type="clnt:GraphT"/>
  <xs:element name="Grid" type="clnt:GridT"/>
  <xs:element name="Image" type="clnt:ImageT"/>
  <xs:element name="Text" type="xs:string"/>
  <xs:element name="Parameter" type="clnt:ParameterT"/>
  <xs:element name="Plugin" type="clnt:PluginT"/>
  <xs:element name="Action" type="clnt:ActionT"/>
  <xs:element name="RowBreak" type="clnt:RowBreakT"/>
  <xs:element name="ColumnBreak" type="clnt:ColumnBreakT"/>
  <xs:element name="Separator" type="clnt:SeparatorT"/>
</xs:choice>
</xs:complexType>
</xs:element>
</xs:sequence>
<xs:attribute name="Style" type="clnt:MenuStyleT" use="optional"/>
<xs:attribute name="NodePath" type="xs:string" use="required"/>
</xs:extension>
</xs:complexContent>
</xs:complexType>

```

The attributes of a MenuT type are described in Table A.36.

Table A.36 – Attributes of MenuT

Attribute	Description
Style	This optional attribute determines the menu style.
NodePath	This is a mandatory attribute which gives the qualified path of the Node in the Device Model.

The elements of a MenuT type are described in Table A.37.

Table A.37 – Elements of MenuT

Element	Description
InitActionList	This optional element specifies the Actions to be executed before the menu is activated.
PreEditActionList	This optional element specifies the Actions to be executed immediately after the element is activated.
PostEditActionList	This optional element specifies the Actions to be executed that after the user has finished processing the element.
ExitActionList	This optional element specifies the Actions to be executed when the menu is deactivated.
Access	This optional element specifies whether the menu shall be used ONLINE or OFFLINE
ItemList	This optional element specifies the base content of the menu.

A.40 MenuReferenceT

This type specifies the layout of a Window, Dialog, Page, Group, Menu, or Table.

The XML schema for a MenuReferenceT type is:

```
<xs:complexType name="MenuReferenceT">
  <xs:complexContent>
    <xs:extension base="clnt:UiElementT">
      <xs:sequence>
        <xs:element name="Review" type="xs:boolean" minOccurs="0"/>
      </xs:sequence>
      <xs:attribute name="Style" type="clnt:MenuStyleT" use="optional"/>
      <xs:attribute name="NodePath" type="xs:string" use="required"/>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

The attributes of a MenuReferenceT type are described in Table A.38.

Table A.38 – Attributes of MenuReferenceT

Attribute	Description
Style	This optional attribute determines the menu style.
NodePath	This is mandatory attribute which gives the qualified path of the Node in the Device Model.

The elements of a MenuReferenceT type are described in Table A.39.

Table A.39 – Elements of MenuReferenceT

Element	Description
Review	This optional element specifies all referenced items of type VARIABLE appearing in the GUI or its subcomponents shall assume the qualifiers as READ ONLY.

A.41 MenuStyleT

This type specifies the style of a menu.

The XML schema for a MenuStyleT enumeration type is:

```
<xs:simpleType name="MenuStyleT">
  <xs:restriction base="xs:string">
    <xs:enumeration value="Menu"/>
    <xs:enumeration value="Group"/>
    <xs:enumeration value="Page"/>
    <xs:enumeration value="Table"/>
    <xs:enumeration value="Window"/>
    <xs:enumeration value="Dialog"/>
    <xs:enumeration value="Record"/>
    <xs:enumeration value="Collection"/>
    <xs:enumeration value="ItemArray"/>
    <xs:enumeration value="EditDisplay"/>
  </xs:restriction>
</xs:simpleType>
```


The enumeration values of a MenuStyleT enumeration type are described in Table A.40.

Table A.40 – Enumerations of MenuStyleT

Enumeration	Description
Menu	EDD Menu Style 'MENU'
Group	EDD Menu Style 'GROUP'
Page	EDD Menu Style 'PAGE'
Table	EDD Menu Style 'TABLE'
Window	EDD Menu Style 'WINDOW'
Dialog	EDD Menu Style 'DIALOG'
Record	Menu represents EDD 'RECORD'
Collection	Menu represents EDD 'COLLECTION'
ItemArray	Menu represents EDD 'ITEMARRAY'
EditDisplay	Menu represents EDD 'EDIT_DISPLAY'

A.42 NumericDataT

This type specifies the data type of an integer, boolean or floating point value.

The XML schema for a NumericDataT enumeration type is:

```
<xs:simpleType name="NumericDataT">
  <xs:restriction base="xs:string">
    <xs:enumeration value="SignedInteger"/>
    <xs:enumeration value="UnsignedInteger"/>
    <xs:enumeration value="FloatingPoint"/>
    <xs:enumeration value="Double"/>
    <xs:enumeration value="Boolean"/>
  </xs:restriction>
</xs:simpleType>
```

The enumeration values of a NumericDataT enumeration type are described in Table A.41.

Table A.41 – Enumerations of NumericDataT

Enumeration	Description
SignedInteger	A signed integer.
UnsignedInteger	An unsigned integer.
FloatingPoint	A floating point number.
Double	A Double point number.
Boolean	A boolean value.

A.43 NumericTemplateT

This type specifies template used to display a numeric parameter value.

The XML schema for a NumericTemplateT type is:

```
<xs:complexType name="NumericTemplateT">
```

```

<xs:sequence>
  <xs:element name="DataType" type="clnt:NumericDataT"/>
  <xs:element name="Options" type="clnt:VariantOptionListT" minOccurs="0"
    maxOccurs="1">
    <xs:unique name="UiTemplateUniqueVariantValue">
      <xs:selector xpath="clnt:Option/clnt:Value/*"/>
      <xs:field xpath="."/>
    </xs:unique>
  </xs:element>
</xs:sequence>
</xs:complexType>

```

The elements of a NumericTemplateT type are described in Table A.42.

Table A.42 – Elements of NumericTemplateT

Element	Description
DataType	This required element specifies the type of the numeric parameter.
Options	This optional element specifies the list of options the user may select for the string.

A.44 OptionListT

This type specifies a list of Option elements. The selection response contains the 0-based index of the selected option.

The XML schema for an OptionListT type is:

```

<xs:complexType name="OptionListT">
  <xs:sequence maxOccurs="unbounded">
    <xs:element name="Option" type="clnt:LabelT"/>
  </xs:sequence>
</xs:complexType>

```

The elements of an OptionListT type are described in Table A.43.

Table A.43 – Elements of OptionListT

Element	Description
Option	An element of the list.

A.45 OrientationT

This type specifies a direction.

The XML schema for an OrientationT enumeration type is:

```

<xs:simpleType name="OrientationT">
  <xs:restriction base="xs:string">
    <xs:enumeration value="Horizontal"/>
    <xs:enumeration value="Vertical"/>
  </xs:restriction>
</xs:simpleType>

```

The enumeration values of an OrientationT enumeration type are described in Table A.44.

Table A.44 – Enumerations of OrientationT

Enumeration	Description
Horizontal	Left to right and right to left.
Vertical	Top to bottom and bottom to top.

A.46 ParameterInputRequestT

This type specifies a request sent from an FDI Server to an FDI Client when the user needs to modify a value during an Action. Once the user has finished modifying the value, the FDI Client sends an InputResponse back to the FDI Server.

The XML schema for a ParameterInputRequestT type is:

```
<xs:complexType name="ParameterInputRequestT">
  <xs:sequence>
    <xs:element name="Prompt" type="xs:string"/>
    <xs:element name="Parameter" type="clnt:ParameterT"/>
  </xs:sequence>
</xs:complexType>
```

The elements of a ParameterInputRequestT type are described in Table A.45.

Table A.45 – Elements of ParameterInputRequestT

Element	Description
Prompt	This required element specifies the prompt to be displayed to the user.
Parameter	This required element specifies the value to be edited by the user.

A.47 ParameterListT

This type specifies a list of Parameter elements.

The XML schema for a ParameterListT type is:

```
<xs:complexType name="ParameterListT">
  <xs:sequence maxOccurs="unbounded">
    <xs:element name="Parameter" type="clnt:ParameterT"/>
  </xs:sequence>
</xs:complexType>
```

The elements of a ParameterListT type are described in Table A.46.

Table A.46 – Elements of ParameterListT

Element	Description
Parameter	An element of the list.

A.48 ParameterT

This type specifies a device or block parameter that is displayed in a user interface element such as a Window or Dialog.

The XML schema for a ParameterT type is:

```
<xs:complexType name="ParameterT">
  <xs:complexContent>
    <xs:extension base="clnt:UiElementSizeableT">
      <xs:sequence>
        <xs:element name="UITemplate" type="clnt:UiTemplateT"/>
        <xs:element name="Unit" type="xs:string" minOccurs="0"/>
        <xs:element name="Handling" type="clnt:HandlingT" default="rw"
          minOccurs="0"/>
        <xs:element name="RangeList" type="clnt:RangeListT" minOccurs="0"/>
        <xs:element name="DisplayFormat" type="clnt:FormatSpecifierT"
          minOccurs="0"/>
        <xs:element name="EditFormat" type="clnt:FormatSpecifierT"
          minOccurs="0"/>
        <xs:element name="TimeFormat" type="xs:string" minOccurs="0"/>
        <xs:element name="TimeScale" type="clnt:TimeScaleT" minOccurs="0"/>
        <xs:element name="DisplayLabel" type="xs:boolean" default="true"
          minOccurs="0"/>
        <xs:element name="DisplayValue" type="xs:boolean" default="true"
          minOccurs="0"/>
        <xs:element name="DisplayUnit" type="xs:boolean" default="true"
          minOccurs="0"/>
        <xs:element name="PreEditActionList" type="clnt:ActionListT"
          minOccurs="0"/>
        <xs:element name="PostEditActionList" type="clnt:ActionListT"
          minOccurs="0"/>
        <xs:element name="ScalingFactor" type="xs:double" minOccurs="0"/>
        <xs:element name="Class" type="clnt:ParameterClassT" minOccurs="0"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

The elements of a ParameterT type are described in Table A.47.

Table A.47 – Elements of ParameterT

Element	Description
UITemplate	This element specifies the complex UI template used to display a parameter. UI templates modify how parameter values are displayed. The default UI template displays a parameter value as plain text.
Unit	This optional element specifies the engineering unit of the parameter. The default is the parameter has no engineering unit.
Handling	This optional element specifies whether or not the parameter may be modified. The default is ReadWrite.
RangeList	This optional element specifies the minimum and maximum values to which the user may set the parameter.
DisplayFormat	This optional element specifies the format to be used when displaying the value of the parameter as specified in IEC 61804-3. The default is dependent upon the data type of the parameter and is determined by the FDI Client.
EditFormat	This optional element specifies the format to be used when modifying the value of the parameter as specified in IEC 61804-3. The default is dependent upon the data type of the parameter and is determined by the FDI Client.
TimeFormat	This optional element specifies the format to be used when displaying and modifying the value of parameters of date or time types.
TimeScale	This optional element specifies the TIME_SCALE of parameters of date or time types.
DisplayLabel	This optional element specifies whether or not the parameter's label should be displayed. The default is true.
DisplayValue	This optional element specifies whether or not the parameter's value should be displayed. The default is true.
DisplayUnit	This optional element specifies whether or not the parameter's unit should be displayed. The default is true.
PreEditActionList	This optional element specifies the Actions to be executed before the parameter is modified by the user.
PostEditActionList	This optional element specifies the Actions to be executed after the parameter is modified by the user.
ScalingFactor	This element specifies the scaling factor to be used when the value of the parameter will be shown to the user. This visible value is the result of the multiplication of the factor and the parameter value, returned by the device.
Class	This optional element specifies the EDD CLASS attribute of VARIABLE.

A.49 PluginT

This type specifies a User Interface Plug-in that is referenced from a user interface element such as a Window or Dialog.

The XML schema for a PluginT type is:

```
<xs:complexType name="PluginT">
  <xs:complexContent>
    <xs:extension base="clnt:UiElementT">
      <xs:sequence>
        <xs:element name="Identifier">
          <xs:simpleType>
            <xs:restriction base="xs:string">
```

```

        <xs:pattern value="[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[0-9a-fA-F]{4}-
        [0-9a-fA-F]{4}-[0-9a-fA-F]{12}"/>
    </xs:restriction>
</xs:simpleType>
</xs:element>
</xs:sequence>
</xs:extension>
</xs:complexContent>
</xs:complexType>

```

The elements of a PluginT type are described in Table A.48.

Table A.48 – Elements of PluginT

Element	Description
Identifier	This required element specifies the Universally Unique Identifier (UUID) that identifies the plug-in.

A.50 RangeListT

This type specifies a list of Range elements.

The XML schema for a RangeListT type is:

```

<xs:complexType name="RangeListT">
  <xs:sequence maxOccurs="unbounded">
    <xs:element name="Range" type="clnt:RangeT"/>
  </xs:sequence>
</xs:complexType>

```

The elements of a RangeListT type are described in Table A.49.

Table A.49 – Elements of RangeListT

Element	Description
Range	An element of the list.

A.51 RangeT

This type specifies a range defined by a minimum and a maximum value.

The XML schema for a RangeT type is:

```

<xs:complexType name="RangeT">
  <xs:sequence>
    <xs:element name="MinimumValue" type="clnt:VariantT"/>
    <xs:element name="MaximumValue" type="clnt:VariantT" minOccurs="0"/>
  </xs:sequence>
</xs:complexType>

```

The elements of a RangeT type are described in Table A.50.

Table A.50 – Elements of RangeT

Element	Description
MinimumValue	This required element specifies the minimum value of the range.
MaximumValue	This required element specifies the maximum value of the range.

A.52 ResponseT

This type specifies the response sent by an FDI Client to an FDI Server as a result of processing an Action request.

The XML schema for a ResponseT type is:

```
<xs:complexType name="ResponseT"/>
```

A.53 RowBreakT

This type specifies a row break.

The XML schema for a RowBreakT type is:

```
<xs:complexType name="RowBreakT"/>
```

A.54 ScalingT

This type specifies a method of scaling values from a lower bound to an upper bound.

The XML schema for a ScalingT enumeration type is:

```
<xs:simpleType name="ScalingT">
  <xs:restriction base="xs:string">
    <xs:enumeration value="Linear"/>
    <xs:enumeration value="Logarithmic"/>
  </xs:restriction>
</xs:simpleType>
```

The enumeration values of a ScalingT enumeration type are described in Table A.51.

Table A.51 – Enumerations of ScalingT

Enumeration	Description
Linear	The values are scaled linearly.
Logarithmic	The values are scaled logarithmically via the natural logarithm.

A.55 SelectionRequestT

This type specifies a request sent from an FDI Server to an FDI Client when the user needs to choose from a list of options during an Action. Once the user has selected one of the options, the FDI Client sends an SelectionResponse back to the FDI Server.

The XML schema for a SelectionRequestT type is:

```
<xs:complexType name="SelectionRequestT">
  <xs:sequence>
    <xs:element name="Prompt" type="xs:string"/>
    <xs:element name="OptionList" type="clnt:OptionListT">
      <xs:unique name="UniqueOptionIdentifier">
        <xs:selector xpath="clnt:Option/clnt:Label"/>
        <xs:field xpath="."/>
      </xs:unique>
    </xs:element>
  </xs:sequence>
</xs:complexType>
```

The elements of a SelectionRequestT type are described in Table A.52.

Table A.52 – Elements of SelectionRequestT

Element	Description
Prompt	This required element specifies the prompt to be displayed to the user.
OptionList	This required element specifies the list of options the user may select from.

A.56 SelectionResponseT

This type specifies the response sent by an FDI Client to an FDI Server as a result of processing an SelectionRequest.

The XML schema for a SelectionResponseT type is:

```
<xs:complexType name="SelectionResponseT">
  <xs:sequence>
    <xs:element name="SelectedOption" type="xs:unsignedLong"/>
  </xs:sequence>
</xs:complexType>
```

The elements of a SelectionResponseT type are described in Table A.53.

Table A.53 – Elements of SelectionResponseT

Element	Description
SelectedOption	The 0-based index of the selected option.

A.57 SeparatorT

This type specifies a separator.

The XML schema for a SeparatorT type is:

```
<xs:complexType name="SeparatorT"/>
```


A.58 SizeT

This type specifies the relative size of an item.

The XML schema for a SizeT enumeration type is:

```
<xs:simpleType name="SizeT">
  <xs:restriction base="xs:string">
    <xs:enumeration value="XXX_SMALL"/>
    <xs:enumeration value="XX_SMALL"/>
    <xs:enumeration value="X_SMALL"/>
    <xs:enumeration value="SMALL"/>
    <xs:enumeration value="MEDIUM"/>
    <xs:enumeration value="LARGE"/>
    <xs:enumeration value="X_LARGE"/>
    <xs:enumeration value="XX_LARGE"/>
  </xs:restriction>
</xs:simpleType>
```

The enumeration values of a SizeT enumeration type are described in Table A.54.

Table A.54 – Enumerations of SizeT

Enumeration	Description
XXX_SMALL	Extra extra extra small.
XX_SMALL	Extra extra small.
X_SMALL	Extra small.
SMALL	Small.
MEDIUM	Medium.
LARGE	Large.
X_LARGE	Extra large.
XX_LARGE	Extra extra large.

A.59 ParameterClassT

This type specifies the EDD CLASS type definition for VARIABLE.

The XML schema for a ParameterClassT enumeration type is:

```
<xs:simpleType name="ParameterClassT">
  <xs:list>
    <xs:simpleType>
      <xs:restriction base="xs:string">
        <xs:enumeration value="ALARM"/>
        <xs:enumeration value="ANALOG_INPUT"/>
        <xs:enumeration value="ANALOG_OUTPUT"/>
        <xs:enumeration value="COMPUTATION"/>
        <xs:enumeration value="CONTAINED"/>
        <xs:enumeration value="CORRECTION"/>
        <xs:enumeration value="DEVICE"/>
        <xs:enumeration value="SPECIALIST"/>
        <xs:enumeration value="DIAGNOSTIC"/>
        <xs:enumeration value="DIGITAL_INPUT"/>
        <xs:enumeration value="DIGITAL_OUTPUT"/>
      </xs:restriction>
    </xs:simpleType>
  </xs:list>
</xs:simpleType>
```

```

<xs:enumeration value="DISCRETE_INPUT"/>
<xs:enumeration value="DISCRETE_OUTPUT"/>
<xs:enumeration value="DYNAMIC"/>
<xs:enumeration value="FREQUENCY_INPUT"/>
<xs:enumeration value="FREQUENCY_OUTPUT"/>
<xs:enumeration value="HART"/>
<xs:enumeration value="INPUT"/>
<xs:enumeration value="IS_CONFIG"/>
<xs:enumeration value="LOCAL"/>
<xs:enumeration value="LOCAL_DISPLAY"/>
<xs:enumeration value="OPERATE"/>
<xs:enumeration value="OPTIONAL"/>
<xs:enumeration value="OUTPUT"/>
<xs:enumeration value="SERVICE"/>
<xs:enumeration value="TEMPORARY"/>
<xs:enumeration value="TUNE"/>
</xs:restriction>
</xs:simpleType>
</xs:list>
</xs:simpleType>

```

The enumeration values of a ParameterClassT enumeration type are described in Table A.55.

Table A.55 – Enumerations of ParameterClassT

Enumeration	Description
ALARM	The VARIABLE contains alarm limits.
ANALOG_INPUT	The VARIABLE is part of an analog input block.
ANALOG_OUTPUT	The VARIABLE is part of an analog output block.
COMPUTATION	The VARIABLE is part of a computation block.
CONTAINED	The VARIABLE represents the physical characteristics of the device.
CORRECTION	The VARIABLE is part of the correction block.
DEVICE	The VARIABLE represents the physical characteristics of the device.
SPECIALIST	The VARIABLE is a configuration parameter. Permission to modify this VARIABLE may only be granted to users that are specialists for this device.
DIAGNOSTIC	The VARIABLE indicates the device status.
DIGITAL_INPUT	The VARIABLE is part of a digital input block.
DIGITAL_OUTPUT	The VARIABLE is part of a digital output block.
DISCRETE_INPUT	The VARIABLE is part of a discrete input block.
DISCRETE_OUTPUT	The VARIABLE is part of a discrete output block.
DYNAMIC	The VARIABLE is modified by the device without stimulus from the network.
FREQUENCY_INPUT	The VARIABLE is part of a frequency input block.
FREQUENCY_OUTPUT	The VARIABLE is part of a frequency output block.
HART	The VARIABLE is part of the HART block which characterizes the HART interface.
INPUT	The VARIABLE is part of an input block.
IS_CONFIG	A modification of the VARIABLE results in setting the revision counter or configuration-changed bit, as applicable.
LOCAL	The VARIABLE is locally used by the EDD application.
LOCAL_DISPLAY	The VARIABLE is part of the local display block.
OPERATE	The VARIABLE is used to control a block's operation.

Enumeration	Description
OPTIONAL	The VARIABLE may not be present in the device.
OUTPUT	The VARIABLE is part of the output block.
SERVICE	The VARIABLE is used when performing routine maintenance.
TEMPORARY	The VARIABLE is initialized to its DEFAULT_VALUE in each EDD application session. LOCAL variables may be persisted by the EDD application; TEMPORARY variables are never persisted.
TUNE	The VARIABLE is used to tune the algorithm of a block.

A.60 ActionClassT

This type specifies the EDD CLASS definition of an Action

The XML schema for a ActionClassT enumeration type is:

```
<xs:simpleType name="ActionClassT">
  <xs:list>
    <xs:simpleType>
      <xs:restriction base="xs:string">
        <xs:enumeration value="ALARM"/>
        <xs:enumeration value="ANALOG_OUTPUT"/>
        <xs:enumeration value="COMPUTATION"/>
        <xs:enumeration value="CONTAINED"/>
        <xs:enumeration value="CORRECTION"/>
        <xs:enumeration value="DEVICE"/>
        <xs:enumeration value="SPECIALIST"/>
        <xs:enumeration value="DIAGNOSTIC"/>
        <xs:enumeration value="DISCRETE"/>
        <xs:enumeration value="FREQUENCY"/>
        <xs:enumeration value="HART"/>
        <xs:enumeration value="INPUT"/>
        <xs:enumeration value="LOCAL_DISPLAY"/>
        <xs:enumeration value="OPERATE"/>
        <xs:enumeration value="OUTPUT"/>
        <xs:enumeration value="SERVICE"/>
        <xs:enumeration value="TUNE"/>
      </xs:restriction>
    </xs:simpleType>
  </xs:list>
</xs:simpleType>
```

The enumeration values of a ActionClassT enumeration type are described in Table A.56.

Table A.56 – Enumerations of ActionClassT

Enumeration	Description
ALARM	The METHOD is associated with an alarm (e.g., specifying alarm limits, indicating alarm status, etc.)
ANALOG_OUTPUT	The METHOD is part of an analog output block.
COMPUTATION	The METHOD is part of a computation block.
CONTAINED	The METHOD represents the physical characteristics of the device.
CORRECTION	The METHOD supports the transducers in the field device. The METHOD may establish transducer limits, provide signal damping, indicate the transducer's value, linearize or calibrate the transducer, etc.
DEVICE	The METHOD specifies the physical characteristics of the device.
SPECIALIST	Permission to execute this METHOD may only be granted to users that are specialists for this device.
DIAGNOSTIC	The METHOD evaluates the device status.
DISCRETE	The METHOD supports the discrete and or digital I/O of the device.
FREQUENCY	The METHOD supports the frequency I/O of the device.
HART	The METHOD is part of the HART block which characterizes the HART interface. There is only one HART block.
INPUT	The METHOD is part of an input block. An input block is a special kind of computation block which does unit conversions, scaling, and damping. The input block parameters can be determined by the output of another block.
LOCAL_DISPLAY	The METHOD is part of the local display block. A local display block contains the VARIABLES associated with the local interface (keyboard, display, etc.) of the device.
OPERATE	The METHOD is used to control a block's operation.
OUTPUT	The METHOD is part of the output block. The values of output parameters may be accessed by another block input.
SERVICE	The METHOD is used when performing routine maintenance.
TUNE	The METHOD is used to tune the algorithm of a block.

A.61 SourceListT

This type specifies a list of Source elements.

The XML schema for a SourceListT type is:

```
<xs:complexType name="SourceListT">
  <xs:sequence maxOccurs="unbounded">
    <xs:element name="Source" type="clnt:SourceT"/>
  </xs:sequence>
</xs:complexType>
```

The elements of a SourceListT type are described in Table A.57.

Table A.57 – Elements of SourceListT

Element	Description
Source	An element of the list.

A.62 SourceT

This type specifies a source of data values displayed by a chart.

The XML schema for a SourceT type is:

```
<xs:complexType name="SourceT">
  <xs:complexContent>
    <xs:extension base="clnt:DiagramLineT">
      <xs:sequence>
        <xs:element name="ParameterList" type="clnt:ParameterListT"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

The elements of a SourceT type are described in Table A.58.

Table A.58 – Elements of SourceT

Element	Description
ParameterList	This required element specifies the parameters to be sampled and displayed.

A.63 StringDataT

This type specifies that data type of a string value.

The XML schema for a StringDataT enumeration type is:

```
<xs:simpleType name="StringDataT">
  <xs:restriction base="xs:string">
    <xs:enumeration value="ASCII"/>
    <xs:enumeration value="BITSTRING"/>
    <xs:enumeration value="EUC"/>
    <xs:enumeration value="PACKED_ASCII"/>
    <xs:enumeration value="PASSWORD"/>
    <xs:enumeration value="VISIBLE"/>
    <xs:enumeration value="OCTET"/>
  </xs:restriction>
</xs:simpleType>
```

The enumeration values of a StringDataT enumeration type are described in Table A.59.

Table A.59 – Enumerations of StringDataT

Enumeration	Description
ASCII	String of ASCII characters.
BITSTRING	Variables of data type BIT_ENUMERATED are string constants which identify the position of a character of the VARIABLE value with its position.
EUC	(Extended Unit code) – is the internal code for specific characters (for example, China: EUC-CN, Taiwan EUC-TW). EUC is used to handle East Asian languages (ISO/IEC 10646-1).
PACKED_ASCII	Is a subset of ASCII produced by removing the two most significant bits of each ASCII character.
PASSWORD	Is a string type and is intended for specifying password strings. Except for how the variable is presented to the user, password and ASCII string types are identical.
VISIBLE	This string is an ordered sequence of characters from the ISO/IEC 10646-1 and ISO/IEC 2375 character set.
OCTET	Is for specifying a sequence of unformatted binary data whose definition is determined by the implementation of the device.

A.64 StringTemplateT

This type specifies the template used to display a string parameter value.

The XML schema for a StringTemplateT type is:

```
<xs:complexType name="StringTemplateT">
  <xs:sequence>
    <xs:element name="DataType" type="clnt:StringDataT"/>
    <xs:element name="Options" type="clnt:StringOptionListT" minOccurs="0"
      maxOccurs="1">
      <xs:unique name="UiTemplateUniqueStringValue">
        <xs:selector xpath="clnt:Option/clnt:Value"/>
        <xs:field xpath="."/>
      </xs:unique>
    </xs:element>
  </xs:sequence>
</xs:complexType>
```

The elements of a StringTemplateT type are described in Table A.60.

Table A.60 – Elements of StringTemplateT

Element	Description
DataType	This required element specifies the type of the string parameter.
Options	This optional element specifies the list of options the user may select for the string.

A.65 StringOptionListT

This type specifies the list of options to choose for the string parameter type.

The XML schema for a StringOptionListT type is:

```
<xs:complexType name="StringOptionListT">
  <xs:sequence maxOccurs="unbounded">
    <xs:element name="Option" type="clnt:StringOptionT"/>
  </xs:sequence>
</xs:complexType>
```

The elements of a StringOptionListT type are described in Table A.61.

Table A.61 – Elements of StringOptionListT

Element	Description
Option	An element of the string options.

A.66 StringOptionT

This type specifies one of the possible values of string options.

The XML schema for a StringOptionT type is:

```
<xs:complexType name="StringOptionT">
  <xs:complexContent>
    <xs:extension base="clnt:LabelHelpT">
      <xs:sequence>
        <xs:element name="Value" type="xs:string"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

The elements of a StringOptionT type are described in Table A.62.

Table A.62 – Elements of StringOptionT

Element	Description
Value	This required element specifies the unique identifier of the list entry in the context of the parent list element.

A.67 StringT

This type specifies the type of a string value.

The XML schema for a StringT type is:

```
<xs:complexType name="StringT">
  <xs:sequence>
    <xs:element name="DataType" type="clnt:StringDataT"/>
    <xs:element name="Length" type="xs:unsignedInt"/>
  </xs:sequence>
</xs:complexType>
```

```

</xs:sequence>
</xs:complexType>

```

The elements of a StringT type are described in Table A.63.

Table A.63 – Elements of StringT

Element	Description
Data Type	This required element specifies the data type of the string.
Length	This required element specifies the length of the string, in characters not in octets.

A.68 TimeScaleT

This enum type specifies the options for the EDD timescale modifier.

The XML schema for a TimeScaleT enumeration type is:

```

<xs:simpleType name="TimeScaleT">
  <xs:restriction base="xs:string">
    <xs:enumeration value="SECONDS"/>
    <xs:enumeration value="MINUTES"/>
    <xs:enumeration value="HOURS"/>
  </xs:restriction>
</xs:simpleType>

```

The enumeration values of a TimeScaleT enumeration type are described in Table A.64.

Table A.64 – Enumerations of TimeScaleT

Enumeration	Description
SECONDS	The access shall be done ONLINE.
MINUTES	The access shall be done OFFLINE.
HOURS	The access shall be done OFFLINE.

A.69 UidLayoutInformation

This element is the root entry element containing the UID.

The XML schema for a UidLayoutInformation element is:

```

<xs:element name="UidLayoutInformation">
  <xs:complexType>
    <xs:sequence>
      <xs:choice>
        <xs:element name="Menu" type="clnt:MenuT"/>
        <xs:element name="ActionRequest" type="clnt:ActionRequestT"/>
        <xs:element name="ActionResponse" type="clnt:ActionResponseT"/>
      </xs:choice>
    </xs:sequence>
  </xs:complexType>
</xs:element>

```


The elements of a UidLayoutInformation element are described in Table A.65.

Table A.65 – Elements of UidLayoutInformation

Element	Description
Menu	This optional element specifies a request from an FDI Server to an FDI Client to display a menu.
ActionRequest	This optional element specifies a request from an FDI Server to an FDI Client for the user to perform a specific action.
ActionResponse	This optional element specifies a request from an FDI Server to an FDI Client for a response to be displayed to the user.

A.70 UidRequestT

This type specifies a request sent from an FDI Server to an FDI Client when an advanced user interface needs to be shown to the user during an Action. The structure of the user interface is specified as a Dialog, Table, or Window. When the user interface is no longer needed, the FDI Client sends a UIDResponse back to the FDI Server.

The XML schema for a UidRequestT type is:

```
<xs:complexType name="UidRequestT">
  <xs:sequence>
    <xs:element name="UidRef" type="xs:string"/>
    <xs:element name="ButtonList" type="clnt:ButtonListT">
      <xs:unique name="UniqueButtonIdentifier">
        <xs:selector xpath="clnt:Button/clnt:Label"/>
        <xs:field xpath="."/>
      </xs:unique>
    </xs:element>
  </xs:sequence>
</xs:complexType>
```

The elements of a UidRequestT type are described in Table A.66.

Table A.66 – Elements of UidRequestT

Element	Description
UidRef	This required element specifies the Node path of the UID containing the definition of the Dialog, Table, or Window to be displayed.
ButtonList	This required element specifies the buttons displayed by the FDI Client that the user may press to indicate the user interface is no longer needed.

A.71 UidResponseT

This type specifies the response sent by an FDI Client to an FDI Server as a result of processing a SelectionRequest.

The XML schema for a UidResponseT type is:

```
<xs:complexType name="UidResponseT">
  <xs:sequence>
    <xs:element name="SelectedButton" type="xs:unsignedLong"/>
  </xs:sequence>
```

```
</xs:complexType>
```

The elements of a `UidResponseT` type are described in Table A.67.

Table A.67 – Elements of `UidResponseT`

Element	Description
SelectedButton	The 0-based index of the selected button. If the button list is not provided in the UID request, Next and Cancel buttons are shown. SelectedButton=0 means that the Next button was pressed.

A.72 `UiElementSizeableT`

The XML schema for a `UiElementSizeableT` type is:

```
<xs:complexType name="UiElementSizeableT">
  <xs:complexContent>
    <xs:extension base="clnt:UiElementT">
      <xs:sequence>
        <xs:element name="Height" type="clnt:SizeT" minOccurs="0"/>
        <xs:element name="Width" type="clnt:SizeT" minOccurs="0"/>
      </xs:sequence>
      <xs:attribute name="NodePath" type="xs:string" use="required"/>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

The attributes of a `UiElementSizeableT` type are described in Table A.68.

Table A.68 – Attributes of `UiElementSizeableT`

Attribute	Description
NodePath	This is a mandatory attribute which specifies the qualified path of the Node in the Device Model. The path description helps in finding a Node in the Information Model.

The elements of a `UiElementSizeableT` type are described in Table A.69.

Table A.69 – Elements of `UiElementSizeableT`

Element	Description
Height	This optional element specifies the relative height of the element.
Width	This optional element specifies the relative width of the element.

A.73 `UiElementT`

This type specifies an extended layout object with a settable visibility.

The XML schema for a `UiElementT` type is:

```
<xs:complexType name="UiElementT">
  <xs:complexContent>
    <xs:extension base="clnt:LabelHelpT">
```

```

    <xs:sequence>
      <xs:element name="Visibility" type="xs:boolean" minOccurs="0"/>
    </xs:sequence>
  </xs:extension>
</xs:complexContent>
</xs:complexType>

```

The elements of a UiElementT type are described in Table A.70.

Table A.70 – Elements of UiElementT

Element	Description
Visibility	This optional element specifies whether the element is visible or not. The default is visible.

A.74 UiTemplateT

This type specifies templates used to display a parameter value. UI templates modify how parameter values are displayed.

The XML schema for a UiTemplateT type is:

```

<xs:complexType name="UiTemplateT">
  <xs:sequence>
    <xs:choice>
      <xs:element name="Enumeration" type="clnt:EnumerationItemListT">
        <xs:unique name="UiTemplateUniqueEnumValue">
          <xs:selector xpath="clnt:EnumerationItem/clnt:Value"/>
          <xs:field xpath="."/>
        </xs:unique>
      </xs:element>
      <xs:element name="BitEnumeration" type="clnt:BitEnumerationItemListT">
        <xs:unique name="UiTemplateUniqueBitMask">
          <xs:selector xpath="clnt:BitEnumerationItem/clnt:Value"/>
          <xs:field xpath="."/>
        </xs:unique>
      </xs:element>
      <xs:element name="String" type="clnt:StringTemplateT"/>
      <xs:element name="Arithmetic" type="clnt:NumericTemplateT"/>
      <xs:element name="DateTime" type="clnt:DateTimeDataT"/>
    </xs:choice>
    <xs:element name="UiTemplateSize" type="xs:int" minOccurs="0"/>
  </xs:sequence>
</xs:complexType>

```

The elements of a UiTemplateT type are described in Table A.71.

Table A.71 – Elements of UiTemplateT

Element	Description
Enumeration	This element specifies that the value shall be displayed with an UI template optimized for enumerations. Each enumeration item defines a description and help text for a parameter value.
BitEnumeration	This element specifies that the value shall be displayed with an UI template optimized for bit enumerations. Each enumeration item defines a description and help text for a bitmask of the parameter value.
String	This element specifies that the value shall be displayed with an UI template optimized for all values of String Types.
Arithmetic	This element specifies that the value shall be displayed with an UI template optimized for all values of Numeric Types.
DateTime	This element specifies that the value shall be displayed with an UI template optimized for all values of Date and Time Types.
UiTemplateSize	This is an optional element used to specify the size of the Arithmetic data types, for example Byte information for Signed and Unsigned integer and to specify the number of characters in case of string datatypes.

A.75 VariantT

This type specifies a value. The value is in raw format and not scaled.

The XML schema for a VariantT type is:

```
<xs:complexType name="VariantT">
  <xs:choice>
    <xs:element name="Float" type="xs:float"/>
    <xs:element name="Double" type="xs:double"/>
    <xs:element name="Integer" type="xs:integer"/>
    <xs:element name="UnsignedInteger" type="xs:nonNegativeInteger"/>
    <xs:element name="Date" type="xs:date"/>
    <xs:element name="DateTime" type="xs:dateTime"/>
    <xs:element name="Time" type="xs:time"/>
    <xs:element name="Duration" type="xs:duration"/>
    <xs:element name="String" type="xs:string"/>
    <xs:element name="Boolean" type="xs:boolean"/>
  </xs:choice>
</xs:complexType>
```

The elements of a VariantT type are described in Table A.72.

Table A.72 – Elements of VariantT

Element	Description
Float	This optional element specifies a single precision floating point value.
Double	This optional element specifies a double precision floating point value.
Integer	This optional element specifies a signed integer value.
UnsignedInteger	This optional element specifies an unsigned integer value.
Date	This optional element specifies a date value.
DateTime	This optional element specifies a date and time value.
Time	This optional element specifies a time value.
Duration	This optional element specifies a length of time.
String	This optional element specifies a string.
Boolean	This optional element specifies a boolean.

A.76 VariantOptionListT

This type specifies the list of options to choose for the numeric parameter type.

The XML schema for a VariantOptionListT type is:

```
<xs:complexType name="VariantOptionListT">
  <xs:sequence maxOccurs="unbounded">
    <xs:element name="Option" type="clnt:VariantOptionT"/>
  </xs:sequence>
</xs:complexType>
```

The elements of a VariantOptionListT type are described in Table A.73.

Table A.73 – Elements of VariantOptionListT

Element	Description
Option	An element of the numeric options.

A.77 VariantOptionT

This type specifies one of the possible values of numeric options.

The XML schema for a VariantOptionT type is:

```
<xs:complexType name="VariantOptionT">
  <xs:complexContent>
    <xs:extension base="clnt:LabelHelpT">
      <xs:sequence>
        <xs:element name="Value" type="clnt:VariantT"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

The elements of a VariantOptionT type are described in Table A.74.

Table A.74 – Elements of VariantOptionT

Element	Description
Value	This required element specifies the unique identifier of the list entry in the context of the parent list element.

A.78 VectorListT

This type specifies a list of Vector elements.

The XML schema for a VectorListT type is:

```
<xs:complexType name="VectorListT">
  <xs:sequence maxOccurs="unbounded">
    <xs:element name="Vector" type="clnt:VectorT"/>
  </xs:sequence>
</xs:complexType>
```

The elements of a VectorListT type are described in Table A.75.

Table A.75 – Elements of VectorListT

Element	Description
Vector	An element of the list.

A.79 VectorT

This type specifies the content of a row or column in a grid.

The XML schema for a VectorT type is:

```
<xs:complexType name="VectorT">
  <xs:sequence>
    <xs:element name="Heading" type="xs:string"/>
    <xs:element name="Items">
      <xs:complexType>
        <xs:sequence minOccurs="0" maxOccurs="unbounded">
          <xs:choice>
            <xs:element name="Parameter" type="clnt:ParameterT"/>
            <xs:element name="Value" type="clnt:VariantT"/>
          </xs:choice>
        </xs:sequence>
      </xs:complexType>
    </xs:element>
  </xs:sequence>
</xs:complexType>
```

The elements of a VectorT type are described in Table A.76.

Table A.76 – Elements of VectorT

Element	Description
Heading	This required element specifies the localized heading displayed along with the data.
Items	This required element specifies the data to be displayed, which may be parameters or static scalar values.

A.80 WaveformListT

This type specifies a list of Waveform elements.

The XML schema for a WaveformListT type is:

```
<xs:complexType name="WaveformListT">
  <xs:sequence maxOccurs="unbounded">
    <xs:element name="Waveform" type="clnt:WaveformT"/>
  </xs:sequence>
</xs:complexType>
```

The elements of a WaveformListT type are described in Table A.77.

Table A.77 – Elements of WaveformListT

Element	Description
Waveform	An element of the list.

A.81 WaveformT

This type specifies a single data set displayed on a graph. A graph may contain one or more waveforms.

The XML schema for a WaveformT type is:

```
<xs:complexType name="WaveformT">
  <xs:complexContent>
    <xs:extension base="clnt:DiagramLineT">
      <xs:sequence>
        <xs:element name="Handling" type="clnt:HandlingT" default="rw"
          minOccurs="0" maxOccurs="1"/>
        <xs:element name="WaveformType" type="clnt:WaveformTypeT" minOccurs="1"
          maxOccurs="1"/>
        <xs:element name="KeyPointList" type="clnt:WaveformKeyPointListT"
          minOccurs="0" maxOccurs="1"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

The elements of a WaveformT type are described in Table A.78.

Table A.78 – Elements of WaveformT

Element	Description
Handling	This optional element specifies whether or not the waveform may be modified. The default is ReadWrite.
WaveformType	
KeyPointList	This optional element specifies a collection of key points that are displayed along with the waveform. The key points may or may not be points on the waveform itself.

A.82 WaveformTypeT

This is the abstract base type of all Waveform Types.

The XML schema for a WaveformTypeT type is:

```
<xs:complexType name="WaveformTypeT" abstract="true"/>
```

A.83 WaveformTypeHorizontalT

From EDD-Spec: The HORIZONTAL attribute specifies a WAVEFORM that contains horizontal lines.

The XML schema for a WaveformTypeHorizontalT type is:

```
<xs:complexType name="WaveformTypeHorizontalT">
  <xs:complexContent>
    <xs:extension base="clnt:WaveformTypeT">
      <xs:sequence>
        <xs:element name="Yvalues" type="clnt:WaveformVectorT" minOccurs="1"
          maxOccurs="1"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

The elements of a WaveformTypeHorizontalT type are described in Table A.79.

Table A.79 – Elements of WaveformTypeHorizontalT

Element	Description
Yvalues	From EDD-Spec: The Y_VALUES attribute specifies the Y coordinate of each horizontal line in the WAVEFORM.

A.84 WaveformTypeVerticalT

From EDD-Spec: The VERTICAL attribute specifies a WAVEFORM that contains vertical lines.

The XML schema for a WaveformTypeVerticalT type is:


```

<xs:complexType name="WaveformTypeVerticalT">
  <xs:complexContent>
    <xs:extension base="clnt:WaveformTypeT">
      <xs:sequence>
        <xs:element name="Xvalues" type="clnt:WaveformVectorT" minOccurs="1"
          maxOccurs="1"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>

```

The elements of a WaveformTypeVerticalT type are described in Table A.80.

Table A.80 – Elements of WaveformTypeVerticalT

Element	Description
Xvalues	From EDD-Spec: The X_VALUES attribute specifies the X coordinate of each vertical line in the WAVEFORM.

A.85 WaveformTypeYTT

From EDD-Spec: The YT attribute specifies a WAVEFORM that contains a list of points that are defined via an initial X coordinate, an X increment between successive points and a list of Y values.

The XML schema for a WaveformTypeYTT type is:

```

<xs:complexType name="WaveformTypeYTT">
  <xs:complexContent>
    <xs:extension base="clnt:WaveformTypeT">
      <xs:sequence>
        <xs:element name="Yvalues" type="clnt:WaveformVectorT" minOccurs="1"
          maxOccurs="1"/>
        <xs:element name="Xinitial" type="clnt:VariantT" minOccurs="1"
          maxOccurs="1"/>
        <xs:element name="Xincrement" type="clnt:VariantT" minOccurs="1"
          maxOccurs="1"/>
        <xs:element name="NumberOfPoints" type="xs:nonNegativeInteger"
          minOccurs="0" maxOccurs="1"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>

```

The elements of a WaveformTypeYTT type are described in Table A.81.

Table A.81 – Elements of WaveformTypeYTT

Element	Description
Yvalues	From EDD-Spec: The Y_VALUES attribute specifies the Y coordinate of each point in the WAVEFORM.
Xinitial	From EDD-Spec: The X_INITIAL attribute specifies the X coordinate of the first point in the WAVEFORM.
Xincrement	From EDD-Spec: The X_INCREMENT attribute specifies difference between the X coordinates of adjacent points in the WAVEFORM.
NumberOfPoints	From EDD-Spec: The NUMBER_OF_POINTS attribute specifies the number of valid data points in X_VALUES. By default, the number of points in the WAVEFORM without a NUMBER_OF_POINTS attribute equals the size of X_VALUES.

A.86 WaveformTypeXYT

From EDD-Spec: The XY attribute specifies a WAVEFORM that contains a list of (x,y) points.

The XML schema for a WaveformTypeXYT type is:

```
<xs:complexType name="WaveformTypeXYT">
  <xs:complexContent>
    <xs:extension base="clnt:WaveformTypeT">
      <xs:sequence>
        <xs:element name="Xvalues" type="clnt:WaveformVectorT" minOccurs="1"
          maxOccurs="1"/>
        <xs:element name="Yvalues" type="clnt:WaveformVectorT" minOccurs="1"
          maxOccurs="1"/>
        <xs:element name="NumberOfPoints" type="xs:nonNegativeInteger"
          minOccurs="0" maxOccurs="1"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

The elements of a WaveformTypeXYT type are described in Table A.82.

Table A.82 – Elements of WaveformTypeXYT

Element	Description
Xvalues	From EDD-Spec: The X_VALUES attribute specifies the X coordinate of each point in the WAVEFORM. For each X coordinate specified in X_VALUES there shall be a corresponding Y coordinate specified in Y_VALUES.
Yvalues	From EDD-Spec: The Y_VALUES attribute specifies the Y coordinate of each point in the WAVEFORM. For each Y coordinate specified in Y_VALUES there shall be a corresponding X coordinate specified in X_VALUES.
NumberOfPoints	From EDD-Spec: The NUMBER_OF_POINTS attribute specifies the number of valid data points in X_VALUES and Y_VALUES. By default, the number of points in the WAVEFORM without a NUMBER_OF_POINTS attribute equals the size of X_VALUES and Y_VALUES.

A.87 WaveformKeyPointListT

From EDD-Spec: The KEY_POINTS attribute specifies key points in the WAVEFORM that should be highlighted by the EDD application. The key points need not directly correspond to the data points specified via the TYPE attribute. The way in which these points are highlighted is defined by the EDD specification. By default, a WAVEFORM without a KEY_POINTS attribute should not have any of its points highlighted.

The XML schema for a WaveformKeyPointListT type is:

```
<xs:complexType name="WaveformKeyPointListT">
  <xs:complexContent>
    <xs:extension base="clnt:WaveformTypeT">
      <xs:sequence>
        <xs:element name="Xvalues" type="clnt:WaveformVectorT" minOccurs="1"
          maxOccurs="1"/>
        <xs:element name="Yvalues" type="clnt:WaveformVectorT" minOccurs="1"
          maxOccurs="1"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

The elements of a WaveformKeyPointListT type are described in Table A.83.

Table A.83 – Elements of WaveformKeyPointListT

Element	Description
Xvalues	From EDD-Spec: The X_VALUES attribute specifies the X coordinate of each point that is to be highlighted. For each X coordinate specified in X_VALUES there shall be a corresponding Y coordinate specified in Y_VALUES.
Yvalues	From EDD-Spec: The Y_VALUES attribute specifies the Y coordinate of each point that is to be highlighted. For each Y coordinate specified in Y_VALUES, there shall be a corresponding X coordinate specified in X_VALUES.

A.88 WaveformVectorT

WaveformVectorT represents the whole information of a waveform vector, for example the nodepath for the array of scalar values and additional information about these values such as Handling, Range, Type.

The XML schema for a WaveformVectorT type is:

```
<xs:complexType name="WaveformVectorT">
  <xs:sequence>
    <xs:element name="WaveformVectorElementList"
      type="clnt:WaveformVectorElementListT" minOccurs="1" maxOccurs="1"/>
  </xs:sequence>
  <xs:attribute name="DataArrayNodePath" type="xs:string" use="required"/>
</xs:complexType>
```

The attributes of a WaveformVectorT type are described in Table A.84.

Table A.84 – Attributes of WaveformVectorT

Attribute	Description
dataArrayNodePath	The nodepath that is used by the client to request the waveform vector.

The elements of a WaveformVectorT type are described in Table A.85.

Table A.85 – Elements of WaveformVectorT

Element	Description
WaveformVectorElementList	WaveformVectorElementList contains static information about each individual element of a waveform vector.

A.89 WaveformVectorElementListT

It represents the static information of a waveform vector, except the nodepath for the array of scalar values. This is the additional information such as Handling, Range, Type.

The XML schema for a WaveformVectorElementListT type is:

```
<xs:complexType name="WaveformVectorElementListT">
  <xs:sequence>
    <xs:element name="WaveformVectorElement" type="clnt:WaveformVectorElementT"
      minOccurs="0" maxOccurs="unbounded"/>
  </xs:sequence>
</xs:complexType>
```

The elements of a WaveformVectorElementListT type are described in Table A.86.

Table A.86 – Elements of WaveformVectorElementListT

Element	Description
WaveformVectorElement	The static information of an individual waveform vector element.

A.90 WaveformVectorElementT

It represents the whole information of a single element in a waveform vector. It contains additional information about one value such as Handling, Range, Type. It belongs to the corresponding element in the result array of DataArrayNodePath.

The XML schema for a WaveformVectorElementT type is:

```
<xs:complexType name="WaveformVectorElementT">
  <xs:sequence>
    <xs:element name="DataType" type="clnt:NumericDataT" default="SignedInteger"
      minOccurs="0"/>
    <xs:element name="RangeList" type="clnt:RangeListT" minOccurs="0"/>
    <xs:element name="Handling" type="clnt:HandlingT" default="rw" minOccurs="0"
      maxOccurs="1"/>
    <xs:element name="PreEditActionsList" type="clnt:ActionListT" minOccurs="0"
      maxOccurs="1"/>
    <xs:element name="PostEditActionsList" type="clnt:ActionListT" minOccurs="0"
      maxOccurs="1"/>
  </xs:sequence>
</xs:complexType>
```

```

    <xs:element name="ScalingFactor" type="xs:double" minOccurs="0"/>
    <xs:element name="DisplayFormat" type="clnt:FormatSpecifierT" minOccurs="0"/>
    <xs:element name="EditFormat" type="clnt:FormatSpecifierT" minOccurs="0"/>
  </xs:sequence>
</xs:complexType>

```

The elements of a WaveformVectorElementT type are described in Table A.87.

Table A.87 – Elements of WaveformVectorElementT

Element	Description
DataType	This required element specifies the data type of the value in the corresponding element of the result array of DataArrayNodePath.
RangeList	This optional element specifies the acceptable range of the value in the corresponding element of the result array of DataArrayNodePath.
Handling	This optional element specifies whether or not the current data value may be modified. The default is ReadWrite.
PreEditActionsList	This optional element specifies the PreEditActions of the value in the corresponding element of the result array of DataArrayNodePath.
PostEditActionsList	This optional element specifies the PostEditActions of the value in the corresponding element of the result array of DataArrayNodePath.
ScalingFactor	This optional element specifies the scaling factor of the value in the corresponding element of the result array of DataArrayNodePath.
DisplayFormat	This optional element specifies the display format of the value in the corresponding element of the result array of DataArrayNodePath.
EditFormat	This optional element specifies the edit format of the value in the corresponding element of the result array of DataArrayNodePath.

Annex B (informative)

Action example

The following is an example of an EDD method being executed by an FDI Server and the resulting interaction with an FDI Client.

The EDDL used by this example is shown below, followed by a sequence diagram and the description of the sequence.

```
VARIABLE device_var1
{
    LABEL "device var1";
    HELP "";
    CLASS DEVICE;
    HANDLING READ & WRITE;
    CONSTANT_UNIT "constUnit";
    TYPE INTEGER;
    PRE_EDIT_ACTIONS
    {
        PreEditAction1
    }
    POST_EDIT_ACTIONS
    {
        PostEditAction1
    }
}

VARIABLE process_value
{
    LABEL "Level";
    HELP "";
    CLASS DYNAMIC;
    HANDLING READ;
    CONSTANT_UNIT "m";
    TYPE FLOAT;
}

VARIABLE newI
{
    LABEL "new i";
    HELP "new value of i";
    CLASS DEVICE;
    HANDLING READ & WRITE;
    TYPE INTEGER;
}

VARIABLE newJ
{
    LABEL "new j";
    HELP "new value of j";
    CLASS DEVICE;
    HANDLING READ & WRITE;
    TYPE INTEGER;
}

MENU MethodMenu
{
    LABEL "MethodMenu";
    HELP "This menu is used in a method";
    STYLE DIALOG;
    ITEMS
    {
        newI,
        newJ
    }
}
```

```

}

METHOD UIReqRespCategories
{
    LABEL "Request Response Categories";
    HELP "This method demonstrates different categories of messages that are passed
        between server and client during a method execution";
    DEFINITION
    {
        int i, j, k, selection;
        add_abort_method(AbortMethod);
        //Acknowledgement
        ACKNOWLEDGE("Please hit OK to acknowledge the start of this method");
                                                                    // A010

        i = 5;
        PUT_MESSAGE("i = %{i}"); //Info // A020
        j = 10;
        PUT_MESSAGE("j = %{j}"); //Info // A030
        k = i+j;
        DELAY(5, "k = %{k}"); //Delay // A040

        GET_DEV_VAR_VALUE("Enter new value for the
            %[L]%[U]{device_var1}%[U]",device_var1); //Input // A050

        selection = SELECT_FROM_LIST("Is the value entered, correct? ", "YES;NO");
                                                                    //selection // A060
        if (selection == 1)
        {
            device_var1 = 0;
            abort(); //abort --- this will trigger the AbortMethod as well; // A070
        }
        k = k + device_var1;

        if (k == i + j)
        {
            display_comm_status(2); //Error - 2 == "Buffer Overflow" -- HART;
                                                                    // A080
        }

        display("Current level is %{process_value}%[U]{process_value}!"); // A090
        MenuDisplay(MethodMenu,"APPLY;DISCARD",selection); //UIDMessage // A100
        if(selection == 0)
        {
            i = newI;
            j = newJ;
            ACKNOWLEDGE("new value of k = %{k}"); // A110
        }
        ACKNOWLEDGE("This concludes the method !!"); // A120
    }
}

METHOD AbortMethod
{
    LABEL "AbortMethod";
    HELP "This is a simple Abort Method";
    DEFINITION
    {
        ACKNOWLEDGE("Method was aborted due to a call to abort()"); // B010
    }
}

METHOD PreEditAction1
{
    LABEL "Action1";
    HELP "This is a simple Pre Edit Warning";
    DEFINITION
    {
        ACKNOWLEDGE("Do you really want to edit this variable"); // C010
    }
}

```

```
    }  
}  
  
METHOD PostEditAction1  
{  
    LABEL "Action2";  
    HELP "This is a simple Post Edit Message";  
    DEFINITION  
    {  
        ACKNOWLEDGE("You actually edited the variable now!!!"); // D010  
    }  
}  
  
MENU FDIActions  
{  
    LABEL "FDI Actions";  
    HELP "This menu contains methods for verifying FDI UID contents";  
    ITEMS  
    {  
        MethodMenu,  
        UIReqRespCategories  
    }  
}
```

The example assumes as a precondition that the FDI Client has already established a Session with the FDI Server, the user has navigated to the Device (MyDevice), and the user has initiated the opening of the function group (FDIActions).

The sequence begins with the FDI Client subscribing to the value of the UID in order to retrieve the UID content from the FDI Server. The FDI Client adds a monitored item and waits for the user to provide the initial value. The sequence is illustrated in Figure B.1 along with the XML string that will be returned from the FDI Server.

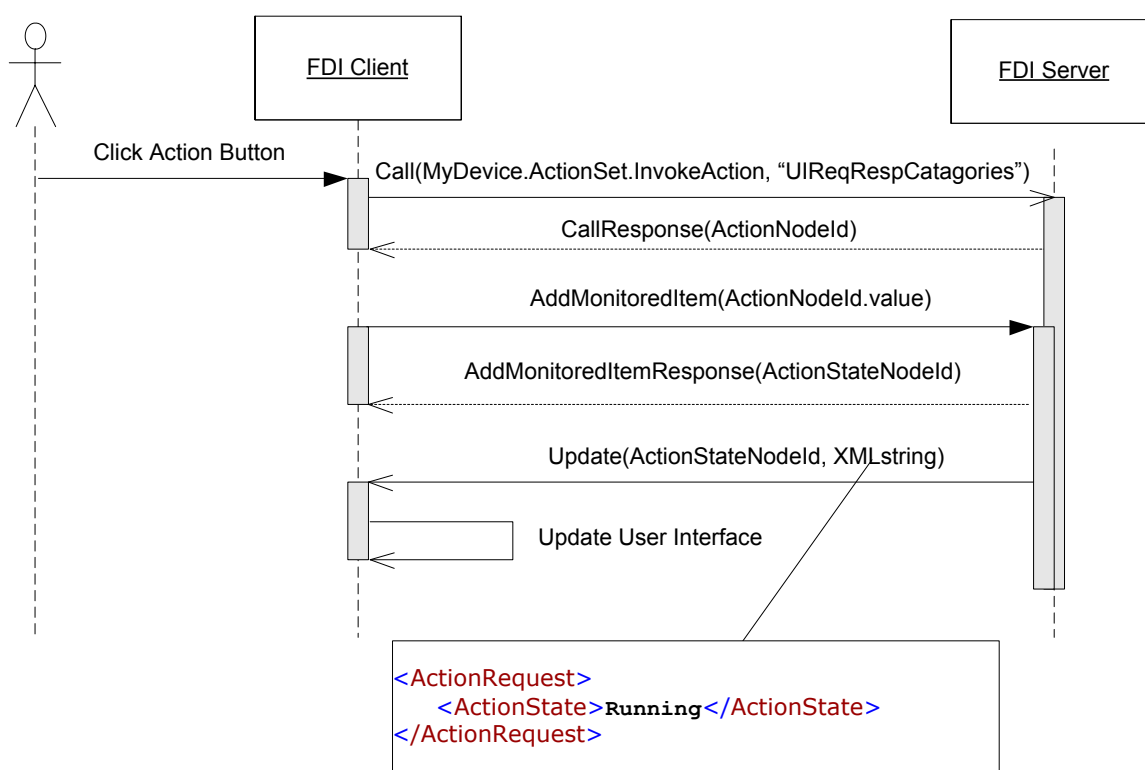


IEC

Figure B.1 – Action example (step 1)

The FDI Client interprets the XML string and presents the menu to the user as four action buttons with each button containing the text defined in the <Label> element. The FDI Client then waits for the user to select one of the actions.

In the next step of the example the sequence begins with the user clicking the action button (UIReqRespCategories) presented in the menu. The FDI Client reacts by initiating an OPC UA Call service request (see IEC 62541-4), specifying the OPC UA method (MyDevice.ActionSet.InvokeAction) and the EDDL method (UIReqRespCategories). The FDI Server responds by locating the EDDL method and initiating execution of it. The FDI Server responds with a unique node Id (Action NodeId1) that represents the running method. The FDI Client subscribes to the value of the method using the node Id (ActionNodeId.value). The FDI Server provides the current value of the method as any XML string. The initial value of this XML string indicates that the method has started. The sequence of this step is illustrated in Figure B.2.



IEC

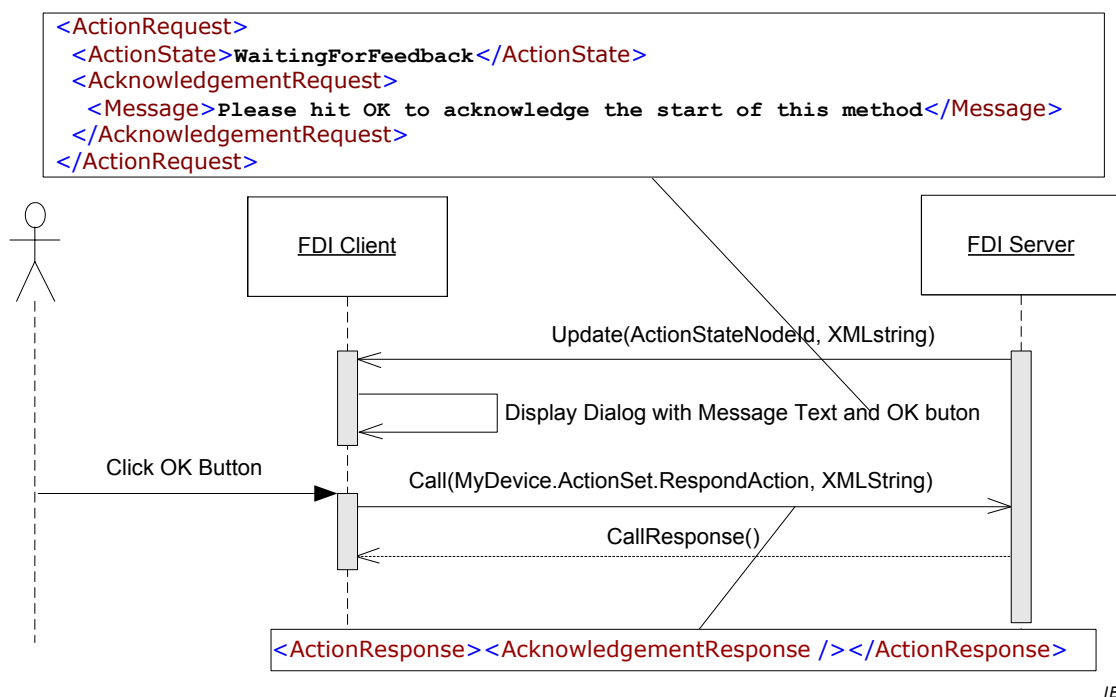
Figure B.2 – Action example (step 2)

In the next step of the example the method has run up to the “ACKNOWLEDGE” statement. The FDI Server processes the statement by setting the ActionNodeId value to indicate that a user acknowledgement is needed before the method can continue. The change to the value results in the FDI Server issuing a subscription update. Upon receiving the value update the FDI Client detects that an acknowledge request is pending. The FDI Client reacts by opening a dialog to present the message received in the value update also with an OK button.

The user reads the message and presses the OK button resulting in the FDI Client issuing a RespondAction method call to the FDI Server.

The FDI Server, receiving the RespondAction call, continues the execution of the method.

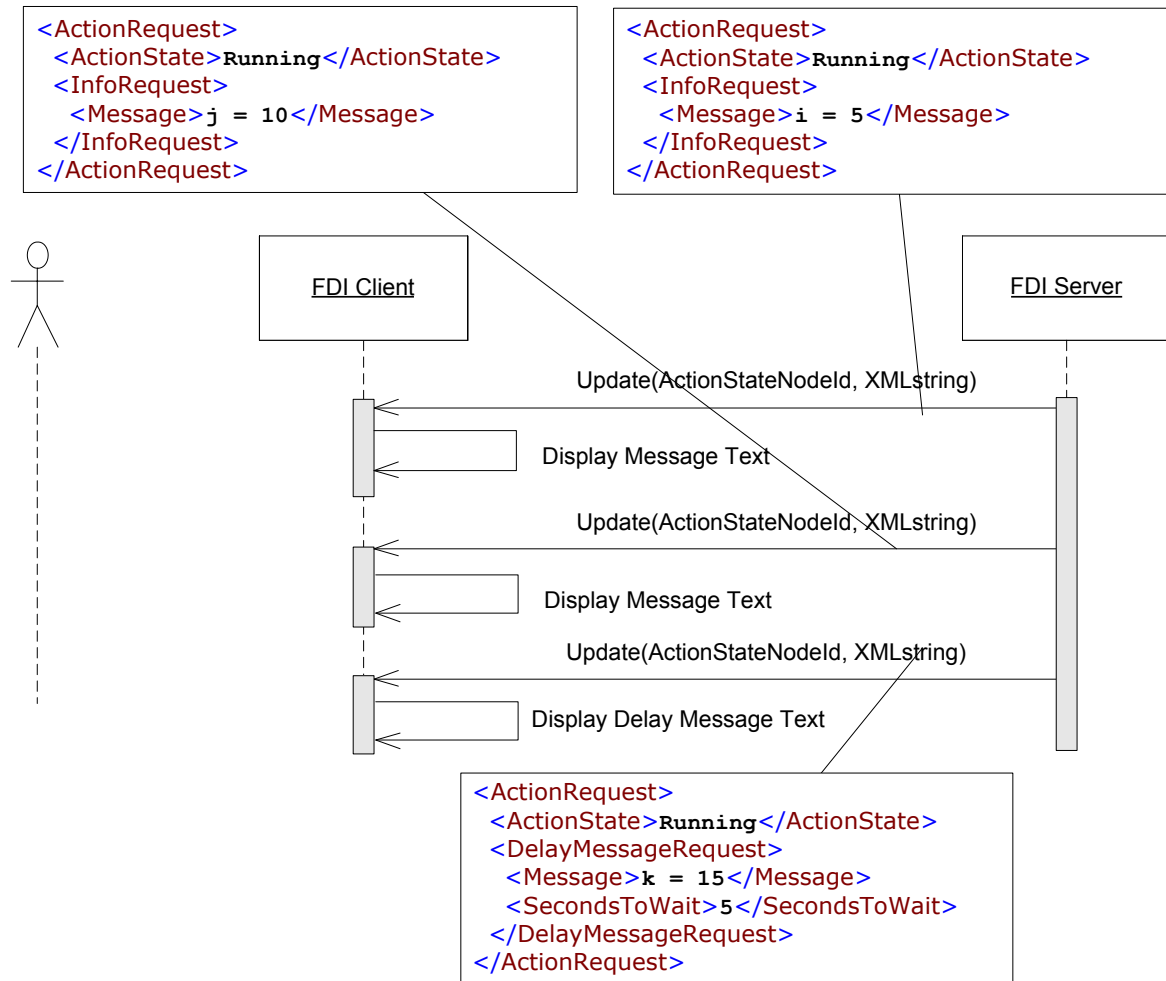
The sequence of this step is illustrated in Figure B.3.



IEC

Figure B.3 – Action example (step 3)

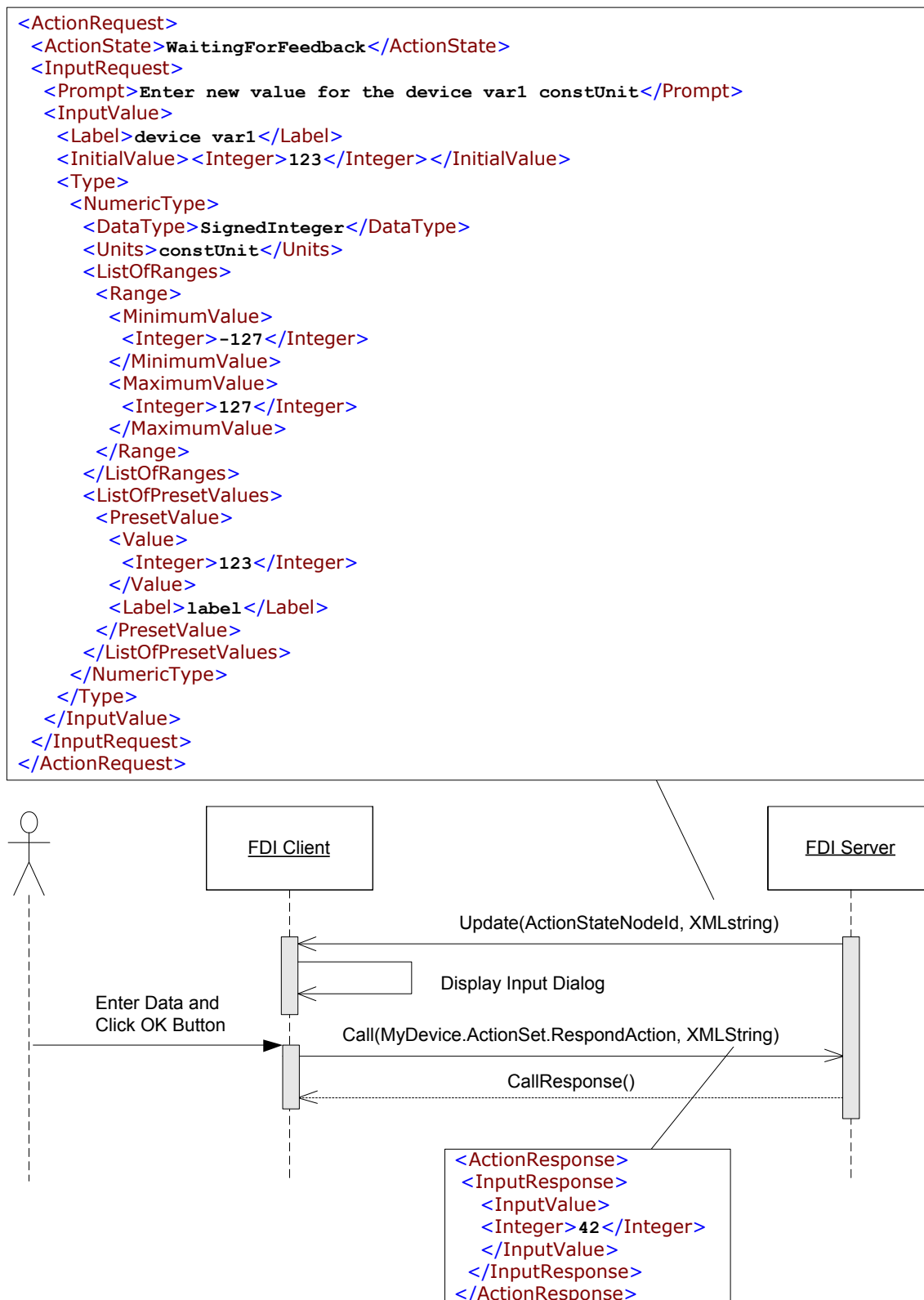
In the next step of the example the method has just executed the “ACKNOWLEDGE” statement and is about to execute the two “PUT_MESSAGE” statements and the “DELAY” statement. The FDI Server processes these statements by setting the `ActionNodeId` value to indicate the notifications. Each statement will generate a value change and be delivered to the FDI Client. The FDI Client reacts by showing the message text in an appropriate user interface component. The sequence of this step is illustrated in Figure B.4.



IEC

Figure B.4 – Action example (step 4)

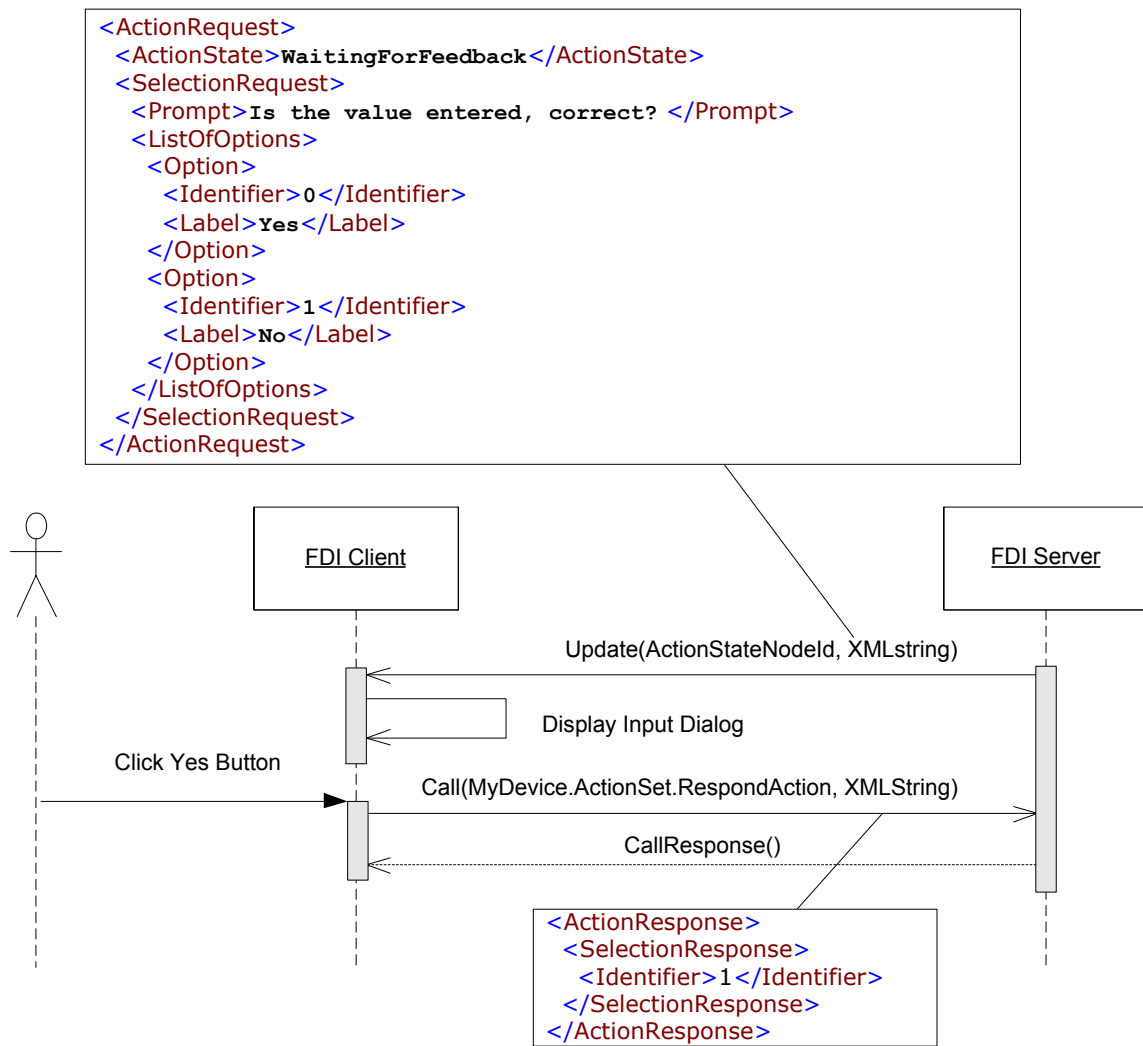
In the next step of the example the method requests the user to input a value for the (device_var1) variable. The FDI Server processes this statement by setting the ActionNodeId value to indicate the data input request. The FDI Client reacts to the value update by opening a dialog, showing the message text and preparing to accept user input. The sequence of this step is illustrated in Figure B.5.



IEC

Figure B.5 – Action example (step 5)

In the next step of the example the method requests the user to verify the value that was inputted for the (device_var1) variable. The FDI Server processes this statement by setting the ActionNodeId value to indicate a response is needed. The FDI Client reacts to the value update by opening a dialog, showing the message text and preparing to accept a response from the user. The sequence of this step is illustrated in Figure B.6.



IEC

Figure B.6 – Action example (step 6)

Annex C (informative)

Typical FDI Client use cases

C.1 General

Annex C describes how typical FDI Client use cases could be implemented.

C.2 Bulk operations

In Clause C.2 the term bulk operation is understood as applying the same operation iteratively to a group of Devices. If the group of Devices consists of Devices of different Device Types, a precondition is the clarification of what the “same” operations are with respect to the different Device Types.

Bulk operations can be performed by an FDI Client or by a UIP. In case of a UIP the group of devices shall be a subset of the Device and its sub-devices because a UIP has only access to these.

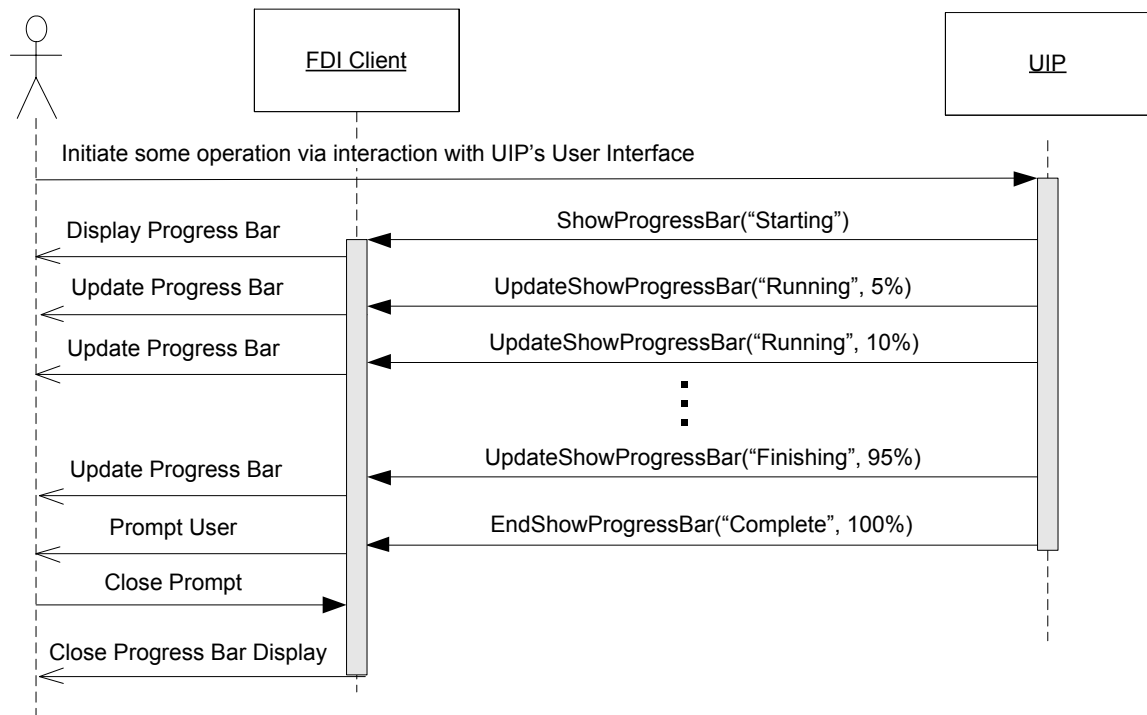
NOTE The following description only refers to an FDI Client. The algorithm is the same for a UIP besides the above mentioned restriction.

First of all an FDI Client has to give the user the ability to define a group of Devices. This is implementation specific and outside the scope of this document. The FDI Client can then use the OPC UA services to apply the same operation to these Devices. For that, OPC UA supports reading and writing of multiple parameters in one service call as well as invoking several methods in one service call. See IEC 62541-4 for details.

C.3 Progress bar support

The following is an example of a UIP using the Progress Bar functionality provided by the FDI Client (see Figure C.1). In this example the user initiates some operations by interacting with the UIP's user interface. The UIP initiates the operation and uses the progress bar functionality to keep the user informed of the progress (see 5.2.2.10 and 5.2.2.11).

In this example the FDI Client opens a small dialog window to present the progress information to the user. Once the UIP informs the FDI Client that the operation is complete, via the EndShowProgressBar service (see 5.2.2.12), the FDI Client prompts the user before closing the dialog to ensure the user has seen the 100 % completion of the operations.



IEC

Figure C.1 – Progress bar support

Bibliography

ISO/IEC 10646-1, *Information technology – Universal Multiple-Octet Coded Character Set (UCS) – Part 1: Architecture and Basic Multilingual Plane*

ISO/IEC 2375, *Information technology – Procedure for registration of escape sequences and coded character sets*

FDI-2021, *FDI Project Technical Specification – Part 1: Overview*
<available at www.fdi-cooperation.com>

FDI-2022, *FDI Project Technical Specification – Part 2: FDI Client*
<available at www.fdi-cooperation.com>

FDI-2023, *FDI Project Technical Specification – Part 3: FDI Server*
<available at www.fdi-cooperation.com>

FDI-2024, *FDI Project Technical Specification – Part 4: FDI Packages*
<available at www.fdi-cooperation.com>

FDI-2025, *FDI Project Technical Specification – Part 5: FDI Information Model*
<available at www.fdi-cooperation.com>

FDI-2026, *FDI Project Technical Specification – Part 6: FDI Technology Mapping*
<available at www.fdi-cooperation.com>

FDI-2027, *FDI Project Technical Specification – Part 7: FDI Communication Devices*
<available at www.fdi-cooperation.com>

SOMMAIRE

AVANT-PROPOS	144
INTRODUCTION.....	146
1 Domaine d'application	147
2 Références normatives	148
3 Termes, définitions, abréviations, acronymes et conventions	149
3.1 Termes et définitions	149
3.1.1 Termes utilisés pour les Services	149
3.1.2 Termes utilisés pour les Services d'Accès à l'Appareil	150
3.2 Abréviations et acronymes	150
3.3 Conventions.....	150
4 Vue d'ensemble	151
5 Client FDI	152
5.1 Services d'Accès à l'Appareil	152
5.1.1 Généralités	152
5.1.2 Modèle d'Appareil	152
5.1.3 Modèle de Nœud	154
5.1.4 Services	160
5.1.5 Services Base Property	164
5.1.6 Services du Modèle d'Appareil.....	165
5.1.7 Services de verrouillage	177
5.1.8 Services d'Accès Direct	178
5.1.9 Types de données	181
5.2 Services d'Hébergement	186
5.2.1 Généralités	186
5.2.2 Services	186
5.2.3 Définitions du Type Paramètre.....	192
6 UIP.....	193
6.1 Services d'UIP	193
6.1.1 Services	193
6.1.2 Définitions du Type Paramètre.....	197
6.2 Règles d'instanciation de l'UIP	199
6.3 Diagramme d'états de l'UIP	199
6.3.1 Etats.....	199
6.3.2 Transitions d'état.....	200
6.4 Permissions de l'UIP	200
6.5 Téléchargements de l'UIP à partir du Serveur FDI	200
7 Actions	202
7.1 Généralités	202
7.2 Diagramme de séquences.....	203
7.3 Définition du schéma d'Action FDI.....	207
8 Description d'Interface Utilisateur (UID, User Interface Description)	207
8.1 Vue d'ensemble	207
8.2 Exécution de l'UID	209
Annexe A (normative) Schéma XML.....	213
A.1 Généralités	213

A.2	AbortRequestT	213
A.3	AccessT	213
A.4	AcknowledgementRequestT	214
A.5	ActionListT	214
A.6	AbortingNotificationT	214
A.7	ActionRequestT	215
A.8	ActionResponseT	216
A.9	ActionT	216
A.10	AxisListT	217
A.11	AxisT	217
A.12	BitEnumerationItemListT	218
A.13	BitEnumerationItemT	219
A.14	ButtonListT	219
A.15	ChartT	220
A.16	ChartTypeT	220
A.17	ColorNameT	221
A.18	ColorT	222
A.19	ColorValueT	222
A.20	ColumnBreakT	223
A.21	DateTimeDataT	223
A.22	DelayMessageRequestT	223
A.23	DiagramLineT	224
A.24	EnumerationItemListT	225
A.25	EnumerationItemT	225
A.26	FormatSpecifierT	226
A.27	GraphT	226
A.28	GridT	227
A.29	HandlingT	227
A.30	ImageT	228
A.31	InfoRequestT	229
A.32	InputRequestT	229
A.33	InputResponseT	230
A.34	InputValueT	230
A.35	InputValueTypeT	231
A.36	LabelHelpT	231
A.37	LabelT	232
A.38	LineTypeT	232
A.39	MenuT	233
A.40	MenuReferenceT	235
A.41	MenuStyleT	235
A.42	NumericDataT	236
A.43	NumericTemplateT	237
A.44	OptionListT	237
A.45	OrientationT	237
A.46	ParameterInputRequestT	238
A.47	ParameterListT	238
A.48	ParameterT	239
A.49	PluginT	240
A.50	RangeListT	241

A.51	RangeT	241
A.52	ResponseT	242
A.53	RowBreakT	242
A.54	ScalingT	242
A.55	SelectionRequestT	243
A.56	SelectionResponseT	243
A.57	SeparatorT	243
A.58	SizeT	244
A.59	ParameterClassT	244
A.60	ActionClassT	246
A.61	SourceListT	247
A.62	SourceT	248
A.63	StringDataT	248
A.64	StringTemplateT	249
A.65	StringOptionListT	250
A.66	StringOptionT	250
A.67	StringT	250
A.68	TimeScaleT	251
A.69	UidLayoutInformation	251
A.70	UidRequestT	252
A.71	UidResponseT	252
A.72	UiElementSizeableT	253
A.73	UiElementT	253
A.74	UiTemplateT	254
A.75	VariantT	255
A.76	VariantOptionListT	256
A.77	VariantOptionT	256
A.78	VectorListT	257
A.79	VectorT	257
A.80	WaveformListT	258
A.81	WaveformT	258
A.82	WaveformTypeT	259
A.83	WaveformTypeHorizontalT	259
A.84	WaveformTypeVerticalT	259
A.85	WaveformTypeYTT	260
A.86	WaveformTypeXYT	261
A.87	WaveformKeyPointListT	262
A.88	WaveformVectorT	262
A.89	WaveformVectorElementListT	263
A.90	WaveformVectorElementT	263
Annexe B (informative)	Exemple d'Action	265
Annexe C (informative)	Cas d'utilisation types du Client FDI	274
C.1	Généralités	274
C.2	Opérations d'ensemble	274
C.3	Prise en charge de la barre de progression	274
Bibliographie		276
Figure 1 – Diagramme de l'architecture FDI		148

Figure 2 – Structure générale d'un Appareil	153
Figure 3 – Structure des Blocs	154
Figure 4 – NodeClasses du Modèle d'Appareil	155
Figure 5 – Exemple: Hiérarchie de la Variable représentant un RECORD	158
Figure 6 – Hiérarchie Variable représentant une VALUE_ARRAY de RECORD	159
Figure 7 – Diagramme d'états de l'UIP	199
Figure 8 – Diagramme de séquences d'Action FDI	205
Figure 9 – Descriptions d'Interface Utilisateur	208
Figure 10 – Diagramme de séquences de la Description d'Interface Utilisateur	211
Figure B.1 – Exemple d'Action (étape 1)	268
Figure B.2 – Exemple d'Action (étape 2)	269
Figure B.3 – Exemple d'Action (étape 3)	270
Figure B.4 – Exemple d'Action (étape 4)	271
Figure B.5 – Exemple d'Action (étape 5)	272
Figure B.6 – Exemple d'Action (étape 6)	273
Figure C.1 – Prise en charge de la barre de progression	275
Tableau 1 – Attributs de BaseNodeClass	155
Tableau 2 – Attributs de la NodeClass Objet	155
Tableau 3 – Attributs de la NodeClass Variable	156
Tableau 4 – Tableau de Définition des Services	160
Tableau 5 – Affectations de bits de StatusCode	162
Tableau 6 – InfoBits de DataValue	163
Tableau 7 – Codes de résultat de service	163
Tableau 8 – Codes de résultat du niveau opération	164
Tableau 9 – Paramètres du Service GetDeviceAccessInterfaceVersion	165
Tableau 10 – Paramètres du Service GetOnlineAccessAvailability	165
Tableau 11 – Paramètres du Service Explorer	166
Tableau 12 – Paramètres du Service CancelBrowse	167
Tableau 13 – Paramètres du Service Read	167
Tableau 14 – Codes de résultat du service Read	168
Tableau 15 – Codes de résultat de l'opération Read	168
Tableau 16 – Paramètres du Service CancelRead	169
Tableau 17 – Paramètres du Service Write	170
Tableau 18 – Codes de résultat de l'opération Write	170
Tableau 19 – Paramètres du Service CancelWrite	171
Tableau 20 – Paramètres du Service CreateSubscription	172
Tableau 21 – Codes de résultat du Service CreateSubscription	172
Tableau 22 – Paramètres du Service Subscribe	172
Tableau 23 – Codes de résultat de l'opération Subscribe	174
Tableau 24 – Paramètres du service Unsubscribe	175
Tableau 25 – Codes de résultat de l'opération Unsubscribe	175
Tableau 26 – Paramètres du Service DeleteSubscription	176

Tableau 27 – Paramètres du Service DataChangeCallback.....	176
Tableau 28 – Codes de résultat de DataChangeCallback.....	176
Tableau 29 – Paramètres du Service InitLock	177
Tableau 30 – Codes de résultat du Service InitLock.....	178
Tableau 31 – Paramètres du Service ExitLock	178
Tableau 32 – Codes de résultat du Service ExitLock.....	178
Tableau 33 – Paramètres du Service InitDirectAccess	179
Tableau 34 – Codes de résultat du Service InitDirectAccess.....	179
Tableau 35 – Paramètres du Service ExitDirectAccess	180
Tableau 36 – Codes de résultat du Service ExitDirectAccess.....	180
Tableau 37 – Paramètres du Service Transfer	181
Tableau 38 – Codes de résultat du Service Transfer	181
Tableau 39 – Types de données de base.....	181
Tableau 40 – Identifiants assignés aux Attributs	182
Tableau 41 – NodeSpecifieur.....	183
Tableau 42 – DataValue	183
Tableau 43 – InnerErrorInfo	184
Tableau 44 – Définition de LocalizedText.....	184
Tableau 45 – Exemples de LocaleId	184
Tableau 46 – Structure du Type de Données Range	185
Tableau 47 – Structure du Type de Données EUInformation	185
Tableau 48 – Définition d'EnumValueType	186
Tableau 49 – Paramètres du Service GetClientTechnologyVersion	186
Tableau 50 – Paramètres du Service OpenUserInterface	187
Tableau 51 – Paramètres du Service LogAuditTrailMessage.....	188
Tableau 52 – Paramètres du Service SaveUserSettings.....	188
Tableau 53 – Paramètres du Service LoadUserSettings.....	189
Tableau 54 – Paramètres du Service Trace	189
Tableau 55 – Paramètres du Service ShowMessageBox.....	190
Tableau 56 – Paramètres du Service ShowProgressBar	190
Tableau 57 – Paramètres du Service UpdateShowProgressBar	191
Tableau 58 – Paramètres du Service EndShowProgressBar	191
Tableau 59 – Paramètres du service StandardUIActionItemsChange	192
Tableau 60 – Paramètres du service SpecificUIActionItemsChange	192
Tableau 61 – Définition de DefaultResult	192
Tableau 62 – Définition de ButtonSet.....	193
Tableau 63 – Définition de AcknStyle.....	193
Tableau 64 – Paramètres du Service Activate.....	194
Tableau 65 – Paramètres du Service Deactivate.....	195
Tableau 66 – Paramètres du Service SetSystemLabel	195
Tableau 67 – Paramètres du Service SetTraceLevel	196
Tableau 68 – Paramètres du Service GetStandardUIActionItems	196
Tableau 69 – Paramètres du service GetSpecificUIActionItems	196

Tableau 70 – Paramètres du Service InvokeStandardUIAction	197
Tableau 71 – Paramètres du Service InvokeSpecificUIAction	197
Tableau 72 – Définition de TraceLevel	197
Tableau 73 – Définition de StandardUIAction	198
Tableau 74 – Définition de StandardUIActionItem	198
Tableau 75 – Définition de SpecificUIActionItem	198
Tableau 76 – Etats de l'UIP	200
Tableau 77 – Transitions d'états de l'UIP	200
Tableau A.1 – Eléments d'AbortRequestT	213
Tableau A.2 – Enumérations d'AccessT	213
Tableau A.3 – Eléments d'AcknowledgementRequestT	214
Tableau A.4 – Eléments d>ActionListT	214
Tableau A.5 – Eléments d>ActionRequestT	215
Tableau A.6 – Eléments d>ActionResponseT	216
Tableau A.7 – Eléments d>ActionT	217
Tableau A.8 – Eléments d'AxisListT	217
Tableau A.9 – Attributs d'AxisT	218
Tableau A.10 – Eléments d'AxisT	218
Tableau A.11 – Eléments de BitEnumerationItemListT	219
Tableau A.12 – Eléments de BitEnumerationItemT	219
Tableau A.13 – Eléments de ButtonListT	219
Tableau A.14 – Eléments de ChartT	220
Tableau A.15 – Enumérations de ChartTypeT	221
Tableau A.16 – Enumérations de ColorNameT	222
Tableau A.17 – Enumérations de DateTimeDataT	223
Tableau A.18 – Eléments de DelayMessageRequestT	224
Tableau A.19 – Attributs de DiagramLineT	224
Tableau A.20 – Eléments de DiagramLineT	224
Tableau A.21 – Eléments d'EnumerationItemListT	225
Tableau A.22 – Eléments d'EnumerationItemT	226
Tableau A.23 – Eléments de GraphT	227
Tableau A.24 – Eléments de GridT	227
Tableau A.25 – Enumérations de HandlingT	228
Tableau A.26 – Attributs d'ImageT	228
Tableau A.27 – Eléments d'ImageT	229
Tableau A.28 – Eléments d'InfoRequestT	229
Tableau A.29 – Eléments d'InputRequestT	230
Tableau A.30 – Eléments d'InputResponseT	230
Tableau A.31 – Eléments d'InputValueT	231
Tableau A.32 – Eléments d'InputValueTypeT	231
Tableau A.33 – Eléments de LabelHelpT	232
Tableau A.34 – Eléments de LabelT	232
Tableau A.35 – Enumérations de LineTypeT	233

Tableau A.36 – Attributs de MenuT	234
Tableau A.37 – Eléments de MenuT	234
Tableau A.38 – Attributs de MenuReferenceT	235
Tableau A.39 – Eléments de MenuReferenceT	235
Tableau A.40 – Enumérations de MenuStyleT	236
Tableau A.41 – Enumérations de NumericDataT	236
Tableau A.42 – Eléments de NumericTemplateT	237
Tableau A.43 – Eléments d'OptionListT	237
Tableau A.44 – Enumérations d'OrientationT	238
Tableau A.45 – Eléments de ParameterInputRequestT	238
Tableau A.46 – Eléments de ParameterListT	239
Tableau A.47 – Eléments de ParameterT	240
Tableau A.48 – Eléments de PluginT	241
Tableau A.49 – Eléments de RangeListT	241
Tableau A.50 – Eléments de RangeT	242
Tableau A.51 – Enumérations de ScalingT	242
Tableau A.52 – Eléments de SelectionRequestT	243
Tableau A.53 – Eléments de SelectionResponseT	243
Tableau A.54 – Enumérations de SizeT	244
Tableau A.55 – Enumérations de ParameterClassT	245
Tableau A.56 – Enumérations d'ActionClassT	247
Tableau A.57 – Eléments de SourceListT	247
Tableau A.58 – Eléments de SourceT	248
Tableau A.59 – Enumérations de StringDataT	249
Tableau A.60 – Eléments de StringTemplateT	249
Tableau A.61 – Eléments de StringOptionListT	250
Tableau A.62 – Eléments de StringOptionT	250
Tableau A.63 – Eléments de StringT	251
Tableau A.64 – Enumérations de TimeScaleT	251
Tableau A.65 – Eléments d'UidLayoutInformation	252
Tableau A.66 – Eléments d'UidRequestT	252
Tableau A.67 – Eléments d'UidResponseT	253
Tableau A.68 – Attributs d'UiElementSizeableT	253
Tableau A.69 – Eléments d'UiElementSizeableT	253
Tableau A.70 – Eléments d'UiElementT	254
Tableau A.71 – Eléments d'UiTemplateT	255
Tableau A.72 – Eléments de VariantT	256
Tableau A.73 – Eléments de VariantOptionListT	256
Tableau A.74 – Eléments de VariantOptionT	257
Tableau A.75 – Eléments de VectorListT	257
Tableau A.76 – Eléments de VectorT	258
Tableau A.77 – Eléments de WaveformListT	258
Tableau A.78 – Eléments de WaveformT	259

Tableau A.79 – Éléments de WaveformTypeHorizontalT	259
Tableau A.80 – Éléments de WaveformTypeVerticalT	260
Tableau A.81 – Éléments de WaveformTypeYTT	261
Tableau A.82 – Éléments de WaveformTypeXYT	261
Tableau A.83 – Éléments de WaveformKeyPointListT	262
Tableau A.84 – Attributs de WaveformVectorT	263
Tableau A.85 – Éléments de WaveformVectorT	263
Tableau A.86 – Éléments de WaveformVectorElementListT	263
Tableau A.87 – Éléments de WaveformVectorElementT	264

COMMISSION ELECTROTECHNIQUE INTERNATIONALE

INTEGRATION DES APPAREILS DE TERRAIN (FDI) –

Partie 2: Client FDI

AVANT-PROPOS

- 1) La Commission Electrotechnique Internationale (IEC) est une organisation mondiale de normalisation composée de l'ensemble des comités électrotechniques nationaux (Comités nationaux de l'IEC). L'IEC a pour objet de favoriser la coopération internationale pour toutes les questions de normalisation dans les domaines de l'électricité et de l'électronique. A cet effet, l'IEC – entre autres activités – publie des Normes internationales, des Spécifications techniques, des Rapports techniques, des Spécifications accessibles au public (PAS) et des Guides (ci-après dénommés "Publication(s) de l'IEC"). Leur élaboration est confiée à des comités d'études, aux travaux desquels tout Comité national intéressé par le sujet traité peut participer. Les organisations internationales, gouvernementales et non gouvernementales, en liaison avec l'IEC, participent également aux travaux. L'IEC collabore étroitement avec l'Organisation Internationale de Normalisation (ISO), selon des conditions fixées par accord entre les deux organisations.
- 2) Les décisions ou accords officiels de l'IEC concernant les questions techniques représentent, dans la mesure du possible, un accord international sur les sujets étudiés, étant donné que les Comités nationaux de l'IEC intéressés sont représentés dans chaque comité d'études.
- 3) Les Publications de l'IEC se présentent sous la forme de recommandations internationales et sont agréées comme telles par les Comités nationaux de l'IEC. Tous les efforts raisonnables sont entrepris afin que l'IEC s'assure de l'exactitude du contenu technique de ses publications; l'IEC ne peut pas être tenue responsable de l'éventuelle mauvaise utilisation ou interprétation qui en est faite par un quelconque utilisateur final.
- 4) Dans le but d'encourager l'uniformité internationale, les Comités nationaux de l'IEC s'engagent, dans toute la mesure possible, à appliquer de façon transparente les Publications de l'IEC dans leurs publications nationales et régionales. Toutes divergences entre toutes Publications de l'IEC et toutes publications nationales ou régionales correspondantes doivent être indiquées en termes clairs dans ces dernières.
- 5) L'IEC elle-même ne fournit aucune attestation de conformité. Des organismes de certification indépendants fournissent des services d'évaluation de conformité et, dans certains secteurs, accèdent aux marques de conformité de l'IEC. L'IEC n'est responsable d'aucun des services effectués par les organismes de certification indépendants.
- 6) Tous les utilisateurs doivent s'assurer qu'ils sont en possession de la dernière édition de cette publication.
- 7) Aucune responsabilité ne doit être imputée à l'IEC, à ses administrateurs, employés, auxiliaires ou mandataires, y compris ses experts particuliers et les membres de ses comités d'études et des Comités nationaux de l'IEC, pour tout préjudice causé en cas de dommages corporels et matériels, ou de tout autre dommage de quelque nature que ce soit, directe ou indirecte, ou pour supporter les coûts (y compris les frais de justice) et les dépenses découlant de la publication ou de l'utilisation de cette Publication de l'IEC ou de toute autre Publication de l'IEC, ou au crédit qui lui est accordé.
- 8) L'attention est attirée sur les références normatives citées dans cette publication. L'utilisation de publications référencées est obligatoire pour une application correcte de la présente publication.

La Norme internationale IEC 62769-2 a été établie par le sous-comité 65E: Les appareils et leur intégration dans les systèmes de l'entreprise, du comité d'études 65 de l'IEC: Mesure, commande et automation dans les processus industriels.

Le texte de cette norme est issu des documents suivants:

CDV	Rapport de vote
65E/345/CDV	65E/422/RVC

Le rapport de vote indiqué dans le tableau ci-dessus donne toute information sur le vote ayant abouti à l'approbation de cette norme.

Cette publication a été rédigée selon les Directives ISO/IEC, Partie 2.

Une liste de toutes les parties de la série IEC 62769, publiées sous le titre général *Intégration des appareils de terrain (FDI)*, peut être consultée sur le site web de l'IEC.

Le comité a décidé que le contenu de cette publication ne sera pas modifié avant la date de stabilité indiquée sur le site web de l'IEC sous "<http://webstore.iec.ch>" dans les données relatives à la publication recherchée. A cette date, la publication sera

- reconduite,
- supprimée,
- remplacée par une édition révisée, ou
- amendée.

IMPORTANT – Le logo "*colour inside*" qui se trouve sur la page de couverture de cette publication indique qu'elle contient des couleurs qui sont considérées comme utiles à une bonne compréhension de son contenu. Les utilisateurs devraient, par conséquent, imprimer cette publication en utilisant une imprimante couleur.

INTRODUCTION

La Commission Electrotechnique Internationale (IEC) attire l'attention sur le fait qu'il est déclaré que la conformité avec les dispositions du présent document peut impliquer l'utilisation de brevets concernant:

- a) la méthode de fourniture et d'installation des fonctionnalités spécifiques aux appareils (cf. famille de brevets DE10357276);
- b) la méthode et l'appareil utilisés pour l'accès à un module fonctionnel du système d'automation (cf. famille de brevets EP2182418);
- c) les méthodes et les appareils utilisés pour diminuer les exigences mémoire relatives aux applications logicielles du système de commande de processus (cf. famille de brevets US2013232186);
- d) modèle d'objet d'appareil extensible (cf. famille de brevets US12/893,680).

L'IEC ne prend pas position quant à la preuve, à la validité et à la portée de ces droits de propriété.

Le détenteur de ces droits de propriété a donné l'assurance à l'IEC qu'il consent à négocier des licences avec des demandeurs du monde entier, soit sans frais soit à des termes et conditions raisonnables et non discriminatoires. A ce propos, la déclaration du détenteur des droits de propriété est enregistrée à l'IEC. Des informations peuvent être demandées à:

- a) ABB Research Ltd
Claes Ryttoft
Affolterstrasse 4
Zurich, 8050
Suisse
- b) Phoenix Contact GmbH & Co KG
Intellectual Property, Licenses & Standards
Flachsmarktstrasse 8, 32825 Blomberg
Allemagne
- c) Fisher Controls International LLC
John Dilger, Emerson Process Management LLLP
301 S. 1st Avenue, Marshalltown, Iowa 50158
Etats-Unis d'Amérique

Rockwell Automation Technologies, Inc.
1 Allen-Bradley Drive
Mayfield Heights, Ohio 44124
Etats-Unis d'Amérique
- d)

L'attention est d'autre part attirée sur le fait que certains des éléments du présent document peuvent faire l'objet de droits de propriété autres que ceux qui ont été mentionnés ci-dessus. L'IEC ne saurait être tenue pour responsable de l'identification de ces droits de propriété en tout ou partie.

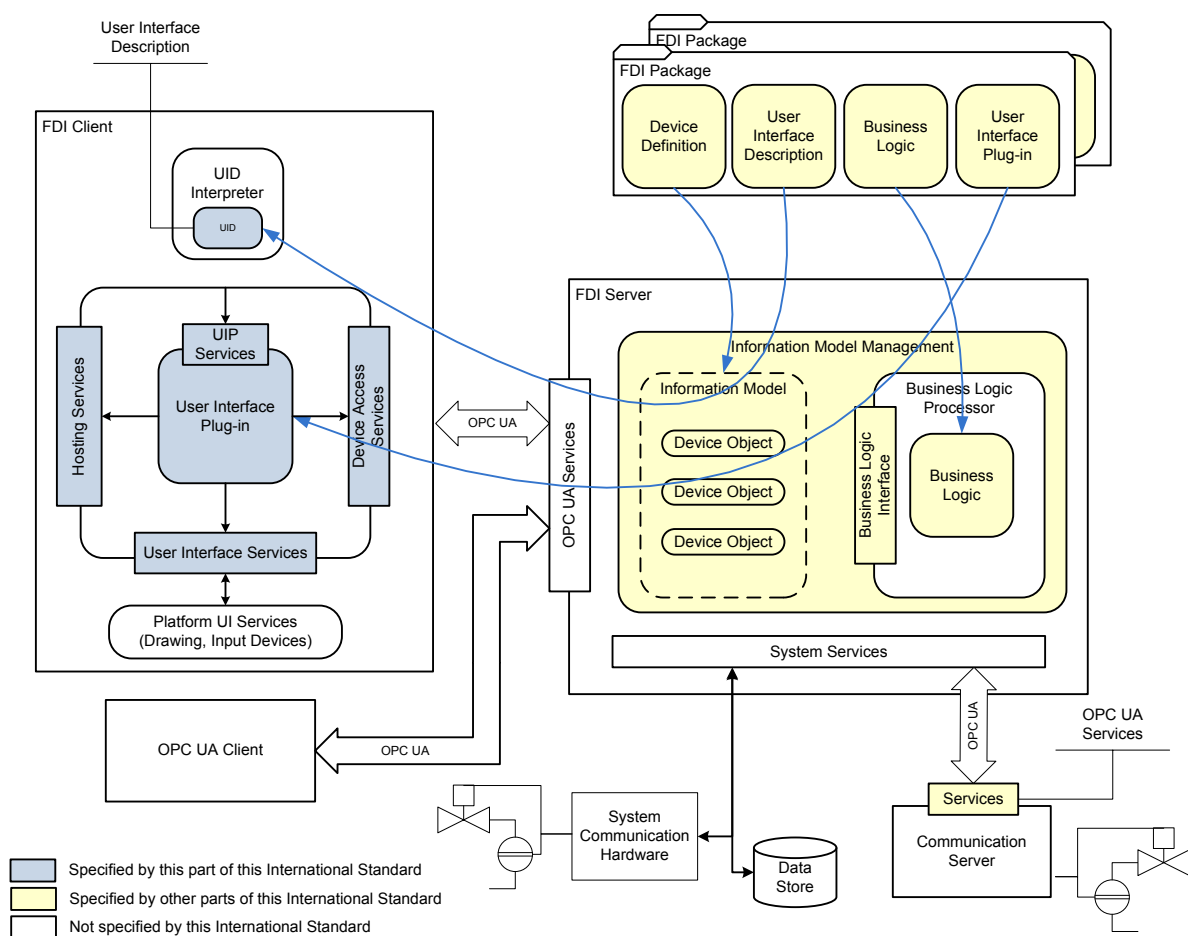
L'ISO (www.iso.org/patents) et l'IEC (<http://patents.iec.ch>) tiennent à jour des bases de données en ligne sur les brevets relatifs à leurs normes. Les utilisateurs sont encouragés à consulter ces bases de données pour obtenir l'information la plus récente concernant les droits de brevet.

INTEGRATION DES APPAREILS DE TERRAIN (FDI) –

Partie 2: Client FDI

1 Domaine d'application

La présente partie de l'IEC 62769 définit le client FDI. L'architecture FDI complète est présentée à la Figure 1. Les composants architecturaux relevant du domaine d'application du présent document ont été mis en évidence dans cette figure.



IEC

Anglais	Français
User Interface Description	Description d'Interface Utilisateur
FDI Client	Client FDI
UID Interpreter	Interprète UID
UID	UID
UIP Services	Services UIP
Hosting Services	Services d'Hébergement
User Interface Plug-in	Plugiciel d'Interface Utilisateur
Device Access Services	Services d'Accès à l'Appareil

Anglais	Français
User Interface Services	Services d'Interface Utilisateur
Platform UI Services (Drawing, Input Devices)	Services d'UI de Plate-Forme (Dessin, Appareils d'Entrée)
OPC UA Client	Client OPC UA
FDI Package	Paquetage FDI
Device Definition	Définition d'Appareil
Business Logic	Logique Applicative
FDI Server	Serveur FDI
OPC UA Services	Services OPC UA
Information Model Management	Gestion du Modèle d'Information
Information Model	Modèle d'Information
Device Object	Objet Device (Appareil)
Business Logic Processor	Processeur de Logique Applicative
Business Logic Interface	Interface de Logique Applicative
System Services	Services Système
System Communication Hardware	Matériel de Communication du Système
Data Store	Magasin de Données
Communication Server	Serveur de Communication
Specified by this part of this International Standard	Spécifié par la présente partie de la présente Norme internationale
Specified by other parts of this International Standard	Spécifié par d'autres parties de la présente Norme internationale
Not specified by this International Standard	Non spécifié par la présente Norme internationale

Figure 1 – Diagramme de l'architecture FDI

2 Références normatives

Les documents suivants sont cités en référence de manière normative, en intégralité ou en partie, dans le présent document et sont indispensables pour son application. Pour les références datées, seule l'édition citée s'applique. Pour les références non datées, la dernière édition du document de référence s'applique (y compris les éventuels amendements).

IEC 62769-1, *Intégration des appareils de terrain (FDI) – Partie 1: Vue d'ensemble*

NOTE L'IEC 62769-1 est techniquement identique à la FDI-2021

IEC 62769-3, *Intégration des appareils de terrain (FDI) – Partie 3: Serveur FDI*

NOTE L'IEC 62769-3 est techniquement identique à la FDI-2023

IEC 62769-4:2015, *Intégration des appareils de terrain (FDI) – Partie 4: Paquetages FDI*

NOTE L'IEC 62769-4 est techniquement identique à la FDI-2024

IEC 62769-5, *Intégration des appareils de terrain (FDI) – Partie 5: Modèle d'Information FDI*

NOTE L'IEC 62769-5 est techniquement identique à la FDI-2025

IEC 62769-6:2015, *Intégration des appareils de terrain (FDI) – Partie 6: Mapping de technologies FDI*

NOTE L'IEC 62769-6 est techniquement identique à la FDI-2026

IEC 62541-3, *Architecture unifiée OPC – Partie 3: Modèle de l'espace d'adressage*

IEC 62541-4, *Architecture unifiée OPC – Partie 4: Services*

ISO 639, *Codes pour la représentation des noms de langue*

ISO 3166, *Codes pour la représentation des noms de pays et de leurs subdivisions*

ISO/IEC 10918-1, *Technologies de l'information – Compression numérique et codage des images fixes de nature photographique: Prescriptions et lignes directrices*

ISO/IEC 15948, *Technologies de l'information – Infographie et traitement d'images – Graphiques de réseau portables (PNG): Spécification fonctionnelle*

IEEE 754, *IEEE Standard for Floating-Point Arithmetic* (disponible en anglais seulement)

IETF RFC 2083, *PNG (Portable Network Graphics) Specification Version 1.0* (disponible en anglais seulement)

IETF RFC 3066, *Tags for the Identification of Languages* (disponible en anglais seulement)

XML Schema-1, *XML Schema: Structures* (disponible à l'adresse <http://www.w3.org/TR/xmlschema-1/>) (disponible en anglais seulement)

XML Schema-2, *XML Schema: Datatypes* (disponible à l'adresse <http://www.w3.org/TR/xmlschema-2/>) (disponible en anglais seulement)

3 Termes, définitions, abréviations, acronymes et conventions

3.1 Termes et définitions

Pour les besoins du présent document, les termes et définitions de l'IEC 62769-1, ainsi que les suivants s'appliquent.

3.1.1 Termes utilisés pour les Services

3.1.1.1

Services de verrouillage

ensemble de Services à travers lesquels l'accès à un appareil est contrôlé

3.1.1.2

Services du Modèle d'Appareil

sous-ensemble des Services d'Accès à l'Appareil à travers lequel un UIP peut accéder aux informations d'un Appareil

3.1.1.3

Services d'Accès Direct

sous-ensemble des Services d'Accès à l'Appareil à travers lequel un UIP peut accéder directement à un Appareil

3.1.2 Termes utilisés pour les Services d'Accès à l'Appareil

3.1.2.1

Attribut

élément d'information d'un Nœud

Note 1 à l'article: Certains Attributs existent pour toutes les NodeClasses et d'autres sont spécifiques à une NodeClass donnée.

Note 2 à l'article: Remplace la définition donnée dans l'IEC 62769-1.

3.1.2.2

Modèle d'Appareil

hiérarchie de Nœuds représentant un Appareil existant

3.1.2.3

Nœud

élément dans un Modèle d'Appareil qui peut être adressé via les Services d'Accès à l'Appareil

Note 1 à l'article: Remplace la définition donnée dans l'IEC 62769-1.

3.1.2.4

NodeClass (Classe de nœud)

peut être un Objet ou une Variable

Note 1 à l'article: Remplace la définition donnée dans l'IEC 62769-1.

3.1.2.5

Objet

instance de la NodeClass Objet

Note 1 à l'article: Remplace la définition donnée dans l'IEC 62769-1.

3.1.2.6

Variable

instance de la NodeClass Variable

Note 1 à l'article: Remplace la définition donnée dans l'IEC 62769-1.

3.2 Abréviations et acronymes

Pour les besoins du présent document, les abréviations et acronymes de l'IEC 62769-1, ainsi que les suivants s'appliquent.

UTC	Coordinated Universal Time (Temps Universel Coordonné)
XML	Extended mark-up language (Langage de balisage extensible)

3.3 Conventions

Les conventions pour les définitions de service sont identiques à l'IEC 62541-4.

Les types de données de base sont définis dans l'IEC 62541-3.

"Paramètre" est toujours un Paramètre du Modèle d'Information. Le terme "paramètre" est utilisé pour l'usage général du mot. Pour toute ambiguïté, du contexte supplémentaire sera apporté.

4 Vue d'ensemble

Un Paquetage FDI (FDI Package) fournit les informations nécessaires à un Type d'Appareil pour permettre la gestion de l'Appareil au sein du système. Il est fourni par un vendeur d'appareils et déployé dans un Serveur FDI. Il peut contenir deux types de composants d'interface utilisateur, dont le Client FDI dispose pour l'affichage à l'utilisateur. Un Paquetage FDI peut contenir un seul type ou les deux. Ces deux types sont appelés Plugiciels d'Interface Utilisateur (UIPfn) et Descriptions d'Interface Utilisateur.

Un Plugiciel d'Interface Utilisateur (UIP) est un élément exécutable. Un UIP est fourni par un Paquetage FDI et transféré au Client FDI par le Serveur FDI. Un UIP fournit un ensemble de Services UIP que le Client FDI utilise pour initialiser et interagir avec l'UIP.

NOTE 1 L'IEC 62769-6 définit les interfaces de programmation d'application pour les services décrits dans le présent document.

Les Descriptions d'Interface Utilisateur (UID) sont définies en utilisant l'EDDL (langage de description d'appareil électronique). Un UID est fourni au Client FDI par le Serveur FDI. Le Client FDI utilise l'interprète UID pour interpréter et exécuter l'UID. Un UID peut utiliser d'autres composants d'UID et d'UIP en tant que sous-composants, afin de fournir une approche modulaire et de tirer le meilleur parti des deux éléments d'interface utilisateur, descriptif et exécutable.

NOTE 2 Les UIP peuvent utiliser d'autres UIP mais ne peuvent pas utiliser d'autres UID.

Le Serveur FDI met les UID et les UIP à la disposition du Client FDI via le Modèle d'Information. Le Modèle d'Information organise les UID et les UIP par Type d'Appareil.

Le Client FDI fournit l'environnement d'exécution pour les UIP. Le Client FDI charge l'UIP à partir du Serveur FDI.

L'environnement d'exécution de l'UIP du Client FDI se compose des ensembles de services suivants, mis à la disposition de l'UIP:

- Services d'Accès à l'Appareil;
- Services d'Hébergement;
- Services d'Interface Utilisateur;
- Services d'Impression (s'ils sont disponibles dans l'environnement d'hébergement).

NOTE 3 Le fait que différents UIP prennent la même ou différentes instances d'interface pour accéder à ces services, est spécifique à l'implémentation. La seule exigence est qu'il n'y ait pas d'effet secondaire si deux UIP utilisent la même instance d'une interface.

De la même manière que le Client FDI, chaque UIP doit également fournir un ensemble de services (Services UIP), à l'aide duquel le Client FDI active, contrôle et arrête les UIP (voir 6.1).

Les Services d'Accès à l'Appareil permettent l'interaction entre l'UIP et le Modèle d'Information maintenu par le Serveur FDI. Le Client FDI prend en charge l'interaction avec le Serveur FDI, libérant l'UIP et lui permettant de se concentrer sur le niveau application uniquement.

L'accès de l'UIP au Modèle d'Information (IM), via les Services d'Accès à l'Appareil, est restreint à l'Appareil et à ses sous-appareils.

Les Services d'Hébergement sont fournis par le Client FDI et sont utilisés par l'UIP. Les Services d'Hébergement comprennent les services relatifs au Client FDI, permettant à l'UIP d'obtenir des informations sur l'environnement.

Les Services d'Interface Utilisateur fournissent les moyens par lesquels l'UIP accède aux services d'interface utilisateur du système d'exploitation sous-jacent. Ces services fournissent l'accès à l'écran, au clavier, souris et aux autres ressources du système d'exploitation. Les Services d'Interface Utilisateur sont définis par la technologie d'implémentation choisie. Le présent document ne comporte donc pas de définition supplémentaire (voir l'IEC 62769-6). Les UIP doivent utiliser les Services d'Hébergement pour afficher les boîtes de dialogue ou les barres de progression. Ils ne doivent en aucun cas utiliser de services comparables fournis par le système d'exploitation sous-jacent.

Aucun service d'impression n'est fourni par le Client. Si un UIP nécessite de générer une impression, il accède aux services d'impression du système d'exploitation sous-jacent. Le présent document ne comporte pas de définition supplémentaire.

Le Client FDI utilise le paramètre culture de l'utilisateur actuellement connecté pour l'environnement d'exécution de l'UIP. Il utilise ce paramètre lors de la création des Sessions OPC UA et il définit la culture pour chaque fil qu'il crée. Les UIP doivent prendre en charge le paramètre culture pour chaque fils qu'ils créent.

Le Client FDI fournit un Interprète UID, utilisé pour interpréter et exécuter les UID. Le schéma XML de l'UID est défini dans le présent document (voir Annexe A).

La Logique Applicative est exécutée dans le Serveur FDI. Certaines Logiques Applicatives peuvent être exposées au Client FDI en tant qu'Actions (voir Article 7) et peuvent être déclenchées par les Clients FDI.

5 Client FDI

5.1 Services d'Accès à l'Appareil

5.1.1 Généralités

Les Services d'Accès à l'Appareil fournissent l'accès aux informations en ligne et hors ligne d'un Appareil ou ses composants, comme défini dans le Paquetage FDI, en particulier pour

- explorer le Modèle d'Appareil,
- lire et écrire des données et s'abonner aux modifications de données,
- contrôler l'accès à l'Appareil, et
- communiquer directement avec l'Appareil.

Le domaine d'application des Services d'Accès à l'Appareil sera un Appareil, un Bloc ou un Serveur de Communication, auquel un UIP est assigné. Le Client FDI est supposé mapper les Services d'Accès à l'Appareil avec les services de l'OPC UA, fournis par le Serveur FDI.

Les principaux services sont les Services du Modèle d'Appareil pour visualiser et accéder aux Paramètres. Les Services Locking (Verrouillage) sont utilisés pour contrôler l'accès simultané à un Appareil. Les Services Direct Access (Accès Direct) permettent à un UIP de communiquer avec l'Appareil.

L'IEC 62769-6 définit la manière dont les différents services sont mappés avec les interfaces réelles. L'IEC 62769-6 spécifie également la manière dont les interfaces sont obtenues.

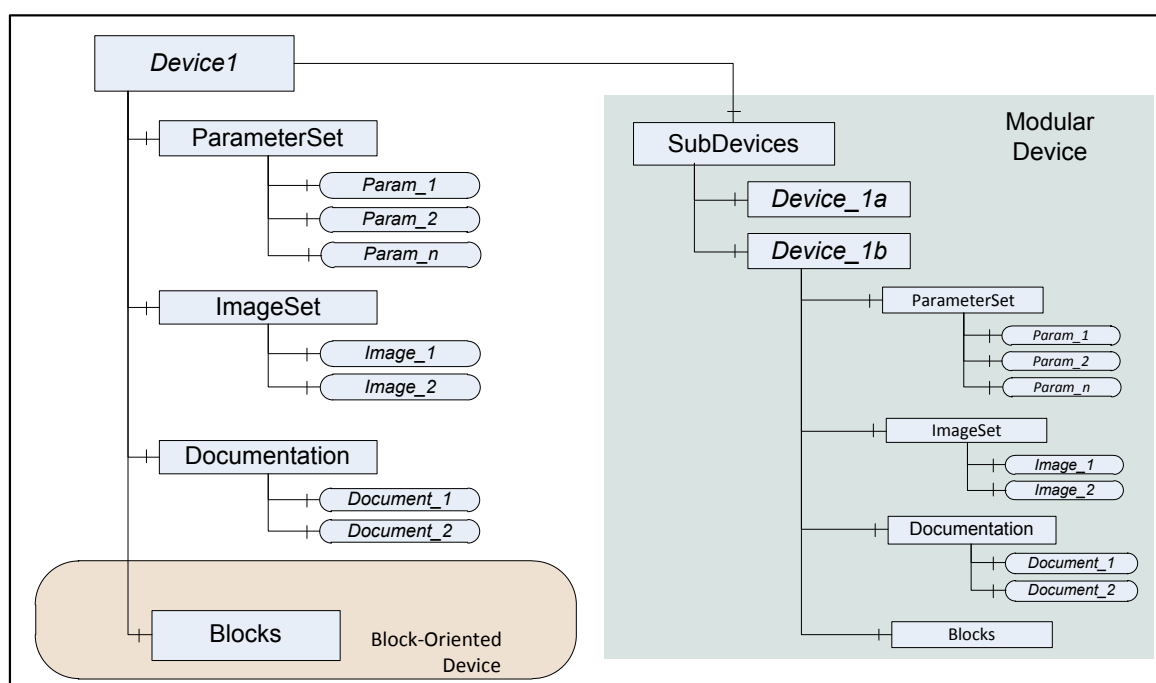
5.1.2 Modèle d'Appareil

Le Modèle d'Appareil (Device Model) définit la structure de toutes les données disponibles pour un UIP. Il est limité à une instance d'appareil unique. Les entités dans la structure (Paramètres, Images et Documents) sont construites à partir des informations du Paquetage

FDI. Les éléments d'Interface Utilisateur, tels que les Menus, Graphiques, Formes d'onde, ne font pas partie du Modèle d'Appareil de l'UIP.

Tous les éléments de l'Appareil sont organisés selon une hiérarchie définie. La racine de la hiérarchie peut être un Appareil ou un Bloc, sous réserve de l'emplacement (par quel MENU) dans lequel l'UIP est référencé dans la Description d'Interface Utilisateur du Paquetage FDI (voir l'IEC 62769-4). Les Nœuds dans la hiérarchie sont des Objets ou des Variables, la principale différence étant que les Variables fournissent une Valeur.

Figure 2 présente la structure générale d'un Appareil. La Figure 3 montre la structure d'un Bloc de manière plus détaillée. Les éléments qui seront réellement disponibles dépendent des contenus du Paquetage FDI respectif. Les rectangles représentent des Nœuds Objet, tandis que les rectangles dont les coins sont arrondis représentent des Nœuds Variable. Ces NodeClasses sont définies en 5.1.3. Le Nœud en haut à gauche est le Nœud racine. Les lignes coupées d'un trait définissent la relation parent-enfant dans la hiérarchie. Par exemple, les enfants de Device1 à la Figure 2 sont ParameterSet, ImageSet, Documentation, Blocks et SubDevices. Les enfants de /SubDevices/Device_1b/ImageSet sont Image_1 et Image_2.

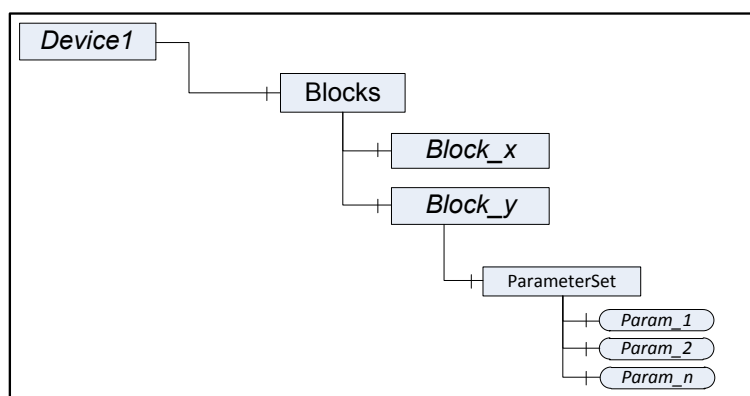


IEC

Anglais	Français
Block-Oriented Device	Appareil orienté Blocs
Modular Device	Appareil Modulaire
Blocks	Blocs
ParameterSet	ParameterSet (Ensemble de paramètres)
SubDevices	SubDevices (Sous-Appareils)
ImageSet	ImageSet (Ensemble d'images)

Figure 2 – Structure générale d'un Appareil

Les noms dans une police d'écriture normale sont définis par les Services d'Accès à l'Appareil; les noms en italiques représentent des réceptacles pour les noms réels, comme défini dans le Paquetage FDI.



IEC

Anglais	Français
Blocks	Blocs
ParameterSet	ParameterSet (Ensemble de paramètres)

Figure 3 – Structure des Blocs

Chaque Nœud dans la hiérarchie est qualifié de manière unique avec son nom de chemin d'accès. Ce nom de chemin d'accès est une concaténation des Attributs Nom de Nœud individuels. Le séparateur est représenté par "/".

EXEMPLE Les exemples suivants décrivent des noms de chemin d'accès qualifiés:

```

/                                     -- le Nœud racine (ici "Device1")

/ParameterSet/Param_2               -- un Nœud Variable

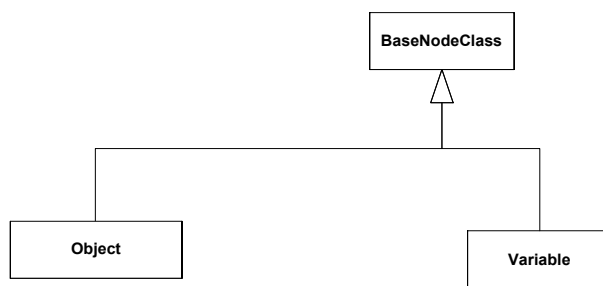
/SubDevices/Device_1b/ImageSet/Image_1  -- une image
    
```

Certaines Variables dans le Paquetage FDI peuvent être considérées comme étant "privées", signifiant qu'elles ne sont pas consultables. Bien qu'elles ne soient pas consultables, elles existent dans le Modèle d'Appareil et peuvent être adressées avec leur nom de chemin d'accès.

5.1.3 Modèle de Nœud

5.1.3.1 Généralités

Les informations dans un Appareil sont organisées sous forme de hiérarchie de Nœuds. Chaque Nœud est un Objet ou une Variable. Les Objets et les Variables sont dérivés de la BaseNodeClass, comme décrit à la Figure 4.



IEC

Anglais	Français
Object	Objet
Variable	Variable
BaseNodeClass	BaseNodeClass (Classe de Nœud de Base)

Figure 4 – NodeClasses du Modèle d'Appareil**5.1.3.2 BaseNodeClass**

Il s'agit d'une NodeClass parente abstraite des Nœuds Objet et Variable. Les Attributs de BaseNodeClass sont donnés dans le Tableau 1. Les Attributs de cette NodeClass sont disponibles dans les NodeClasses Objet et Variable.

Tableau 1 – Attributs de BaseNodeClass

Attribut	DataType	Description
NodePath	String	Chemin qualifié du Nœud dans le Modèle d'Appareil. Cet Attribut est renvoyé par le Service Browse (Explorer) et ne peut ni être lu ni être écrit.
Name	String	Nom du Nœud selon la documentation spécifique de l'appareil.
Label	LocalizedText	Étiquette d'un Nœud lisible par l'homme.
Description (facultatif)	LocalizedText	Chaîne d'aide lisible par l'homme pour décrire le Nœud.

Les Attributs supplémentaires seront disponibles pour les NodeClasses dérivées. Par exemple, une Variable aura des Attributs définissant le DataType (Type de Données) et les AccessRights (Droits d'Accès).

5.1.3.3 NodeClass Objet

Le Tableau 2 contient la liste des Attributs pour les Objets autres de ceux hérités de la BaseNodeClass:

Tableau 2 – Attributs de la NodeClass Objet

Attribut	DataType	Description
LockedStatus	Boolean	Sur "true", cet Attribut indique que cet Objet est actuellement verrouillé. Bad_AttributeInvalid signifie que le verrouillage n'est pas pris en charge par cet Objet.

5.1.3.4 NodeClass Variable**5.1.3.4.1 Généralités**

Les Variables sont utilisées pour représenter des Paramètres, des images et des documents. Lors de la lecture de Variables représentant des images ou des documents, le système les fournira sous la forme d'une ByteString (Chaîne d'Octets). Pour les documents, l'Attribut Name consistera en un nom de fichier ainsi que son extension, qui peut être utilisée pour identifier le type de document. La FDI prend en charge les fichiers en ".pdf et .txt". Pour la représentation des images, voir 5.1.3.4.2.

Le Tableau 3 contient la liste des Attributs pour les Variables autres de celles héritées de la BaseNodeClass:

Tableau 3 – Attributs de la NodeClass Variable

Attribut	DataType	Description
Value (Valeur)	Variant	Valeur de la Variable renvoyée par l'appareil (sans application de l'Attribut ScalingFactor). Le Datatype Variant est spécifié en 5.1.9.4.
DataType (Type de données)	UInt32	L'Attribut DataType spécifie le type de données de l'Attribut Value. Un des types de données spécifié en 5.1.9. L'IEC 62769-6 spécifie l'affectation d'identifiants uniques pour chaque type.
ValueRank (Rang de valeur)	Int32	Indique si l'Attribut Value est une matrice. Il peut prendre les valeurs suivantes: >1 (Plusieurs Dimensions) – la valeur est une matrice ayant le nombre spécifié de dimensions. 1 (Unidimensionnelle) – la valeur est une matrice unidimensionnelle. 0 (Une Ou Plusieurs Dimensions) – la valeur est une matrice à une ou plusieurs dimensions. -1 (Scalaire) – la valeur n'est pas une matrice. -2 (Indifférente) – la valeur peut être scalaire ou une matrice ayant n'importe quel nombre de dimensions. -3 (Scalaire Ou Unidimensionnelle) – la valeur peut être scalaire ou une matrice unidimensionnelle.
ArrayDimensions (Dimensions de matrice) (facultatif)	UInt32[]	Spécifie la longueur de chaque dimension pour une valeur matricielle. L'Attribut permet de décrire la capacité de la Variable et non la taille courante. Le nombre d'éléments doit être égal à la valeur de l'Attribut ValueRank. Doit être nul si ValueRank <= 0. Une valeur 0 pour une dimension particulière indique que la dimension a une longueur Variable. Par exemple, si une Variable est définie par la matrice C suivante: Int32 myArray[346]; alors ce Type de Données de la Variable pointerait vers une Int32, le ValueRank du Type de Variable prend la valeur 1 et l'Attribut ArrayDimensions indique une matrice dont l'une des entrées est à la valeur 346.
AccessRights	Byte	Les droits d'accès pour la Valeur. Une énumération avec l'une des valeurs suivantes: NONE_0 La valeur de la Variable ne peut pas être consultée READ_1 La valeur de la Variable peut être lue WRITE_2 La valeur de la Variable peut être écrite READORWRITE_3 La valeur de la Variable peut être lue ou écrite
UserAccessRights	Byte	Cet Attribut spécifie les droits d'accès à la Valeur pour l'utilisateur actuellement authentifié. Ils peuvent être inférieurs aux droits d'accès potentiels. La même énumération que pour AccessRights est utilisée.
ScalingFactor (facultatif)	Double	Cet Attribut spécifie un facteur scalaire suggéré. Noter que l'Attribut Value contient la valeur brute renvoyée depuis l'appareil. Il est admis que la valeur (brute) soit multipliée par ce facteur avant d'être affichée.
EngineeringUnits (facultatif)	EUInformation	EngineeringUnits spécifie les unités pour la valeur (ex.: °C, hertz, secondes). Voir 5.1.9.3.8 pour la définition du type de données EUInformation.
Attributs des Variables analogiques (Variables représentant des quantités physiques continuellement variables (ex.: pression, température).		
EURange (facultatif)	Range[]	Définit une ou plusieurs plages de valeurs qui peuvent être obtenues en fonctionnement normal. Destiné à un usage tel que la mise à l'échelle automatique d'un affichage en diagramme en bâtons. La défaillance ou la désactivation du capteur ou de l'instrument peut impliquer le renvoi d'une valeur d'élément en dehors de cette plage. Le logiciel UIP doit pouvoir traiter ce cas. Voir 5.1.9.3.7 pour la définition du type de données Range. Les plages peuvent changer pendant l'opération, par exemple, lors du changement du mode d'opération d'un instrument. Comme l'Attribut Value lui-même, les Attributs Range ne sont jamais échelonnés (sans application de l'Attribut ScalingFactor).
Attributs pour Variables discrètes (énumérées) (pour les données pouvant prendre uniquement un certain nombre de valeurs possibles (ex.: OPENING, OPEN, CLOSING, CLOSED).		
CurrentLabel	String	Les Variables énumérées exposent l'état numérique actuel de leur Attribut Value. L'Attribut CurrentLabel fournit le nom de la valeur d'énumération actuelle.
EnumValues	EnumValuesType[]	EnumValues est une matrice de {StateValue, Enumeration Name et Help

Attribut	Data Type	Description
		Information}. Voir 5.1.9.3.9 pour la définition de ce type. Les Clients FDI/UIP peuvent lire cet Attribut en avance et le stocker pour garder en référence le nom ou l'aide, lorsqu'ils reçoivent la représentation numérique.
Attributs pour Variable énumérée en bits (pour les données représentant un masque binaire).		
OptionNames	String[]	Les Variables énumérées en bits transmettent un masque binaire encodé dans un entier non signé d'une longueur suffisante pour représenter tous les bits. L'Attribut OptionNames fournit une représentation lisible par l'homme pour chaque bit valide du masque binaire. L'ordre des bits du masque binaire pointe vers une position de la matrice de Chaînes dans l'Attribut OptionNames, c'est-à-dire que le premier bit pointe vers la première entrée dans la matrice, et ainsi de suite. La matrice contient une Chaîne vide pour chaque bit n'ayant pas de signification spécifique.

5.1.3.4.2 Représentation des images

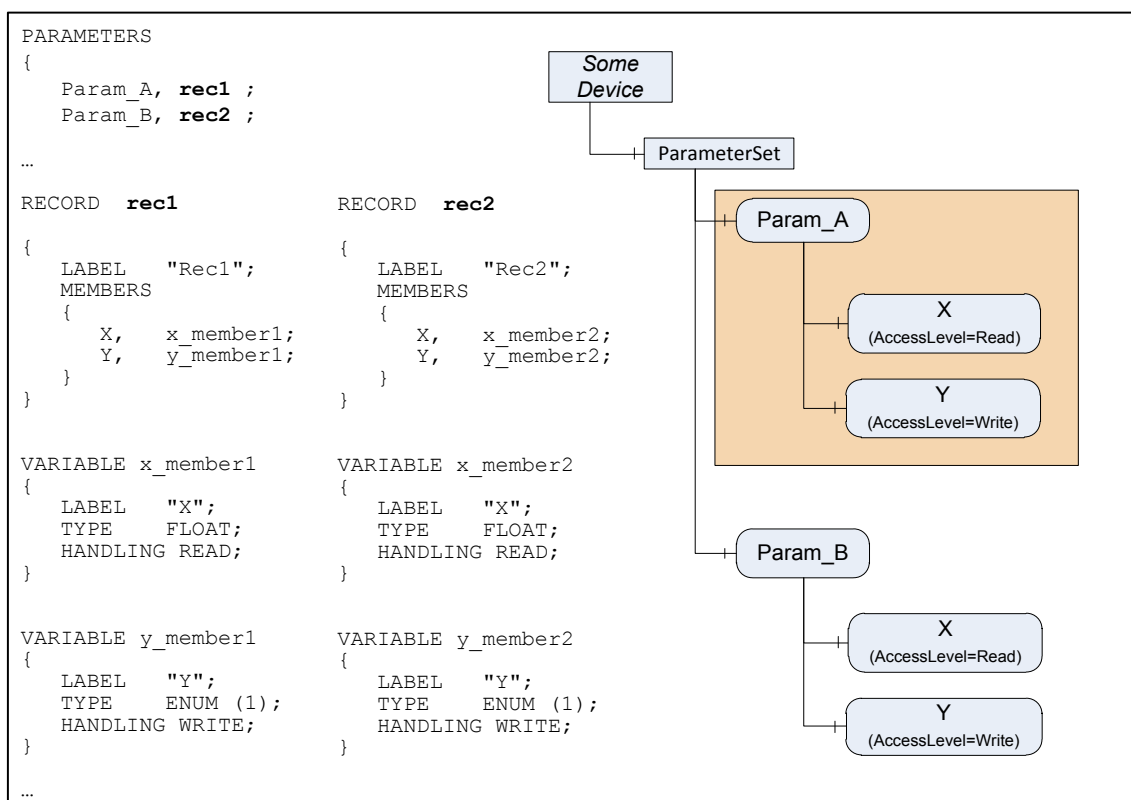
Toutes les images possèdent un Type de Données (DataType) et sont transférées sous la forme d'une ByteString (Chaîne d'octets). La FDI prend en charge trois formats d'image. Afin d'identifier le format de l'image fournie dans la ByteString, les premiers octets doivent être analysés comme indiqué dans le tableau suivant.

Type d'image	Description																		
GIF	Définit une image au format GIF (Graphics Interchange Format). Le format GIF est spécifié à l'adresse http://www.w3.org/Graphics/GIF/spec-gif89a.txt . Les premiers octets d'une image GIF sont les suivants: <table><tr><td>Byte</td><td>1</td><td>2</td><td>3</td></tr><tr><td>Hex</td><td>47</td><td>49</td><td>46</td></tr></table>	Byte	1	2	3	Hex	47	49	46										
Byte	1	2	3																
Hex	47	49	46																
JPG	Définit une image au format JPG (Joint Photographic Experts Group). Le format JPG est défini dans l'ISO/IEC 10918-1. Les premiers octets d'une image JPG sont les suivants: <table><tr><td>Byte</td><td>1</td><td>2</td><td>3</td><td>4</td></tr><tr><td>Hex</td><td>FF</td><td>D8</td><td>FF</td><td>E0</td></tr></table>	Byte	1	2	3	4	Hex	FF	D8	FF	E0								
Byte	1	2	3	4															
Hex	FF	D8	FF	E0															
PNG	Définit une image au format PNG (Portable Network Graphics format). Le format PNG est défini dans l'IETF RFC 2083 et l'ISO/IEC 15948. Les premiers octets d'une image PNG sont les suivants: <table><tr><td>Byte</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td></tr><tr><td>Hex</td><td>89</td><td>50</td><td>4E</td><td>47</td><td>0D</td><td>0A</td><td>1A</td><td>0A</td></tr></table>	Byte	1	2	3	4	5	6	7	8	Hex	89	50	4E	47	0D	0A	1A	0A
Byte	1	2	3	4	5	6	7	8											
Hex	89	50	4E	47	0D	0A	1A	0A											

5.1.3.4.3 Représentation des enregistrements

La hiérarchie de Variable est utilisée pour représenter les Paramètres RECORD de l'EDDL. La Variable racine représente l'enregistrement (record) lui-même. Elle comportera des composants Variable représentant des RECORD MEMBERS de l'EDDL (les RECORD MEMBERS de l'EDDL sont définis dans l'EDDL au moyen d'une référence à une VARIABLE EDDL).

La manière dont un enregistrement est représenté dans le Modèle d'Appareil est expliquée à la Figure 5.



IEC

Anglais	Français
PARAMETERS	PARAMETRES
RECORD	ENREGISTREMENT
LABEL	ETIQUETTE
MEMBERS	MEMBRES
VARIABLE	VARIABLE
TYPE FLOAT	TYPE FLOAT
HANDLING WRITE	TRAITEMENT ECRITURE
HANDLING READ	TRAITEMENT LECTURE
AccessLevel=Read	Niveau d'accès = Lecture
AccessLevel=Write	Niveau d'accès = Ecriture
Some Device	Un Appareil
ParameterSet	ParameterSet (Ensemble de paramètres)

Figure 5 – Exemple: Hiérarchie de la Variable représentant un RECORD

Les Attributs Name et Label de la Variable racine sont définis sur le nom de RECORD et l'Attribut LABEL, respectivement. L'Attribut DataType de la Variable "racine" est Variant (Variante). L'Attribut ValueRank est utilisé pour spécifier que la Valeur contient une matrice. L'Attribut Value représente les valeurs de tous les membres, dans l'ordre défini pour le RECORD. Selon l'exemple montré à la Figure 5, la première variante sera une valeur FLOAT et la seconde sera une valeur ENUM.

Pour chaque composant de Variable représentant un RECORD MEMBER de l'EDDL:

- l'Attribut Name est défini sur l'identifiant de la VARIABLE EDDL correspondante;
- le Label (étiquette) est l'Attribut LABEL de la VARIABLE EDDL correspondante;
- la Description est l'Attribut HELP de la VARIABLE EDDL correspondante;

- L'Attribut AccessRights est dérivé de l'Attribut HANDLING EDDL.

Chaque membre de l'enregistrement peut être accessible par son nom de chemin d'accès, comme spécifié ci-dessus. La navigation peut également être utilisée.

EXEMPLE Exemple de nom de chemin d'accès: /ParameterSet/Param_A/X.

5.1.3.4.4 Représentation des matrices et listes des membres avec types de données simples

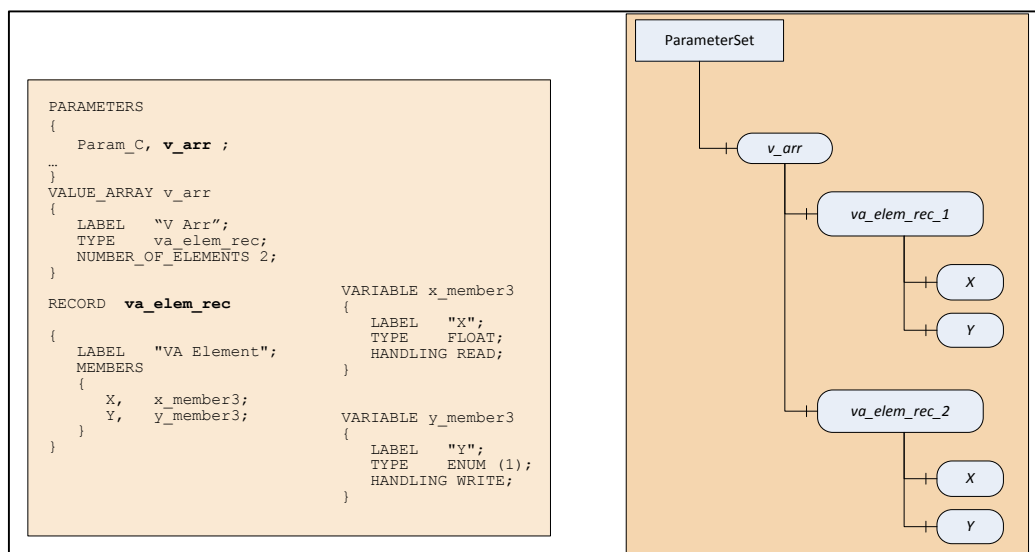
Une Variable unique représentera un élément VALUE_ARRAY ou LIST de l'EDDL lorsque le type de données de l'élément de matrice référencé comporte un type de données simple.

L'Attribut DataType est défini sur l'un des types de données de base (voir 5.1.9.2).

L'Attribut ValueRank est utilisé pour spécifier que la Valeur contient une matrice. Pour une VALUE_ARRAY de l'EDDL, le nombre des éléments est exposé via l'Attribut ArrayDimensions. Pour une LIST de l'EDDL, le nombre des éléments n'est pas spécifié, puisque la taille peut changer dynamiquement.

5.1.3.4.5 Représentation des matrices et des listes de RECORD

Les matrices ou listes de valeur des Paramètres de l'EDDL non simples seront représentées sous la forme d'une matrice de hiérarchies Variable. La Figure 6 présente le code d'échantillonnage EDDL d'une VALUE_ARRAY des RECORD et de la hiérarchie Variable correspondante.



IEC

Anglais	Français
ParameterSet	ParameterSet (Ensemble de paramètres)
PARAMETERS	PARAMETRES
RECORD	ENREGISTREMENT
LABEL	ETIQUETTE
MEMBERS	MEMBRES

Figure 6 –Hiérarchie Variable représentant une VALUE_ARRAY de RECORD

Les Attributs Name (Nom) et Label (Etiquette) de la Variable racine sont définis sur le nom d'ARRAY et l'Attribut LABEL, respectivement. L'Attribut DataType de la DataVariable "racine" est Variant (Variante). L'Attribut ValueRank est utilisé pour spécifier que la Valeur contient une matrice. L'Attribut Value représente toutes les entrées VALUE_ARRAY. La première Variant correspond à la première entrée de la matrice et ainsi de suite. Chaque Variant à son tour contient une matrice. Il peut s'agir soit d'une matrice de types simples ou d'une matrice de Variants. Un RECORD est toujours représenté sous la forme d'une matrice de Variants.

L'élément de VALUE_ARRAY, qui est en fait un RECORD, est représenté sous la forme d'un composant de hiérarchie Variable. Les Attributs Name et Label, de chaque Variable racine représentant un RECORD, sont définis sur le nom de RECORD et l'Attribut LABEL, respectivement. L'indice de la matrice (_1, _2) est accolé afin de permettre l'identification unique. A noter que l'indice commence toujours par '1'.

Les RECORD MEMBERS sont également représentés sous la forme de composants de Variables, comme spécifié en 5.1.3.4.3.

Les membres de chaque enregistrement peuvent être accessibles avec un nom de chemin d'accès. La navigation peut également être utilisée.

EXEMPLE Exemple de nom de chemin d'accès: /ParameterSet/v_arr/va_elem_rec_2/Y.

5.1.4 Services

5.1.4.1 Généralités

Tous les Services spécifiés dans la présente partie de l'IEC 62769 reposent sur les conventions définies en 5.1.4.2. Ils sont spécifiés d'une manière abstraite avec les paramètres request (demande) et response (réponse) dans un tableau unique.

L'accès programmé aux services peut être synchrone ou asynchrone (voir l'IEC 62769-6). Cependant, l'asynchronisme existe afin de maintenir la réactivité de l'Interface Utilisateur. Il est supposé que l'exécution d'un service doit toujours être séquentielle.

5.1.4.2 Conventions pour les définitions de service

Les spécifications des services utilisent des tableaux pour décrire les paramètres des services, comme indiqué au Tableau 4. Dans ce tableau, les paramètres sont organisés sous la forme de paramètres de demande et de paramètres de réponse.

Tableau 4 – Tableau de Définition des Services

Nom	Type	Description
Request (Demande)		Définit les paramètres de demande relatifs au Service
Nom de paramètre simple		Description de ce paramètre
Nom de paramètre construit	structure	Description du paramètre construit
Nom de paramètre composant		Description du paramètre composant
Response (Réponse)		Définit les paramètres de réponse relatifs au Service

Les colonnes Nom, Type et Description donnent le nom, le type de donnée et la description de chaque paramètre. Tous les paramètres sont obligatoires, même si certains peuvent être inutilisés dans certaines circonstances. La colonne Description spécifie la valeur à fournir lorsqu'un paramètre est inutilisé. Les noms de paramètres commencent toujours par une lettre minuscule. Cela permet de différencier un nom et un type, s'ils ont la même dénomination, par exemple, nom = "nodeId", type = "NodeId".

Deux types de paramètres sont définis dans ces tableaux, à savoir simple et construit. Les paramètres simples ont un type de donnée simple, tel que Boolean (Booléen) ou String (Chaîne).

Les paramètres construits sont des paramètres composés de deux ou plusieurs paramètres, qui peuvent être simples ou construits. Les noms des paramètres composants sont indentés sous le nom du paramètre construit.

Les types de données utilisés dans ces tableaux peuvent être des types de bases ou des types spécifiques à un service. Les types de données de base sont énumérés en 5.1.9. Les types de données qui sont spécifiques à un Service sont définis dans le tableau de paramètres du service en question.

5.1.4.3 Audit (Vérification)

La vérification est une exigence pour de nombreux systèmes. Elle fournit le moyen de suivre les activités qui se produisent dans le cadre du fonctionnement normal du système. Elle fournit également le moyen de surveiller les comportements anormaux. Il s'agit également d'une exigence en ce qui concerne la sécurité.

Lorsqu'une piste de vérification est maintenue par le système, les enregistrements de la piste de vérification pour le Service Write, invoqués par l'UIP, seront implicitement créés par le système.

De plus, les UIP possèdent le moyen de fournir des informations supplémentaires sur le contexte de la vérification, concernant des éléments que le système ne peut pas connaître:

- Ils peuvent appeler le service LogAuditTrailMessage (voir 5.2.2.5). Cela doit être accompli lorsque les Services DirectAccess sont utilisés. L'UIP doit inclure suffisamment d'informations dans le message pour décrire précisément l'activité exécutée.
- Ils peuvent fournir un texte de contexte lors de l'utilisation des Services de verrouillage (voir 5.1.7) ou des Services d'Accès Direct (voir 5.1.8).

5.1.4.4 Services et opérations

Plusieurs Services du Modèle d'Appareil (par exemple Read et Write) autorisent la spécification d'une matrice d'éléments qui doivent être traités. Le traitement de chaque élément individuel est appelé "opération". Il s'agit d'une différenciation importante pour la prise en charge des erreurs, puisque l'exécution d'un service est considérée comme étant réussie même en cas d'échec d'opérations individuelles. La liste suivante explique les différences entre les codes de résultat du service et de l'opération

- Code de résultat de service
Si le service réussit, le code de résultat indique "Good" et les paramètres de réponse sont valides. S'il échoue, le code de résultat de service est l'un des codes de défaillance du service, "Bad_...", définis en 5.1.4.6. Dans ce cas, un InnerErrorInfo peut être également fourni (voir 5.1.9.3.4). L'accès programmé aux codes de défaillance du service est spécifique à la technologie et peut être basé sur des exceptions (voir l'IEC 62769-6).
- Code de résultat de l'opération
Pour chaque opération, le code de résultat est renvoyé comme partie des paramètres de réponse spécifiques au service (voir 5.1.4.7 pour la liste des codes de résultat d'opération disponibles).

Les opérations peuvent renvoyer un InnerErrorInfo pour chaque code de résultat différent de "Good", si la demande de service indique returnInnerErrorInfo="true".

5.1.4.5 StatusCode (Code de statut)

Le StatusCode, utilisé pour les résultats de service ou les résultats opérationnels, rapporte le résultat d'une opération. Il s'agit d'un entier non signé de 32 bits. Les 16 bits de poids fort représentent la valeur numérique du code qui doit être utilisée pour détecter des erreurs ou des conditions spécifiques. Les 16 bits de poids faible sont des fanions binaires qui contiennent des informations supplémentaires mais n'altèrent pas la signification du StatusCode.

Tous les UIP doivent toujours vérifier le StatusCode associé à un résultat avant de l'utiliser. Les résultats auxquels est associé un statut incertain/warning (incertain/avertissement) doivent être utilisés avec prudence, car ces résultats peuvent ne pas être valides pour toutes les situations. Les résultats avec un statut bad/failed (mauvais/défaillant) ne doivent jamais être utilisés.

Les affectations exactes des bits sont montrées dans le Tableau 5. L'IEC 62769-6 fournit les fonctions contribuant à l'évaluation du StatusCode.

Tableau 5 – Affectations de bits de StatusCode

Champ	Plage de bit	Description															
Sévérité	30 .. 31	Indique si le StatusCode représente un état good (correct), bad (mauvais) ou incertain (incertain). Ces bits ont les significations suivantes: <table border="1"> <thead> <tr> <th>Sévérité</th><th>Bits</th><th>Description</th></tr> </thead> <tbody> <tr> <td>Good Success</td><td>00</td><td>L'opération s'est déroulée correctement; les résultats peuvent être utilisés.</td></tr> <tr> <td>Uncertain Warning</td><td>01</td><td>L'opération s'est en partie déroulée correctement; les résultats peuvent dans certains cas ne pas être appropriés.</td></tr> <tr> <td>Bad Failure</td><td>10</td><td>L'opération ne s'est pas déroulée correctement et les résultats associés ne peuvent pas être utilisés.</td></tr> <tr> <td>Reserved</td><td>11</td><td>Réservé pour usage ultérieur. Il convient de le traiter comme "Bad".</td></tr> </tbody> </table>	Sévérité	Bits	Description	Good Success	00	L'opération s'est déroulée correctement; les résultats peuvent être utilisés.	Uncertain Warning	01	L'opération s'est en partie déroulée correctement; les résultats peuvent dans certains cas ne pas être appropriés.	Bad Failure	10	L'opération ne s'est pas déroulée correctement et les résultats associés ne peuvent pas être utilisés.	Reserved	11	Réservé pour usage ultérieur. Il convient de le traiter comme "Bad".
Sévérité	Bits	Description															
Good Success	00	L'opération s'est déroulée correctement; les résultats peuvent être utilisés.															
Uncertain Warning	01	L'opération s'est en partie déroulée correctement; les résultats peuvent dans certains cas ne pas être appropriés.															
Bad Failure	10	L'opération ne s'est pas déroulée correctement et les résultats associés ne peuvent pas être utilisés.															
Reserved	11	Réservé pour usage ultérieur. Il convient de le traiter comme "Bad".															
Reserved	29 .. 28	Réservé pour usage ultérieur. Doit toujours être zéro.															
SubCode	16 .. 27	Le code est une valeur numérique affectée pour représenter différents états. Chaque code comporte un nom symbolique et une valeur numérique. Toutes les descriptions issues de la présente spécification se réfèrent au nom symbolique. L'IEC 62769-6 mappe les noms symboliques avec une valeur numérique.															
Reserved	12 .. 15	Réservé pour usage ultérieur. Doit toujours être zéro.															
InfoType	10 .. 11	Le type d'informations contenu dans les bits d'informations. Ces bits ont les significations suivantes: <table border="1"> <thead> <tr> <th>InfoType</th><th>Bits</th><th>Description</th></tr> </thead> <tbody> <tr> <td>NotUsed</td><td>00</td><td>Les bits d'information ne sont pas utilisés et doivent être positionnés à zéro.</td></tr> <tr> <td>DataValue</td><td>01</td><td>Le StatusCode et ses bits d'information sont associés avec une valeur de données renvoyée par le serveur FDI.</td></tr> <tr> <td>Reserved</td><td>1X</td><td>Réservé pour usage ultérieur. Les bits d'information doivent être ignorés.</td></tr> </tbody> </table>	InfoType	Bits	Description	NotUsed	00	Les bits d'information ne sont pas utilisés et doivent être positionnés à zéro.	DataValue	01	Le StatusCode et ses bits d'information sont associés avec une valeur de données renvoyée par le serveur FDI.	Reserved	1X	Réservé pour usage ultérieur. Les bits d'information doivent être ignorés.			
InfoType	Bits	Description															
NotUsed	00	Les bits d'information ne sont pas utilisés et doivent être positionnés à zéro.															
DataValue	01	Le StatusCode et ses bits d'information sont associés avec une valeur de données renvoyée par le serveur FDI.															
Reserved	1X	Réservé pour usage ultérieur. Les bits d'information doivent être ignorés.															
InfoBits	0 .. 9	Les bits d'information supplémentaires qui dépendent du champ Info Type.															

Le Tableau 6 décrit la structure des InfoBits lorsque l'Info Type est mis à DataValue (01).

Tableau 6 – InfoBits de DataValue

Info Type	Plage de bit	Description															
LimitBits	8 .. 9	Les bits de limites associés à la valeur de données. Les bits de limites ont les significations suivantes: <table border="1"> <thead> <tr> <th>Limite</th><th>Bits</th><th>Description</th></tr> </thead> <tbody> <tr> <td>Aucune</td><td>00</td><td>La valeur peut être modifiée.</td></tr> <tr> <td>Faible</td><td>01</td><td>La valeur est à la limite inférieure pour la source de données.</td></tr> <tr> <td>Haute</td><td>10</td><td>La valeur est à la limite supérieure pour la source de données.</td></tr> <tr> <td>Constante</td><td>11</td><td>La valeur est constante et ne peut pas être modifiée.</td></tr> </tbody> </table>	Limite	Bits	Description	Aucune	00	La valeur peut être modifiée.	Faible	01	La valeur est à la limite inférieure pour la source de données.	Haute	10	La valeur est à la limite supérieure pour la source de données.	Constante	11	La valeur est constante et ne peut pas être modifiée.
Limite	Bits	Description															
Aucune	00	La valeur peut être modifiée.															
Faible	01	La valeur est à la limite inférieure pour la source de données.															
Haute	10	La valeur est à la limite supérieure pour la source de données.															
Constante	11	La valeur est constante et ne peut pas être modifiée.															
Overflow	7	Si ce bit est positionné, les changements détectés n'ont pas tous été renvoyés depuis que le tampon de file d'attente du Serveur FDI pour la Variable d'abonnement a atteint sa limite et a dû purger des données.															
Reserved	0 .. 6	Réservé pour usage ultérieur. Doit toujours être zéro.															

5.1.4.6 Codes de défaillance de service

Le Tableau 7 définit les codes de résultat de service. La colonne Service dans le Tableau 7 répertorie quel code peut être renvoyé par quel service. Les codes de résultat spécifiques à un service individuel sont également spécifiés avec le service.

Tableau 7 – Codes de résultat de service

Code de nom Exception	Description	Service
Bad_AlreadyLocked	Le Nœud est déjà verrouillé par un autre Client FDI.	InitLock
Bad_LockRequired	Le Nœud transmis n'est pas encore verrouillé.	Write, DirectAccess
Bad_MaxAgeInvalid	Le paramètre âge maximal n'est pas valide.	Read (Lecture)
Bad_NodeInvalid	L'identifiant ne se réfère pas à un Nœud valide dans le Modèle d'Appareil. Ce code de résultat est utilisé comme code de résultat du niveau service et du niveau opération.	Browse, InitLock, ExitLock
Bad_NothingToDo	Il n'y avait aucune tâche à réaliser, car l'appelant a transmis une liste d'opérations sans éléments.	Read, Write, Subscribe, Unsubscribe
Bad_NotSupported	Le Service n'est pas pris en charge pour le Nœud spécifié ou pour l'Appareil/Bloc en général.	Locking, DirectAccess
Bad_RequestCancelled	La requête a été annulée par le Client/l'UIP.	All
Bad_SubscriptionIdInvalid	L'identifiant d'abonnement n'est pas valide.	Subscribe, Unsubscribe, DeleteSubscription
Bad_Timeout	L'opération s'est interrompue après le délai.	All
Bad_TooManySubscriptions	Le Serveur FDI a atteint le nombre maximal d'abonnements.	CreateSubscription
Bad_InvalidState	Le Nœud spécifié est dans un état ne permettant pas cette opération.	Services pour Locking
Bad_TooManyOperations	La requête n'a pas pu être traitée, car elle spécifiait trop d'opérations.	Read, Write, Subscribe
Bad_UnexpectedError	Une erreur inattendue s'est produite.	All

5.1.4.7 Codes de statut opérationnels

Le Tableau 8 définit les codes de statut pour tous les résultats du niveau opération (pour les services ayant des opérations individuelles, telles que Read, Write ou Subscribe). Chaque service définit le sous-ensemble applicable et contiendra des références avec ce tableau, à la place d'une description.

NOTE La valeur dans le Tableau 8 ne fait pas référence à la qualité de communication, mais à la valeur Paramètre.

Tableau 8 – Codes de résultat du niveau opération

Code de résultat du niveau opération	Description
Good	L'opération s'est déroulée avec succès.
Good_LocalOverride	La valeur a été remplacée.
Good_PostActionFailed	La valeur d'une variable a été lue ou écrite avec succès, mais l'une des post-actions a échoué.
Good_Edited	La Valeur a été modifiée dans l'EditContext et n'est pas encore dans l'Appareil.
Good_DependentValueChanged	La Valeur d'une Variable dépendante a été modifiée mais pas encore appliquée.
Uncertain	L'opération s'est terminée, cependant ses résultats peuvent ne pas être utilisables.
Uncertain_NoCommunicationLastValue	La communication vers la source de données a échoué. La valeur de Variable est la dernière valeur dont la qualité est bonne.
Uncertain_LastUsableValue	Quel que soit le moyen par lequel cette valeur a été mise à jour, ledit moyen n'assurera plus son rôle.
Uncertain_SubstituteValue	La valeur est une valeur opérationnelle qui a été écrasée en écriture manuellement.
Uncertain_InitialValue	La valeur est la valeur initiale d'une Variable qui reçoit normalement sa valeur d'une autre Variable.
Uncertain_SensorNotAccurate	La valeur est sur une des limites du capteur.
Uncertain_EngineeringUnitsExceeded	La valeur est située hors de la plage de valeurs définie pour ce paramètre.
Uncertain_SubNormal	La valeur est dérivée de sources multiples et comporte un nombre de sources correctes inférieur au nombre exigé.
Uncertain_DominantValueChanged	Une modification d'une valeur dominante (ex.: unité technique) a provoqué l'invalidité d'une valeur dépendante.
Bad	L'opération a échoué.
Bad_AttributeInvalid	L'identifiant d'Attribut n'est pas pris en charge pour le Nœud spécifié.
Bad_ConfigurationError	Un problème avec la configuration compromet l'utilité de la valeur.
Bad_DeviceFailure	Une défaillance de l'appareil / la source de données a généré la valeur.
Bad_IndexRangeInvalid	Le paramètre IndexRange comporte une syntaxe non valide.
Bad_NodeInvalid	L'identifiant ne se réfère pas à un Nœud valide dans le Modèle d'Appareil. Ce code de résultat est utilisé comme code de résultat du niveau service et du niveau opération.
Bad_NotConnected	Il convient que la Variable reçoive sa valeur à partir d'une autre Variable, mais n'a pas été configurée pour ce faire.
Bad_NotReadable	Le niveau d'accès ne permet pas la lecture ou l'abonnement au Nœud.
Bad_NotWritable	Le niveau d'accès ne permet pas l'écriture sur le Nœud.
Bad_OutOfRange	La valeur était hors plage.
Bad_OutOfService	La source des données n'est pas opérationnelle.
Bad_SensorFailure	Une défaillance s'est produite au niveau du capteur à partir duquel la valeur est dérivée par l'appareil / la source de données.
Bad_TypeMismatch	La valeur fournie n'est pas du même type que la valeur de la Variable.
Bad_UIPHandleInvalid	Le traitement ne fait pas référence à un Attribut Nœud d'abonnement.
Bad_UserAccessDenied	L'utilisateur n'a pas la permission d'effectuer l'opération demandée.
Bad_WaitingForInitialData	En attente de l'obtention des valeurs par le Serveur FDI de la part de la source de données sous-jacente. Après l'abonnement aux Variables, un certain temps peut être nécessaire pour délivrer les valeurs. Dans ces cas, une mise à jour initiale peut être envoyée avec ce statut avant la Notification avec la première valeur valide.

5.1.5 Services Base Property

5.1.5.1 Vue d'ensemble

Les Services Base Property (Propriété de base) fournissent l'accès aux propriétés de base d'Accès à l'Appareil.

5.1.5.2 Get DeviceAccess Interface Version

5.1.5.2.1 Description

Ce service renvoie la version de l'interface utilisée par l'UIP.

5.1.5.2.2 Paramètres

Le Tableau 9 définit les paramètres du service.

Tableau 9 – Paramètres du Service GetDeviceAccessInterfaceVersion

Nom	Type	Description
Request (Demande)		
Response (Réponse)		
version	String	Version de Technologie FDI de l'interface DeviceAccess. Le format de la valeur est xx.yy.zz, comme défini dans l'IEC 62769-4.

5.1.5.2.3 Résultats de service

Il n'y a pas de résultats de service autres que les codes communs spécifiés en 5.1.4.6.

5.1.5.3 GetOnlineAccessAvailability

5.1.5.3.1 Description

Ce service renvoie un élément de réponse pour savoir si l'accès en ligne est disponible en principe. Il s'agit d'une information générale. Il ne précise pas si un appareil spécifique peut être accessible en ligne.

5.1.5.3.2 Paramètres

Le Tableau 10 définit les paramètres du service.

Tableau 10 – Paramètres du Service GetOnlineAccessAvailability

Nom	Type	Description
Request (Demande)		
Response (Réponse)		
onlineAccess	Boolean	Sur "false", cette valeur spécifie que l'accès en ligne n'est pas disponible. "true" indique une disponibilité de base.

5.1.5.3.3 Résultats de service

Il n'y a pas de résultats de service autres que les codes communs spécifiés en 5.1.4.6.

5.1.6 Services du Modèle d'Appareil

5.1.6.1 Vue d'ensemble

Les Services du Modèle d'Appareil comprennent:

- Browse (Explorer)
- Read (Lecture)
- Write (Ecriture)

- Subscription (Abonnement)
 - CreateSubscription (Créer un abonnement)
 - Subscribe (S'abonner)
 - Unsubscribe (Se désabonner)
 - DeleteSubscription (Supprimer un abonnement)

Les services fournissent l'accès à la représentation hors ligne et en ligne de l'Appareil. Les Nœuds en ligne d'un Appareil sont toujours présents. Cependant, l'accès aux données dont la communication est exigée peut être rejeté avec les codes de statut adéquats, tels que Bad_NotConnected.

NOTE Le modèle en ligne ne contient pas SubDevices. Si l'UIP utilise un chemin comprenant SubDevices pour accéder au Nœud en ligne, ce chemin sera évalué dans la hiérarchie hors ligne.

Les services Cancel (Annuler) sont disponibles pour Browse, Read et Write. La définition et les mécaniques exactes dépendent de la technologie utilisée pour l'implémentation de ces services.

5.1.6.2 Browse (Explorer)

5.1.6.2.1 Description

Permet de parcourir un niveau unique dans la hiérarchie du Modèle d'Appareil et renvoie les Attributs pour tous les enfants du Nœud spécifié.

5.1.6.2.2 Paramètres

Le Tableau 11 définit les paramètres du service.

Tableau 11 – Paramètres du Service Explorer

Nom	Type	Description
Request (Demande)		
nodeToBrowse	NodeSpecifieur	L'identifiant du Nœud à explorer. Voir 5.1.9.3.2 pour la définition du type de NodeSpecifieur.
Response (Réponse)		
browseResult[]	structure	Liste des Nœuds situés dans le prochain niveau inférieur de la hiérarchie. Pour chaque Nœud, les Attributs suivants seront renvoyés.
nodePath	String	Voir BaseNodeClass en 5.1.3.2.
name	String	Voir BaseNodeClass en 5.1.3.2.
label	LocalizedText	Voir BaseNodeClass en 5.1.3.2.

5.1.6.2.3 Résultats de service

Aucun code de résultat de Service spécifique n'est défini pour Browse. Les StatusCodes communs sont définis dans le Tableau 7.

5.1.6.3 CancelBrowse

5.1.6.3.1 Description

L'appel de CancelBrowse indique que l'UIP ne s'intéresse plus aux résultats de ce service. L'exécution sera interrompue dès que possible.

Cancel (Annuler) est une suggestion pour le système. Dû à son exécution asynchrone, le service peut déjà avoir été entièrement ou partiellement complété.

5.1.6.3.2 Paramètres

Le Tableau 12 définit les paramètres du service.

Tableau 12 – Paramètres du Service CancelBrowse

Nom	Type	Description
Request (Demande)		
serviceld	<dépendant de la technologie>	L'identifiant du service à annuler.
Response (Réponse)		

5.1.6.3.3 Résultats de service

Aucun code de résultat de Service spécifique n'est défini pour CancelBrowse. Les requêtes de Browse annulées avec succès doivent répondre à Bad_RequestCancelled. Les StatusCodes communs sont définis dans le Tableau 7.

5.1.6.4 Read (Lecture)

5.1.6.4.1 Description

Ce service est utilisé pour lire les Attributs des Nœuds Objet ou Variable. Les UIP, devant surveiller les Attributs de Variable pour modifications, doivent utiliser les services Subscription (Abonnement) ou lieu de Read (Lire).

5.1.6.4.2 Paramètres

Le Tableau 13 définit les paramètres du service.

Tableau 13 – Paramètres du Service Read

Nom	Type	Description
Request (Demande)		
returnInnerErrorInfo	Boolean	Une valeur "true" demande que les informations relatives aux erreurs, provenant d'appels vers un système sous-jacent, soient renvoyées lorsque disponibles. "false" indique que ces informations ne doivent pas être renvoyées.
attributesToRead[]	Structure	Les Attributs à lire.
nœud	NodeSpecifieur	L'identifiant du Nœud contenant l'Attribut à lire. Voir 5.1.9.3.2 pour la définition du type de NodeSpecifieur.
attributeld	Attributeld	Identifiant numérique de l'Attribut à lire. Voir 5.1.9.3.3.
indexRange	NumericRange	Ce paramètre est utilisé pour identifier un élément simple d'une matrice ou une plage simple d'indices pour les matrices. Si une plage d'éléments est spécifiée, les valeurs sont renvoyées sous forme de composés. Le premier élément est identifié par l'indice 0 (zéro). Ce paramètre est ignoré si l'Attribut spécifié n'est pas une matrice ou une structure. Cependant, si l'Attribut spécifié est une matrice ou une structure, et que ce paramètre est nul (Nul ou Chaîne vide), alors tous les éléments doivent être inclus dans la plage. Pour obtenir une définition détaillée, voir 5.1.9.3.6.
maxAge	UInt32	Âge maximal de la valeur à lire en millisecondes. Si le Serveur FDI comporte une valeur plus récente que maxAge en mémoire cache, il renverra la valeur placée en cache au lieu de demander une nouvelle valeur à l'appareil. Si maxAge est mis à 0, le Serveur FDI doit lire une nouvelle valeur dans la source de données. Les valeurs supérieures à $2^{31} - 1$ (0x7fff ffff) ne sont pas valides pour maxAge.

Nom	Type	Description
Response (Réponse)		
readResult []	DataValue	Le StatusCode, la Valeur et les Timestamps (Marqueurs temporels) pour chaque Attribut Nœud ayant été lu. L'ordre de cette liste correspond à l'ordre du paramètre de demande attributesToRead. L'élément DataValue est spécifié en 5.1.9.3.3.
innerErrorInfos []	InnerErrorInfo	Liste des informations relatives aux erreurs, provenant d'appels d'un système sous-jacent. Voir 5.1.9.3.4. Correspond à la taille et à l'ordre du paramètre de demande attributesToRead. Cette liste est vide si les informations d'erreurs internes n'ont pas été demandées ou si aucune information n'a été rencontrée dans le traitement de la demande.

5.1.6.4.3 Résultats de service

Le Tableau 14 définit les valeurs pour le code de résultat de service. D'autres StatusCodes communs sont définis dans le Tableau 7.

Tableau 14 – Codes de résultat du service Read

Codes de résultat	Description
Bad_MaxAgeInvalid	Le paramètre âge maximal n'est pas valide.

5.1.6.4.4 Codes de résultat de l'opération

Le Tableau 15 définit les valeurs pour le code de statut de niveau opération contenu dans la DataValue de chaque élément readResult. Tous les codes de statut opérationnels avec leur description sont présents dans le Tableau 8.

Tableau 15 – Codes de résultat de l'opération Read

Codes de résultat
Good
Good_LocalOverride
Good_PostActionFailed
Good_DependentValueChanged
Uncertain
Uncertain_NoCommunicationLastValue
Uncertain_LastUsableValue
Uncertain_SubstituteValue
Uncertain_InitialValue
Uncertain_SensorNotAccurate
Uncertain_EngineeringUnitsExceeded
Uncertain_SubNormal
Uncertain_DominantValueChanged
Bad
Bad_UserAccessDenied
Bad_ConfigurationError
Bad_NotConnected
Bad_DeviceFailure
Bad_SensorFailure
Bad_OutOfRange
Bad_OutOfService
Bad_NodeInvalid
Bad_AttributeInvalid
Bad_IndexRangeInvalid
Bad_NotReadable

5.1.6.5 CancelRead

5.1.6.5.1 Description

L'appel de CancelRead indique que l'UIP ne s'intéresse plus aux résultats de ce service. L'exécution sera interrompue dès que possible.

Cancel (Annuler) est une suggestion pour le système. Dû à son exécution asynchrone, le service peut déjà avoir été entièrement ou partiellement complété.

5.1.6.5.2 Paramètres

Le Tableau 16 définit les paramètres du service.

Tableau 16 – Paramètres du Service CancelRead

Nom	Type	Description
Request (Demande)		
serviceld	<dépendant de la technologie>	L'identifiant du service à annuler.
Response (Réponse)		

5.1.6.5.3 Résultats de service

Aucun code de résultat de Service spécifique n'est défini pour CancelRead. Les requêtes de Read annulées avec succès doivent répondre à Bad_RequestCancelled. Les StatusCodes communs sont définis dans le Tableau 7.

5.1.6.6 Write (Ecriture)

5.1.6.6.1 Description

Ce Service est utilisé pour écrire des valeurs dans une ou plusieurs Variables. Pour les valeurs matricielles, ce service autorise l'écriture de la matrice entière, l'écriture des éléments individuels ou l'écriture des plages d'éléments.

Le verrouillage explicite est exigé (voir 5.1.7). Si le Nœud n'est pas verrouillé, la demande sera rejetée avec Bad_LockRequired.

La réponse de service n'est pas renvoyée avant l'exécution par le système de l'opération d'écriture. Le retour en arrière est du ressort du client FDI / UIP.

Les valeurs doivent avoir le type de données correct.

NOTE Aucune traduction automatique du type de données n'est communiquée (voir le code de résultat de l'opération Bad_TypeMismatch).

5.1.6.6.2 Paramètres

Le Tableau 17 définit les paramètres du service.

Tableau 17 – Paramètres du Service Write

Nom	Type	Description
Request (Demande)		
returnInnerErrorInfo	Boolean	Une valeur "true" demande que les informations relatives aux erreurs, provenant d'appels vers un système sous-jacent, soient renvoyées lorsque disponibles. "false" indique que ces informations ne doivent pas être renvoyées.
variablesToWrite[]	structure	Liste des Variables à écrire. Il convient de ne poser aucune hypothèse sur l'ordre de traitement de cette liste. Si l'ordre a une importance, des demandes de service séparées doivent être utilisées.
nœud	NodeSpecifieur	L'identifiant du Nœud contenant l'Attribut à écrire. Voir 5.1.9.3.2 pour la définition du type de NodeSpecifieur.
indexRange	NumericRange	Pour obtenir une définition détaillée, voir 5.1.9.3.6.
value	Variant	Valeur à écrire.
Response (Réponse)		
writeResult[]	UInt32	Les codes de statut pour les résultats d'opération sont définis dans le Tableau 18. L'ordre de cette liste correspond à l'ordre du paramètre de demande variablesToWrite.
innerErrorInfos []	InnerErrorInfo	Liste des informations relatives aux erreurs, provenant d'appels d'un système sous-jacent. Voir 5.1.9.3.4. Correspond à la taille et à l'ordre du paramètre de demande variablesToWrite. Cette liste est vide si les informations d'erreurs internes n'ont pas été demandées ou si aucune information n'a été rencontrée dans le traitement de la demande.

5.1.6.6.3 Résultats de service

Il n'y a pas de résultats de service autres que les codes communs spécifiés en 5.1.4.6.

5.1.6.6.4 Codes de résultat de l'opération

Le Tableau 18 définit les valeurs pour le code de statut de niveau opération contenu dans les éléments writeResult. Tous les codes de statut opérationnels avec leur description sont présents dans le Tableau 8.

Tableau 18 – Codes de résultat de l'opération Write

Codes de résultat
Good
Good_PostActionFailed
Bad
Bad_UserAccessDenied
Bad_NodeInvalid
Bad_IndexRangeInvalid
Bad_TypeMismatch
Bad_OutOfRange
Bad_NotWritable

5.1.6.7 CancelWrite

5.1.6.7.1 Description

L'appel de CancelWrite indique que l'UIP ne s'intéresse plus aux résultats de ce service. L'exécution sera interrompue dès que possible.

Cancel (Annuler) est une suggestion pour le système. Dû à son exécution asynchrone, le service peut déjà avoir été entièrement ou partiellement complété.

5.1.6.7.2 Paramètres

Le Tableau 19 définit les paramètres du service.

Tableau 19 – Paramètres du Service CancelWrite

Nom	Type	Description
Request (Demande)		
serviceld	<dépendant de la technologie>	L'identifiant du service à annuler.
Response (Réponse)		

5.1.6.7.3 Résultats de service

Aucun code de résultat de Service spécifique n'est défini pour CancelWrite. Les requêtes de Write annulées avec succès doivent répondre à Bad_RequestCancelled. Les StatusCodes communs sont définis dans le Tableau 7.

5.1.6.8 Subscriptions (Abonnements)

5.1.6.8.1 Mécanisme d'Abonnement

Les Abonnements permettent à l'UIP de recevoir des rappels (callbacks) non sollicités de la part du Client FDI, lorsque les Attributs Nœud d'abonnement changent. Les Abonnements représentent une manière plus efficace pour obtenir des mises à jour périodiques des données, qu'en effectuant des appels répétés au service Read, c'est-à-dire l'interrogation. L'UIP crée un abonnement en appelant le service CreateSubscription (voir 5.1.6.8.2). Lorsque le service CreateSubscription est appelé, l'UIP doit fournir un paramètre de rappel avec le service DataChangeCallback spécifique à l'UIP (voir 5.1.6.8.6). Après avoir reçu l'identifiant d'abonnement dans la réponse CreateSubscription, l'UIP doit ajouter les Attributs Nœud d'intérêt à l'Abonnement en appelant le service Subscribe (s'Abonner).

Une fois que les valeurs initiales ont été reportées, le service DataChangeCallback est appelé uniquement lorsque les Attributs Nœud changent et seuls les Attributs Nœud ayant été modifiés sont reportés dans le rappel. L'UIP contrôle la fréquence maximale, pour laquelle il convient d'invoquer le rappel, en spécifiant une fréquence en millisecondes.

Une fois l'abonnement aux Attributs Nœud effectué, leurs valeurs peuvent ne pas être disponibles immédiatement et peuvent le devenir à des moments différents. A cet effet, le rappel initial peut ne pas inclure tous les Attributs Nœud d'abonnement. En outre, la mise à jour initiale d'un Attribut Nœud d'abonnement pourra renvoyer un StatusCode Bad ou Uncertain.

En outre, la durée entre le moment où un changement est effectué et l'invocation du rappel peut dépendre de l'endroit où est maintenue l'information à changer. Certaines valeurs sont maintenues par le système, ce qui permet la notification immédiate; d'autres sont situées dans l'appareil physique, la communication doit donc être établie pour accéder à ces valeurs.

5.1.6.8.2 Service CreateSubscription

5.1.6.8.2.1 Description

Ce Service est utilisé pour créer un abonnement.

5.1.6.8.2.2 Paramètres

Le Tableau 20 définit les paramètres du service.

Tableau 20 – Paramètres du Service CreateSubscription

Nom	Type	Description
Request (Demande)		
requestedUpdateRate	UInt32	La fréquence la plus rapide, en millisecondes, à laquelle l'UIP demande à être rappelé pour les modifications de données, spécifiée par la période la plus courte en millisecondes s'écoulant entre les mises à jour. Un rappel est effectué uniquement si des données sont modifiées, peu importe la fréquence demandée. Une fréquence de "0" indique que l'appelant demande à être notifié des modifications dès que possible. Le service renverra un résultat avec la fréquence la plus rapide possible de revisedUpdateRate.
dataChangeCallback	DataChangeCallback	Rappel pour l'envoi vers l'UIP de mises à jour de modifications de données. Voir 5.1.6.8.6. DataChangeCallback est un service implémenté et fourni par l'UIP.
Response (Réponse)		
revisedUpdateRate	UInt32	La fréquence actuelle que le Serveur FDI utilisera, exprimée par la période minimale en millisecondes s'écoulant entre les mises à jour (si les données ont été modifiées depuis la dernière mise à jour).
subscriptionId	SubscriptionId	L'identifiant assigné à l'appel pour l'abonnement. Le type SubscriptionId dépend de la technologie.

5.1.6.8.2.3 Résultats de service

Le Tableau 21 définit les valeurs pour le résultat de service. Les résultats communs sont définis dans le Tableau 7.

Tableau 21 – Codes de résultat du Service CreateSubscription

Codes de résultat	Description
Bad_TooManySubscriptions	Le Serveur FDI a atteint le nombre maximal d'abonnements.

5.1.6.8.3 Service Subscribe (S'abonner)

5.1.6.8.3.1 Description

Ce service est utilisé pour ajouter un ou plusieurs Attributs Nœud à un abonnement existant.

L'abonnement peut être utilisé pour tous les Attributs Nœud.

5.1.6.8.3.2 Paramètres

Le Tableau 22 définit les paramètres du service.

Tableau 22 – Paramètres du Service Subscribe

Nom	Type	Description
Request (Demande)		
subscriptionId	SubscriptionId	L'identifiant d'un abonnement existant ayant été renvoyé par le service CreateSubscription.
returnInnerErrorInfo	Boolean	Une valeur "true" demande que les informations relatives aux erreurs, provenant d'appels vers un système sous-jacent, soient transmises dans DataChangeCallbacks, lorsque disponibles. "false" indique que ces informations ne doivent pas être renvoyées.

Nom	Type	Description
Request (Demande)		
monitoredItemsToAdd[]	structure	Les Attributs à ajouter à l'abonnement.
nœud	NodeSpecifier	L'identifiant du Nœud contenant l'Attribut auquel s'abonner. Voir 5.1.9.3.2 pour la définition du type de NodeSpecifier.
attributId	AttributId	Identifiant numérique de l'Attribut auquel s'abonner. Voir 5.1.9.3.3.
indexRange	NumericRange	Ce paramètre est utilisé pour identifier un élément simple d'une matrice ou une plage simple d'indices pour les matrices. Si une plage d'éléments est spécifiée, les valeurs sont renvoyées sous forme de composés. Le premier élément est identifié par l'indice 0 (zéro). Ce paramètre est ignoré si l'Attribut spécifié n'est pas une matrice ou une structure. Cependant, si l'Attribut spécifié est une matrice ou une structure, et que ce paramètre est nul, alors tous les éléments doivent être inclus dans la plage. Pour obtenir une définition détaillée, voir 5.1.9.3.6.
samplingInterval	Int32	L'intervalle qui définit la fréquence la plus rapide à laquelle il convient d'accéder et d'évaluer l'Attribut. Cet intervalle est défini en millisecondes. La valeur 0 indique qu'il convient que le Serveur FDI utilise la fréquence pratique la plus rapide. La valeur -1 indique que l'intervalle d'échantillonnage par défaut défini par l'UpdateRate d'un Abonnement est utilisé. Voir 5.1.6.8.3.3 pour plus de détails sur l'intervalle d'échantillonnage.
uiPHandle	UInt32	Un identifiant (handle) fourni par l'UIP pour l'Attribut Nœud d'abonnement. Cet identifiant sera transmis avec les données dans le service DataChangeCallback, afin que l'UIP puisse facilement associer chaque valeur modifiée avec l'Attribut Nœud d'abonnement. L'uiPHandle est probablement un index dans un tableau situé quelque part. Il ne doit pas forcément être unique (plusieurs éléments d'abonnement pourraient pointer vers la même entrée de tableau).
Response (Réponse)		
subscribeResult []	structure	Liste des résultats pour les Attributs d'abonnement. La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de demande attributesToSubscribe.
statusCode	UInt32	Le code de statut pour l'Attribut respectif auquel s'abonner, comme défini dans le Tableau 23.
monitoredItemId	UInt32	Identifiant assigné au Serveur FDI pour l'Attribut d'abonnement. Cet identifiant est unique au sein de l'Abonnement et doit être utilisé en appelant Unsubscribe. Ce paramètre est présent seulement si le statusCode indique que l'abonnement à l'Attribut s'est déroulé avec succès.
revisedSamplingInterval	Int32	L'intervalle d'échantillonnage actuel qui sera utilisé. Cette valeur est fondée sur plusieurs facteurs, y compris les fonctions du système sous-jacent.

L'Abonnement sera réussi même si l'utilisateur actuel n'est pas autorisé à accéder à l'Attribut Nœud. Si c'est le cas, le rappel initial renverra le résultat d'opération "Bad_UserAccessDenied". Une fois ce refus traité (ex.: après avoir fourni l'identité d'un utilisateur plus puissant), l'UIP recevra les modifications de données pour cet Attribut Nœud.

N'importe quel Attribut, et pas uniquement la Valeur, peut être sujet à un abonnement. Bien que cela puisse ne pas avoir de sens pour tous les Attributs, la surveillance de certains Attributs apporte des possibilités supplémentaires. Certains exemples peuvent être cités:

- surveiller le LockedStatus pour déterminer lorsque le verrou est retiré par un autre client;
- surveiller le CurrentLabel des Enumérations pour recevoir un nom pouvant être affiché, plutôt qu'une valeur numérique.

5.1.6.8.3.3 Intervalle d'échantillonnage

Chaque élément d'abonnement reçoit un intervalle d'échantillonnage qui lui est affecté et qui est hérité d'updateRate de l'Abonnement, ou alors défini spécifiquement pour se substituer à

cette fréquence. L'intervalle d'échantillonnage indique la fréquence la plus rapide à laquelle il convient que la valeur soit échantillonnée dans l'appareil pour les modifications de données.

L'intervalle d'échantillonnage affecté définit une fréquence cyclique de "meilleur effort", utilisée pour échantillonner l'élément à partir de sa source. "Meilleur effort" signifie dans ce contexte que le système fait de son mieux pour échantillonner à cette fréquence. L'échantillonnage à des fréquences plus rapides que celle-ci est acceptable, mais n'est pas indispensable pour satisfaire aux besoins de l'UIP. La façon dont le système traite la fréquence d'échantillonnage et la fréquence à laquelle il sonde effectivement ses sources de données en interne sont un détail d'implémentation du système. Cependant, le délai entre les valeurs renvoyées vers l'UIP doit être supérieur ou égal à l'intervalle d'échantillonnage.

Le Client FDI peut aussi spécifier 0 pour l'intervalle d'échantillonnage, ce qui indique qu'il convient pour le système d'utiliser la fréquence pratique la plus rapide. Il est escompté que les systèmes prendront en charge uniquement un jeu limité d'intervalles d'échantillonnage pour optimiser leur fonctionnement. Si l'intervalle exact demandé par l'UIP n'est pas pris en charge, l'intervalle le plus approprié, tel que déterminé par le système, sera alors attribué et renvoyé vers l'UIP.

Les données peuvent être recueillies sur la base d'un modèle d'échantillonnage ou générées sur la base d'un modèle piloté par des exceptions. L'intervalle d'échantillonnage pris en charge le plus rapide peut être égal à 0, ceci indique que l'élément de données est piloté par des exceptions au lieu d'être échantillonné périodiquement. Lorsqu'il est piloté par des exceptions, le système sous-jacent n'exige pas d'échantillonnage.

Pour de nombreux cas, le système n'a pas connaissance de la logique de mise à jour des données. Dans ce cas, même si le système échantillonne à la fréquence négociée, les données pourraient être mises à jour par le système sous-jacent à une fréquence beaucoup plus lente. Dans ce cas, les modifications ne peuvent être détectées qu'à cette fréquence plus lente.

Il convient que les UIP sachent que l'échantillonnage par le système et le cycle de mise à jour de l'appareil ne sont pas, en général, synchronisés, ce qui peut induire des retards supplémentaires.

5.1.6.8.3.4 Résultats de service

Il n'y a pas de résultats de service autres que les codes communs spécifiés en 5.1.4.6.

5.1.6.8.3.5 Codes de résultat de l'opération

Le Tableau 23 définit les valeurs pour le code de statut de niveau opération contenu dans les résultats. Tous les codes de statut opérationnels avec leur description sont présents dans le Tableau 8.

Tableau 23 – Codes de résultat de l'opération Subscribe

Codes de résultat
Good
Bad
Bad_NodeInvalid
Bad_AttributeInvalid
Bad_NotReadable
Bad_UserAccessDenied

5.1.6.8.4 Service Unsubscribe (se désabonner)

5.1.6.8.4.1 Description

Ce service est utilisé pour se désabonner d'un ou de plusieurs Attributs Nœud.

5.1.6.8.4.2 Paramètres

Le Tableau 24 définit les paramètres du service.

Tableau 24 – Paramètres du service Unsubscribe

Nom	Type	Description
Request (Demande)		
subscriptionId	SubscriptionId	L'identifiant d'un abonnement existant ayant été renvoyé par le service CreateSubscription.
monitoredItemIds[]	UInt32	Identifiant pour les Attributs d'abonnement ayant été renvoyés par le service Subscribe.
Response (Réponse)		
unsubscribeResult[]	UInt32	Les codes de statut pour les résultats d'opération sont définis dans le Tableau 25. L'ordre de cette liste correspond à l'ordre du paramètre de demande monitoredItemIds.

5.1.6.8.4.3 Résultats de service

Il n'y a pas de résultats de service autres que les codes communs spécifiés en 5.1.4.6.

5.1.6.8.4.4 Codes de résultat de l'opération

Le Tableau 25 définit les valeurs pour le code de statut de niveau opération contenu dans l'élément unsubscribeResult. Tous les codes de statut opérationnels avec leur description sont présents dans le Tableau 8.

Tableau 25 – Codes de résultat de l'opération Unsubscribe

Codes de résultat
Good
Bad
Bad_UIPHandleInvalid

5.1.6.8.5 Service DeleteSubscription

5.1.6.8.5.1 Description

Ce Service est utilisé pour supprimer un abonnement. De par la nature asynchrone des rappels, l'UIP peut recevoir des rappels supplémentaires pour un abonnement, après la suppression de l'abonnement.

5.1.6.8.5.2 Paramètres

Le Tableau 26 définit les paramètres du service.

Tableau 26 – Paramètres du Service DeleteSubscription

Nom	Type	Description
Request (Demande)		
subscriptionId	SubscriptionId	L'identifiant d'un abonnement existant ayant été renvoyé par le service CreateSubscription.
Response (Réponse)		

5.1.6.8.5.3 Résultats de service

Il n'y a pas de résultats de service autres que les codes communs spécifiés en 5.1.4.6.

5.1.6.8.6 Service DataChangeCallback

5.1.6.8.6.1 Description

Ce service est utilisé pour envoyer des mises à jour de modification de données vers l'UIP. Ce service est implémenté et fourni par l'UIP lors de l'appel du service CreateSubscription.

De par la nature asynchrone des rappels, l'UIP peut recevoir des rappels supplémentaires pour un abonnement, après la suppression de l'abonnement.

5.1.6.8.6.2 Paramètres

Le Tableau 27 définit les paramètres du service.

Tableau 27 – Paramètres du Service DataChangeCallback

Nom	Type	Description
Request (Demande)		
subscriptionId	SubscriptionId	Identifiant renvoyé par le service CreateSubscription.
dataChangeData[]	structure	Données ayant été modifiées. Aucun ordre spécifique n'est assuré pour les éléments matriciels.
uiPHandle	UInt32	Le traitement fourni par l'UIP dans le service Subscribe.
value	DataValue	Le StatusCode, la Valeur et les Timestamps de l'Attribut Nœud d'abonnement. L'élément DataValue est spécifié en 5.1.9.3.3.
innerErrorInfos []	InnerErrorInfo	Liste des informations relatives aux erreurs, provenant d'appels d'un système sous-jacent. Voir 5.1.9.3.4. Correspond à la taille et à l'ordre du paramètre de demande dataChangeData. Cette liste est vide si les informations d'erreurs internes n'ont pas été demandées ou si aucune information n'a été rencontrée dans le traitement de la demande.
Response (Réponse)		

5.1.6.8.6.3 Codes de résultat de l'opération

Le Tableau 28 définit les valeurs pour le code de statut de niveau opération contenu dans la structure DataChangeData de chaque élément des valeurs. De plus, tous les codes de statut de niveau opération Read s'appliquent également (voir Tableau 15). Tous les codes de statut opérationnels avec leur description sont présents dans le Tableau 8.

Tableau 28 – Codes de résultat de DataChangeCallback

Codes de résultat
Bad_WaitingForInitialData

5.1.7 Services de verrouillage

5.1.7.1 Vue d'ensemble

Le Locking (verrouillage) est le moyen d'éviter des modifications simultanées sur un Appareil ou un Bloc et ses Paramètres. Les UIP doivent utiliser les services de verrouillage avant d'effectuer des modifications (ex.: les opérations d'écriture et les Services d'Accès Direct).

Le verrou s'applique toujours à la fois sur les versions en ligne et hors ligne.

Lors du verrouillage d'un Appareil Modulaire, le verrou s'applique à l'Appareil complet (y compris tous les modules). De la même manière, lors du verrouillage d'un Appareil orienté Bloc, le verrou s'applique à l'Appareil complet (y compris tous les Blocs).

Si aucun verrou n'est appliqué à l'Appareil de niveau supérieur (Appareil Modulaire ou Appareil orienté Bloc), les sous-Appareils ou les Blocs, respectivement, peuvent être verrouillés indépendamment.

Avec le verrouillage appliqué, les demandes provenant d'autres Clients FDI pour écrire des Paramètres, ou pour réaliser un Accès Direct, seront rejetées.

Le verrou est retiré en appelant ExitLock.

5.1.7.2 Service InitLock

5.1.7.2.1 Description

InitLock réserve l'Appareil ou le Bloc spécifié. Lorsqu'un verrou sera appliqué, les autres Clients FDI ne pourront pas écrire les Paramètres de cet élément. Un verrou appliqué sur un élément déjà verrouillé par un autre Client FDI sera rejeté. Un UIP peut s'abonner à l'Attribut LockedStatus afin d'être informé lors du retrait du verrou par un autre Client.

Les Clients FDI doivent permettre les appels InitLock "imbriqués" provenant du même UIP. Le Client FDI prévoit le même nombre d'appels ExitLock avant de retirer effectivement le verrou.

Les Clients FDI sont chargés d'empêcher des verrous simultanés, vers un Appareil ou un Bloc, par des composants individuels hébergés par ce Client. De tels composants incluent le Client lui-même, des UIP indépendants ou l'interprète UID.

5.1.7.2.2 Paramètres

Le Tableau 29 définit les paramètres du service.

Tableau 29 – Paramètres du Service InitLock

Nom	Type	Description
Request (Demande)		
nœud	NodeSpecifier	L'identifiant du Nœud (représentant un appareil ou un bloc) à verrouiller. Voir 5.1.9.3.2 pour la définition du type de NodeSpecifieur.
context	String	Une chaîne utilisée pour fournir les informations de contexte à propos de l'activité actuelle dans l'UIP. Elle sera utilisée pour améliorer la piste de vérification.
Response (Réponse)		

5.1.7.2.3 Résultats de service

Le Tableau 30 définit les valeurs pour le code de résultat de service. D'autres StatusCodes communs sont définis dans le Tableau 7.

Tableau 30 – Codes de résultat du Service InitLock

Codes de résultat	Description
Bad_NotSupported	Le Nœud ne prend pas en charge le verrouillage.
Bad_AlreadyLocked	Le Nœud est déjà verrouillé par un autre Client FDI ou par un autre composant individuel au sein du Client FDI.

5.1.7.3 Service ExitLock

5.1.7.3.1 Description

ExitLock supprime le verrou.

5.1.7.3.2 Paramètres

Le Tableau 31 définit les paramètres du service.

Tableau 31 – Paramètres du Service ExitLock

Nom	Type	Description
Request (Demande)		
nœud	NodeSpecifier	L'identifiant du Nœud (représentant un appareil ou un bloc) à déverrouiller. Voir 5.1.9.3.2 pour la définition du type de NodeSpecifier.
Response (Réponse)		

5.1.7.3.3 Résultats de service

Le Tableau 32 définit les valeurs pour le code de résultat de service. D'autres StatusCodes communs sont définis dans le Tableau 7.

Tableau 32 – Codes de résultat du Service ExitLock

Codes de résultat	Description
Bad_InvalidState	Le Nœud n'est pas verrouillé.

5.1.8 Services d'Accès Direct

5.1.8.1 Vue d'ensemble

Les Services Direct Access (Accès Direct) fournissent la communication directe avec un Appareil. Cela peut être utilisé pour les opérations qui ne peuvent pas, ou difficilement, être effectuées à travers les Services du Modèle d'Appareil. Les cas d'utilisation comprennent la transmission de paquets de données de grande taille depuis ou vers l'Appareil, par exemple, des données historiques ou un microprogramme. Il convient que les Services d'Accès Direct n'influencent pas l'intégrité de la structure et des données du Modèle d'Appareil. La prise en charge des Services d'Accès Direct est obligatoire pour les Hôtes FDI. Néanmoins, les Systèmes Hôtes doivent fournir des moyens de désactiver/d'activer les Services d'Accès Direct sur demande. Les ingénieurs de mise en service ou les opérateurs d'usine peuvent alors interdire l'usage de l'Accès Direct dans certains cas. Cette interdiction peut être temporaire ou même permanente et pourrait s'appliquer à une partie ou à l'ensemble des éléments de l'usine. Par conséquent, les UIP ne doivent pas dépendre de la disponibilité de

L'Accès Direct pour la configuration initiale d'un Appareil (ex.: nécessaire pour la mise en service).

Le comportement suivant est appliqué lors de l'utilisation de Direct Access:

- Seul un Client FDI ou un UIP peut utiliser DirectAccess sur une période donnée. En mode Accès Direct, les autres Clients ne pourront pas accéder à cet Appareil (même en mode Lecture).
- Cet Appareil doit être verrouillé avant de pouvoir entrer dans ce mode.
- Si les Attributs de Variable peuvent avoir été modifiés, dû à DirectAccess, l'UIP doit définir InvalidateCache sur "true" dans l'argument EndDirectAccess.

Les Services Direct Access comprennent:

- InitDirectAccess
- Transfer
- ExitDirectAccess

Ligne directrice:

Ces services ne doivent pas être utilisés comme alternative à l'accès au Modèle d'Information. A la place, DirectAccess est utilisé pour le transfert de données qui ne sont pas réfléchies dans le Modèle d'Information.

5.1.8.2 InitDirectAccess

5.1.8.2.1 Description

Ce service initialise l'Appareil pour l'utilisation de DirectAccess.

5.1.8.2.2 Paramètres

Le Tableau 33 définit les paramètres du service.

Tableau 33 – Paramètres du Service InitDirectAccess

Nom	Type	Description
Request (Demande)		
context	String	Une chaîne utilisée pour fournir les informations de contexte à propos de l'activité actuelle dans l'UIP. Elle sera utilisée pour améliorer la piste de vérification.
Response (Réponse)		

5.1.8.2.3 Résultats de service

Le Tableau 34 définit les valeurs pour le code de résultat de service. D'autres StatusCodes communs sont définis dans le Tableau 7.

Tableau 34 – Codes de résultat du Service InitDirectAccess

Codes de résultat	Description
Bad_NotSupported	DirectAccess n'est (actuellement) pas pris en charge.
Bad_LockRequired	Le Nœud n'a pas été verrouillé.
Bad_InvalidState	DirectAccess est déjà initialisé.

5.1.8.3 ExitDirectAccess

5.1.8.3.1 Description

Ce service met un terme à l'utilisation de DirectAccess.

5.1.8.3.2 Paramètres

Le Tableau 35 définit les paramètres du service.

Tableau 35 – Paramètres du Service ExitDirectAccess

Nom	Type	Description
Request (Demande)		
InvalidateCache	Boolean	Sur "true", n'importe quelle valeur mise en cache pour les Paramètres de l'Appareil sera invalidée. Cela signifie que ces Paramètres seront lus à nouveau depuis l'Appareil lors de leur prochaine utilisation.
Response (Réponse)		

5.1.8.3.3 Résultats de service

Le Tableau 36 définit les valeurs pour le code de résultat de service. D'autres StatusCodes communs sont définis dans le Tableau 7.

Tableau 36 – Codes de résultat du Service ExitDirectAccess

Codes de résultat	Description
Bad_InvalidState	L'appareil n'est pas dans le mode DirectAccess.

5.1.8.4 Transfer

5.1.8.4.1 Description

Ce service est utilisé pour transmettre des données vers et depuis l'Appareil. Le format des données reçues ou envoyées est spécifique au protocole.

De par sa nature, Direct Access ne permet pas la génération automatique d'informations pour la Piste de vérification, conformes aux nombreuses réglementations. Ainsi, les UIP doivent invoquer le service LogAuditTrailMessage (voir 5.2.2.5) afin de fournir des informations explicites sur les éléments transmis.

Pour les autres appels de service affectant indirectement l'Appareil, via le Modèle d'Appareil (Write), les paramètres du service fournissent suffisamment d'informations.

5.1.8.4.2 Paramètres

Le Tableau 37 définit les paramètres du service.

Tableau 37 – Paramètres du Service Transfer

Nom	Type	Description
Request (Demande)		
sendData	String	Document XML basé sur le TransferSendDataT comme spécifié dans le schéma XML spécifique au profil de communication.
Response (Réponse)		
receiveData	String	Document XML basé sur le TransferResultDataT comme spécifié dans le schéma XML spécifique au profil de communication.

5.1.8.4.3 Résultats de service

Le Tableau 38 définit les valeurs pour le code de résultat de service. D'autres StatusCodes communs sont définis dans le Tableau 7.

Tableau 38 – Codes de résultat du Service Transfer

Codes de résultat	Description
Bad_InvalidState	L'appareil n'est pas dans le mode DirectAccess.

5.1.9 Types de données

5.1.9.1 Généralités

Le Paragraphe 5.1.9 spécifie les types de données utilisés pour les paramètres de Service, les valeurs de Variable et les valeurs d'autres Attributs Nœud. Elles peuvent prendre la forme scalaire ou de matrice.

5.1.9.2 Type de données de base

Le Tableau 39 répertorie les types de données de base, c'est-à-dire les types généralement pris en charge nativement par les langages de programmation.

Tableau 39 – Types de données de base

Data Type	Description
Boolean	Définit une valeur qui peut être positionnée sur "true" ou "false".
String	Représente du texte sous la forme de séries de caractères Unicode. La représentation réelle de la chaîne dépend du mapping de la technologie. Voir IEC 62769-6.
ByteString (Chaîne d'octets)	Une valeur représentant une séquence de valeurs d'octets précédée par une longueur de 32 bits.
UtcTime	Une valeur de DateTime (Date et Heure) utilisée pour définir le Temps Universel Coordonné (UTC). Toutes les valeurs de temps sont des valeurs UTC. Les Clients FDI doivent fournir les éventuelles conversions entre UTC et heure locale. UtcTime est un entier signé 64 bits représentant le nombre d'intervalles de 100 nanosecondes depuis le 1 ^{er} janvier 1601. Il correspond à la structure FILETIME de Windows.
Int8	Un entier signé compris entre -128 et 127 inclus.
Int16	Un entier signé compris entre -32 768 et 32 767 inclus.
Int32	Un entier signé compris entre -2 147 483 648 et 2 147 483 647 inclus.
Int64	Un entier signé compris entre -9 223 372 036 854 775 808 et 9 223 372 036 854 775 807 inclus.
Byte	Une valeur dans une plage de 0 à 255.
UInt16	Un entier signé compris entre 0 et 65 535 inclus.
UInt32	Un entier signé compris entre 0 et 4 294 967 295 inclus.

Data Type	Description
UInt64	Un entier signé compris entre 0 et 18 446 744 073 709 551 615 inclus.
Float	Définit une valeur qui doit être conforme à la définition du type de données à simple précision de la norme IEEE 754.
Double	Définit une valeur qui doit être conforme à la définition du type de données à double précision de la norme IEEE 754.
Duration	Même représentation que Double.

5.1.9.3 Types spéciaux

5.1.9.3.1 Attributelds

Les Attributelds sont représentés sous la forme d'UInt32. Le Tableau 40 répertorie les Attributs et leurs identifiants.

Tableau 40 – Identifiants assignés aux Attributs

Attribut	Identifiant
Name (Nom)	10
Label (Etiquette)	11
Description	12
	...
LockedStatus (Statut verrouillé)	30
	...
Value (Valeur)	100
Data Type (Type de données)	101
ValueRank (Rang de valeur)	102
ArrayDimensions (Dimensions de matrice)	103
AccessRights (Droits d'Accès)	105
UserAccessRights (Droits d'Accès Utilisateur)	106
ScalingFactor (Facteur d'échelle)	107
	...
EURange (Plage d'Unités Techniques)	111
EngineeringUnits (Unités Techniques)	112
	...
EnumValues (Valeurs d'Enumération)	120
CurrentLabel (Etiquette Actuelle)	121
OptionNames (Noms facultatifs)	122

5.1.9.3.2 NodeSpecifier

Chaque Nœud dans le Modèle d'Appareil (voir 5.1.2) est adressable uniquement avec son nom de chemin d'accès qualifié par un spécificateur en ligne/hors ligne. Pour la spécification en ligne, le service doit fonctionner dans la version en ligne du Modèle d'Appareil. Pour la spécification hors ligne, il doit fonctionner dans le modèle hors ligne.

Le nom de chemin d'accès suit la hiérarchie du modèle d'appareil, comme présenté en 5.1.2. Il est le résultat de la concaténation des noms individuels, séparés par des barres obliques ("/"). Voir 5.1.2 pour des exemples de noms de chemin d'accès. Tous les paramètres sont identifiés via `"/ParameterSet/<ParamName>"`, toutes les images via `"/ImageSet/<ImageName>"`, et ainsi de suite.

Les Paragraphes 5.1.3.4.3 et 5.1.3.4.5 spécifient la représentation des paramètres qui détiennent des valeurs d'enregistrement ou des matrices de valeurs d'enregistrement. Un nom de chemin d'accès pour les éléments d'enregistrement est construit en prolongeant le chemin jusqu'au Paramètre. Des exemples sont disponibles dans les sections référencées. Les composants de ce paramètre sont définis dans le Tableau 42.

Tableau 41 – NodeSpecifie

Nom	Type	Description
NodeSpecifie	structure	Spécifie un Nœud dans le modèle d'appareil.
nodePath	String	La description du chemin permettant de trouver un Nœud dans le Modèle d'Information.
useOnline	Boolean	Sur "true", le chemin adresse un Nœud dans le modèle en ligne; sur "false", un Nœud dans le modèle hors ligne est référencé.

Un nom de chemin d'accès vide identifie la Racine.

5.1.9.3.3 DataValue

Ce type décrit la valeur d'un Attribut Nœud dans la réponse de Read et dans DataChangeCallback. Les composants de ce paramètre sont définis dans le Tableau 42.

Tableau 42 – DataValue

Nom	Type	Description
DataValue	structure	La valeur et les informations associées.
value	Variant	La valeur de l'Attribut Nœud. Pour la définition de Variant, voir 5.1.9.4.
statusCode	UInt32	Le StatusCode qui définit la capacité à accéder/fournir la valeur. Le type de StatusCode est spécifié en 5.1.4.5.
sourceTimestamp	UtcTime	Marqueur temporel de la source pour la valeur.
serverTimestamp	UtcTime	Marqueur temporel du serveur pour la valeur.

Le statusCode est utilisé pour indiquer les conditions dans lesquelles une valeur d'Attribut Nœud a été générée et, de ce fait, peut être utilisé comme un indicateur de validité de la valeur.

Le StatusCode de tous les résultats doit être vérifié (au minimum sa Sévérité) avant d'accéder à sa valeur et de l'utiliser.

Le sourceTimestamp reflète le marqueur temporel qui avait été appliqué par la source de données. Le sourceTimestamp est seulement renvoyé avec l'Attribut Value des Nœuds Variable. Pour tous les autres Attributs, l'élément sourceTimestamp renvoyé est configuré sur nul.

Dans le cas d'un statut incertain ou mauvais, sourceTimestamp est utilisé pour refléter l'heure à laquelle la source a reconnu le statut autre que good (correct) ou l'heure de la dernière fois que le Serveur FDI a essayé une reprise sur statut incertain ou mauvais.

Le serverTimestamp est utilisé pour refléter l'heure à laquelle le Serveur FDI a reçu une valeur de Variable ou a su qu'elle était exacte. Dans le cas d'un statut incertain ou mauvais, serverTimestamp est utilisé pour refléter l'heure à laquelle le Serveur FDI a reçu le statut ou l'heure correspondant à la dernière fois que le Serveur FDI a essayé une reprise sur statut incertain ou mauvais.

Le serverTimestamp est mis à jour pour chaque nouvelle valeur reçue.

5.1.9.3.4 InnerErrorInfo

Les Services décrits en 5.1 renvoient des StatusCode des niveaux service et opération. Ces StatusCodes sont indépendants vis-à-vis du protocole et de l'appareil. Dans le cas où un StatusCode provient d'un appel vers le système sous-jacent (ex.: un système, un appareil de communication), l'état du système sous-jacent peut être rapporté via InnerErrorInfo.

Les composants de ce paramètre sont définis dans le Tableau 43.

Tableau 43 – InnerErrorInfo

Nom	Type	Description
InnerErrorInfo	structure	Informations spécifiques à la communication ou à l'Appareil.
symbolicId	String	Cette chaîne doit être utilisée afin d'identifier le résultat de certaines opérations internes. La longueur maximale de cette chaîne est de 32 caractères. Il convient que les systèmes souhaitant renvoyer un code de retour numérique convertissent le code de retour en chaîne et utilisent cette chaîne en tant que symbolicId (ex.: "0xC0040007" ou "-4").
errorText	LocalizedText	Une représentation textuelle de l'identifiant symbolique. La longueur maximale de cette chaîne textuelle est de 256 caractères.

5.1.9.3.5 LocalizedText

Ce type de données définit une structure contenant une Chaîne dans une traduction spécifique à une locale, définie dans l'identifiant de la locale. Ses éléments sont définis dans le Tableau 44.

Tableau 44 – Définition de LocalizedText

Nom	Type	Description
LocalizedText	structure	—
texte	String	Le texte localisé.
locale	String	L'identifiant de la locale (par exemple "en-US").

La locale de l'élément doit être une chaîne constituée d'un composant langue et d'un composant pays/région, conformément à la norme RFC 3066. Le composant <pays/région> est toujours précédé d'un tiret. Le format de la chaîne de caractères de LocaleId est présenté ci-dessous:

<langue>[-<pays/région>], dans lequel
 <langue> doit être un code à deux lettres identifiant une langue conformément à l'ISO 639,
 <pays/région> doit être un code à deux lettres identifiant le pays/la région conformément à l'ISO 3166.

Les règles de construction de LocaleIds doivent être conformes à la norme RFC 3066 et doivent être limitées de la manière suivante:

- La présente spécification permet uniquement l'utilisation du zéro ou d'un composant <pays/région> à la suite d'un composant <langue>.
- La présente spécification permet également les codes <pays/région> à trois lettres "-CHS" et "-CHT" pour les paramètres de localisation chinois "Simplifiés" et "Traditionnels".
- La présente spécification permet également l'utilisation d'autres codes <pays/région> considérés comme étant nécessaires par le Client FDI ou le Serveur FDI.

Le Tableau 45 montre des exemples d'identificateurs de locale.

Tableau 45 – Exemples de LocaleId

Locale	LocaleId d'OPC UA
Anglais	en
Anglais (Etats-Unis)	en-US
Allemand	de
Allemand (Allemagne)	de-DE
Allemand (Autrichien)	de-AT

Une chaîne de caractères vide ou nulle (NULL) indique que la locale est inconnue.

5.1.9.3.6 Plage Numérique

La Plage Numérique est représentée sous la forme d'une Chaîne (String). La syntaxe pour la Chaîne contient l'une des deux constructions suivantes. La première construction est la représentation sous forme de Chaîne d'un nombre entier individuel. Par exemple, "6" est valide, mais "6,0" et "3,2" ne le sont pas. Les valeurs minimales et maximales qui peuvent être exprimées sont définies par l'utilisation de ce paramètre, et non par la définition du type de ce paramètre. La seconde construction est une plage représentée par deux nombres entiers séparés par le caractère deux-points (":"). Le premier nombre entier doit toujours avoir une valeur inférieure à celle du second. Par exemple, "5:7" est valide, alors que "7:5" et "5:5" ne le sont pas. Aucun autre caractère, y compris les caractères espace, ne peut être utilisé.

Tous les indices commencent à 0. L'indice maximal est inférieur de 1 à la longueur de la matrice.

Lors de la lecture avec une plage numérique en dehors des limites de la matrice, le Serveur FDI doit renvoyer un résultat partiel si certains éléments existent au sein de la plage. Le Serveur FDI doit renvoyer l'élément `Bad_OutOfRange` si aucun élément n'existe dans la plage.

Lors de l'écriture d'une valeur, la plage numérique doit être comprise au sein de la matrice.

Une plage numérique peut aussi être utilisée pour spécifier des sous-chaînes pour les valeurs `ByteString` et `String`.

5.1.9.3.7 Range

Cette structure définit la structure de plage nécessaire pour l'Attribut `EURange`. L'élément est défini dans le Tableau 46.

Tableau 46 – Structure du Type de Données Range

Nom	Type	Description
Range	structure	"low" et "high" peuvent contenir n'importe quel type de données adéquat pour le type de données l'Attribut Variable Value.
low	Variant	Valeur la plus faible dans la plage.
high	Variant	Valeur la plus haute dans la plage.

5.1.9.3.8 EUInformation

Cette structure contient les informations à propos des `EngineeringUnits` (Unités techniques). Ses éléments sont définis dans le Tableau 47.

Tableau 47 – Structure du Type de Données EUInformation

Nom	Type	Description
EUInformation	structure	—
unitId	UInt32	Identifiant pour l'évaluation programmée. 0 est utilisé si un unitId n'est pas disponible.
displayName	LocalizedText	Le displayName de l'unité technique correspond généralement à l'abréviation de l'unité technique, par exemple, "h" pour l'heure ou "m/s" pour mètre par seconde.
description	LocalizedText	Contient le nom complet de l'unité technique, tel que hour (heure) ou meter per second (mètre par seconde). Un champ textuel vide indique qu'aucune description n'est disponible.

5.1.9.3.9 EnumValueType

Ce Type de Données Structuré est utilisé pour donner la représentation d'une Enumération lisible par l'homme. Ses éléments sont définis dans le Tableau 48. Lorsque ce type est utilisé

dans une matrice représentant des représentations d'une énumération lisible par l'homme, chaque valeur d'une IntegerRepresentation sera unique dans cette matrice.

Tableau 48 – Définition d'EnumValueType

Nom	Type	Description
EnumValueType	structure	—
value	Int64	La représentation en Entiers d'une énumération
displayName	LocalizedText	Représentation lisible par l'homme de la représentation en nombres entiers d'une Enumération.
description	LocalizedText	La description localisée d'une valeur énumération. Ce champ peut contenir une Chaîne vide si aucune description n'est disponible.

5.1.9.4 Variant

Une Variant est une réunion de tous les types de données. Les Variants peuvent également contenir des matrices de ces différents types.

Les Variants peuvent être vides. Une Variant vide est décrite comme ayant une valeur Nulle. Une valeur Nulle dans une Variant n'équivaut pas à une Chaîne nulle.

Les Variants peuvent contenir des matrices de Variants mais ne peuvent pas contenir directement une autre Variant.

5.2 Services d'Hébergement

5.2.1 Généralités

Les Services d'Hébergement sont fournis par le Client FDI et sont utilisés par l'UIP. Les Services d'Hébergement comprennent les services relatifs à l'environnement du Client FDI, permettant à l'UIP d'obtenir des informations sur l'environnement. Les Services d'Hébergement comprennent également des services pour lancer d'autres UIP ainsi que pour afficher un retour d'information.

5.2.2 Services

5.2.2.1 Généralités

L'accès programmé aux services peut être synchrone ou asynchrone.

5.2.2.2 GetClientTechnology Version

5.2.2.2.1 Description

Ce service renvoie la Version de Technologie du Client FDI qui héberge l'UIP.

5.2.2.2.2 Paramètres

Le Tableau 49 définit les paramètres du service.

Tableau 49 – Paramètres du Service GetClientTechnologyVersion

Nom	Type	Description
Request (Demande)		
Response (Réponse)		
version	String	La Version de Technologie FDI prise en charge par le Client FDI. Le format de la valeur est xx.yy.zz, comme défini dans l'IEC 62769-4.

5.2.2.2.3 Résultats de service

Ce service se déroule toujours correctement.

5.2.2.3 Service OpenUserInterface

5.2.2.3.1 Description

Ce service est utilisé par un UIP pour demander au client FDI d'ouvrir un autre UIP. L'UIP est ouvert dans une fenêtre modale ou non modale de la manière suivante:

- Les UIP sont toujours invoqués dans une fenêtre modale si l'UIP appelant est exécuté dans une fenêtre modale ou s'il est de style dialogue.
- Les UIP sont invoqués dans une fenêtre non modale si leur style = fenêtre, mais également si l'UIP appelant est exécuté en mode non modal.

Si l'UIP est déjà ouvert, ce service doit afficher la fenêtre respective au premier plan.

5.2.2.3.2 Paramètres

Le Tableau 50 définit les paramètres du service.

Tableau 50 – Paramètres du Service OpenUserInterface

Nom	Type	Description
Request (Demande)		
uiPID	String	Identification de l'UIP à ouvrir. Cette chaîne est un UUID (Identifiant Universel Unique) qui définit un Nœud dans le Modèle d'Information. La valeur de cet UUID est définie dans le Paquetage FDI correspondant à l'UIP à ouvrir (voir l'IEC 62769-4).
Response (Réponse)		
Errors		

5.2.2.4 Service CloseUserInterface

5.2.2.4.1 Description

Ce service est utilisé par un UIP pour se fermer lui-même. L'UIP doit terminer toutes les fonctions en attente ou en cours d'exécution qui sont encore ouvertes.

Les UIP appellent ce Service uniquement sur requête de l'utilisateur (Ok, Fermer ou Annuler).

5.2.2.4.2 Paramètres

Il n'y a pas de paramètre spécifique pour le service.

5.2.2.5 Service LogAuditTrailMessage

5.2.2.5.1 Description

Ce service est utilisé par un UIP afin d'enregistrer un message de piste de vérification.

5.2.2.5.2 Paramètres

Le Tableau 51 définit les paramètres du service.

Tableau 51 – Paramètres du Service LogAuditTrailMessage

Nom	Type	Description
Request (Demande)		
message	String	Le message peut être enregistré dans la piste de vérification.
Response (Réponse)		Aucune.

5.2.2.6 Service SaveUserSettings

5.2.2.6.1 Description

Ce service est utilisé par un UIP pour donner au client FDI l'instruction de sauvegarder les paramètres utilisateur fournis. Les paramètres sont stockés par type d'UIP, dans un magasin persistant contrôlé par le Client FDI. Tous les paramètres utilisateur, précédemment sauvegardés, de ce type d'UIP sont remplacés. Le Client FDI ne doit pas modifier les paramètres utilisateur demandés dans le stockage.

Les paramètres utilisateurs incluent généralement des informations relatives à la structure de l'affichage ou d'autres préférences utilisateur. Ils ne sont pas conçus pour être utilisés pour des données spécifiques à une instance.

Les modifications partielles des paramètres utilisateur ne sont pas prises en charge. L'UIP doit charger tous les paramètres, les mettre à jour et sauvegarder l'ensemble complet.

La structure et le contenu de la chaîne paramètre utilisateur sont spécifiques au type d'UIP. Par exemple, un UIP pourrait utiliser des paires valeur-nom. Les problèmes de version de ces données sont à la responsabilité de l'UIP. Différents UIP du même type peuvent réécrire tous les autres paramètres.

La structure des données est entièrement sous le contrôle de l'UIP. Cependant, les problèmes de version concernant les différentes versions d'UIP sont également de la responsabilité de l'UIP.

5.2.2.6.2 Paramètres

Le Tableau 52 définit les paramètres du service.

Tableau 52 – Paramètres du Service SaveUserSettings

Nom	Type	Description
Request (Demande)		
userSetting[]	String	Liste des paramètres utilisateur. Le contenu de ces chaînes est spécifique au type d'UIP.
Response (Réponse)		

5.2.2.7 Service LoadUserSettings

5.2.2.7.1 Description

Ce service est utilisé par un UIP pour donner au client FDI l'instruction de retrouver les paramètres utilisateur précédemment sauvegardés.

5.2.2.7.2 Paramètres

Le Tableau 53 définit les paramètres du service.

Tableau 53 – Paramètres du Service LoadUserSettings

Nom	Type	Description
Request (Demande)		
Response (Réponse)		
userSetting[]	String	Liste des paramètres utilisateur. Le contenu de ces chaînes est spécifique à l'UIP.

5.2.2.8 Service Trace

5.2.2.8.1 Description

Ce service doit être utilisé par l'UIP afin de fournir au Client FDI les informations à propos des événements internes à l'UIP. Les messages de trace sont généralement utilisés pour le diagnostic des problèmes. L'UIP doit appeler ce service en fonction des paramètres du niveau de traçage spécifiés dans le Service SetTraceLevel de l'UIP (voir 6.1.1.4).

5.2.2.8.2 Paramètres

Le Tableau 54 définit les paramètres du service.

Tableau 54 – Paramètres du Service Trace

Nom	Type	Description
Request (Demande)		
eventType	TraceLevel	Sévérité du message de trace. L'une des valeurs qui spécifient la sévérité des données de traçage (voir 6.1.2).
classification	String	Classification spécifique à l'UIP du message de trace.
message	String	Message de trace. La langue du message de trace doit être l'Anglais. Le texte intégré pourrait être localisé.
Response (Réponse)		

5.2.2.9 Service ShowMessageBox

5.2.2.9.1 Description

Ce service ouvre une boîte de dialogue et attend que l'utilisateur presse l'un des boutons indiqués. Une boîte de dialogue ouverte doit bloquer toutes les activités liées à l'instance d'appareil. Il convient de ne pas modifier les interactions simultanées avec d'autres instances d'Appareil.

5.2.2.9.2 Paramètres

Le Tableau 55 définit les paramètres du service.

Tableau 55 – Paramètres du Service ShowMessageBox

Nom	Type	Description
Request (Demande)		
acknStyle	AcknStyle	Voir Tableau 63.
message	String	Le message devant être affiché à l'utilisateur.
caption	String	Légende de la barre de titre à afficher.
buttonSet	ButtonSet	Voir Tableau 62.
defaultResult	Entier	Identifiant du bouton par défaut; voir Tableau 61.
Response (Réponse)		
buttonSelected	Entier	Identifiant du bouton sélectionné; voir Tableau 61.

5.2.2.10 Service ShowProgressBar

5.2.2.10.1 Description

Ce service ouvre une barre de progression. La barre de progression reste sur l'interface utilisateur après que le service a renvoyé un résultat. Les informations affichées peuvent être mises à jour avec le service UpdateShowProgressBar (voir 5.2.2.11). La barre de progression peut être fermée avec le service EndShowProgressBar (voir 5.2.2.12).

Une seule barre de progression par UIP peut apparaître à la fois.

5.2.2.10.2 Paramètres

Le Tableau 56 définit les paramètres du service.

Tableau 56 – Paramètres du Service ShowProgressBar

Nom	Type	Description
Request (Demande)		
message	String	Le message devant être affiché à l'utilisateur.
callback	CancelCallback	Service appelé par le Client afin d'informer l'UIP que l'utilisateur a annulé l'opération. Voir 0. Le service CancelCallback est implémenté et fourni par l'UIP.
Response (Réponse)		

5.2.2.11 Service UpdateShowProgressBar

5.2.2.11.1 Description

Ce service met à jour une barre de progression déjà ouverte.

5.2.2.11.2 Paramètres

Le Tableau 57 définit les paramètres du service.

NOTE Un appel de ce service est rejeté si le service ShowProgressBar n'a pas été appelé auparavant.

Tableau 57 – Paramètres du Service UpdateShowProgressBar

Nom	Type	Description
Request (Demande)		
message	String	Le message mis à jour devant être affiché à l'utilisateur.
pourcentage	Entier	Le pourcentage mis à jour de la progression devant être affichée à l'utilisateur.
Response (Réponse)		

5.2.2.12 Service EndShowProgressBar**5.2.2.12.1 Description**

Ce service ferme une barre de progression ouverte. Ce service attendra que l'utilisateur presse un bouton ou renverra immédiatement un résultat si l'utilisateur a déjà pressé le bouton.

5.2.2.12.2 Paramètres

Le Tableau 58 définit les paramètres du service.

Tableau 58 – Paramètres du Service EndShowProgressBar

Nom	Type	Description
Request (Demande)		
message	String	Le message mis à jour devant être affiché à l'utilisateur.
pourcentage	Entier	Le pourcentage mis à jour de la progression devant être affichée à l'utilisateur.
Response (Réponse)		

5.2.2.13 Service CancelCallback**5.2.2.13.1 Description**

Avec ce service, le Client informe l'UIP que l'Utilisateur a demandé l'annulation de l'opération. Ce service est implémenté et fourni par l'UIP lors de l'appel du service ShowProgressBar. L'UIP est chargé de fermer l'élément ProgressBar.

De par la nature asynchrone des rappels, le service CancelCallback pourrait être appelé, même après que l'UIP a appelé le service EndShowProgressBar.

5.2.2.13.2 Paramètres

Il n'y a pas de paramètre spécifique pour le service.

5.2.2.14 Service StandardUIActionItemsChangeCallback**5.2.2.14.1 Description**

Ce service est utilisé par l'UIP afin de notifier le Client FDI sur les changements de l'état des éléments d'action UI normalisés (activé/désactivé). Voir 6.1.2.2 pour des éléments d'action UI normalisés.

5.2.2.14.2 Paramètres

Le Tableau 59 définit les paramètres du service.

Tableau 59 – Paramètres du service StandardUIActionItemsChange

Nom	Type	Description
Request (Demande)		
Response (Réponse)		
StandardUIActionItems[]	StandardUIActionItem	Liste mise à jour des Actions Normalisées fournies par l'UIP.

5.2.2.15 Service SpecificUIActionItemsChangeCallback

5.2.2.15.1 Description

Ce service est utilisé par l'UIP afin de notifier le Client FDI sur les changements des éléments d'action de l'UI spécifiques à cet UIP. L'UIP doit utiliser ce rappel pour chaque ajout ou retrait d'éléments d'action ou à chaque changement de l'état d'un élément d'action (c'est-à-dire, activé/désactivé). Voir 6.1.2.4 pour des actions UI spécifiques.

5.2.2.15.2 Paramètres

Le Tableau 60 définit les paramètres du service.

Tableau 60 – Paramètres du service SpecificUIActionItemsChange

Nom	Type	Description
Request (Demande)		
Response (Réponse)		
SpecificUIActionItems[]	SpecificUIActionItem	Liste mise à jour des éléments d'action UI spécifiques à l'UIP.

5.2.3 Définitions du Type Paramètre

5.2.3.1 Définition de DefaultResult

Les composants de ce paramètre sont définis dans le Tableau 61.

Tableau 61 – Définition de DefaultResult

Nom	Type	Description
DefaultResult	Entier	Définition d'un bouton dans une boîte de dialogue. Utilisé pour spécifier le bouton par défaut et le bouton sélectionné par l'utilisateur. Cette valeur est une énumération avec l'une des valeurs suivantes: BUTTONNONE_0 BUTTONOK_1 BUTTONCANCEL_2 BUTTONYES_3 BUTTONNO_4

5.2.3.2 ButtonSet

Les composants de ce paramètre sont définis dans le Tableau 62.

Tableau 62 – Définition de ButtonSet

Nom	Type	Description
ButtonSet	Entier	Définition des boutons dans une boîte de dialogue ou de la barre de progression affichée pour l'utilisateur. Cette valeur est une énumération avec l'une des valeurs suivantes: BUTTONSETOK_0 Seul le bouton Ok est affiché BUTTONSETOKCANCEL_1 Les boutons Ok et Cancel (annuler) sont affichés BUTTONSETYESNOCANCEL_2 Les boutons Yes (Oui), No (Non), et Cancel sont affichés BUTTONSETYESNO_3 Les boutons Yes et No sont affichés

5.2.3.3 AcknStyle

Les composants de ce paramètre sont définis dans le Tableau 63.

Tableau 63 – Définition de AcknStyle

Nom	Type	Description
AcknStyle	Entier	Le style de la boîte de dialogue ou de la barre de progression affichée pour l'utilisateur. Par exemple, ce paramètre peut influencer l'icône de la boîte de dialogue. Cette valeur est une énumération avec l'une des valeurs suivantes: ACKNSTYLEINFO_0 Style des informations ACKNSTYLEWARNING_1 Style des avertissements ACKNSTYLEERROR_2 Style des erreurs

6 UIP

6.1 Services d'UIP

6.1.1 Services

6.1.1.1 Service Activate (Activer)

6.1.1.1.1 Description

Ce service est utilisé par le Client FDI afin d'initialiser un UIP (voir 6.1.2). Le Client FDI doit appeler ce service après la création d'une instance d'UIP. Les règles suivantes définissent le type de fenêtre:

- Une fenêtre modale est utilisée si le style de l'UIP = dialogue ou si le parent invoqué (UID ou UIP) est modal.
- Une fenêtre non modale est utilisée si le style de l'UIP = fenêtre, mais également si le parent invoqué est exécuté dans un mode non modal.

6.1.1.1.2 Paramètres

Le Tableau 64 définit les paramètres du service.

Tableau 64 – Paramètres du Service Activate

Nom	Type	Description
Request (Demande)		
hostingInterface	Interface	Le Client FDI hébergeant l'interface devant être utilisée par l'UIP (voir 5.2).
deviceAccessInterface	Interface	L'interface d'accès à l'appareil du Client FDI devant être utilisée par l'UIP (voir 5.1).
context	UInteger	Spécifie le contexte dans lequel l'UIP est activé: 0_ONLINE 1_OFFLINE Le Client doit obtenir le contexte à partir du FunctionalGroup (groupe fonctionnel) dans lequel cet UIP a été extrait; c'est-à-dire, si le Groupe Fonctionnel fait partie de la représentation d'appareil Hors ligne ou En ligne. Si un UIP est invoqué à partir d'un autre UIP avec le service OpenUserInterface ou OpenModalUserInterface, le contexte en est hérité.
localeSetting	<dépendant de la technologie>	La description d'une locale est constituée au minimum d'un identifiant de langue, d'informations d'affichage et du pays. Le format de la valeur d'une propriété dépend de la technologie.
Response (Réponse)		

Le localeSetting (Paramètre de la locale) ne doit pas dépendre des paramètres du système d'exploitation. Les paramètres du système d'exploitation peuvent être utilisés par défaut, mais ils doivent être modifiables par l'utilisateur.

L'identifiant de langue doit indiquer la langue des informations textuelles localisées.

Les informations d'affichage doivent être utilisées pour contrôler les couleurs (ex.: couleur de l'arrière-plan, du texte pour les différents statuts), ainsi que le format des éléments spécifiques (ex.: pour les chiffres et les dates).

Le paramètre Pays doit être utilisé pour appliquer la réglementation spécifique aux différents pays, comme les unités techniques admises. Pour ce faire, l'UIP modifiera l'Attribut EngineeringUnit des Nœuds de Variable associés, afin de correspondre au pays spécifié. Le Serveur FDI est alors responsable du reformatage des valeurs en conséquence.

6.1.1.2 Service Deactivate (Désactiver)

6.1.1.2.1 Description

Ce service est utilisé par le Client FDI afin de désactiver un UIP. L'UIP doit libérer toutes les références à d'autres composants et terminer toutes les fonctions en attente ou en cours d'exécution, y compris les UIP encore ouvertes.

L'UIP peut indiquer qu'il n'est pas prêt à être désactivé. Cela doit être utilisé si l'annulation de fonctions en attente ou en cours d'exécution implique des effets secondaires importants.

6.1.1.2.2 Paramètres

Le Tableau 65 définit les paramètres du service.

Tableau 65 – Paramètres du Service Deactivate

Nom	Type	Description
Request (Demande)		
Response (Réponse)		
deactivateCancelled	Boolean	Indication permettant de savoir si l'UIP a refusé la demande Deactivate. deactivateCancelled = "true" indique que l'UIP a refusé la requête Deactivate. deactivateCancelled = "false" indique que l'UIP a accepté la requête Deactivate.

6.1.1.3 Service SetSystemLabel

6.1.1.3.1 Description

Ce service est utilisé par le Client FDI afin de déterminer un identifiant lisible par l'homme de l'instance d'UIP au sein du Client FDI.

L'étiquette doit être utilisée pour les éléments d'interface utilisateur affichés et gérés par l'UIP (ex.: boîte de dialogue) et garantira que l'utilisateur peut uniquement identifier l'Appareil vers lequel est dirigé l'élément d'interface utilisateur. Selon les situations, l'UIP peut étendre cette étiquette avec des informations spécifiques.

Le Client FDI doit appeler ce service avant le service Activate (voir 6.1.1.1). S'il n'a pas été appelé, l'UIP doit utiliser la partie textuelle par défaut de l'Attribut Label de l'Appareil (voir 5.1.3.2).

6.1.1.3.2 Paramètres

Le Tableau 66 définit les paramètres du service.

Tableau 66 – Paramètres du Service SetSystemLabel

Nom	Type	Description
Request (Demande)		
systemLabel	String	Etiquette lisible par l'homme qui identifie l'UIP au sein du Client FDI.
Response (Réponse)		Aucune.

6.1.1.4 Service SetTraceLevel

6.1.1.4.1 Description

Ce service est utilisé par le Client FDI afin de notifier l'UIP sur les types de messages Trace qu'il convient d'enregistrer via le service Trace (voir 5.2.2.8). Plusieurs types peuvent être définis, tels que Critical (Critique), Error (Erreur) et Warning (Avertissement).

6.1.1.4.2 Paramètres

Le Tableau 67 définit les paramètres du service.

Tableau 67 – Paramètres du Service SetTraceLevel

Nom	Type	Description
Request (Demande)		
traceLevel	TraceLevel	Niveau de Trace qui doit contrôler le type d'information (voir Tableau 72).
Response (Réponse)		

6.1.1.5 Service GetStandardUIActionItems

6.1.1.5.1 Description

Ce service est utilisé par le Client FDI pour obtenir les actions UI normalisées disponibles dans l'UIP (ex.: Fermer, Appliquer, Aide). Le Client FDI fournit la représentation uniforme de ces actions UI normalisées, dans la partie UI du Client FDI.

6.1.1.5.2 Paramètres

Le Tableau 68 définit les paramètres du service.

Tableau 68 – Paramètres du Service GetStandardUIActionItems

Nom	Type	Description
Request (Demande)		
Response (Réponse)		
StandardUIActionItems[]	StandardUIActionItem	Liste des Actions Normalisées fournies par l'UIP.

6.1.1.6 Service GetSpecificUIActionItems

6.1.1.6.1 Description

Ce service est utilisé par le Client FDI pour obtenir les actions UI disponibles et spécifiques à l'UIP. Le Client FDI fournit la représentation uniforme de ces actions UI, dans la partie UI du Client FDI.

6.1.1.6.2 Paramètres

Le Tableau 69 définit les paramètres du service.

Tableau 69 – Paramètres du service GetSpecificUIActionItems

Nom	Type	Description
Request (Demande)		
Response (Réponse)		
SpecificUIActionItems[]	SpecificUIActionItem	Liste d'actions UI spécifiques à l'UIP.

6.1.1.7 Service InvokeStandardUIAction

6.1.1.7.1 Description

Ce service est utilisé par le Client FDI pour notifier l'UIP d'exécuter une action normalisée (ex.: Fermer, Appliquer, Aide). Le Client FDI transmet le type de l'action normalisée à exécuter dans ce service.

6.1.1.7.2 Paramètres

Le Tableau 70 définit les paramètres du service.

Tableau 70 – Paramètres du Service InvokeStandardUIAction

Nom	Type	Description
Request (Demande)		
actionId	StandardUIAction	L'une des valeurs définies en 6.1.2.2.
Response (Réponse)		

6.1.1.8 Service InvokeSpecificUIAction

6.1.1.8.1 Description

Ce service est utilisé par le Client FDI pour demander à l'UIP d'exécuter une action UI spécifique à cet UIP. Le Client FDI transmet l'identifiant de l'action à exécuter dans ce service.

6.1.1.8.2 Paramètres

Le Tableau 71 définit les paramètres du service.

Tableau 71 – Paramètres du Service InvokeSpecificUIAction

Nom	Type	Description
Request (Demande)		
actionId	UInteger	Il s'agit de l'identifiant de l'action UI spécifique à exécuter
Response (Réponse)		

6.1.2 Définitions du Type Paramètre

6.1.2.1 TraceLevel

L'élément TraceLevel est défini dans le Tableau 72.

Tableau 72 – Définition de TraceLevel

Nom	Type	Description
TraceLevel	UInteger	Le niveau de sévérité qui contrôle le type des informations transmises au service Trace (voir 5.2.2.8). Cette valeur est une énumération de bits avec l'une des valeurs suivantes: NONE_0 Désactivation complète du traçage CRITICAL_1 Erreur fatale ou crash de l'application ERROR_2 Erreur récupérable WARNING_4 Erreur non critique INFO_8 Message informationnel VERBOSE_16 Trace de débogage

6.1.2.2 StandardUIAction

Les actions normalisées sont définies afin de permettre l'uniformité de la présentation et du traitement. Elles seront demandées par le Client d'hébergement. Ces actions ne sont pas

censées être mappées en 1:1 avec les boutons de l'interface utilisateur. Le Client préférera afficher OK, CANCEL (Annuler) et APPLY (Appliquer).

En cliquant sur OK, CANCEL ou APPLY, l'utilisateur provoquera l'appel de ces actions.

- OK déclenchera un appel de l'action Apply suivi par un appel de l'action Close (Fermer).
- APPLY déclenchera un appel de l'action Apply.
- CANCEL déclenchera un appel vers l'action Close. Si des modifications sont toujours en suspens, l'UIP doit ouvrir une boîte de dialogue demandant la confirmation de l'utilisateur.

L'énumération de StandardUIAction est spécifiée dans le Tableau 73.

Tableau 73 – Définition de StandardUIAction

Nom	Type	Description
StandardUIAction	UInteger	Identifiant pour l'élément d'action UI normalisé avec l'une des valeurs suivantes: APPLY_0 Apply Standard UI Action (Appliquer l'Action UI Normalisée) • Avec cette action, le Client demandera que les modifications de paramètre dans l'interface utilisateur soient appliquées à la source. CLOSE_1 Close Standard UI Action (Fermer l'Action UI Normalisée) • Avec cette action, le Client demande la fermeture de l'UIP. Si les modifications des paramètres n'ont pas été appliquées, l'UIP doit ouvrir une boîte de dialogue demandant la confirmation de l'utilisateur. HELP_2 Help Standard UI Action (Aide de l'Action UI Normalisée) • Demande l'affichage d'une aide en ligne décrivant la fonctionnalité de l'UIP (un document d'aide). Elle ne peut pas se substituer à une aide contextuelle, qui sera généralement fournie par l'intermédiaire d'infobulles.

6.1.2.3 StandardUIActionItem

Les composants de ce paramètre sont définis dans le Tableau 74.

Tableau 74 – Définition de StandardUIActionItem

Nom	Type	Description
StandardUIActionItem	Structure	
actionId	StandardUIAction	Identifiant de l'élément d'action UI normalisé.
enabled	Boolean	Indique l'état de l'élément d'action. "true" indique que l'élément est activé. "false" indique que l'élément est désactivé.

6.1.2.4 SpecificUIActionItem

Les composants de ce paramètre sont définis dans le Tableau 75.

Tableau 75 – Définition de SpecificUIActionItem

Nom	Type	Description
SpecificUIActionItem	structure	
actionId	UInteger	Identifiant de l'élément d'action spécifique à l'UIP.
descriptor	String	Une chaîne lisible par l'homme qui fournit des informations à propos de l'action.
enabled	Boolean	Indique l'état de l'élément d'Action. "true" indique que l'élément est activé. "false" indique que l'élément est désactivé.
label	String	Étiquette de l'élément action.

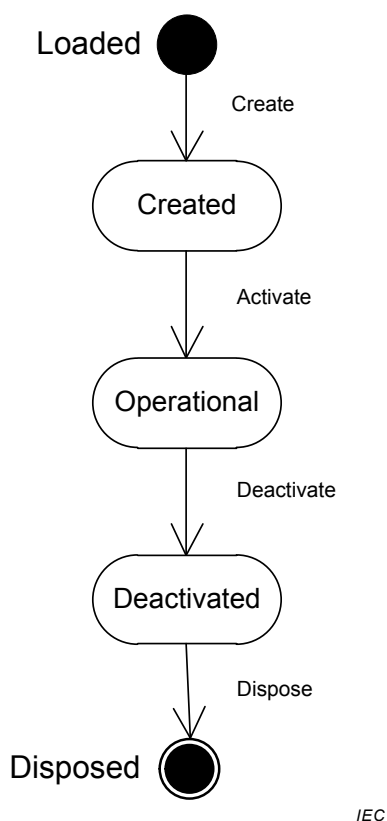
6.2 Règles d'instanciation de l'UIP

Chaque instance d'appareil peut contenir un maximum de deux instances du même type d'UIP, une pour la version hors ligne et une pour la version en ligne.

6.3 Diagramme d'états de l'UIP

6.3.1 Etats

La Figure 7 présente le diagramme d'états de l'UIP. Si les Services UIP (voir 6.1) déclenchent des modifications d'état, celles-ci sont mentionnées sur les côtés du diagramme d'états.



Anglais	Français
Loaded	Chargé
Created	Créé
Operational	Opérationnel
Deactivated	Désactivé
Disposed	Supprimé
Create	Créer
Activate	Activer
Deactivate	Désactiver
Dispose	Supprimer

NOTE Create (Créer) et Dispose (Rejeter) sont des services dépendants de la technologie décrits dans l'IEC 62769-6. Create utilise la Propriété StartElementName (voir l'IEC 62769-5) de l'UIP afin de créer l'UIP approprié.

Figure 7 – Diagramme d'états de l'UIP

Les états de l'UIP sont spécifiés dans le Tableau 76.

Tableau 76 – Etats de l'UIP

Etat	Description
Created (Créée)	L'état initial après la création de l'instance d'UIP par le Serveur FDI.
Operational	L'état d'exécution normal.
Deactivated	L'état final avant qu'un UIP soit rejeté.

6.3.2 Transitions d'état

Les transitions d'état de l'UIP sont spécifiées dans le Tableau 77.

Tableau 77 – Transitions d'états de l'UIP

Etat source	Evénement	Etat final
Loaded	Le Client FDI a créé l'instance UIP.	Created (Créée)
Created (Créée)	Le service Activate (Activer) a été appelé pour l'instance UIP.	Operational
Operational	Le service Deactivate (Désactiver) a été appelé avec succès pour l'instance UIP.	Deactivated
Deactivated	Le Client FDI a rejeté l'instance UIP.	Disposed

6.4 Permissions de l'UIP

Un UIP se voit accorder les permissions suivantes, concernant les ressources du système d'exploitation.

- Accès au système de fichiers (lecture/écriture) (obligatoire).
- Accès aux imprimantes (obligatoire – si une imprimante est disponible dans l'environnement d'hébergement).
- Accès au réseau (facultatif).
- Utilisation des contrôles ActiveX (facultatif).

NOTE 1 L'installation des Contrôles ActiveX ne relève pas du domaine d'application de la présente norme.

La mention "obligatoire" signifie qu'un Client FDI doit accorder les permissions, celles-ci étant accordées par le système d'exploitation. Cela ne signifie pas qu'un UIP dispose d'un accès libre au système de fichiers complet. Les permissions pour accéder au système de fichiers et aux imprimantes sont définies par le système de configuration.

NOTE 2 Le paramétrage des permissions pour les ressources du système d'exploitation d'une configuration système ne relève pas du domaine d'application de la présente norme.

La mention "facultatif" signifie que le Client FDI peut restreindre les permissions par rapport aux permissions données par la configuration système. Les moyens par lesquels un Client FDI peut restreindre l'accès au réseau et/ou l'utilisation des contrôles ActiveX sont spécifiés dans l'IEC 62769-6.

Les UIP ne doivent jamais bloquer le Client, par exemple en effectuant des appels synchrones afin de potentiellement bloquer les ressources du système d'exploitation. Les fils de travail sont un moyen commun d'éviter les situations de blocage.

6.5 Téléchargements de l'UIP à partir du Serveur FDI

Les scénarios suivants exigent qu'un Client FDI récupère un UIP.

- L'UID spécifie un UIP dans son Plugiciel d'élément XML (voir Annexe A).
- L'UIP ouvre un autre UIP avec le service OpenUserInterface (voir 5.2.2.3).

Dans tous les scénarios, le Client FDI obtient un Uipld comme identifiant pour l'UIP. Un Uipld est un UUID (identifiant universel unique). Un UIP peut avoir plusieurs Variants UIP, qui diffèrent dans leur plate-forme et leur support d'exécution (voir l'IEC 62769-4:2015, 5.3.3.2.2).

NOTE Un Uipld ne contient pas d'information de version. La résolution d'un Uipld pour la version appropriée de l'UIP est effectuée par le Serveur FDI sur la base des informations du Paquetage FDI.

Le même Uipld peut être résolu pour différentes versions UIP pour différents Appareils. A différents moments dans le temps et pour le même Appareil, le même Uipld peut être résolu de façon différente si une mise à jour de l'UIP s'est produite dans le Serveur FDI entre-temps.

Avec l'Uipld, le Client FDI peut récupérer les informations suivantes à propos de toutes les Variants UIP correspondantes (voir l'IEC 62769-5), depuis le Serveur FDI:

- RuntimeId et PlatformId
- UIPVariantVersion
- FDITechnologyVersion

Pour ce faire, le Client FDI appelle le service OPC UA TranslateBrowsePathToNodeIds avec la liste suivante de noms relatifs:

```
"UIPSet/<Uipld>"
"UIPSet/<Uipld >/RuntimeId"
"UIPSet/<Uipld >/PlatformId"
"UIPSet/<Uipld >/FDITechnologyVersion"
"UIPSet/<Uipld >/Style"
"UIPSet/<Uipld >/StartElementName"
"UIPSet/<Uipld >/UIPVariantVersion"
```

Le Serveur FDI renvoie des matrices de NodeIds pour chaque nom relatif. Le nombre d'entrées dans chaque matrice correspond au nombre de Variants UIP pour l'Uipld. Le Client FDI peut ensuite lire les valeurs de propriété.

Si plusieurs Variants UIP sont disponibles, le Client FDI choisit la Variant UIP la plus appropriée, sur la base de FDITechnologyVersion, RuntimeId, et PlatformId.

Le Client FDI peut maintenir en mémoire cache les Variants UIP déjà téléchargées depuis le Serveur FDI. Si Uipld, RuntimeId, PlatformId, FDITechnologyVersion et UIPVariantVersion, provenant de la Variant UIP sélectionnée, correspondent à une Variant UIP parmi celles en mémoire cache, cette Variant UIP peut être utilisée et aucun téléchargement depuis le Serveur FDI n'est exigé.

Si aucune mémoire cache n'est maintenue ou qu'aucune Variant UIP correspondante n'est trouvée dans la mémoire cache, le Client FDI doit lire l'élément FDITechnologyVersion (voir l'IEC 62769-5) de la Variant UIP sélectionnée. Le paramètre FDITechnologyVersion permet d'aider le Client FDI à déterminer s'il peut prendre en charge l'exécution de l'UIP. Le Client FDI doit exécuter un UIP de la même version majeure, indépendamment de la version mineure ou de révision.

Enfin, le Client FDI peut télécharger l'UIP sélectionné depuis le Serveur FDI en lisant la valeur de la Variant UIP.

Un Client FDI peut implémenter une autre séquence que celle décrite ici. Par exemple, il peut d'abord vérifier FDITechnologyVersion afin de déterminer s'il peut exécuter l'UIP. FDITechnologyVersion est le même pour toutes les Variants UIP d'une version spécifique d'un UIP.

Le Client FDI peut implémenter des stratégies d'optimisation. Par exemple, il peut mettre en mémoire cache les versions des UIP (en plus des versions des Variants UIP). Si la version de l'UIP est identique à la version de l'UIP en mémoire cache, il n'est pas nécessaire de lire les versions des Variants UIP, car elles doivent être les mêmes pour une version d'UIP identique (voir l'IEC 62769-4).

7 Actions

7.1 Généralités

L'EDD (Description d'Appareil Electronique) contenue dans le Paquetage FDI, tel que défini dans l'IEC 62769-4, peut comporter des méthodes EDD. Certaines méthodes EDD peuvent être exposées au Client FDI en tant qu'Actions et peuvent être déclenchées par les Clients FDI.

NOTE Ces Actions n'ont aucune relation avec la construction EDD ACTION.

Les Actions sont exécutées dans le Serveur FDI. Le diagramme d'états Action est donné dans l'IEC 62769-3.

Un Client FDI peut invoquer une Action en appelant la méthode InvokeAction OPC UA (voir l'IEC 62769-5). Les modifications d'états du diagramme d'états d'Action correspondant sont envoyées au Client FDI via le mécanisme d'abonnement (voir l'IEC 62541-4). Des données supplémentaires, pouvant provenir d'une modification d'état, sont transférées sous la forme d'un document XML.

Une Action peut impliquer des interactions de l'utilisateur. Le résultat d'une interaction de l'utilisateur est envoyé au Serveur FDI avec le service RespondAction (voir l'IEC 62769-5). Les données envoyées avec ce service sont transférées sous la forme d'un document XML.

Une Action peut être abandonnée par le Client ou par le Serveur.

Si le Serveur abandonne une Action, il envoie d'abord une AbortingNotification (Notification d'Abandon). Le Client doit répondre avec une AbortNotificationConfirmation (Confirmation de Notification d'Abandon), mais il continue de traiter les ActionRequests (Requêtes d'Actions). Suite à la réception de l'élément AbortNotification (Notification d'Abandon), le Client doit désactiver le CancelButton (Bouton Annuler). A la fin du processus d'abandon, le Serveur enverra un élément AbortRequest (Requête d'Abandon) au Client. Le Client répond avec un élément AbortResponse (Réponse à un Abandon) et ferme l'élément ActionWindow (Fenêtre Action).

Un Client FDI peut abandonner une Action en procédant de la manière suivante:

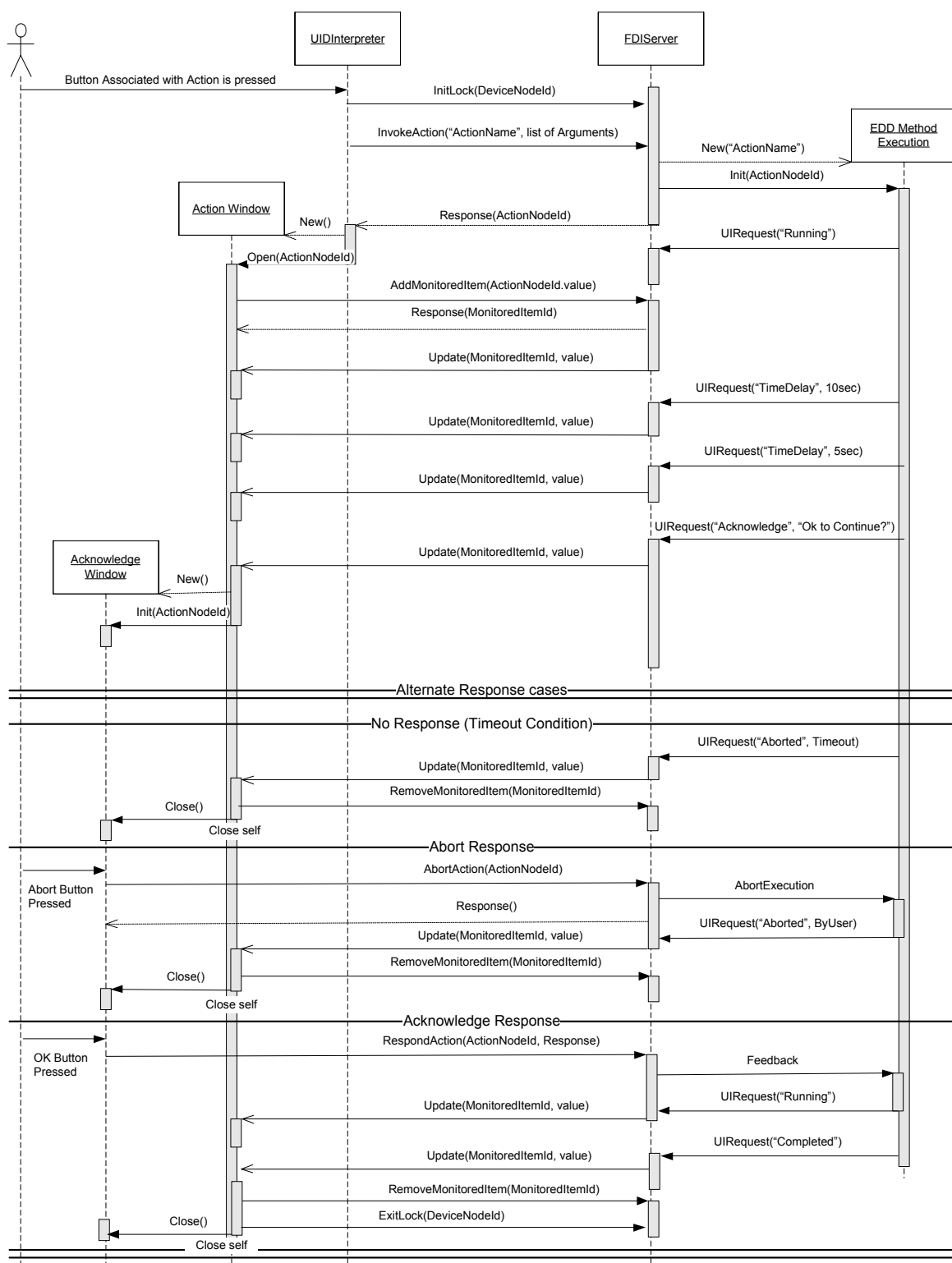
- 1) Envoyer une demande au service AbortAction (Abandon de l'Action) (voir l'IEC 62769-5).
- 2) Répondre à une ActionRequest existante en envoyant une ActionResponse (Réponse à une Action) valide.
- 3) Continuer le traitement des ActionRequests.
- 4) Le Serveur envoie une AbortingNotification une fois qu'il a démarré le processus d'abandon. Le Client doit répondre avec une AbortNotificationConfirmation, mais il continue de traiter les ActionRequests. Suite à la réception de l'élément AbortNotification, le Client doit désactiver le CancelButton.
- 5) L'Action est abandonnée à la réception d'un élément AbortingRequest (Requête d'Abandon) envoyé par le Serveur. Le Client doit répondre avec un élément AbortResponse et ferme l'élément ActionWindow.

L'Annexe B contient un exemple d'une méthode EDD exécutée par un Serveur FDI ainsi que de l'interaction qui en résulte avec un Client FDI. Il inclut des diagrammes d'états et les messages échangés (extraits XML).

7.2 Diagramme de séquences

Le diagramme de séquences à la Figure 8 montre l'interaction client/serveur d'une Action. Ce diagramme considère comme une condition préalable que l'Action puisse être démarrée depuis une interface utilisateur, affichée par le Client FDI ou par un UIP.

NOTE 1 Le diagramme montre UIDInterpreter, ActionWindow et AcknowledgeWindow en tant que sous-systèmes du Client FDI. Il montre également EDDMethodExecution en tant que sous-système du Serveur FDI. Ceci à des fins explicatives uniquement.



IEC

Anglais	Français
Button associated with Action is pressed	Le bouton associé à l'Action est pressé
UIDInterpreter	Interprète UID
FDIServer	Serveur FDI
EDD Method Execution	Exécution de la Méthode EDD
Action Window	Fenêtre Action
Acknowledge Window	Fenêtre de Confirmation

Anglais	Français
InvokeAction("ActionName", list of Arguments)	InvokeAction("ActionName", liste des Arguments)
UIRequest("Acknowledge", "Ok to Continue?")	UIRequest("Confirmer", "Ok pour continuer?")
Alternate Response Cases	Cas de Réponse Alternative
No Response (Timeout Condition)	Aucune Réponse (Condition de Temporisation)
Abort Response	Abandonner la Réponse
Abort Button pressed	Bouton Abandonner pressé
Acknowledge Response	Confirmer la Réponse
OK Button pressed	Bouton OK pressé
Close self	Auto-fermeture

Figure 8 – Diagramme de séquences d'Action FDI

L'Action est initiée depuis l'interface utilisateur. S'il n'est pas déjà verrouillé, l'élément InitLock doit d'abord être appelé. Le Client FDI appelle alors la méthode InvokeAction OPC UA (voir l'IEC 62769-5). Le nom de l'Action qui va être invoquée, ainsi que tous les arguments de l'Action nécessaires, est donné en tant qu'arguments de la méthode OPC UA.

NOTE 2 Cette description est identique, que l'Action soit initiée depuis un UIP ou une UID. La seule différence est qu'un UIP n'invoque pas l'Action dans le Serveur FDI directement, mais utilise les services correspondants dans l'interface DeviceAccess. Le Client FDI appelle alors l'Action sur le Serveur FDI de la part de l'UIP.

NOTE 3 L'interaction entre le Client FDI et le Serveur FDI, telle que l'abonnement à ActionNodeId et le rendu des interfaces utilisateur (si demandé pendant l'exécution de l'Action), est toujours à la responsabilité du Client FDI, peu importe si l'Action a été initiée par une UID ou un UIP. Par conséquent, un UIP peut ne pas connaître les échanges de documents XML (tels que définis par le schéma XML à l'Annexe A), entre le Client FDI et le Serveur FDI.

Le Serveur FDI répond à l'appel InvokeAction en créant un diagramme d'états d'exécution de l'Action et répond au Client FDI en renvoyant un ActionNodeId représentant le diagramme d'états dans le Modèle d'Information.

Le Serveur FDI est responsable de l'exécution du diagramme d'états (tel que défini dans l'IEC 62769-3) et de la mise à jour de ses états dans le Modèle d'Information à l'aide de l'ActionNodeId fourni. L'ActionNodeId est également utilisé par le Client FDI afin d'établir les abonnements avec le Serveur FDI, dans le but de surveiller l'exécution de l'Action. Pendant que l'Action est en cours d'exécution, le Serveur FDI met à jour le diagramme d'états et le Nœud correspondant dans le Modèle d'Information. S'il existe des abonnements pour l'ActionNodeId, le Serveur FDI traitera ces modifications et préviendra le Client FDI de ces modifications.

Pendant l'exécution de l'Action, les opérations telles que les notifications, la lecture et l'écriture (également vers d'autres Nœuds que celui comportant l'ActionNodeId) ne sont pas bloquées.

Le Serveur FDI commence une Action en paramétrant l'état sur "Running" (En cours), puis exécute la méthode EDD. Le Client FDI utilise le service AddMonitoredItem OPC UA (voir l'IEC 62541-4) afin d'établir un abonnement à l'aide de l'ActionNodeId fourni. Le Serveur FDI répond à l'abonnement nouvellement créé en prévenant le Client FDI de l'état actuel du diagramme d'états de l'Action. Dans ce diagramme de séquences, l'abonnement a été établi avant que l'exécution de l'Action atteigne un état notable, tel que l'état TimeDelay. Cependant, la première notification pour le Client FDI indiquerait "Running". Si l'Action avait atteint l'état TimeDelay avant l'établissement de l'abonnement, la première notification pour le Client FDI indiquerait "TimeDelay", puisqu'il s'agirait de l'état actuel.

Le Client FDI ne doit pas maintenir un diagramme d'états d'Action.

NOTE 4 Les informations envoyées au Client FDI comprennent les informations d'états et la dernière demande UI. Il n'y a donc pas de situation de compétition en fonction du moment où le Client FDI ajoute le Nœud d'état en tant qu'élément contrôlé. Même si le diagramme d'états d'Action a connu plusieurs transitions d'état, le Client FDI

peut ne pas les connaître. Le comportement OPC UA normalisé veut que les informations actuelles soient envoyées lorsqu'un Nœud est ajouté en tant qu'élément contrôlé.

Lorsque le Serveur FDI entre dans l'état TimeDelay, l'élément ActionNodeId est mis à jour au début du retard puis à l'expiration du retard. Des mises à jour intermédiaires peuvent également être appliquées en fonction du délai de retard. La fréquence réelle des mises à jour intermédiaires est déterminée par la conception du Serveur FDI et correspond généralement à quelques secondes. Toutes les mises à jour d'ActionNodeId seront soumises au Client FDI via le mécanisme d'abonnement.

EXEMPLE 1 A la Figure 8, le Serveur FDI est entré dans le délai de retard de dix secondes et envoie des mises à jour toutes les cinq secondes.

Lorsque le Serveur FDI passe à l'état WaitingForFeedback, le Modèle d'Information est mis à jour, au même titre que l'invite utilisateur. Le diagramme d'états met en pause l'exécution de l'Action jusqu'à ce qu'une réponse de l'utilisateur soit fournie ou jusqu'à expiration d'une temporisation. Une modification du diagramme d'états donne lieu à l'envoi d'une notification vers le Client FDI. Lorsque le Client FDI détecte que le nouvel état est WaitingForFeedback (en attente d'un retour d'information), il ouvre une boîte de dialogue présentant l'invite fournie dans l'information d'état, ainsi que les moyens de délivrer un retour d'information.

EXEMPLE 2 A la Figure 8, le Client FDI présente la manière de confirmer ou d'abandonner la méthode avec les boutons respectifs "OK" ou "Abandon".

Le diagramme de séquences montre trois cas de réponses alternatives.

La réponse "Timeout": Si l'utilisateur ne répond pas à l'appel avant expiration du délai du serveur, le Serveur FDI abandonnera l'exécution de l'Action. L'état dans le Modèle d'Information est défini sur "Aborted" (Abandonné), avec une indication expliquant que cela est dû à l'expiration d'une temporisation. L'exécution de l'Action se termine à ce point, mais le Serveur FDI maintient le Nœud d'état et l'état final jusqu'à ce que les abonnements soient résiliés, soit par le Client FDI en supprimant l'élément contrôlé soit par l'expiration générale de la session du Client FDI avec le Serveur FDI.

Cas de réponse "Abort": Si l'utilisateur répond au Serveur FDI avec une réponse "abort", l'exécution de la méthode est abandonnée. L'état dans le Modèle d'Information est défini sur "Aborted" (abandonné), avec une indication expliquant que cela est dû à une action de l'utilisateur. Le Serveur FDI maintient le Nœud d'état et l'état final jusqu'à ce que les abonnements soient résiliés, soit par le Client FDI en supprimant l'élément contrôlé soit par l'expiration générale de la session du Client FDI avec le Serveur FDI. Le Nœud d'état est également préservé si l'état "Completed" ou "Aborted" du diagramme d'états de l'Action est atteint avant que le Client FDI ait établi un abonnement ou ait ajouté le Nœud d'état comme élément contrôlé.

Cas de réponse "Positive feedback": Si l'utilisateur répond au Serveur FDI avec un retour d'information positif, l'exécution de l'Action remet l'état sur "Running" dans le Modèle d'Information et continue l'exécution normale de l'Action.

EXEMPLE 3 A la Figure 8, aucune modification d'état ne se présente jusqu'à la fin de l'Action, la prochaine modification d'état étant donc l'état final "Completed".

Le diagramme d'états se termine après que l'état final est fixé. Le Serveur FDI maintient le Nœud d'état et l'état final jusqu'à ce que les abonnements soient résiliés, soit par le Client FDI en supprimant l'élément contrôlé soit par l'expiration générale de la session du Client FDI avec le Serveur FDI.

NOTE 5 Aucune méthode KeepAlive séparée n'est nécessaire. La supervision de la session indique au Serveur FDI que le Client FDI est actif.

Après le terme de l'exécution de l'Action et après avoir retiré l'élément contrôlé, le verrou peut être retiré.

7.3 Définition du schéma d'Action FDI

Le schéma XML pour le XML envoyé entre le Client FDI et le Serveur FDI est défini à l'Annexe A.

NOTE Le schéma XML de l'Annexe A inclut également les définitions des UID.

ActionRequest est l'élément racine des documents XML échangés depuis le Serveur FDI vers le Client FDI, lors de l'exécution de l'Action. Ces documents XML sont envoyés au Client FDI avec le service Update (voir l'IEC 62541-4). La partie obligatoire de ces documents est l'élément ActionState, qui spécifie l'état actuel du diagramme d'états de l'Action correspondante (voir l'IEC 62769-3). Des données supplémentaires sont spécifiées si le Client FDI doit présenter une interface utilisateur de la part de l'Action. Le style et le contenu de l'interface utilisateur sont spécifiés dans ces données supplémentaires.

ActionResponse est l'élément racine des documents XML échangés depuis le Client FDI vers le Serveur FDI. Ces documents sont utilisés lorsque l'utilisateur fournit un retour d'information pour une demande d'UI. Le diagramme d'états de l'Action (tel que défini dans l'IEC 62769-3) doit être à l'état WaitingForFeedback ou WaitingForFeedbackA. Le Serveur FDI remet l'état sur Running ou Aborting après réception du retour d'information de l'utilisateur. Ces documents XML sont envoyés avec le service RespondAction (tel que défini dans l'IEC 62769-5).

Les arguments des Actions sont également spécifiés par des documents XML. Les arguments sont spécifiés avec le type ListOfActionArguments sous la forme de paires valeur-nom.

8 Description d'Interface Utilisateur (UID, User Interface Description)

8.1 Vue d'ensemble

Comme défini dans l'IEC 62769-5, le FunctionalGroup (Groupe fonctionnel) de niveau supérieur dans le Modèle d'Information peut contenir un Nœud UIDescription (UID) ou un ensemble de Nœuds UIPlugin (UIP). L'Attribut Value d'un Nœud UID est une chaîne et le Schéma XML de l'UID définit les contenus de ladite chaîne. Le schéma XML est défini à l'Annexe A.

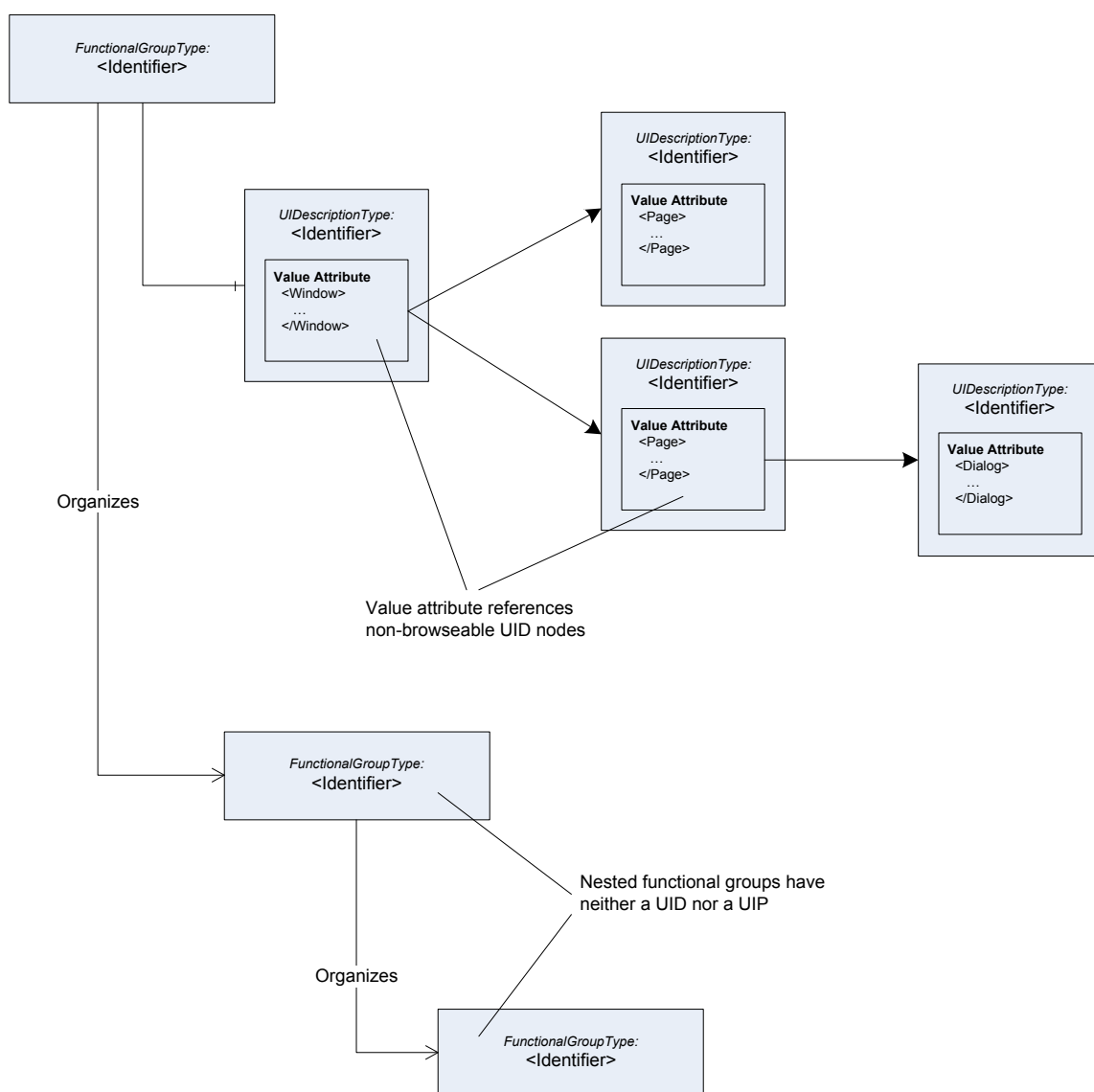
NOTE 1 Le schéma XML de l'Annexe A inclut également les définitions des Actions FDI.

En plus des Nœuds UID exposés dans le Modèle d'Information via les Groupes Fonctionnels, des Nœuds UID non consultables peuvent être rencontrés. Comme présenté à la Figure 9, les Nœuds UID non consultables sont liés à un Nœud UID parent via l'attribut NodePath. Le Nœud UID parent peut être soit un Nœud consultable soit un Nœud non consultable.

L'élément racine de l'Attribut Value d'un nœud UID d'un Groupe Fonctionnel doit être un élément Fenêtre, Dialogue, Menu ou Tableau. L'élément racine de l'Attribut Value d'un Nœud UID non consultable doit être un élément Fenêtre, Dialogue, Page, Menu ou Tableau.

L'Attribut Value du Nœud UID d'un Groupe Fonctionnel doit contenir suffisamment d'informations pour permettre au Client FDI de restituer les parties visibles de la vue. Le Serveur FDI peut omettre les parties de la vue qui ne sont pas visibles. À la place des informations omises, le Serveur FDI doit fournir une référence (via l'attribut NodePath) vers un Nœud UID non consultable contenant les informations manquantes. Le Client FDI peut utiliser le NodePath spécifié afin d'obtenir les informations manquantes à partir du Serveur FDI, si cela est jugé nécessaire.

NOTE 2 La seconde page d'une boîte de dialogue à onglets serait un exemple de partie non visible d'une fenêtre. L'étiquette de la seconde page est visible, mais le contenu de cette page ne l'est pas.



Anglais	Français
Organizes	Organise
Value attribute references non-browseable UID nodes	L'attribut Value référence des nœuds d'UID non consultables
Nested functional groups have neither a UID nor a UIP	Les groupes fonctionnels imbriqués ne comportent ni UID ni UIP

IEC

Figure 9 – Descriptions d'Interface Utilisateur

Le XML renvoyé du Serveur FDI vers le Client FDI contient des éléments relatifs à la structure de l'affichage et au contenu. Les éléments d'affichage définissent l'organisation, le positionnement et la structure visuelle de l'interface utilisateur. Les éléments de contenu sont des blocs fonctionnels de base de l'interface utilisateur.

NOTE 3 Le XML inclut uniquement des éléments "valides". De nouveaux documents XML seront renvoyés si la validité est modifiée.

Les éléments d'affichage sont les suivants:

- ColumnBreak (Césure de Colonne)
- Dialog (Boîte de Dialogue)
- EditDisplay (Editer l'Affichage)

- Group (Groupe)
- Menu
- Page
- RowBreak (Césure de Ligne)
- Table (Tableau)
- Window (Fenêtre)

Les éléments de contenu sont les suivants:

- Action
- Chart (Diagramme)
- Graph (Graphique)
- Grid (Grille)
- Image
- Parameter (Paramètre)
- Plugin (Plugiciel)
- Text (Texte)

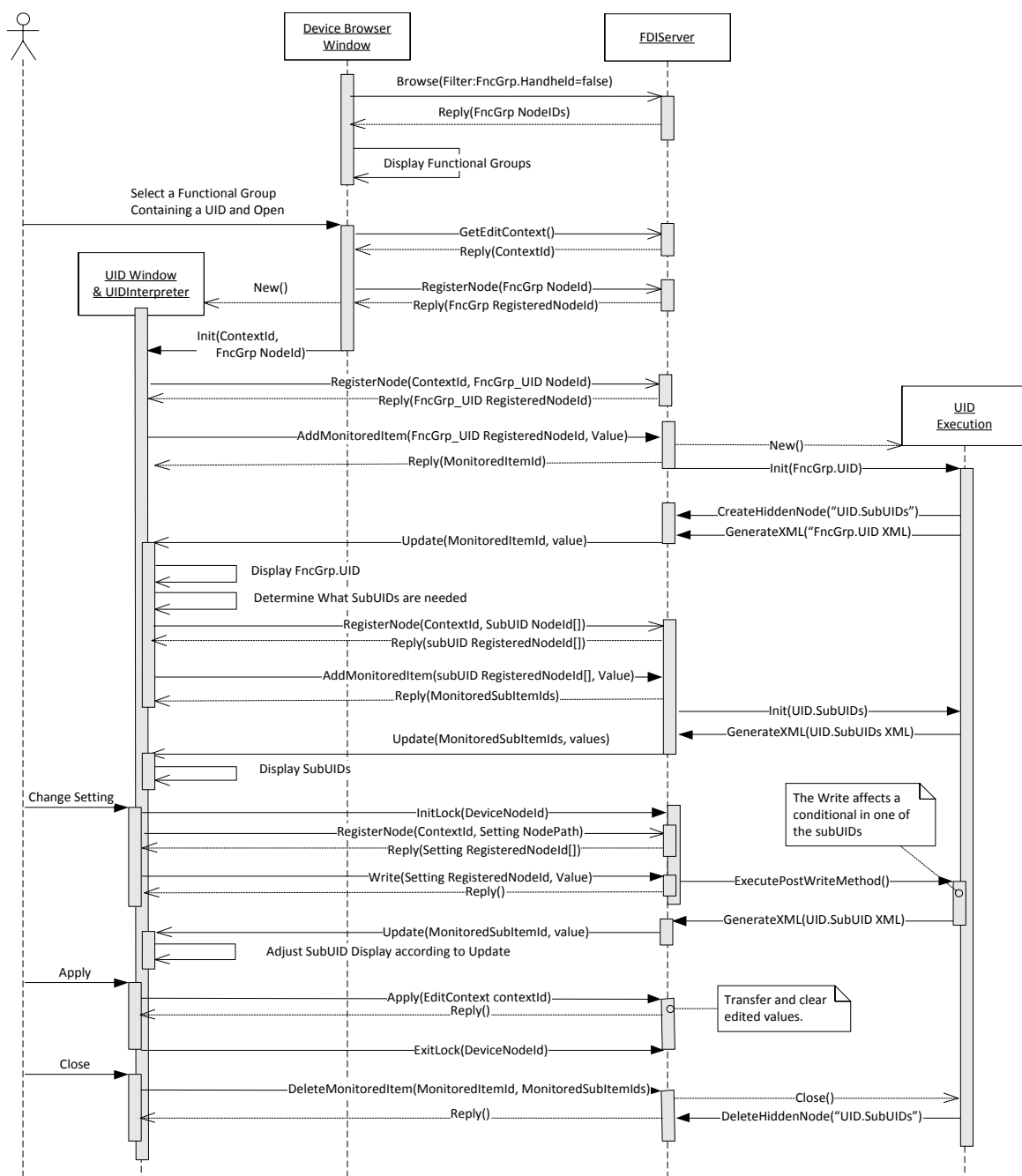
L'algorithme pour restituer ces éléments sur l'écran de l'ordinateur est spécifié dans l'IEC 61804-4.

NOTE 4 La définition des UID est fortement influencée par l'IEC 61804-3 et l'IEC 61804-4.

8.2 Exécution de l'UID

La Figure 10 présente un exemple de la séquence d'étapes utilisée par un Client FDI pour appeler et exécuter une Description d'Interface Utilisateur (UID). Cet exemple assume la condition préalable que le Client FDI a établi une session avec le Serveur FDI, que l'utilisateur a navigué vers un Appareil en naviguant ou en consultant le Modèle d'Information et que le Client FDI a présenté à l'utilisateur une liste de Groupes Fonctionnels. Cet exemple comprend une sous-UID référencée dans le XML de l'UID et contenant du contenu supplémentaire. L'exemple définit la modification d'un Paramètre utilisé par le sous-UID afin de calculer le contenu supplémentaire.

NOTE 1 Le diagramme montre UIDWindow, UIDInterpreter et Device Browser Window en tant que sous-systèmes du Client FDI. Il montre également UIDExecution en tant que sous-système du Serveur FDI. Ceci à des fins explicatives uniquement.



IEC

Anglais	Français
Device Browser Window	Fenêtre Exploration de l'Appareil
FDIServer	Serveur FDI
Display Functional Groups	Afficher les Groupes Fonctionnels
Select a Functional Group containing a UID and open	Sélectionner un Groupe Fonctionnel contenant un UID et l'Ouvrir
UID Window & UIDInterpreter	Fenêtre UID et Interprète UID
UID Execution	Exécution de l'UID
Display FncGrp.UID	Afficher FncGrp.UID
Determine What SubUIDs are needed	Déterminer quels Sous-UID sont nécessaires
Display SubUIDs	Afficher les Sous-UID
Change Setting	Modifier les paramètres

Anglais	Français
The Write affects a conditional in one of the subUIDs	Write affecte une conditionnelle dans l'un des sous-UID
Ajust SubUID Display according to Update	Ajuster l’Affichage du Sous-UID en fonction de la Mise à jour
Apply	Appliquer
Transfer and clear edited values	Transférer et effacer les valeurs éditées
Close	Fermer

Figure 10 – Diagramme de séquences de la Description d'Interface Utilisateur

La séquence à la Figure 10 commence avec l'utilisateur sélectionnant l'un des Groupes Fonctionnels pour l'ouvrir. La Fenêtre Exploration de l'Appareil (Device Browser Window) initialise l'ouverture de l'UID associée avec le Groupe Fonctionnel sélectionné, par l'acquisition d'un EditContext et par la création d'une nouvelle fenêtre UID et d'un interprète UID qui sera utilisé pour présenter l'UID à l'utilisateur. La fenêtre UID nouvellement créée est initialisée en lui fournissant l'EditContext et le NodeID du Groupe Fonctionnel sélectionné dans EditContext. Pour retrouver ce NodeId, la Méthode RegisterNodesByRelativePath est appelée en transmettant le BrowsePath du Groupe Fonctionnel et l'EditContext en tant qu'entrée.

L'élément Verrouillage est exigé pour le Service Write.

NOTE 2 L'exemple inclut donc les appels pour verrouiller (InitLock) et déverrouiller (ExitLock) l'Appareil.

La fenêtre UID commence par établir un abonnement à la valeur de l'UID contenu dans le Groupe Fonctionnel référencé en utilisant le service AddMonitoredItem OPC UA, afin d'obtenir le contenu (c'est-à-dire la représentation XML) de l'UID. Le Serveur répond à la demande d'abonnement en créant un diagramme d'Exécution d'UID et en l'initialisant en transmettant un identifiant interne de l'UID sélectionné. Le diagramme d'Exécution d'UID interprète la définition de l'UID, crée les Nœuds de sous-UID non consultables référencés dans le Modèle d'Information et génère la représentation XML de l'UID de niveau supérieur. La représentation XML générée est maintenue dans le Modèle d'Information pour prendre en charge l'abonnement de la Fenêtre UID. Les NodePath des sous-UID sont inclus dans le XML des UID parents. La méthode RegisterNodesByRelativePath permet de traduire les NodeSpecifier en NodeId. La Fenêtre UID peut utiliser ces NodeId afin de créer des abonnements ultérieurs. Le Serveur FDI fournit le contenu de l'UID de niveau supérieur auquel s'est abonnée la Fenêtre UID.

La Fenêtre UID interprète le contenu XML de la valeur UID mise à jour, fournie par l'abonnement, et restitue l'interface utilisateur. Elle collecte également les références aux sous-UID et émet une seconde demande de service AddMonitoredItem OPC UA pour inclure les sous-UID dans l'abonnement. Le Serveur FDI répond au service AddMonitoredItem en initialisant les Nœuds de sous-UID résultants dans la génération des représentations XML du contenu de sous-UID. Le Serveur FDI met à jour la Fenêtre UID avec la valeur générée de chacun des sous-UID inclus dans la demande AddMonitoredItem OPC UA.

NOTE 3 L'exemple utilise plusieurs sous-UID (mais ce nombre n'est pas spécifié). Par exemple, AddMonitoredItem(UID.SubUIDs.value) ajoute les Nœuds de tous les sous-UID à l'abonnement.

La Fenêtre UID répond aux mises à jour de l'abonnement en restituant les sous-UID dans l'interface utilisateur. En plus des sous-UID, les UID contiennent également les NodePath faisant référence aux paramètres et aux actions. RegisterNodesByRelativePath permet de traduire ces NodePath en NodeId et la Fenêtre UID utilise le service AddMonitoredItem pour s'abonner aux valeurs des paramètres. Le Serveur FDI fournit les valeurs à la Fenêtre UID. L'UID est maintenant entièrement affichée et préparée.

Lorsque l'utilisateur commence à modifier une valeur, la Fenêtre UID demande un verrou avec la Méthode InitLock.

Il existe différentes stratégies lorsqu'il est nécessaire d'obtenir un verrou. Au plus tard, un verrou doit être obtenu avant que des actions de pré-édition soient exécutées ou que la valeur soit écrite sur le Serveur FDI.

Si une action de pré-édition est définie pour le paramètre modifié, la Fenêtre UID appelle cette action.

Lorsque l'utilisateur a fini de modifier la valeur, celle-ci est écrite sur EditContext avec le service Write. Si une action de post-édition est définie pour le paramètre modifié, la Fenêtre UID appelle cette action.

L'Exécution de l'UID détermine que la modification de la valeur résulte dans la modification d'une conditionnelle contenue dans un des sous-UID. La valeur XML d'un des sous-UID qui est affecté, est régénérée et le Modèle d'Information est mis à jour. Le Serveur FDI réagit à la mise à jour du Modèle d'Information en notifiant la Fenêtre UID de la modification.

La Fenêtre UID traite la modification de la valeur du sous-UID et effectue les ajustements nécessaires sur l'interface utilisateur afin de répercuter la modification.

L'utilisateur complète la séquence de l'exemple en cliquant sur un bouton "appliquer", qui indique à la Fenêtre UID de se fermer. La Fenêtre UID appelle la Méthode Apply pour ses EditContext et les modifications sont appliquées à l'Appareil. Toutes les actions de pré- et de post-écriture sont exécutées. La Fenêtre UID supprime les abonnements en appelant le service DeleteMonitoredItem OPC UA.

Le Serveur FDI traite l'appel DeleteMonitoredItem OPC UA en supprimant les éléments de l'abonnement. Ce après quoi la Fenêtre UID appelle la Méthode Discard sur l'EditContext et le Serveur FDI supprime les Nœuds de sous-UID non consultables et ferme l'Exécution de l'UID.

Annexe A (normative)

Schéma XML

A.1 Généralités

Ce qui suit concerne les définitions de schéma XML pour les UID et les Actions. Les types sont énumérés dans l'ordre alphabétique. L'espace de nom "xs:" dans les éléments fait référence au Schéma XML W3C.

A.2 AbortRequestT

Ce Type spécifie une demande envoyée depuis un Serveur FDI vers un Client FDI, lorsqu'une Action est abandonnée. Le Client FDI peut informer l'utilisateur sur les raisons de l'abandon de l'Action. Après acquittement de l'utilisateur, le Client FDI renvoie un ActionResponse au Serveur FDI.

Le schéma XML pour un type AbortRequestT est le suivant:

```
<xs:complexType name="AbortRequestT">
  <xs:sequence>
    <xs:element name="Message" type="xs:string"/>
  </xs:sequence>
</xs:complexType>
```

Les éléments de type AbortRequestT sont décrits dans le Tableau A.1.

Tableau A.1 – Eléments d'AbortRequestT

Élément	Description
Message	Cet élément exigé spécifie la raison pour laquelle l'action a été abandonnée.

A.3 AccessT

Ce type spécifie si l'accès doit être EN LIGNE ou HORS LIGNE.

Le schéma XML pour un type d'énumération AccessT est le suivant:

```
<xs:simpleType name="AccessT">
  <xs:restriction base="xs:string">
    <xs:enumeration value="ONLINE"/>
    <xs:enumeration value="OFFLINE"/>
  </xs:restriction>
</xs:simpleType>
```

Les valeurs de type d'énumération AccessT sont décrites dans le Tableau A.2.

Tableau A.2 – Enumérations d'AccessT

Enumération	Description
ONLINE	L'accès doit être réalisé EN LIGNE.

OFFLINE	L'accès doit être réalisé HORS LIGNE.
---------	---------------------------------------

A.4 AcknowledgementRequestT

Ce type spécifie une demande envoyée depuis un Serveur FDI vers un Client FDI lorsque l'utilisateur a besoin d'acquitter une condition ou un état au sein d'une Action. Après acquittement de l'utilisateur, le Client FDI renvoie un AcknowledgementResponse au Serveur FDI. AcknowledgementRequest doit être utilisé uniquement pour acquitter des conditions de fonctionnement normal. AbortRequest doit être utilisé afin d'acquitter une condition qui aboutit à l'abandon de l'Action.

Le schéma XML pour un type AcknowledgementRequestT est le suivant:

```
<xs:complexType name="AcknowledgementRequestT">
  <xs:sequence>
    <xs:element name="Message" type="xs:string"/>
  </xs:sequence>
</xs:complexType>
```

Les éléments de type AcknowledgementRequestT sont décrits dans le Tableau A.3.

Tableau A.3 – Eléments d'AcknowledgementRequestT

Élément	Description
Message	Cet élément exigé spécifie la condition pour laquelle l'acquittement de l'utilisateur est demandé, sous la forme d'un message envoyé à l'utilisateur.

A.5 ActionListT

Ce type spécifie une liste des éléments Action.

Le schéma XML pour un type ActionListT est le suivant:

```
<xs:complexType name="ActionListT">
  <xs:sequence maxOccurs="unbounded">
    <xs:element name="Action" type="clnt:ActionT"/>
  </xs:sequence>
</xs:complexType>
```

Les éléments de type ActionListT sont décrits dans le Tableau A.4.

Tableau A.4 – Eléments d'ActionListT

Élément	Description
Action	Un élément de la liste.

A.6 AbortingNotificationT

Ce type est utilisé dans l'élément ActionRequest pour signaler au client que l'action a été annulée sur le serveur.

Le schéma XML pour un type AbortingNotificationT est le suivant:

```
<xs:complexType name="AbortingNotificationT"/>
```

A.7 ActionRequestT

Ce type spécifie une demande d'action provenant du Serveur FDI.

Le schéma XML pour un type ActionRequestT est le suivant:

```
<xs:complexType name="ActionRequestT">
  <xs:sequence>
    <xs:element name="EditContext" type="xs:string"/>
    <xs:choice>
      <xs:element name="AbortingNotification" type="clnt:AbortingNotificationT"/>
      <xs:element name="AcknowledgementRequest"
        type="clnt:AcknowledgementRequestT"/>
      <xs:element name="AbortRequest" type="clnt:AbortRequestT"/>
      <xs:element name="UIDRequest" type="clnt:UidRequestT"/>
      <xs:element name="SelectionRequest" type="clnt:SelectionRequestT"/>
      <xs:element name="InputRequest" type="clnt:InputRequestT"/>
      <xs:element name="ParameterInputRequest"
        type="clnt:ParameterInputRequestT"/>
      <xs:element name="InfoRequest" type="clnt:InfoRequestT"/>
      <xs:element name="DelayMessageRequest" type="clnt:DelayMessageRequestT"/>
    </xs:choice>
  </xs:sequence>
</xs:complexType>
```

Les éléments de type ActionRequestT sont décrits dans le Tableau A.5.

Tableau A.5 – Eléments d'ActionRequestT

Elément	Description
EditContext	Cet élément spécifie l'élément EditContext à utiliser lors de la modification de Variables pour cette Action.
AbortingNotification	Cet élément précise que l'action a été abandonnée sur le serveur. Pour garantir le bon déroulement du processus d'abandon, l'Interface Utilisateur ne doit pas permettre l'annulation d'autres requêtes d'action.
AcknowledgementRequest	Cet élément facultatif spécifie une demande depuis un Serveur FDI vers un Client FDI pour que l'utilisateur acquitte une condition. Le Client ne répondra pas au Serveur FDI avant que l'utilisateur ait acquitté la condition.
AbortRequest	Cet élément facultatif spécifie une demande depuis un Serveur FDI vers un Client FDI pour informer l'utilisateur que l'Action va être abandonnée.
UIDRequest	Cet élément facultatif spécifie une demande depuis un Serveur FDI vers un Client FDI pour afficher une interface utilisateur complexe.
SelectionRequest	Cet élément facultatif spécifie une demande depuis un Serveur FDI vers un Client FDI pour que l'utilisateur choisisse différentes alternatives à partir d'une liste.
InputRequest	Cet élément facultatif spécifie une demande depuis un Serveur FDI vers un Client FDI pour que l'utilisateur puisse éditer des données.
ParameterInputRequest	Cet élément facultatif spécifie une demande depuis un Serveur FDI vers un Client FDI pour que l'utilisateur édite un paramètre.
InfoRequest	Cet élément facultatif spécifie une demande depuis un Serveur FDI vers un Client FDI pour qu'un message soit affiché pour l'utilisateur. Ce message n'exige pas d'acquiescement de l'utilisateur et le Client FDI répondra immédiatement après avoir affiché le message.

Elément	Description
DelayMessageRequest	Ce type spécifie une demande depuis un Serveur FDI vers un Client FDI lorsqu'un délai de retard est demandé au cours d'une Action. Le Client FDI informe l'utilisateur du motif et de la durée du retard. Le Client FDI estime la durée du délai de retard demandé et renvoie un élément DelayMessageResponse (Réponse à un Message de Retard) au Serveur FDI une fois que le délai alloué est écoulé.

A.8 ActionResponseT

Ce type spécifie une réponse d'action vers le Serveur FDI.

Le schéma XML pour un type ActionResponseT est le suivant:

```
<xs:complexType name="ActionResponseT">
  <xs:choice>
    <xs:element name="AbortingNotificationConfirmation" type="clnt:ResponseT"/>
    <xs:element name="AcknowledgementResponse" type="clnt:ResponseT"/>
    <xs:element name="AbortResponse" type="clnt:ResponseT"/>
    <xs:element name="UidResponse" type="clnt:UidResponseT"/>
    <xs:element name="SelectionResponse" type="clnt:SelectionResponseT"/>
    <xs:element name="InputResponse" type="clnt:InputResponseT"/>
    <xs:element name="ParameterInputResponse" type="clnt:ResponseT"/>
    <xs:element name="InfoResponse" type="clnt:ResponseT"/>
    <xs:element name="DelayMessageResponse" type="clnt:ResponseT"/>
  </xs:choice>
</xs:complexType>
```

Les éléments de type ActionResponseT sont décrits dans le Tableau A.6.

Tableau A.6 – Eléments d'ActionResponseT

Elément	Description
AbortingNotificationConfirmation	Cet élément facultatif spécifie la réponse d'un Client FDI à une demande AbortingNotification d'un Serveur FDI.
AcknowledgementResponse	Cet élément facultatif spécifie la réponse d'un Client FDI à une demande AcknowledgementRequest par le Serveur FDI.
AbortResponse	Cet élément facultatif spécifie la réponse d'un Client FDI à une demande AbortRequest par le Serveur FDI.
UidResponse	Cet élément facultatif spécifie la réponse d'un Client FDI à une demande UIDRequest par le Serveur FDI.
SelectionResponse	Cet élément facultatif spécifie la réponse d'un Client FDI à une demande SelectionRequest par le Serveur FDI.
InputResponse	Cet élément facultatif spécifie la réponse d'un Client FDI à une demande InputRequest par le Serveur FDI.
ParameterInputResponse	Cet élément facultatif spécifie la réponse d'un Client FDI à une demande ParameterInputRequest d'un Serveur FDI.
InfoResponse	Cet élément facultatif spécifie la réponse d'un Client FDI à une demande InfoRequest par le Serveur FDI.
DelayMessageResponse	Cet élément facultatif spécifie la réponse d'un Client FDI à une demande DelayMessageRequest par le Serveur FDI.

A.9 ActionT

Ce type spécifie une Action, qui représente une séquence d'étapes nécessitant la collaboration entre un Client FDI et un Serveur FDI.

Le schéma XML pour un type ActionT est le suivant:

```
<xs:complexType name="ActionT">
  <xs:complexContent>
    <xs:extension base="clnt:UiElementT">
      <xs:sequence>
        <xs:element name="Name" type="xs:string"/>
        <xs:element name="Access" type="clnt:AccessT" minOccurs="0"/>
        <xs:element name="Class" type="clnt:ActionClassT" minOccurs="0"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

Les éléments de type ActionT sont décrits dans le Tableau A.7.

Tableau A.7 – Éléments d'ActionT

Élément	Description
Name	Cet élément exigé indique le nom de l'Action, à laquelle la méthode InvokeAction d'un Nœud UID peut être appliquée dans un Serveur FDI.
Access	Cet élément facultatif spécifie si l'action doit être réalisée EN LIGNE ou HORS LIGNE.
Class	Cet élément facultatif spécifie l'attribut EDD CLASS de l'action.

A.10 AxisListT

Ce type présente la liste des éléments d'axe désignés d'un graphique ou d'un diagramme.

Le schéma XML pour un type AxisListT est le suivant:

```
<xs:complexType name="AxisListT">
  <xs:sequence minOccurs="0" maxOccurs="unbounded">
    <xs:element name="Axis" type="clnt:AxisT"/>
  </xs:sequence>
</xs:complexType>
```

Les éléments de type AxisListT sont décrits dans le Tableau A.8.

Tableau A.8 – Éléments d'AxisListT

Élément	Description
Axe	Un élément d' AxisListT.

A.11 AxisT

Ce type spécifie un axe dans un graphique ou un diagramme.

Le schéma XML pour un type AxisT est le suivant:

```
<xs:complexType name="AxisT">
  <xs:complexContent>
    <xs:extension base="clnt:LabelHelpT">
```

```

<xs:sequence>
  <xs:element name="MaximumValue" type="clnt:VariantT" minOccurs="0"/>
  <xs:element name="MinimumValue" type="clnt:VariantT" minOccurs="0"/>
  <xs:element name="DisplayedRange" minOccurs="0">
    <xs:complexType>
      <xs:attribute name="NodePathViewMinimum" type="xs:string"
        use="required"/>
      <xs:attribute name="NodePathViewMaximum" type="xs:string"
        use="required"/>
    </xs:complexType>
  </xs:element>
  <xs:element name="Scaling" type="clnt:ScalingT" default="Linear"
    minOccurs="0"/>
  <xs:element name="Unit" type="xs:string" minOccurs="0"/>
</xs:sequence>
<xs:attribute name="Name" type="xs:string" use="required"/>
</xs:extension>
</xs:complexContent>
</xs:complexType>

```

Les attributs de type AxisT sont décrits dans le Tableau A.9.

Tableau A.9 – Attributs d'AxisT

Attribut	Description
Name	Nom unique de l'axe dans le contexte du Diagramme/Graphique dudit axe.

Les éléments de type AxisT sont décrits dans le Tableau A.10.

Tableau A.10 – Eléments d'AxisT

Elément	Description
MaximumValue	Cet élément exigé spécifie la valeur la plus grande située entre les limites de l'axe.
MinimumValue	Cet élément exigé spécifie la valeur la plus petite située entre les limites de l'axe.
DisplayedRange	Sn1p1t Sn1p1t
Scaling	Cet élément facultatif spécifie la manière dont il convient d'échelonner la valeur. La valeur par défaut est Linear (Linéaire).
Unit	Cet élément facultatif spécifie l'unité technique de l'axe.

A.12 BitEnumerationItemListT

Ce type spécifie le contenu d'une énumération de bits.

Le schéma XML pour un type BitEnumerationItemListT est le suivant:

```

<xs:complexType name="BitEnumerationItemListT">
  <xs:sequence maxOccurs="unbounded">
    <xs:element name="BitEnumerationItem" type="clnt:BitEnumerationItemT"/>
  </xs:sequence>
</xs:complexType>

```

Les éléments de type BitEnumerationItemListT sont décrits dans le Tableau A.11.

Tableau A.11 – Eléments de BitEnumerationItemT

Elément	Description
BitEnumerationItem	Un élément de l'énumération de bits.

A.13 BitEnumerationItemT

Ce type spécifie la signification d'un bit simple d'une valeur en mode point.

Le schéma XML pour un type BitEnumerationItemT est le suivant:

```
<xs:complexType name="BitEnumerationItemT">
  <xs:complexContent>
    <xs:extension base="clnt:LabelHelpT">
      <xs:sequence>
        <xs:element name="Value" type="xs:unsignedLong"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

Les éléments de type BitEnumerationItemT sont décrits dans le Tableau A.12.

Tableau A.12 – Eléments de BitEnumerationItemT

Elément	Description
Value (Valeur)	Cet élément exigé spécifie un bit sous la forme d'un masque binaire, c'est-à-dire, 0x1 spécifie le bit le moins significatif, 0x2 spécifie le second bit le moins significatif, 0x4 spécifie le troisième bit le plus significatif, et ainsi de suite.

A.14 ButtonListT

Ce type spécifie une liste des éléments Button (Bouton). La réponse UID contient l'index basé sur 0 du bouton sélectionné.

Le schéma XML pour un type ButtonListT est le suivant:

```
<xs:complexType name="ButtonListT">
  <xs:sequence maxOccurs="unbounded">
    <xs:element name="Button" type="clnt:LabelT"/>
  </xs:sequence>
</xs:complexType>
```

Les éléments de type ButtonListT sont décrits dans le Tableau A.13.

Tableau A.13 – Eléments de ButtonListT

Elément	Description
Button	Un élément de la liste de boutons.

A.15 ChartT

Ce type spécifie un graphique qui affiche les données d'un appareil. Les données sont lues à partir de l'appareil de manière périodique et continue. Il est affiché lors de l'arrivée de nouvelles données.

Le schéma XML pour un type ChartT est le suivant:

```
<xs:complexType name="ChartT">
  <xs:complexContent>
    <xs:extension base="clnt:UiElementSizeableT">
      <xs:sequence>
        <xs:element name="Length" type="xs:nonNegativeInteger" default="600000"
          minOccurs="0"/>
        <xs:element name="Type" type="clnt:ChartTypeT" default="Strip"
          minOccurs="0"/>
        <xs:element name="CycleTime" type="xs:nonNegativeInteger" default="1000"
          minOccurs="0"/>
        <xs:element name="AxisList" type="clnt:AxisListT">
          <xs:key name="AxisKey">
            <xs:selector xpath="clnt:Axis"/>
            <xs:field xpath="@Name"/>
          </xs:key>
        </xs:element>
        <xs:element name="SourceList" type="clnt:SourceListT"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

Les éléments de type ChartT sont décrits dans le Tableau A.14.

Tableau A.14 – Eléments de ChartT

Elément	Description
Length	Cet élément facultatif spécifie la longueur du temps, en millisecondes, qui est affichée par le diagramme. Le nombre d'échantillons affichés par un diagramme peut être calculé en divisant Length (Longueur) par "CycleTime". La valeur par défaut est 600 000.
Type	Cet élément facultatif spécifie le type du diagramme. La valeur par défaut est Strip.
CycleTime	Cet élément facultatif spécifie la fréquence, en millisecondes, pour laquelle les données du diagramme sont mises à jour. La valeur par défaut est 1 000.
AxisList	Cet élément exigé indique les données de l'axe vertical affichées par le diagramme.
SourceList	Cet élément exigé spécifie les données affichées par le diagramme.

A.16 ChartTypeT

Ce type spécifie l'apparence et le comportement général d'un diagramme.

Le schéma XML pour un type ChartTypeT est le suivant:

```
<xs:simpleType name="ChartTypeT">
  <xs:restriction base="xs:string">
```

```

    <xs:enumeration value="Gauge"/>
    <xs:enumeration value="HorizontalBar"/>
    <xs:enumeration value="Scope"/>
    <xs:enumeration value="Strip"/>
    <xs:enumeration value="Sweep"/>
    <xs:enumeration value="VerticalBar"/>
  </xs:restriction>
</xs:simpleType>

```

Les valeurs de type d'énumération ChartTypeT sont décrites dans le Tableau A.15.

Tableau A.15 – Enumérations de ChartTypeT

Enumération	Description
Gauge	Une valeur source unique est affichée sous la forme de jauge, dont la représentation graphique est similaire celle d'une jauge d'essence d'un véhicule.
HorizontalBar	Les valeurs sources sont affichées sous la forme de barres horizontales.
Scope	Les valeurs sources sont affichées sous la forme d'une courbe allant de la gauche vers la droite. Lorsque les valeurs sources atteignent l'extrémité droite de la zone affichée, la zone d'affichage est effacée et les nouvelles valeurs sources sont affichées à partir de l'extrémité gauche.
Strip	Les valeurs sources sont affichées sous la forme d'une courbe allant de la gauche vers la droite. Lorsque les valeurs sources atteignent l'extrémité droite de la zone affichée, la zone d'affichage défile avec les valeurs sources qui disparaissent au fur et à mesure qu'elles arrivent à l'extrémité gauche et les nouvelles valeurs sources apparaissent à la droite.
Sweep	Les valeurs sources sont affichées sous la forme d'une courbe allant de la gauche vers la droite. Lorsque les valeurs sources atteignent l'extrémité droite de la zone affichée, les nouvelles valeurs sources sont affichées à gauche, mais au contraire de "Scope", seule la portion de la zone d'affichage nécessaire pour afficher les nouvelles valeurs sources est effacée.
VerticalBar	Les valeurs sources sont affichées sous forme de barres verticales.

A.17 ColorNameT

Ce type spécifie le nom d'une couleur prédéfinie.

Le schéma XML pour un type ColorNameT est le suivant:

```

<xs:simpleType name="ColorNameT">
  <xs:restriction base="xs:string">
    <xs:enumeration value="Aqua"/>
    <xs:enumeration value="Black"/>
    <xs:enumeration value="Blue"/>
    <xs:enumeration value="Fuchsia"/>
    <xs:enumeration value="Gray"/>
    <xs:enumeration value="Green"/>
    <xs:enumeration value="Lime"/>
    <xs:enumeration value="Maroon"/>
    <xs:enumeration value="Navy"/>
    <xs:enumeration value="Olive"/>
    <xs:enumeration value="Purple"/>
    <xs:enumeration value="Red"/>
    <xs:enumeration value="Silver"/>
    <xs:enumeration value="Teal"/>
  </xs:restriction>
</xs:simpleType>

```

```

    <xs:enumeration value="White"/>
    <xs:enumeration value="Yellow"/>
  </xs:restriction>
</xs:simpleType>

```

Les valeurs de type d'énumération ColorNameT sont décrites dans le Tableau A.16.

Tableau A.16 – Enumérations de ColorNameT

Enumération	Description
Agua	Valeur RGB #00FFFF.
Black	Valeur RGB #000000.
Blue	Valeur RGB #0000FF.
Fuchsia	Valeur RGB #FF00FF.
Gray	Valeur RGB #808080.
Green	Valeur RGB #008000.
Lime	Valeur RGB #00FF00.
Maroon	Valeur RGB #800000.
Navy	Valeur RGB #000080.
Olive	Valeur RGB #808000.
Purple	Valeur RGB #800080.
Red	Valeur RGB #FF0000.
Silver	Valeur RGB #C0C0C0.
Teal	Valeur RGB #008080.
White	Valeur RGB #FFFFFF.
Yellow	Valeur RGB #FFFF00.

A.18 ColorT

Ce type spécifie une couleur, soit avec un nom ou une valeur RGB.

Le schéma XML pour un type ColorT est le suivant:

```

<xs:simpleType name="ColorT">
  <xs:union memberTypes="clnt:ColorNameT clnt:ColorValueT"/>
</xs:simpleType>

```

A.19 ColorValueT

Ce type spécifie une valeur RGB consistant en un caractère dièse (#) suivi de trois ou six chiffres hexadécimaux. Si trois chiffres hexadécimaux sont spécifiés, chaque caractère est répété, par exemple, #F0F équivaut à #FF00FF.

Le schéma XML pour un type ColorValueT est le suivant:

```

<xs:simpleType name="ColorValueT">
  <xs:restriction base="xs:string">
    <xs:pattern value="#[0-9a-fA-F]{3}"/>
    <xs:pattern value="#[0-9a-fA-F]{6}"/>
  </xs:restriction>
</xs:simpleType>

```


A.20 ColumnBreakT

Ce type représente un saut de colonne.

Le schéma XML pour un type ColumnBreakT est le suivant:

```
<xs:complexType name="ColumnBreakT"/>
```

A.21 DateTimeDataT

Ce type spécifie le type de données d'une valeur date/heure.

Le schéma XML pour un type d'énumération DateTimeDataT est le suivant:

```
<xs:simpleType name="DateTimeDataT">
  <xs:restriction base="xs:string">
    <xs:enumeration value="Date"/>
    <xs:enumeration value="Time"/>
    <xs:enumeration value="DateTime"/>
    <xs:enumeration value="Duration"/>
    <xs:enumeration value="TimeValue"/>
  </xs:restriction>
</xs:simpleType>
```

Les valeurs de type d'énumération DateTimeDataT sont décrites dans le Tableau A.17.

Tableau A.17 – Enumérations de DateTimeDataT

Enumération	Description
Date	Une valeur représentant une date sans l'heure.
Time	Une valeur représentant l'heure sans la date.
DateTime	Une valeur représentant une date et l'heure.
Duration	Une valeur représentant une durée de temps.
TimeValue	Une valeur représentant une heure de la journée.

A.22 DelayMessageRequestT

Ce type spécifie une demande envoyée depuis un Serveur FDI vers un Client FDI, lorsqu'un retard apparaît au sein d'une Action. Le Client FDI informe l'utilisateur de la raison du retard et de sa durée prévue. Le Client FDI renvoie ensuite un élément ActionResponse au Serveur FDI. Le Serveur FDI peut envoyer DelayMessageRequests de manière fréquente, avec des informations à jour concernant la durée du retard. Lorsque le retard est fini, le Serveur FDI envoie DelayMessageRequest avec l'élément SecondsToWait mis sur zéro.

Le schéma XML pour un type DelayMessageRequestT est le suivant:

```
<xs:complexType name="DelayMessageRequestT">
  <xs:sequence>
    <xs:element name="Message" type="xs:string"/>
    <xs:element name="SecondsToWait" type="xs:unsignedLong"/>
  </xs:sequence>
</xs:complexType>
```

Les éléments de type DelayMessageRequestT sont décrits dans le Tableau A.18.

Tableau A.18 – Eléments de DelayMessageRequestT

Elément	Description
Message	Cet élément exigé spécifie le message devant être affiché à l'utilisateur.
SecondsToWait	Cet élément exigé spécifie le nombre de secondes que le retard est censé durer.

A.23 DiagramLineT

Ce type spécifie une source abstraite.

Le schéma XML pour un type DiagramLineT est le suivant:

```
<xs:complexType name="DiagramLineT" abstract="true">
  <xs:complexContent>
    <xs:extension base="clnt:UiElementT">
      <xs:sequence>
        <xs:element name="Emphasis" type="xs:boolean" default="false"
          minOccurs="0"/>
        <xs:element name="LineColor" type="clnt:ColorT" minOccurs="0"/>
        <xs:element name="LineType" type="clnt:LineTypeT" minOccurs="0"/>
        <xs:element name="VerticalAxis" minOccurs="0">
          <xs:complexType>
            <xs:attribute name="AxisRef" type="xs:string"/>
          </xs:complexType>
        </xs:element>
        <xs:element name="InitActionList" type="clnt:ActionListT" minOccurs="0"/>
        <xs:element name="RefreshActionList" type="clnt:ActionListT"
          minOccurs="0"/>
        <xs:element name="ExitActionList" type="clnt:ActionListT" minOccurs="0"/>
      </xs:sequence>
      <xs:attribute name="Name" type="xs:string" use="required"/>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

Les attributs de type DiagramLineT sont décrits dans le Tableau A.19.

Tableau A.19 – Attributs de DiagramLineT

Attribut	Description
Name	Nom unique de DiagramlineT dans le contexte du Diagramme/Graphique dudit élément DiagramlineT.

Les éléments de type DiagramLineT sont décrits dans le Tableau A.20.

Tableau A.20 – Eléments de DiagramLineT

Elément	Description
Emphasis	Cet élément facultatif spécifie s'il convient de mettre en avant ces données de manière graphique. La valeur par défaut est "false".
LineColor	Cet élément facultatif spécifie la couleur dans laquelle sont affichées les données. La valeur par défaut est déterminée par le Client FDI.

LineType	Cet élément facultatif spécifie le type de ligne à afficher. Il convient d'afficher chaque donnée, dont l'élément LineType est le même, avec la même structure (motif de ligne, épaisseur de ligne, etc.).
VerticalAxis	Cet élément facultatif spécifie l'apparence de l'axe vertical. Sn1p1t
InitActionList	Cet élément facultatif spécifie les Actions devant être exécutées avant l'affichage des données.
RefreshActionList	Cet élément facultatif spécifie les Actions devant être exécutées après la lecture des données par l'appareil, mais avant qu'elles soient affichées.
ExitActionList	Cet élément facultatif spécifie les Actions devant être exécutées lors de la fermeture du graphique ou du diagramme contenant les données.

A.24 EnumerationItemListT

Ce type spécifie le contenu d'une énumération.

Le schéma XML pour un type EnumerationItemListT est le suivant:

```
<xs:complexType name="EnumerationItemListT">
  <xs:sequence maxOccurs="unbounded">
    <xs:element name="EnumerationItem" type="clnt:EnumerationItemT"/>
  </xs:sequence>
</xs:complexType>
```

Les éléments de type EnumerationItemListT sont décrits dans le Tableau A.21.

Tableau A.21 – Eléments d'EnumerationItemListT

Elément	Description
EnumerationItem	Un élément de l'énumération.

A.25 EnumerationItemT

Ce type spécifie l'une des valeurs possibles d'une valeur énumérée.

Le schéma XML pour un type EnumerationItemT est le suivant:

```
<xs:complexType name="EnumerationItemT">
  <xs:complexContent>
    <xs:extension base="clnt:LabelHelpT">
      <xs:sequence>
        <xs:element name="Value" type="xs:unsignedLong"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

Les éléments de type EnumerationItemT sont décrits dans le Tableau A.22.

Tableau A.22 – Éléments d'EnumerationItemT

Élément	Description
Value (Valeur)	Cet élément exigé spécifie l'identifiant unique de l'entrée d'une liste dans le contexte d'un élément de liste parente.

A.26 FormatSpecifieiT

Ce type définit le spécificateur de format de paramètres qui est admis (compatible avec ANSI C).

Le schéma XML pour un type FormatSpecifieiT est le suivant:

```
<xs:simpleType name="FormatSpecifieiT">
  <xs:restriction base="xs:string">
    <xs:pattern value="%?[#-0+ ]*\d*(\.\d*)?[hlL]?[dioxXucsfeEgGpn%]" />
  </xs:restriction>
</xs:simpleType>
```

A.27 GraphT

Ce type spécifie un graphique affichant graphiquement un ensemble fini de données depuis un appareil.

Le schéma XML pour un type GraphT est le suivant:

```
<xs:complexType name="GraphT">
  <xs:complexContent>
    <xs:extension base="clnt:UiElementSizeableT">
      <xs:sequence>
        <xs:element name="CycleTime" type="xs:nonNegativeInteger" default="0"
          minOccurs="0"/>
        <xs:element name="AxisList" type="clnt:AxisListT">
          <xs:key name="GraphAxisKey">
            <xs:selector xpath="clnt:Axis"/>
            <xs:field xpath="@Name"/>
          </xs:key>
        </xs:element>
        <xs:element name="HorizontalAxis" minOccurs="0">
          <xs:complexType>
            <xs:attribute name="AxisRef" type="xs:string"/>
          </xs:complexType>
        </xs:element>
        <xs:element name="WaveformList" type="clnt:WaveformListT"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

Les éléments de type GraphT sont décrits dans le Tableau A.23.

Tableau A.23 – Éléments de GraphT

Élément	Description
CycleTime	Cet élément facultatif spécifie la fréquence, en millisecondes, à laquelle l'ensemble de données est relu depuis l'appareil et affiché de nouveau. Si l'élément CycleTime est à 0, le graphique n'est pas rafraîchi. La valeur par défaut est 0.
AxisList	Cet élément exigé spécifie tous les axes auxquels le graphique ou les formes d'onde font référence.
HorizontalAxis	Sn1p1t
WaveformList	Cet élément exigé spécifie les formes d'onde affichées sur le graphique.

A.28 GridT

Ce type spécifie une grille qui affiche des données dans une grille ayant une forme de tableau.

Le schéma XML pour un type GridT est le suivant:

```
<xs:complexType name="GridT">
  <xs:complexContent>
    <xs:extension base="clnt:UiElementSizeableT">
      <xs:sequence>
        <xs:element name="Handling" type="clnt:HandlingT" default="rw"
          minOccurs="0"/>
        <xs:element name="Orientation" type="clnt:OrientationT"
          default="Vertical" minOccurs="0"/>
        <xs:element name="VectorList" type="clnt:VectorListT"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

Les éléments de type GridT sont décrits dans le Tableau A.24.

Tableau A.24 – Éléments de GridT

Élément	Description
Handling	Cet élément facultatif spécifie si les données au sein de la grille peuvent être modifiées. La valeur par défaut est ReadWrite.
Orientation	Cet élément facultatif spécifie si les vecteurs sont affichés horizontalement ou verticalement. La valeur par défaut est Vertical.
VectorList	Cet élément exigé spécifie les données affichées au sein de la grille.

A.29 HandlingT

Ce type spécifie la manière par laquelle un élément peut être accessible (ro = lecture seule, wo = écriture seule, rw = lecture et écriture).

Le schéma XML pour un type d'énumération HandlingT est le suivant:

```
<xs:simpleType name="HandlingT">
```

```

<xs:restriction base="xs:string">
  <xs:enumeration value="rw"/>
  <xs:enumeration value="wo"/>
  <xs:enumeration value="ro"/>
</xs:restriction>
</xs:simpleType>

```

Les valeurs de type d'énumération HandlingT sont décrites dans le Tableau A.25.

Tableau A.25 – Enumérations de HandlingT

Enumération	Description
rw	L'élément peut uniquement être lu.
wo	L'élément peut uniquement être écrit.
ro	L'élément peut être lu et écrit.

A.30 ImageT

Ce type spécifie une entité visuelle telle qu'une image ou une illustration.

Le schéma XML pour un type ImageT est le suivant:

```

<xs:complexType name="ImageT">
  <xs:complexContent>
    <xs:extension base="clnt:UiElementT">
      <xs:sequence>
        <xs:element name="Inline" type="xs:boolean" default="false"
          minOccurs="0"/>
        <xs:element name="AlignLeft" type="xs:boolean" default="false"
          minOccurs="0"/>
        <xs:element name="AlignRight" type="xs:boolean" default="false"
          minOccurs="0"/>
        <xs:element name="Link" minOccurs="0">
          <xs:complexType>
            <xs:choice>
              <xs:element name="Menu" type="clnt:MenuReferenceT"/>
              <xs:element name="Plugin" type="clnt:PluginT"/>
              <xs:element name="Action" type="clnt:ActionT"/>
            </xs:choice>
          </xs:complexType>
        </xs:element>
      </xs:sequence>
      <xs:attribute name="NodePath" type="xs:string" use="required"/>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>

```

Les attributs de type ImageT sont décrits dans le Tableau A.26.

Tableau A.26 – Attributs d'ImageT

Attribut	Description
NodePath	Il s'agit d'un attribut obligatoire qui donne le chemin qualifié du Nœud dans le Modèle d'Appareil.

Les éléments de type ImageT sont décrits dans le Tableau A.27.

Tableau A.27 – Eléments d'ImageT

Elément	Description
Inline	Si l'image est dans une fenêtre, une boîte de dialogue, une page ou un groupe, et que le qualificateur en ligne n'est pas spécifié, il convient de couvrir l'image avec la largeur de MENU; autrement, il convient d'afficher l'image en ligne avec d'autres éléments d'affichage "layoutitems" de "itemlist".
AlignLeft	Si l'image est dans une fenêtre, une boîte de dialogue, une page ou un groupe, et que le qualificateur en ligne n'est pas spécifié, il convient de couvrir l'image avec la largeur de MENU; autrement, il convient d'afficher l'image en ligne avec d'autres éléments d'affichage "layoutitems" de "itemlist".
AlignRight	Si l'image est dans une fenêtre, une boîte de dialogue, une page ou un groupe, et que le qualificateur en ligne n'est pas spécifié, il convient de couvrir l'image avec la largeur de MENU; autrement, il convient d'afficher l'image en ligne avec d'autres éléments d'affichage "layoutitems" de "itemlist".
Link	Cet élément facultatif spécifie un lien vers les éléments Window, Dialog, Menu, Table, ou Action relatifs à l'image.

A.31 InfoRequestT

Ce type spécifie une demande envoyée depuis un Serveur FDI vers un Client FDI lorsqu'un message doit être affiché selon les besoins de l'utilisateur pendant une Action. Contrairement à AcknowledgementRequest, à la réception d'un InfoRequest, le Client FDI n'attend pas l'acquiescement du message par l'utilisateur. Une fois le message affiché, le Client FDI renvoie immédiatement un InfoResponse vers le Serveur FDI.

Le schéma XML pour un type InfoRequestT est le suivant:

```
<xs:complexType name="InfoRequestT">
  <xs:sequence>
    <xs:element name="Message" type="xs:string"/>
  </xs:sequence>
</xs:complexType>
```

Les éléments de type InfoRequestT sont décrits dans le Tableau A.28.

Tableau A.28 – Eléments d'InfoRequestT

Elément	Description
Message	Cet élément exigé spécifie le message devant être affiché à l'utilisateur.

A.32 InputRequestT

Ce type spécifie une demande envoyée depuis un Serveur FDI vers un Client FDI lorsque l'utilisateur a besoin de modifier une valeur lors d'une Action. Une fois que l'utilisateur a fini de modifier la valeur, le Client FDI renvoie un InputResponse vers le Serveur FDI.

Le schéma XML pour un type InputRequestT est le suivant:

```
<xs:complexType name="InputRequestT">
  <xs:sequence>
```

```

    <xs:element name="Prompt" type="xs:string"/>
    <xs:element name="InputValue" type="clnt:InputValueT"/>
  </xs:sequence>
</xs:complexType>

```

Les éléments de type InputRequestT sont décrits dans le Tableau A.29.

Tableau A.29 – Eléments d'InputRequestT

Élément	Description
Prompt	Cet élément exigé spécifie l'invite devant être affichée à l'utilisateur.
InputValue	Cet élément exigé spécifie la valeur devant être éditée par l'utilisateur.

A.33 InputResponseT

La réponse envoyée par un Client FDI à un Serveur FDI, résultant du traitement d'InputRequest.

Le schéma XML pour un type InputResponseT est le suivant:

```

<xs:complexType name="InputResponseT">
  <xs:sequence>
    <xs:element name="InputValue" type="clnt:VariantT"/>
  </xs:sequence>
</xs:complexType>

```

Les éléments de type InputResponseT sont décrits dans le Tableau A.30.

Tableau A.30 – Eléments d'InputResponseT

Élément	Description
InputValue	Cet élément exigé décrit la valeur spécifiée par l'utilisateur.

A.34 InputValueT

Ce type spécifie la valeur devant être modifiée par l'utilisateur pendant InputRequest.

Le schéma XML pour un type InputValueT est le suivant:

```

<xs:complexType name="InputValueT">
  <xs:complexContent>
    <xs:extension base="clnt:LabelHelpT">
      <xs:sequence>
        <xs:element name="InitialValue" type="clnt:VariantT"/>
        <xs:element name="Type" type="clnt:InputValueTypeT" minOccurs="0"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>

```


Les éléments de type InputValueType sont décrits dans le Tableau A.31.

Tableau A.31 – Eléments d'InputValueType

Elément	Description
InitialValue	Cet élément exigé spécifie la valeur initiale affichée à l'utilisateur.
Type	Cet élément facultatif spécifie le type de la valeur.

A.35 InputValueType

Ce type spécifie le type de la valeur modifiée par l'utilisateur pendant InputRequest.

Le schéma XML pour un type InputValueType est le suivant:

```
<xs:complexType name="InputValueType">
  <xs:choice>
    <xs:element name="NumericType" type="clnt:NumericDataT"/>
    <xs:element name="StringType" type="clnt:StringT"/>
    <xs:element name="Enumeration" type="clnt:EnumerationItemListT">
      <xs:unique name="InputValueUniqueEnumValue">
        <xs:selector xpath="clnt:EnumerationItem/clnt:Value"/>
        <xs:field xpath="."/>
      </xs:unique>
    </xs:element>
    <xs:element name="BitEnumeration" type="clnt:BitEnumerationItemListT">
      <xs:unique name="InputValueUniqueBitMask">
        <xs:selector xpath="clnt:BitEnumerationItem/clnt:Value"/>
        <xs:field xpath="."/>
      </xs:unique>
    </xs:element>
    <xs:element name="DateTimeType" type="clnt:DateTimeDataT"/>
  </xs:choice>
</xs:complexType>
```

Les éléments de type InputValueType sont décrits dans le Tableau A.32.

Tableau A.32 – Eléments d'InputValueType

Elément	Description
NumericType	Cet élément facultatif spécifie une valeur de type entier ou à virgule flottante.
StringType	Cet élément facultatif spécifie une valeur de type chaîne.
Énumération	Cet élément spécifie une énumération. Chaque élément de l'énumération définit une description et un texte d'aide pour une valeur d'un paramètre.
BitEnumeration	Cet élément spécifie une énumération. Chaque élément de l'énumération définit une description et un texte d'aide pour un masque de bits de la valeur d'un paramètre.
DateTimeType	Cet élément spécifie une valeur de date ou d'heure.

A.36 LabelHelpT

Ce type spécifie un objet de structure d'affichage étendu avec un texte d'aide localisé supplémentaire.

Le schéma XML pour un type LabelHelpT est le suivant:

```
<xs:complexType name="LabelHelpT" abstract="true">
  <xs:complexContent>
    <xs:extension base="clnt:LabelT">
      <xs:sequence>
        <xs:element name="Help" type="xs:string" minOccurs="0"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

Les éléments de type LabelHelpT sont décrits dans le Tableau A.33.

Tableau A.33 – Eléments de LabelHelpT

Élément	Description
Help	Cet élément facultatif spécifie le texte d'aide localisé de l'élément.

A.37 LabelT

Ce type spécifie l'objet de structure d'affichage avec une description localisée.

Le schéma XML pour un type LabelT est le suivant:

```
<xs:complexType name="LabelT">
  <xs:sequence>
    <xs:element name="Label" type="xs:string"/>
  </xs:sequence>
</xs:complexType>
```

Les éléments de type LabelT sont décrits dans le Tableau A.34.

Tableau A.34 – Eléments de LabelT

Élément	Description
Label	Cet élément exigé spécifie le nom localisé de l'élément.

A.38 LineTypeT

Ce type spécifie le type de ligne dans un diagramme ou un graphique. Toutes les lignes d'un même type doivent avoir la même apparence visuelle (ex.: toutes les lignes Data0 et toutes les lignes Data1 s'afficheront respectivement de la même manière), mais les deux ensembles de lignes doivent être visuellement distincts.

Le schéma XML pour un type d'énumération LineTypeT est le suivant:

```
<xs:simpleType name="LineTypeT">
  <xs:restriction base="xs:string">
    <xs:enumeration value="Data"/>
    <xs:enumeration value="Data0"/>
    <xs:enumeration value="Data1"/>
    <xs:enumeration value="Data2"/>
    <xs:enumeration value="Data3"/>
  </xs:restriction>
</xs:simpleType>
```

```

<xs:enumeration value="Data4"/>
<xs:enumeration value="Data5"/>
<xs:enumeration value="Data6"/>
<xs:enumeration value="Data7"/>
<xs:enumeration value="Data8"/>
<xs:enumeration value="Data9"/>
<xs:enumeration value="LowLimit"/>
<xs:enumeration value="LowLowLimit"/>
<xs:enumeration value="HighLimit"/>
<xs:enumeration value="HighHighLimit"/>
<xs:enumeration value="Transparent"/>
</xs:restriction>
</xs:simpleType>

```

Les valeurs de type d'énumération LineTypeT sont décrites dans le Tableau A.35.

Tableau A.35 – Enumérations de LineTypeT

Enumération	Description
Data	La ligne représente des données.
Data0	La ligne représente des données.
Data1	La ligne représente des données.
Data2	La ligne représente des données.
Data3	La ligne représente des données.
Data4	La ligne représente des données.
Data5	La ligne représente des données.
Data6	La ligne représente des données.
Data7	La ligne représente des données.
Data8	La ligne représente des données.
Data9	La ligne représente des données.
LowLimit	La ligne représente une limite basse.
LowLowLimit	La ligne représente une limite très basse.
HighLimit	La ligne représente une limite haute.
HighHighLimit	La ligne représente une limite très haute.
Transparent	Représente un type de ligne transparente.

A.39 MenuT

Ce type spécifie la structure d'affichage d'une Fenêtre (Window), d'une Boîte de Dialogue (Dialog), d'une Page (Page), d'un Groupe (Group), d'un Menu (Menu) ou d'un Tableau (Table).

Le schéma XML pour un type MenuT est le suivant:

```

<xs:complexType name="MenuT">
  <xs:complexContent>
    <xs:extension base="clnt:UiElementT">
      <xs:sequence>
        <xs:element name="InitActionList" type="clnt:ActionListT" minOccurs="0"/>
        <xs:element name="PreEditActionList" type="clnt:ActionListT"
          minOccurs="0"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>

```

```

<xs:element name="PostEditActionList" type="clnt:ActionListT"
  minOccurs="0"/>
<xs:element name="ExitActionList" type="clnt:ActionListT" minOccurs="0"/>
<xs:element name="Access" type="clnt:AccessT" minOccurs="0"/>
<xs:element name="ItemList">
  <xs:complexType>
    <xs:choice minOccurs="0" maxOccurs="unbounded">
      <xs:element name="Menu" type="clnt:MenuReferenceT"/>
      <xs:element name="Chart" type="clnt:ChartT"/>
      <xs:element name="Graph" type="clnt:GraphT"/>
      <xs:element name="Grid" type="clnt:GridT"/>
      <xs:element name="Image" type="clnt:ImageT"/>
      <xs:element name="Text" type="xs:string"/>
      <xs:element name="Parameter" type="clnt:ParameterT"/>
      <xs:element name="Plugin" type="clnt:PluginT"/>
      <xs:element name="Action" type="clnt:ActionT"/>
      <xs:element name="RowBreak" type="clnt:RowBreakT"/>
      <xs:element name="ColumnBreak" type="clnt:ColumnBreakT"/>
      <xs:element name="Separator" type="clnt:SeparatorT"/>
    </xs:choice>
  </xs:complexType>
</xs:element>
</xs:sequence>
<xs:attribute name="Style" type="clnt:MenuStyleT" use="optional"/>
<xs:attribute name="NodePath" type="xs:string" use="required"/>
</xs:extension>
</xs:complexContent>
</xs:complexType>

```

Les attributs de type MenuT sont décrits dans le Tableau A.36.

Tableau A.36 – Attributs de MenuT

Attribut	Description
Style	Cet attribut facultatif détermine le style de menu.
NodePath	Il s'agit d'un attribut obligatoire qui donne le chemin qualifié du Nœud dans le Modèle d'Appareil.

Les éléments de type MenuT sont décrits dans le Tableau A.37.

Tableau A.37 – Eléments de MenuT

Elément	Description
InitActionList	Cet élément facultatif spécifie les Actions devant être exécutées avant l'activation du menu.
PreEditActionList	Cet élément facultatif spécifie les Actions devant être exécutées immédiatement après l'activation de l'élément.
PostEditActionList	Cet élément facultatif spécifie les Actions devant être exécutées juste après que l'utilisateur a fini de traiter l'élément.
ExitActionList	Cet élément facultatif spécifie les Actions devant être exécutées lorsque le menu est désactivé.
Access	Cet élément facultatif spécifie si le menu doit être utilisé EN LIGNE ou HORS LIGNE.
ItemList	Cet élément facultatif spécifie le contenu de base du menu.

A.40 MenuReferenceT

Ce type spécifie la structure d'affichage d'une Fenêtre (Window), d'une Boîte de Dialogue (Dialog), d'une Page (Page), d'un Groupe (Group), d'un Menu (Menu) ou d'un Tableau (Table).

Le schéma XML pour un type MenuReferenceT est le suivant:

```
<xs:complexType name="MenuReferenceT">
  <xs:complexContent>
    <xs:extension base="clnt:UiElementT">
      <xs:sequence>
        <xs:element name="Review" type="xs:boolean" minOccurs="0"/>
      </xs:sequence>
      <xs:attribute name="Style" type="clnt:MenuStyleT" use="optional"/>
      <xs:attribute name="NodePath" type="xs:string" use="required"/>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

Les attributs de type MenuReferenceT sont décrits dans le Tableau A.38.

Tableau A.38 – Attributs de MenuReferenceT

Attribut	Description
Style	Cet attribut facultatif détermine le style de menu.
NodePath	Il s'agit d'un attribut obligatoire qui donne le chemin qualifié du Nœud dans le Modèle d'Appareil.

Les éléments de type MenuReferenceT sont décrits dans le Tableau A.39.

Tableau A.39 – Eléments de MenuReferenceT

Elément	Description
Review	Cet élément facultatif spécifie que tous les éléments référencés du type VARIABLE apparaissant dans le GUI (Interface Utilisateur Graphique) ou ses sous-composants doivent supposer que les qualificateurs sont en lecture seule (READ ONLY).

A.41 MenuStyleT

Ce type spécifie le style de menu.

Le schéma XML pour un type d'énumération MenuStyleT est le suivant:

```
<xs:simpleType name="MenuStyleT">
  <xs:restriction base="xs:string">
    <xs:enumeration value="Menu"/>
    <xs:enumeration value="Group"/>
    <xs:enumeration value="Page"/>
    <xs:enumeration value="Table"/>
    <xs:enumeration value="Window"/>
    <xs:enumeration value="Dialog"/>
    <xs:enumeration value="Record"/>
    <xs:enumeration value="Collection"/>
  </xs:restriction>
</xs:simpleType>
```

```

    <xs:enumeration value="ItemArray"/>
    <xs:enumeration value="EditDisplay"/>
  </xs:restriction>
</xs:simpleType>

```

Les valeurs de type d'énumération MenuStyleT sont décrites dans le Tableau A.40.

Tableau A.40 – Enumérations de MenuStyleT

Enumération	Description
Menu	Style de Menu EDD "MENU"
Group (Groupe)	Style de Menu EDD "GROUP"
Page	Style de Menu EDD "PAGE"
Table (Tableau)	Style de Menu EDD "TABLE"
Window (Fenêtre)	Style de Menu EDD "WINDOW"
Dialog (Boîte de Dialogue)	Style de Menu EDD "DIALOG"
Record (Enregistrement)	Le Menu représente l'élément "RECORD" de l'EDD
Collection	Le Menu représente l'élément "COLLECTION" de l'EDD
ItemArray	Le Menu représente l'élément "ITEMARRAY" de l'EDD
EditDisplay	Le Menu représente l'élément "EDIT_DISPLAY" de l'EDD

A.42 NumericDataT

Ce type spécifie un type de données d'une valeur entière, booléenne ou à virgule flottante.

Le schéma XML pour un type d'énumération NumericDataT est le suivant:

```

<xs:simpleType name="NumericDataT">
  <xs:restriction base="xs:string">
    <xs:enumeration value="SignedInteger"/>
    <xs:enumeration value="UnsignedInteger"/>
    <xs:enumeration value="FloatingPoint"/>
    <xs:enumeration value="Double"/>
    <xs:enumeration value="Boolean"/>
  </xs:restriction>
</xs:simpleType>

```

Les valeurs de type d'énumération NumericDataT sont décrites dans le Tableau A.41.

Tableau A.41 – Enumérations de NumericDataT

Enumération	Description
SignedInteger	Un entier signé.
UnsignedInteger	Un entier non signé.
FloatingPoint	Un nombre à virgule flottante.
Double	Un nombre à Double point.
Boolean	Une valeur booléenne.

A.43 NumericTemplateT

Ce type spécifie le modèle utilisé pour afficher une valeur de paramètre numérique.

Le schéma XML pour un type NumericTemplateT est le suivant:

```
<xs:complexType name="NumericTemplateT">
  <xs:sequence>
    <xs:element name="DataType" type="clnt:NumericDataT"/>
    <xs:element name="Options" type="clnt:VariantOptionListT" minOccurs="0"
      maxOccurs="1">
      <xs:unique name="UiTemplateUniqueVariantValue">
        <xs:selector xpath="clnt:Option/clnt:Value/*"/>
        <xs:field xpath="."/>
      </xs:unique>
    </xs:element>
  </xs:sequence>
</xs:complexType>
```

Les éléments de type NumericTemplateT sont décrits dans le Tableau A.42.

Tableau A.42 – Eléments de NumericTemplateT

Elément	Description
DataType (Type de données)	Cet élément exigé spécifie le type de paramètre numérique.
Options	Cet élément facultatif spécifie la liste d'options que l'utilisateur peut choisir pour la chaîne.

A.44 OptionListT

Ce type spécifie une liste des éléments Option. La réponse de sélection contient l'index basé sur 0 de l'option sélectionnée.

Le schéma XML pour un type OptionListT est le suivant:

```
<xs:complexType name="OptionListT">
  <xs:sequence maxOccurs="unbounded">
    <xs:element name="Option" type="clnt:LabelT"/>
  </xs:sequence>
</xs:complexType>
```

Les éléments de type OptionListT sont décrits dans le Tableau A.43.

Tableau A.43 – Eléments d'OptionListT

Elément	Description
Option	Un élément de la liste.

A.45 OrientationT

Ce type spécifie une direction.

Le schéma XML pour un type d'énumération OrientationT est le suivant:

```
<xs:simpleType name="OrientationT">
  <xs:restriction base="xs:string">
    <xs:enumeration value="Horizontal"/>
    <xs:enumeration value="Vertical"/>
  </xs:restriction>
</xs:simpleType>
```

Les valeurs de type d'énumération OrientationT sont décrites dans le Tableau A.44.

Tableau A.44 – Enumérations d'OrientationT

Enumération	Description
Horizontal	De gauche à droite et de droite à gauche.
Vertical	Du haut vers le bas et le bas vers le haut.

A.46 ParameterInputRequestT

Ce type spécifie une demande envoyée depuis un Serveur FDI vers un Client FDI lorsque l'utilisateur a besoin de modifier une valeur lors d'une Action. Une fois que l'utilisateur a fini de modifier la valeur, le Client FDI renvoie un InputResponse vers le Serveur FDI.

Le schéma XML pour un type ParameterInputRequestT est le suivant:

```
<xs:complexType name="ParameterInputRequestT">
  <xs:sequence>
    <xs:element name="Prompt" type="xs:string"/>
    <xs:element name="Parameter" type="clnt:ParameterT"/>
  </xs:sequence>
</xs:complexType>
```

Les éléments de type ParameterInputRequestT sont décrits dans le Tableau A.45.

Tableau A.45 – Eléments de ParameterInputRequestT

Elément	Description
Prompt	Cet élément exigé spécifie l'invite devant être affichée à l'utilisateur.
Parameter (Paramètre)	Cet élément exigé spécifie la valeur devant être éditée par l'utilisateur.

A.47 ParameterListT

Ce type spécifie une liste d'éléments Paramètre.

Le schéma XML pour un type ParameterListT est le suivant:

```
<xs:complexType name="ParameterListT">
  <xs:sequence maxOccurs="unbounded">
    <xs:element name="Parameter" type="clnt:ParameterT"/>
  </xs:sequence>
</xs:complexType>
```


Les éléments de type `ParameterListT` sont décrits dans le Tableau A.46.

Tableau A.46 – Eléments de `ParameterListT`

Elément	Description
Parameter (Paramètre)	Un élément de la liste.

A.48 `ParameterT`

Ce type spécifie un paramètre d'appareil ou de bloc qui est affiché dans un élément d'interface utilisateur, tel qu'une Fenêtre ou une Boîte de Dialogue.

Le schéma XML pour un type `ParameterT` est le suivant:

```
<xs:complexType name="ParameterT">
  <xs:complexContent>
    <xs:extension base="clnt:UiElementSizeableT">
      <xs:sequence>
        <xs:element name="UITemplate" type="clnt:UITemplateT"/>
        <xs:element name="Unit" type="xs:string" minOccurs="0"/>
        <xs:element name="Handling" type="clnt:HandlingT" default="rw"
          minOccurs="0"/>
        <xs:element name="RangeList" type="clnt:RangeListT" minOccurs="0"/>
        <xs:element name="DisplayFormat" type="clnt:FormatSpecifieiT"
          minOccurs="0"/>
        <xs:element name="EditFormat" type="clnt:FormatSpecifieiT"
          minOccurs="0"/>
        <xs:element name="TimeFormat" type="xs:string" minOccurs="0"/>
        <xs:element name="TimeScale" type="clnt:TimeScaleT" minOccurs="0"/>
        <xs:element name="DisplayLabel" type="xs:boolean" default="true"
          minOccurs="0"/>
        <xs:element name="DisplayValue" type="xs:boolean" default="true"
          minOccurs="0"/>
        <xs:element name="DisplayUnit" type="xs:boolean" default="true"
          minOccurs="0"/>
        <xs:element name="PreEditActionList" type="clnt:ActionListT"
          minOccurs="0"/>
        <xs:element name="PostEditActionList" type="clnt:ActionListT"
          minOccurs="0"/>
        <xs:element name="ScalingFactor" type="xs:double" minOccurs="0"/>
        <xs:element name="Class" type="clnt:ParameterClassT" minOccurs="0"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

Les éléments de type ParameterT sont décrits dans le Tableau A.47.

Tableau A.47 – Éléments de ParameterT

Élément	Description
UITemplate	Cet élément spécifie le modèle d'UI complexe utilisé pour afficher un paramètre. Les modèles d'UI modifient la manière dont les valeurs des paramètres sont affichées. Le modèle d'UI par défaut affiche une valeur de paramètre en texte clair.
Unit	Cet élément facultatif spécifie l'unité technique du paramètre. Par défaut, le paramètre n'a pas d'unité technique.
Handling	Cet élément facultatif spécifie si le paramètre peut être modifié. La valeur par défaut est ReadWrite.
RangeList	Cet élément facultatif spécifie les valeurs, minimale et maximale, auxquelles l'utilisateur peut définir le paramètre.
DisplayFormat	Cet élément facultatif spécifie le format qui doit être utilisé pour l'affichage de la valeur du paramètre, tel que spécifié dans l'IEC 61804-3. La valeur par défaut dépend du type de données du paramètre. Elle est déterminée par le Client FDI.
EditFormat	Cet élément facultatif spécifie le format qui doit être utilisé pour la modification de la valeur du paramètre, tel que spécifié dans l'IEC 61804-3. La valeur par défaut dépend du type de données du paramètre. Elle est déterminée par le Client FDI.
TimeFormat	Cet élément facultatif spécifie le format qui doit être utilisé pour l'affichage et la modification de la valeur des paramètres de type date ou heure.
TimeScale	Cet élément facultatif spécifie l'élément TIME_SCALE des paramètres de type date ou heure.
DisplayLabel	Cet élément facultatif spécifie s'il convient d'afficher l'étiquette du paramètre. La valeur par défaut est "true".
DisplayValue	Cet élément facultatif spécifie s'il convient d'afficher la valeur du paramètre. La valeur par défaut est "true".
DisplayUnit	Cet élément facultatif spécifie s'il convient d'afficher l'unité du paramètre. La valeur par défaut est "true".
PreEditActionList	Cet élément facultatif spécifie les Actions devant être exécutées avant que le paramètre soit modifié par l'utilisateur.
PostEditActionList	Cet élément facultatif spécifie les Actions devant être exécutées après que le paramètre a été modifié par l'utilisateur.
ScalingFactor	Cet élément spécifie le facteur d'échelle qui doit être utilisé lorsque la valeur du paramètre sera affichée à l'utilisateur. Cette valeur visible est le résultat de la multiplication du facteur par la valeur du paramètre renvoyée par l'appareil.
Class	Cet élément facultatif spécifie l'attribut EDD CLASS de l'élément VARIABLE.

A.49 PluginT

Ce type spécifie un Plugiciel d'Interface Utilisateur qui est référencé à partir d'un élément d'interface utilisateur, tel qu'une Fenêtre ou une Boîte de Dialogue.

Le schéma XML pour un type PluginT est le suivant:

```
<xs:complexType name="PluginT">
  <xs:complexContent>
    <xs:extension base="clnt:UiElementT">
      <xs:sequence>
        <xs:element name="Identifiant">
          <xs:simpleType>
```

```

        <xs:restriction base="xs:string">
            <xs:pattern value="[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[0-9a-fA-F]{4}-[0-9a-fA-F]{4}-[0-9a-fA-F]{12}"/>
        </xs:restriction>
    </xs:simpleType>
</xs:element>
</xs:sequence>
</xs:extension>
</xs:complexContent>
</xs:complexType>

```

Les éléments de type PluginT sont décrits dans le Tableau A.48.

Tableau A.48 – Eléments de PluginT

Elément	Description
Identifier	Cet élément exigé spécifie un Identifiant Universel Unique (Universally Unique Identifier, UUID) qui identifie le logiciel.

A.50 RangeListT

Ce type spécifie une liste d'éléments Range (Plage).

Le schéma XML pour un type RangeListT est le suivant:

```

<xs:complexType name="RangeListT">
    <xs:sequence maxOccurs="unbounded">
        <xs:element name="Range" type="clnt:RangeT"/>
    </xs:sequence>
</xs:complexType>

```

Les éléments de type RangeListT sont décrits dans le Tableau A.49.

Tableau A.49 – Eléments de RangeListT

Elément	Description
Range	Un élément de la liste.

A.51 RangeT

Ce type spécifie une plage définie par une valeur minimale et une valeur maximale.

Le schéma XML pour un type RangeT est le suivant:

```

<xs:complexType name="RangeT">
    <xs:sequence>
        <xs:element name="MinimumValue" type="clnt:VariantT"/>
        <xs:element name="MaximumValue" type="clnt:VariantT" minOccurs="0"/>
    </xs:sequence>
</xs:complexType>

```

Les éléments de type RangeT sont décrits dans le Tableau A.50.

Tableau A.50 – Éléments de RangeT

Élément	Description
MinimumValue	Cet élément exigé spécifie la valeur minimale de la plage.
MaximumValue	Cet élément exigé spécifie la valeur maximale de la plage.

A.52 ResponseT

Ce type spécifie la réponse envoyée par un Client FDI à un Serveur FDI, résultant d'une demande Action.

Le schéma XML pour un type ResponseT est le suivant:

```
<xs:complexType name="ResponseT"/>
```

A.53 RowBreakT

Ce type représente un saut de ligne.

Le schéma XML pour un type RowBreakT est le suivant:

```
<xs:complexType name="RowBreakT"/>
```

A.54 ScalingT

Ce type spécifie une méthode d'échelonnage des valeurs à partir d'une limite basse jusqu'à une limite haute.

Le schéma XML pour un type d'énumération ScalingT est le suivant:

```
<xs:simpleType name="ScalingT">
  <xs:restriction base="xs:string">
    <xs:enumeration value="Linear"/>
    <xs:enumeration value="Logarithmic"/>
  </xs:restriction>
</xs:simpleType>
```

Les valeurs de type d'énumération ScalingT sont décrites dans le Tableau A.51.

Tableau A.51 – Enumérations de ScalingT

Enumération	Description
Linear	Les valeurs sont échelonnées de manière linéaire.
Logarithmic	Les valeurs sont échelonnées de manière logarithmique via le logarithme naturel.

A.55 SelectionRequestT

Ce type spécifie une demande envoyée depuis un Serveur FDI vers un Client FDI lorsque l'utilisateur a besoin de choisir parmi une liste d'options lors d'une Action. Une fois que l'utilisateur a sélectionné une des options, le Client FDI renvoie un élément SelectionResponse vers le Serveur FDI.

Le schéma XML pour un type SelectionResponseT est le suivant:

```
<xs:complexType name="SelectionRequestT">
  <xs:sequence>
    <xs:element name="Prompt" type="xs:string"/>
    <xs:element name="OptionList" type="clnt:OptionListT">
      <xs:unique name="UniqueOptionIdentifier">
        <xs:selector xpath="clnt:Option/clnt:Label"/>
        <xs:field xpath="."/>
      </xs:unique>
    </xs:element>
  </xs:sequence>
</xs:complexType>
```

Les éléments de type SelectionRequestT sont décrits dans le Tableau A.52.

Tableau A.52 – Eléments de SelectionRequestT

Elément	Description
Prompt	Cet élément exigé spécifie l'invite devant être affichée à l'utilisateur.
OptionList	Cet élément exigé spécifie la liste d'options que l'utilisateur peut choisir.

A.56 SelectionResponseT

Ce type spécifie la réponse envoyée par un Client FDI à un Serveur FDI, résultant du traitement de SelectionRequest.

Le schéma XML pour un type SelectionResponseT est le suivant:

```
<xs:complexType name="SelectionResponseT">
  <xs:sequence>
    <xs:element name="SelectedOption" type="xs:unsignedLong"/>
  </xs:sequence>
</xs:complexType>
```

Les éléments de type SelectionResponseT sont décrits dans le Tableau A.53.

Tableau A.53 – Eléments de SelectionResponseT

Elément	Description
SelectedOption	L'index basé sur 0 de l'option sélectionnée.

A.57 SeparatorT

Ce type définit un séparateur.

Le schéma XML pour un type SeparatorT est le suivant:

```
<xs:complexType name="SeparatorT"/>
```

A.58 SizeT

Ce type spécifie la taille relative d'un élément.

Le schéma XML pour un type d'énumération SizeT est le suivant:

```
<xs:simpleType name="SizeT">
  <xs:restriction base="xs:string">
    <xs:enumeration value="XXX_SMALL"/>
    <xs:enumeration value="XX_SMALL"/>
    <xs:enumeration value="X_SMALL"/>
    <xs:enumeration value="SMALL"/>
    <xs:enumeration value="MEDIUM"/>
    <xs:enumeration value="LARGE"/>
    <xs:enumeration value="X_LARGE"/>
    <xs:enumeration value="XX_LARGE"/>
  </xs:restriction>
</xs:simpleType>
```

Les valeurs de type d'énumération SizeT sont décrites dans le Tableau A.54.

Tableau A.54 – Enumérations de SizeT

Enumération	Description
XXX_SMALL	Très très très petit.
XX_SMALL	Très très petit.
X_SMALL	Très petit.
SMALL	Petit.
MEDIUM	Moyen.
LARGE	Grand.
X_LARGE	Très grand.
XX_LARGE	Très très grand.

A.59 ParameterClassT

Ce type spécifie la définition du type EDD CLASS pour l'élément VARIABLE.

Le schéma XML pour un type d'énumération ParameterClassT est le suivant:

```
<xs:simpleType name="ParameterClassT">
  <xs:list>
    <xs:simpleType>
      <xs:restriction base="xs:string">
        <xs:enumeration value="ALARM"/>
        <xs:enumeration value="ANALOG_INPUT"/>
        <xs:enumeration value="ANALOG_OUTPUT"/>
        <xs:enumeration value="COMPUTATION"/>
        <xs:enumeration value="CONTAINED"/>
        <xs:enumeration value="CORRECTION"/>
      </xs:restriction>
    </xs:simpleType>
  </xs:list>
</xs:simpleType>
```

```

<xs:enumeration value="DEVICE"/>
<xs:enumeration value="SPECIALIST"/>
<xs:enumeration value="DIAGNOSTIC"/>
<xs:enumeration value="DIGITAL_INPUT"/>
<xs:enumeration value="DIGITAL_OUTPUT"/>
<xs:enumeration value="DISCRETE_INPUT"/>
<xs:enumeration value="DISCRETE_OUTPUT"/>
<xs:enumeration value="DYNAMIC"/>
<xs:enumeration value="FREQUENCY_INPUT"/>
<xs:enumeration value="FREQUENCY_OUTPUT"/>
<xs:enumeration value="HART"/>
<xs:enumeration value="INPUT"/>
<xs:enumeration value="IS_CONFIG"/>
<xs:enumeration value="LOCAL"/>
<xs:enumeration value="LOCAL_DISPLAY"/>
<xs:enumeration value="OPERATE"/>
<xs:enumeration value="OPTIONAL"/>
<xs:enumeration value="OUTPUT"/>
<xs:enumeration value="SERVICE"/>
<xs:enumeration value="TEMPORARY"/>
<xs:enumeration value="TUNE"/>
</xs:restriction>
</xs:simpleType>
</xs:list>
</xs:simpleType>

```

Les valeurs de type d'énumération ParameterClassT sont décrites dans le Tableau A.55.

Tableau A.55 – Enumérations de ParameterClassT

Enumération	Description
ALARM	VARIABLE contient des limites d'alarme.
ANALOG_INPUT	VARIABLE fait partie d'un bloc d'entrée analogique.
ANALOG_OUTPUT	VARIABLE fait partie d'un bloc de sortie analogique.
COMPUTATION	VARIABLE fait partie d'un bloc de calcul.
CONTAINED	VARIABLE représente les caractéristiques physiques de l'appareil.
CORRECTION	VARIABLE fait partie du bloc de correction.
DEVICE	VARIABLE représente les caractéristiques physiques de l'appareil.
SPECIALIST	VARIABLE est un paramètre de configuration. La permission de modifier cette VARIABLE peut uniquement être accordée aux utilisateurs qui sont spécialistes de cet appareil.
DIAGNOSTIC	VARIABLE indique le statut de l'appareil.
DIGITAL_INPUT	VARIABLE fait partie d'un bloc d'entrée numérique.
DIGITAL_OUTPUT	VARIABLE fait partie d'un bloc de sortie numérique.
DISCRETE_INPUT	VARIABLE fait partie d'un bloc d'entrée discret.
DISCRETE_OUTPUT	VARIABLE fait partie d'un bloc de sortie discret.
DYNAMIC	VARIABLE est modifiée par l'appareil sans stimulus provenant du réseau.
FREQUENCY_INPUT	VARIABLE fait partie d'un bloc d'entrée de fréquence.
FREQUENCY_OUTPUT	VARIABLE fait partie d'un bloc de sortie de fréquence.
HART	Cette VARIABLE fait partie du bloc HART qui caractérise l'interface HART.
INPUT	VARIABLE fait partie d'un bloc d'entrée.
IS_CONFIG	Une modification des résultats de VARIABLE lors de la définition du compteur de révision ou du bit modifié par la configuration, comme

	applicable.
LOCAL	VARIABLE est utilisé localement par l'application EDD.
LOCAL_DISPLAY	VARIABLE fait partie du bloc d'affichage local.
OPERATE	VARIABLE est utilisé pour contrôler l'exploitation d'un bloc.
OPTIONAL	VARIABLE peut ne pas être présent dans l'appareil.
OUTPUT	VARIABLE fait partie du bloc de sortie.
SERVICE	VARIABLE est utilisé lors de la conduite de maintenance de routine.
TEMPORARY	VARIABLE est initialisé à DEFAULT_VALUE dans chaque session d'application EDD. Les variables LOCAL peuvent être rendues persistantes par l'application EDD; les variables TEMPORARY ne sont jamais persistantes.
TUNE	VARIABLE est utilisé pour régler l'algorithme d'un bloc.

A.60 ActionClassT

Ce type spécifie la définition d'EDD CLASS d'une Action.

Le schéma XML pour un type d'énumération ActionClassT est le suivant:

```
<xs:simpleType name="ActionClassT">
  <xs:list>
    <xs:simpleType>
      <xs:restriction base="xs:string">
        <xs:enumeration value="ALARM"/>
        <xs:enumeration value="ANALOG_OUTPUT"/>
        <xs:enumeration value="COMPUTATION"/>
        <xs:enumeration value="CONTAINED"/>
        <xs:enumeration value="CORRECTION"/>
        <xs:enumeration value="DEVICE"/>
        <xs:enumeration value="SPECIALIST"/>
        <xs:enumeration value="DIAGNOSTIC"/>
        <xs:enumeration value="DISCRETE"/>
        <xs:enumeration value="FREQUENCY"/>
        <xs:enumeration value="HART"/>
        <xs:enumeration value="INPUT"/>
        <xs:enumeration value="LOCAL_DISPLAY"/>
        <xs:enumeration value="OPERATE"/>
        <xs:enumeration value="OUTPUT"/>
        <xs:enumeration value="SERVICE"/>
        <xs:enumeration value="TUNE"/>
      </xs:restriction>
    </xs:simpleType>
  </xs:list>
</xs:simpleType>
```


Les valeurs de type d'énumération ActionClassT sont décrites dans le Tableau A.56.

Tableau A.56 – Enumérations d'ActionClassT

Enumération	Description
ALARM	METHOD est associé à une alarme (ex.: spécification des limites d'une alarme, indication de l'état d'alarme, etc.)
ANALOG_OUTPUT	METHOD fait partie d'un bloc de sortie analogique.
COMPUTATION	METHOD fait partie d'un bloc de calcul.
CONTAINED	METHOD représente les caractéristiques physiques de l'appareil.
CORRECTION	La METHOD prend en charge les transducteurs dans l'appareil de champ. METHOD peut établir des limites de transducteur, fournir un amortissement de signal, indiquer la valeur du transducteur, linéariser ou étalonner le transducteur, etc.
DEVICE	METHOD spécifie les caractéristiques physiques de l'appareil.
SPECIALIST	La permission d'exécuter cet élément METHOD peut uniquement être accordée aux utilisateurs qui sont spécialistes de cet appareil.
DIAGNOSTIC	METHOD évalue l'état de l'appareil.
DISCRETE	METHOD prend en charge l'E/S discrète et/ou numérique de l'appareil.
FREQUENCY	METHOD prend en charge l'E/S de fréquence de l'appareil.
HART	Cette METHOD fait partie du bloc HART qui caractérise l'interface HART. Il y a un seul bloc HART.
INPUT	METHOD fait partie d'un bloc d'entrée. Un bloc d'entrée est un type particulier de bloc de calcul qui effectue des conversions d'unité, une mise à l'échelle et un amortissement. Les paramètres de bloc d'entrée peuvent être déterminés par la sortie d'un autre bloc.
LOCAL_DISPLAY	METHOD fait partie du bloc d'affichage local. Un bloc d'affichage local contient les Attributs VARIABLE associés à l'interface locale (clavier, affichage, etc.) de l'appareil.
OPERATE	METHOD est utilisé pour contrôler l'exploitation d'un bloc.
OUTPUT	METHOD fait partie du bloc de sortie. Les valeurs de paramètres de sortie peuvent être accessibles par un autre bloc d'entrée.
SERVICE	METHOD est utilisé lors de la conduite de maintenance de routine.
TUNE	METHOD est utilisé pour régler l'algorithme d'un bloc.

A.61 SourceListT

Ce type spécifie une liste d'éléments Source.

Le schéma XML pour un type SourceListT est le suivant:

```
<xs:complexType name="SourceListT">
  <xs:sequence maxOccurs="unbounded">
    <xs:element name="Source" type="clnt:SourceT"/>
  </xs:sequence>
</xs:complexType>
```

Les éléments de type SourceListT sont décrits dans le Tableau A.57.

Tableau A.57 – Eléments de SourceListT

Elément	Description
Source	Un élément de la liste.

A.62 SourceT

Ce type spécifie une source de valeurs de données affichées par un diagramme.

Le schéma XML pour un type SourceT est le suivant:

```
<xs:complexType name="SourceT">
  <xs:complexContent>
    <xs:extension base="clnt:DiagramLineT">
      <xs:sequence>
        <xs:element name="ParameterList" type="clnt:ParameterListT"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

Les éléments de type SourceT sont décrits dans le Tableau A.58.

Tableau A.58 – Eléments de SourceT

Elément	Description
ParameterList	Cet élément exigé spécifie les paramètres devant être échantillonnés et affichés.

A.63 StringDataT

Ce type spécifie le type de données d'une valeur de chaîne.

Le schéma XML pour un type d'énumération StringDataT est le suivant:

```
<xs:simpleType name="StringDataT">
  <xs:restriction base="xs:string">
    <xs:enumeration value="ASCII"/>
    <xs:enumeration value="BITSTRING"/>
    <xs:enumeration value="EUC"/>
    <xs:enumeration value="PACKED_ASCII"/>
    <xs:enumeration value="PASSWORD"/>
    <xs:enumeration value="VISIBLE"/>
    <xs:enumeration value="OCTET"/>
  </xs:restriction>
</xs:simpleType>
```

Les valeurs de type d'énumération StringDataT sont décrites dans le Tableau A.59.

Tableau A.59 – Enumérations de StringDataT

Enumération	Description
ASCII	Chaîne de caractères ASCII.
BITSTRING	Les Variables du type de données BIT_ENUMERATED sont des constantes du type chaîne qui identifient la position d'un caractère de la valeur VARIABLE avec sa position.
EUC	(Code d'Unité Étendu) – est le code interne pour des caractères spécifiques (par exemple, Chine: EUC-CN, Taiwan:EUC-TW). EUC est utilisé pour les langues d'Asie de l'Est (ISO/IEC 10646-1).
PACKED_ASCII	Est un sous-ensemble d'ASCII produit en retirant les deux bits les plus significatifs de chaque caractère ASCII.
PASSWORD	Est un type de chaîne, destiné à spécifier les chaînes de mot de passe. Excepté pour la manière dont la variable est présentée à l'utilisateur, le mot de passe et les types de chaînes ASCII sont identiques.
VISIBLE	Cette chaîne est une séquence ordonnée de caractères de l'ensemble de caractères de l'ISO/IEC 10646-1 et de l'ISO/IEC 2375.
OCTET	Pour la spécification d'une séquence de données binaires non formatées dont la définition est déterminée par l'implémentation de l'appareil.

A.64 StringTemplateT

Ce type spécifie le modèle utilisé pour afficher une valeur de paramètre de chaîne.

Le schéma XML pour un type StringTemplateT est le suivant:

```
<xs:complexType name="StringTemplateT">
  <xs:sequence>
    <xs:element name="DataType" type="clnt:StringDataT"/>
    <xs:element name="Options" type="clnt:StringOptionListT" minOccurs="0"
      maxOccurs="1">
      <xs:unique name="UiTemplateUniqueStringValue">
        <xs:selector xpath="clnt:Option/clnt:Value"/>
        <xs:field xpath="."/>
      </xs:unique>
    </xs:element>
  </xs:sequence>
</xs:complexType>
```

Les éléments de type StringTemplateT sont décrits dans le Tableau A.60.

Tableau A.60 – Eléments de StringTemplateT

Elément	Description
DataType (Type de données)	Cet élément exigé spécifie le type de paramètre de chaîne.
Options	Cet élément facultatif spécifie la liste d'options que l'utilisateur peut choisir pour la chaîne.

A.65 StringOptionListT

Ce type spécifie la liste d'options à choisir pour le type de paramètre de chaîne.

Le schéma XML pour un type StringOptionListT est le suivant:

```
<xs:complexType name="StringOptionListT">
  <xs:sequence maxOccurs="unbounded">
    <xs:element name="Option" type="clnt:StringOptionT"/>
  </xs:sequence>
</xs:complexType>
```

Les éléments de type StringOptionListT sont décrits dans le Tableau A.61.

Tableau A.61 – Eléments de StringOptionListT

Elément	Description
Option	Un élément des options de chaîne.

A.66 StringOptionT

Ce type spécifie l'une des valeurs possibles des options de chaîne.

Le schéma XML pour un type StringOptionT est le suivant:

```
<xs:complexType name="StringOptionT">
  <xs:complexContent>
    <xs:extension base="clnt:LabelHelpT">
      <xs:sequence>
        <xs:element name="Value" type="xs:string"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

Les éléments de type StringOptionT sont décrits dans le Tableau A.62.

Tableau A.62 – Eléments de StringOptionT

Elément	Description
Value (Valeur)	Cet élément exigé spécifie l'identifiant unique de l'entrée d'une liste dans le contexte d'un élément de liste parente.

A.67 StringT

Ce type spécifie le type d'une valeur d'une chaîne.

Le schéma XML pour un type StringT est le suivant:

```
<xs:complexType name="StringT">
  <xs:sequence>
    <xs:element name="DataType" type="clnt:StringDataT"/>
    <xs:element name="Length" type="xs:unsignedInt"/>
  </xs:sequence>
</xs:complexType>
```

```
</xs:sequence>
</xs:complexType>
```

Les éléments de type StringT sont décrits dans le Tableau A.63.

Tableau A.63 – Eléments de StringT

Elément	Description
DataType (Type de données)	Cet élément exigé spécifie le type de données de la chaîne.
Length	Cet élément exigé spécifie la longueur de la chaîne, en caractères et non pas en octets.

A.68 TimeScaleT

Ce type d'énumération spécifie les options pour le modificateur d'échelle de temps EDD.

Le schéma XML pour un type d'énumération TimeScaleT est le suivant:

```
<xs:simpleType name="TimeScaleT">
  <xs:restriction base="xs:string">
    <xs:enumeration value="SECONDS"/>
    <xs:enumeration value="MINUTES"/>
    <xs:enumeration value="HOURS"/>
  </xs:restriction>
</xs:simpleType>
```

Les valeurs de type d'énumération TimeScaleT sont décrites dans le Tableau A.64.

Tableau A.64 – Enumérations de TimeScaleT

Enumération	Description
SECONDS	L'accès doit être réalisé EN LIGNE.
MINUTES	L'accès doit être réalisé HORS LIGNE.
HOURS	L'accès doit être réalisé HORS LIGNE.

A.69 UidLayoutInformation

Cet élément est l'élément d'entrée racine qui contient l'UID.

Le schéma XML pour l'élément UidLayoutInformation est le suivant:

```
<xs:element name="UidLayoutInformation">
  <xs:complexType>
    <xs:sequence>
      <xs:choice>
        <xs:element name="Menu" type="clnt:MenuT"/>
        <xs:element name="ActionRequest" type="clnt:ActionRequestT"/>
        <xs:element name="ActionResponse" type="clnt:ActionResponseT"/>
      </xs:choice>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

Les éléments de type `UidLayoutInformation` sont décrits dans le Tableau A.65.

Tableau A.65 – Éléments d'UidLayoutInformation

Élément	Description
Menu	Cet élément optionnel spécifie une demande d'un serveur FDI vers un client FDI pour afficher un menu.
ActionRequest	Cet élément optionnel spécifie une demande d'un serveur FDI vers un client FDI d'effectuer une action spécifique pour l'utilisateur.
ActionResponse	Cet élément optionnel spécifie une demande d'un serveur FDI vers un client FDI pour qu'une réponse soit affichée pour l'utilisateur.

A.70 UidRequestT

Ce type spécifie une demande envoyée depuis un Serveur FDI vers un Client FDI lorsqu'une interface utilisateur avancée doit être affichée à l'utilisateur pendant une Action. La structure de l'interface utilisateur est spécifiée en tant que Boîte de Dialogue, Tableau ou Fenêtre. Lorsque l'interface utilisateur n'est plus nécessaire, le Client FDI renvoie une `UIDResponse` vers le Serveur FDI.

Le schéma XML pour un type `UidRequestT` est le suivant:

```
<xs:complexType name="UidRequestT">
  <xs:sequence>
    <xs:element name="UidRef" type="xs:string"/>
    <xs:element name="ButtonList" type="clnt:ButtonListT">
      <xs:unique name="UniqueButtonIdentifier">
        <xs:selector xpath="clnt:Button/clnt:Label"/>
        <xs:field xpath="."/>
      </xs:unique>
    </xs:element>
  </xs:sequence>
</xs:complexType>
```

Les éléments de type `UidRequestT` sont décrits dans le Tableau A.66.

Tableau A.66 – Éléments d'UidRequestT

Élément	Description
UidRef	Cet élément exigé spécifie le chemin d'accès au Nœud de l'UID contenant la définition de la Boîte de Dialogue, le Tableau ou la Fenêtre à afficher.
ButtonList	Cet élément exigé spécifie les boutons affichés par le Client FDI, que l'utilisateur peut presser pour indiquer que l'interface utilisateur n'est plus nécessaire.

A.71 UidResponseT

Ce type spécifie la réponse envoyée par un Client FDI à un Serveur FDI, résultant du traitement de `SelectionRequest`.

Le schéma XML pour un type `UidResponseT` est le suivant:

```
<xs:complexType name="UidResponseT">
  <xs:sequence>
```

```

        <xs:element name="SelectedButton" type="xs:unsignedLong"/>
    </xs:sequence>
</xs:complexType>

```

Les éléments de type `UidResponseT` sont décrits dans le Tableau A.67.

Tableau A.67 – Éléments d'UidResponseT

Élément	Description
SelectedButton	L'index basé sur 0 du bouton sélectionné. Si la liste de boutons n'est pas fournie dans la demande d'UID, les boutons Next (suivant) et Cancel (Annuler) sont affichés. SelectedButton=0 signifie que le bouton Next a été pressé.

A.72 UiElementSizeableT

Le schéma XML pour un type `UiElementSizeableT` est le suivant:

```

<xs:complexType name="UiElementSizeableT">
  <xs:complexContent>
    <xs:extension base="clnt:UiElementT">
      <xs:sequence>
        <xs:element name="Height" type="clnt:SizeT" minOccurs="0"/>
        <xs:element name="Width" type="clnt:SizeT" minOccurs="0"/>
      </xs:sequence>
      <xs:attribute name="NodePath" type="xs:string" use="required"/>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>

```

Les attributs de type `UiElementSizeableT` sont décrits dans le Tableau A.68.

Tableau A.68 – Attributs d'UiElementSizeableT

Attribut	Description
NodePath	Il s'agit d'un attribut obligatoire qui spécifie le chemin qualifié du Nœud dans le Modèle d'Appareil. La description du chemin permet de trouver un Nœud dans le Modèle d'Information.

Les éléments de type `UiElementSizeableT` sont décrits dans le Tableau A.69.

Tableau A.69 – Éléments d'UiElementSizeableT

Élément	Description
Height	Cet élément facultatif spécifie la hauteur relative de l'élément.
Width	Cet élément facultatif spécifie la largeur relative de l'élément.

A.73 UiElementT

Ce type spécifie un objet de structure d'affichage étendu avec une visibilité paramétrable.

Le schéma XML pour un type `UiElementT` est le suivant:

```

<xs:complexType name="UiElementT">
  <xs:complexContent>
    <xs:extension base="clnt:LabelHelpT">
      <xs:sequence>
        <xs:element name="Visibility" type="xs:boolean" minOccurs="0"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>

```

Les éléments de type UiElementT sont décrits dans le Tableau A.70.

Tableau A.70 – Eléments d'UiElementT

Elément	Description
Visibility	Cet élément facultatif spécifie si l'élément est visible ou non. Il est visible par défaut.

A.74 UiTemplateT

Ce type spécifie les modèles utilisés pour afficher une valeur de paramètre. Les modèles d'UI modifient la manière dont les valeurs des paramètres sont affichées.

Le schéma XML pour un type UiTemplateT est le suivant:

```

<xs:complexType name="UiTemplateT">
  <xs:sequence>
    <xs:choice>
      <xs:element name="Enumeration" type="clnt:EnumerationItemListT">
        <xs:unique name="UiTemplateUniqueEnumValue">
          <xs:selector xpath="clnt:EnumerationItem/clnt:Value"/>
          <xs:field xpath="."/>
        </xs:unique>
      </xs:element>
      <xs:element name="BitEnumeration" type="clnt:BitEnumerationItemListT">
        <xs:unique name="UiTemplateUniqueBitMask">
          <xs:selector xpath="clnt:BitEnumerationItem/clnt:Value"/>
          <xs:field xpath="."/>
        </xs:unique>
      </xs:element>
      <xs:element name="String" type="clnt:StringTemplateT"/>
      <xs:element name="Arithmetic" type="clnt:NumericTemplateT"/>
      <xs:element name="DateTime" type="clnt:DateTimeDataT"/>
    </xs:choice>
    <xs:element name="UiTemplateSize" type="xs:int" minOccurs="0"/>
  </xs:sequence>
</xs:complexType>

```


Les éléments de type UiTemplateT sont décrits dans le Tableau A.71.

Tableau A.71 – Eléments d'UiTemplateT

Elément	Description
Enumeration	Cet élément spécifie que la valeur doit être affichée avec un modèle d'UI optimisé pour les énumérations. Chaque élément de l'énumération définit une description et un texte d'aide pour une valeur d'un paramètre.
BitEnumeration	Cet élément spécifie que la valeur doit être affichée avec un modèle d'UI optimisé pour les énumérations de bits. Chaque élément de l'énumération définit une description et un texte d'aide pour un masque de bits de la valeur d'un paramètre.
String	Cet élément spécifie que la valeur doit être affichée avec un modèle d'UI optimisé pour toutes les valeurs de Types de Chaînes.
Arithmetic	Cet élément spécifie que la valeur doit être affichée avec un modèle d'UI optimisé pour toutes les valeurs de Types Numériques.
DateTime	Cet élément spécifie que la valeur doit être affichée avec un modèle d'UI optimisé pour toutes les valeurs de Types de Date et d'Heure.
UiTemplateSize	Il s'agit d'un élément facultatif utilisé pour spécifier la taille des types de données Arithmétiques pour, par exemple, les informations d'Octets pour des entiers Signés et Non Signés, ainsi que pour spécifier le nombre de caractères pour des types de données de chaîne.

A.75 VariantT

Ce type spécifie une valeur. Le format de la valeur est brut et non échelonné.

Le schéma XML pour un type VariantT est le suivant:

```
<xs:complexType name="VariantT">
  <xs:choice>
    <xs:element name="Float" type="xs:float"/>
    <xs:element name="Double" type="xs:double"/>
    <xs:element name="Integer" type="xs:integer"/>
    <xs:element name="UnsignedInteger" type="xs:nonNegativeInteger"/>
    <xs:element name="Date" type="xs:date"/>
    <xs:element name="DateTime" type="xs:dateTime"/>
    <xs:element name="Time" type="xs:time"/>
    <xs:element name="Duration" type="xs:duration"/>
    <xs:element name="String" type="xs:string"/>
    <xs:element name="Boolean" type="xs:boolean"/>
  </xs:choice>
</xs:complexType>
```

Les éléments de type VariantT sont décrits dans le Tableau A.72.

Tableau A.72 – Eléments de VariantT

Elément	Description
Float	Cet élément facultatif spécifie une valeur à virgule flottante à simple précision.
Double	Cet élément facultatif spécifie une valeur à virgule flottante à double précision.
Integer	Cet élément facultatif spécifie une valeur entière signée.
UnsignedInteger	Cet élément facultatif spécifie une valeur entière non signée.
Date	Cet élément facultatif spécifie une valeur de date.
DateTime	Cet élément facultatif spécifie une valeur de date et d'heure.
Time	Cet élément facultatif spécifie une valeur d'heure.
Duration	Cet élément facultatif spécifie une longueur d'une heure.
String	Cet élément facultatif spécifie une chaîne.
Boolean	Cet élément facultatif spécifie une valeur booléenne.

A.76 VariantOptionListT

Ce type spécifie la liste d'options à choisir pour le type de paramètre numérique.

Le schéma XML pour un type VariantOptionListT est le suivant:

```
<xs:complexType name="VariantOptionListT">
  <xs:sequence maxOccurs="unbounded">
    <xs:element name="Option" type="clnt:VariantOptionT"/>
  </xs:sequence>
</xs:complexType>
```

Les éléments de type VariantOptionListT sont décrits dans le Tableau A.73.

Tableau A.73 – Eléments de VariantOptionListT

Elément	Description
Option	Un élément des options numériques.

A.77 VariantOptionT

Ce type spécifie l'une des valeurs possibles des options numériques.

Le schéma XML pour un type VariantOptionT est le suivant:

```
<xs:complexType name="VariantOptionT">
  <xs:complexContent>
    <xs:extension base="clnt:LabelHelpT">
      <xs:sequence>
        <xs:element name="Value" type="clnt:VariantT"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

Les éléments de type VariantOptionT sont décrits dans le Tableau A.74.

Tableau A.74 – Eléments de VariantOptionT

Elément	Description
Value (Valeur)	Cet élément exigé spécifie l'identifiant unique de l'entrée d'une liste dans le contexte d'un élément de liste parente.

A.78 VectorListT

Ce type spécifie une liste d'éléments Vector (Vecteur).

Le schéma XML pour un type VectorListT est le suivant:

```
<xs:complexType name="VectorListT">
  <xs:sequence maxOccurs="unbounded">
    <xs:element name="Vector" type="clnt:VectorT"/>
  </xs:sequence>
</xs:complexType>
```

Les éléments de type VectorListT sont décrits dans le Tableau A.75.

Tableau A.75 – Eléments de VectorListT

Elément	Description
Vector	Un élément de la liste.

A.79 VectorT

Ce type spécifie le contenu d'une ligne ou d'une colonne dans une grille.

Le schéma XML pour un type VectorT est le suivant:

```
<xs:complexType name="VectorT">
  <xs:sequence>
    <xs:element name="Heading" type="xs:string"/>
    <xs:element name="Items">
      <xs:complexType>
        <xs:sequence minOccurs="0" maxOccurs="unbounded">
          <xs:choice>
            <xs:element name="Parameter" type="clnt:ParameterT"/>
            <xs:element name="Value" type="clnt:VariantT"/>
          </xs:choice>
        </xs:sequence>
      </xs:complexType>
    </xs:element>
  </xs:sequence>
</xs:complexType>
```

Les éléments de type VectorT sont décrits dans le Tableau A.76.

Tableau A.76 – Éléments de VectorT

Élément	Description
Heading	Cet élément exigé spécifie l'en-tête localisé affiché avec les données.
Items	Cet élément exigé spécifie les données devant être affichées, il peut s'agir de paramètres ou de valeurs scalaires statiques.

A.80 WaveformListT

Ce type spécifie une liste d'éléments Waveform (Forme d'onde).

Le schéma XML pour un type WaveformListT est le suivant:

```
<xs:complexType name="WaveformListT">
  <xs:sequence maxOccurs="unbounded">
    <xs:element name="Waveform" type="clnt:WaveformT"/>
  </xs:sequence>
</xs:complexType>
```

Les éléments de type WaveformListT sont décrits dans le Tableau A.77.

Tableau A.77 – Éléments de WaveformListT

Élément	Description
Waveform	Un élément de la liste.

A.81 WaveformT

Ce type spécifie un ensemble de données simple affiché dans un graphique. Un graphique peut contenir une ou plusieurs formes d'onde.

Le schéma XML pour un type WaveformT est le suivant:

```
<xs:complexType name="WaveformT">
  <xs:complexContent>
    <xs:extension base="clnt:DiagramLineT">
      <xs:sequence>
        <xs:element name="Handling" type="clnt:HandlingT" default="rw"
          minOccurs="0" maxOccurs="1"/>
        <xs:element name="WaveformType" type="clnt:WaveformTypeT" minOccurs="1"
          maxOccurs="1"/>
        <xs:element name="KeyPointList" type="clnt:WaveformKeyPointListT"
          minOccurs="0" maxOccurs="1"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

Les éléments de type WaveformT sont décrits dans le Tableau A.78.

Tableau A.78 – Eléments de WaveformT

Elément	Description
Handling	Cet élément facultatif spécifie si la forme d'onde peut être modifiée. La valeur par défaut est ReadWrite.
WaveformType	
KeyPointList	Cet élément facultatif spécifie une collection de points clés qui sont affichés avec la forme d'onde. Les points clés peuvent ou peuvent ne pas être des points sur la forme d'onde elle-même.

A.82 WaveformTypeT

Il s'agit du type de base abstrait de tous les Types de Formes d'onde.

Le schéma XML pour un type WaveformTypeT est le suivant:

```
<xs:complexType name="WaveformTypeT" abstract="true"/>
```

A.83 WaveformTypeHorizontalT

D'après la spécification de l'EDD: L'attribut HORIZONTAL spécifie un attribut WAVEFORM qui contient des lignes horizontales.

Le schéma XML pour un type WaveformTypeHorizontalT est le suivant:

```
<xs:complexType name="WaveformTypeHorizontalT">
  <xs:complexContent>
    <xs:extension base="clnt:WaveformTypeT">
      <xs:sequence>
        <xs:element name="Yvalues" type="clnt:WaveformVectorT" minOccurs="1"
          maxOccurs="1"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

Les éléments de type WaveformTypeHorizontalT sont décrits dans le Tableau A.79.

Tableau A.79 – Eléments de WaveformTypeHorizontalT

Elément	Description
Yvalues	D'après la spécification de l'EDD: L'attribut Y_VALUES spécifie la coordonnée Y de chaque ligne horizontale dans la WAVEFORM.

A.84 WaveformTypeVerticalT

D'après la spécification de l'EDD: L'attribut VERTICAL spécifie un attribut WAVEFORM qui contient des lignes verticales.

Le schéma XML pour un type WaveformTypeVerticalT est le suivant:

```

<xs:complexType name="WaveformTypeVerticalT">
  <xs:complexContent>
    <xs:extension base="clnt:WaveformTypeT">
      <xs:sequence>
        <xs:element name="Xvalues" type="clnt:WaveformVectorT" minOccurs="1"
          maxOccurs="1"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>

```

Les éléments de type WaveformTypeVerticalT sont décrits dans le Tableau A.80.

Tableau A.80 – Eléments de WaveformTypeVerticalT

Elément	Description
Xvalues	D'après la spécification de l'EDD: L'attribut X_VALUES spécifie la coordonnée X de chaque ligne verticale dans la WAVEFORM.

A.85 WaveformTypeYTT

D'après la spécification de l'EDD: L'attribut YT spécifie une WAVEFORM qui contient une liste de points qui sont définis par une coordonnée X initiale, un incrément X entre des points successifs et une liste de valeurs Y.

Le schéma XML pour un type WaveformTypeYTT est le suivant:

```

<xs:complexType name="WaveformTypeYTT">
  <xs:complexContent>
    <xs:extension base="clnt:WaveformTypeT">
      <xs:sequence>
        <xs:element name="Yvalues" type="clnt:WaveformVectorT" minOccurs="1"
          maxOccurs="1"/>
        <xs:element name="Xinitial" type="clnt:VariantT" minOccurs="1"
          maxOccurs="1"/>
        <xs:element name="Xincrement" type="clnt:VariantT" minOccurs="1"
          maxOccurs="1"/>
        <xs:element name="NumberOfPoints" type="xs:nonNegativeInteger"
          minOccurs="0" maxOccurs="1"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>

```

Les éléments de type WaveformTypeYTT sont décrits dans le Tableau A.81.

Tableau A.81 – Eléments de WaveformTypeYTT

Elément	Description
Yvalues	D'après la spécification de l'EDD: L'attribut Y_VALUES (valeurs Y) spécifie la coordonnée Y de chaque point dans la WAVEFORM.
Xinitial	D'après la spécification de l'EDD: L'attribut X_INITIAL (X initial) spécifie la coordonnée X du premier point dans la WAVEFORM.
Xincrement	D'après la spécification de l'EDD: L'attribut X_INCREMENT (incrément X) spécifie la différence entre les coordonnées X de points adjacents dans la WAVEFORM.
NumberOfPoints	D'après la spécification de l'EDD: L'attribut NUMBER_OF_POINTS spécifie le nombre de points de données valides dans X_VALUES. Par défaut, le nombre de points dans un attribut WAVEFORM sans attribut NUMBER_OF_POINTS est égal à la taille de X_VALUES.

A.86 WaveformTypeXYT

D'après la spécification de l'EDD: L'attribut XY spécifie un attribut WAVEFORM qui contient une liste de points (x,y).

Le schéma XML pour un type WaveformTypeXYT est le suivant:

```
<xs:complexType name="WaveformTypeXYT">
  <xs:complexContent>
    <xs:extension base="clnt:WaveformTypeT">
      <xs:sequence>
        <xs:element name="Xvalues" type="clnt:WaveformVectorT" minOccurs="1"
          maxOccurs="1"/>
        <xs:element name="Yvalues" type="clnt:WaveformVectorT" minOccurs="1"
          maxOccurs="1"/>
        <xs:element name="NumberOfPoints" type="xs:nonNegativeInteger"
          minOccurs="0" maxOccurs="1"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

Les éléments de type WaveformTypeXYT sont décrits dans le Tableau A.82.

Tableau A.82 – Eléments de WaveformTypeXYT

Elément	Description
Xvalues	D'après la spécification de l'EDD: L'attribut X_VALUES (valeurs X) spécifie la coordonnée X de chaque point dans la WAVEFORM. Chaque coordonnée X spécifiée dans X_VALUES doit correspondre à une coordonnée Y spécifiée dans Y_VALUES.
Yvalues	D'après la spécification de l'EDD: L'attribut Y_VALUES (valeurs Y) spécifie la coordonnée Y de chaque point dans la WAVEFORM. Chaque coordonnée Y spécifiée dans Y_VALUES doit correspondre à une coordonnée X spécifiée dans X_VALUES.
NumberOfPoints	D'après la spécification de l'EDD: L'attribut NUMBER_OF_POINTS spécifie le nombre de points de données valides dans X_VALUES et Y_VALUES. Par défaut, le nombre de points dans une WAVEFORM sans attribut NUMBER_OF_POINTS est égal à la taille de X_VALUES et Y_VALUES.

A.87 WaveformKeyPointListT

D'après la spécification de l'EDD: L'attribut KEY_POINTS spécifie les points clés dans l'attribut WAVEFORM qu'il convient de mettre en évidence à l'aide de l'application EDD. Les points clés peuvent ne pas correspondre directement aux points de données spécifiés via l'attribut TYPE. La façon selon laquelle ces points sont mis en surbrillance est définie par la spécification de l'EDD. Par défaut, il convient de ne pas mettre de point de WAVEFORM sans attribut KEY_POINTS en évidence.

Le schéma XML pour un type WaveformKeyPointListT est le suivant:

```
<xs:complexType name="WaveformKeyPointListT">
  <xs:complexContent>
    <xs:extension base="clnt:WaveformTypeT">
      <xs:sequence>
        <xs:element name="Xvalues" type="clnt:WaveformVectorT" minOccurs="1"
          maxOccurs="1"/>
        <xs:element name="Yvalues" type="clnt:WaveformVectorT" minOccurs="1"
          maxOccurs="1"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

Les éléments de type WaveformKeyPointListT sont décrits dans le Tableau A.83.

Tableau A.83 – Eléments de WaveformKeyPointListT

Elément	Description
Xvalues	D'après la spécification de l'EDD: L'attribut X_VALUES spécifie la coordonnée X de chaque point qui doit être mise en évidence. Chaque coordonnée X spécifiée dans X_VALUES doit correspondre à une coordonnée Y spécifiée dans Y_VALUES.
Yvalues	D'après la spécification de l'EDD: L'attribut Y_VALUES spécifie la coordonnée Y de chaque point qui doit être mise en évidence. Chaque coordonnée Y spécifiée dans Y_VALUES doit correspondre à une coordonnée X spécifiée dans X_VALUES.

A.88 WaveformVectorT

WaveformVectorT regroupe l'ensemble des informations relatives à un vecteur de forme d'onde, par exemple, le chemin du nœud pour la matrice des valeurs scalaires et les informations supplémentaires sur ces valeurs, notamment sur les éléments Handling, Range, Type.

Le schéma XML pour un type WaveformVectorT est le suivant:

```
<xs:complexType name="WaveformVectorT">
  <xs:sequence>
    <xs:element name="WaveformVectorElementList"
      type="clnt:WaveformVectorElementListT" minOccurs="1" maxOccurs="1"/>
  </xs:sequence>
  <xs:attribute name="DataArrayNodePath" type="xs:string" use="required"/>
</xs:complexType>
```

Les attributs de type WaveformVectorT sont décrits dans le Tableau A.84.

Tableau A.84 – Attributs de WaveformVectorT

Attribut	Description
DataRowNodePath	Le chemin du nœud qui est utilisé par le client pour sa demande de vecteur de forme d'onde.

Les éléments de type WaveformVectorT sont décrits dans le Tableau A.85.

Tableau A.85 – Eléments de WaveformVectorT

Elément	Description
WaveformVectorElementList	WaveformVectorElementList contient des informations statiques sur chaque élément individuel d'un vecteur de forme d'onde.

A.89 WaveformVectorElementListT

Cet élément regroupe l'information statique relative à un vecteur de forme d'onde, hormis le chemin du nœud pour la matrice des valeurs scalaires. Il s'agit d'informations supplémentaires (ex.: éléments Handling, Range, Type)

Le schéma XML pour un type WaveformVectorElementListT est le suivant:

```
<xs:complexType name="WaveformVectorElementListT">
  <xs:sequence>
    <xs:element name="WaveformVectorElement" type="clnt:WaveformVectorElementT"
      minOccurs="0" maxOccurs="unbounded"/>
  </xs:sequence>
</xs:complexType>
```

Les éléments de type WaveformVectorElementListT sont décrits dans le Tableau A.86.

Tableau A.86 – Eléments de WaveformVectorElementListT

Elément	Description
WaveformVectorElement	Les informations statiques au sujet d'un élément de vecteur de forme d'onde individuelle.

A.90 WaveformVectorElementT

Cet élément regroupe la totalité des informations relatives à un élément unique dans un vecteur de forme d'onde. Il contient des informations supplémentaires sur une valeur (ex.: Handling, Range, Type). Il appartient à l'élément correspondant de la matrice de résultats de l'élément DataRowNodePath.

Le schéma XML pour un type WaveformVectorElementT est le suivant:

```
<xs:complexType name="WaveformVectorElementT">
  <xs:sequence>
    <xs:element name="DataType" type="clnt:NumericDataT" default="SignedInteger"
      minOccurs="0"/>
    <xs:element name="RangeList" type="clnt:RangeListT" minOccurs="0"/>
    <xs:element name="Handling" type="clnt:HandlingT" default="rw" minOccurs="0"
      maxOccurs="1"/>
  </xs:sequence>
</xs:complexType>
```

```

<xs:element name="PreEditActionsList" type="clnt:ActionListT" minOccurs="0"
  maxOccurs="1"/>
<xs:element name="PostEditActionsList" type="clnt:ActionListT" minOccurs="0"
  maxOccurs="1"/>
<xs:element name="ScalingFactor" type="xs:double" minOccurs="0"/>
<xs:element name="DisplayFormat" type="clnt:FormatSpecifierT" minOccurs="0"/>
<xs:element name="EditFormat" type="clnt:FormatSpecifierT" minOccurs="0"/>
</xs:sequence>
</xs:complexType>

```

Les éléments de type `WaveformVectorElementT` sont décrits dans le Tableau A.87.

Tableau A.87 – Éléments de `WaveformVectorElementT`

Élément	Description
<code>DataType</code> (Type de données)	Cet élément exigé spécifie le type de données de la valeur de l'élément correspondant de la matrice de résultats de l'élément <code>dataArrayNodePath</code> .
<code>RangeList</code>	Cet élément facultatif spécifie la plage acceptable de la valeur de l'élément correspondant de la matrice de résultats de l'élément <code>dataArrayNodePath</code> .
<code>Handling</code>	Cet élément facultatif spécifie si la valeur des données actuelles peut être modifiée. La valeur par défaut est <code>ReadWrite</code> .
<code>PreEditActionsList</code>	Cet élément facultatif spécifie l'élément <code>PreEditActions</code> de la valeur de l'élément correspondant de la matrice de résultats de l'élément <code>dataArrayNodePath</code> .
<code>PostEditActionsList</code>	Cet élément facultatif spécifie l'élément <code>PostEditActions</code> de la valeur de l'élément correspondant de la matrice de résultats de l'élément <code>dataArrayNodePath</code> .
<code>ScalingFactor</code>	Cet élément facultatif spécifie le facteur d'échelle de la valeur de l'élément correspondant de la matrice de résultats de l'élément <code>dataArrayNodePath</code> .
<code>DisplayFormat</code>	Cet élément facultatif spécifie le format d'affichage de la valeur de l'élément correspondant de la matrice de résultats de l'élément <code>dataArrayNodePath</code> .
<code>EditFormat</code>	Cet élément facultatif spécifie le format d'édition de la valeur de l'élément correspondant de la matrice de résultats de l'élément <code>dataArrayNodePath</code> .

Annexe B (informative)

Exemple d'Action

Ce qui suit est un exemple d'une méthode EDD exécutée par un Serveur FDI ainsi que de l'interaction qui en résulte avec un Client FDI.

L'EDDL utilisé dans cet exemple est montré ci-dessous, suivi par un diagramme de séquences et la description de la séquence.

```
VARIABLE device_var1
{
    LABEL "device var1";
    HELP  "";
    CLASS DEVICE;
    HANDLING READ & WRITE;
    CONSTANT_UNIT "constUnit";
    TYPE INTEGER;
    PRE_EDIT_ACTIONS
    {
        PreEditAction1
    }
    POST_EDIT_ACTIONS
    {
        PostEditAction1
    }
}

VARIABLE process_value
{
    LABEL "Level";
    HELP  "";
    CLASS DYNAMIC;
    HANDLING READ;
    CONSTANT_UNIT "m";
    TYPE FLOAT;
}

VARIABLE newI
{
    LABEL "new i";
    HELP  "new value of i";
    CLASS DEVICE;
    HANDLING READ & WRITE;
    TYPE INTEGER;
}

VARIABLE newJ
{
    LABEL "new j";
    HELP  "new value of j";
    CLASS DEVICE;
    HANDLING READ & WRITE;
    TYPE INTEGER;
}

MENU MethodMenu
{
    LABEL "MethodMenu";
    HELP  "This menu is used in a method";
    STYLE DIALOG;
    ITEMS
    {
        newI,
        newJ
    }
}
```

```

}

METHOD UIReqRespCategories
{
    LABEL "Request Response Categories";
    HELP "This method demonstrates different categories of messages that are passed
        between server and client during a method execution";
    DEFINITION
    {
        int i, j, k, selection;
        add_abort_method(AbortMethod);
        //Acknowledgement
        ACKNOWLEDGE("Please hit OK to acknowledge the start of this method");
                                                                    // A010

        i = 5;
        PUT_MESSAGE("i = %{i}"); //Info // A020
        j = 10;
        PUT_MESSAGE("j = %{j}"); //Info // A030
        k = i+j;
        DELAY(5, "k = %{k}"); //Delay // A040

        GET_DEV_VAR_VALUE("Enter new value for the
            %[L]%[U]{device_var1}%[U]",device_var1); //Input // A050

        selection = SELECT_FROM_LIST("Is the value entered, correct? ", "YES;NO");
                                                                    //selection // A060
        if (selection == 1)
        {
            device_var1 = 0;
            abort(); //abort --- this will trigger the AbortMethod as well; // A070
        }
        k = k + device_var1;

        if (k == i + j)
        {
            display_comm_status(2); //Error - 2 == "Buffer Overflow" -- HART;
                                                                    // A080
        }

        display("Current level is %{process_value}%[U]{process_value}!"); // A090
        MenuDisplay(MethodMenu,"APPLY;DISCARD",selection); //UIDMessage // A100
        if(selection == 0)
        {
            i = newI;
            j = newJ;
            ACKNOWLEDGE("new value of k = %{k}"); // A110
        }
        ACKNOWLEDGE("This concludes the method !!"); // A120
    }
}

METHOD AbortMethod
{
    LABEL "AbortMethod";
    HELP "This is a simple Abort Method";
    DEFINITION
    {
        ACKNOWLEDGE("Method was aborted due to a call to abort()"); // B010
    }
}

METHOD PreEditAction1
{
    LABEL "Action1";
    HELP "This is a simple Pre Edit Warning";
    DEFINITION
    {
        ACKNOWLEDGE("Do you really want to edit this variable"); // C010
    }
}

```

```
    }  
}  
  
METHOD PostEditAction1  
{  
    LABEL "Action2";  
    HELP "This is a simple Post Edit Message";  
    DEFINITION  
    {  
        ACKNOWLEDGE("You actually edited the variable now!!!"); // D010  
    }  
}  
  
MENU FDIActions  
{  
    LABEL "FDI Actions";  
    HELP "This menu contains methods for verifying FDI UID contents";  
    ITEMS  
    {  
        MethodMenu,  
        UIReqRespCategories  
    }  
}
```

Cet exemple suppose comme condition préalable que le Client FDI a déjà établi une Session avec le Serveur FDI, l'utilisateur a navigué vers l'Appareil (MyDevice), et l'utilisateur a initié l'ouverture de la fonction groupe (FDIActions).

La séquence commence avec l'abonnement du Client FDI à la valeur de l'UID, afin de retrouver le contenu de l'UID, depuis le Serveur FDI. Le Client FDI ajoute un élément contrôlé et attend la valeur initiale fournie par l'utilisateur. La séquence est donnée à la Figure B.1 avec la chaîne XML qui sera renvoyée depuis le Serveur FDI.

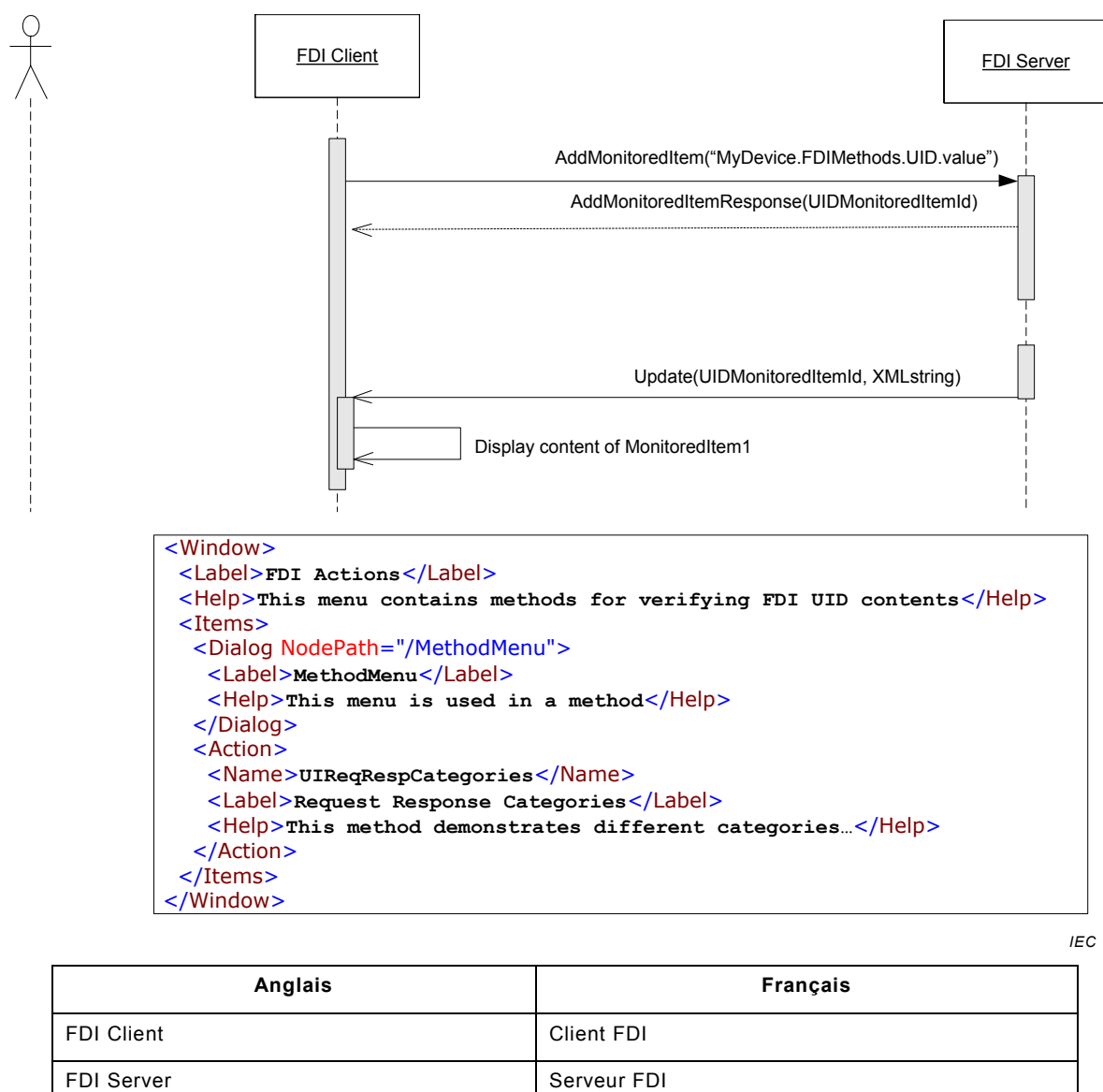
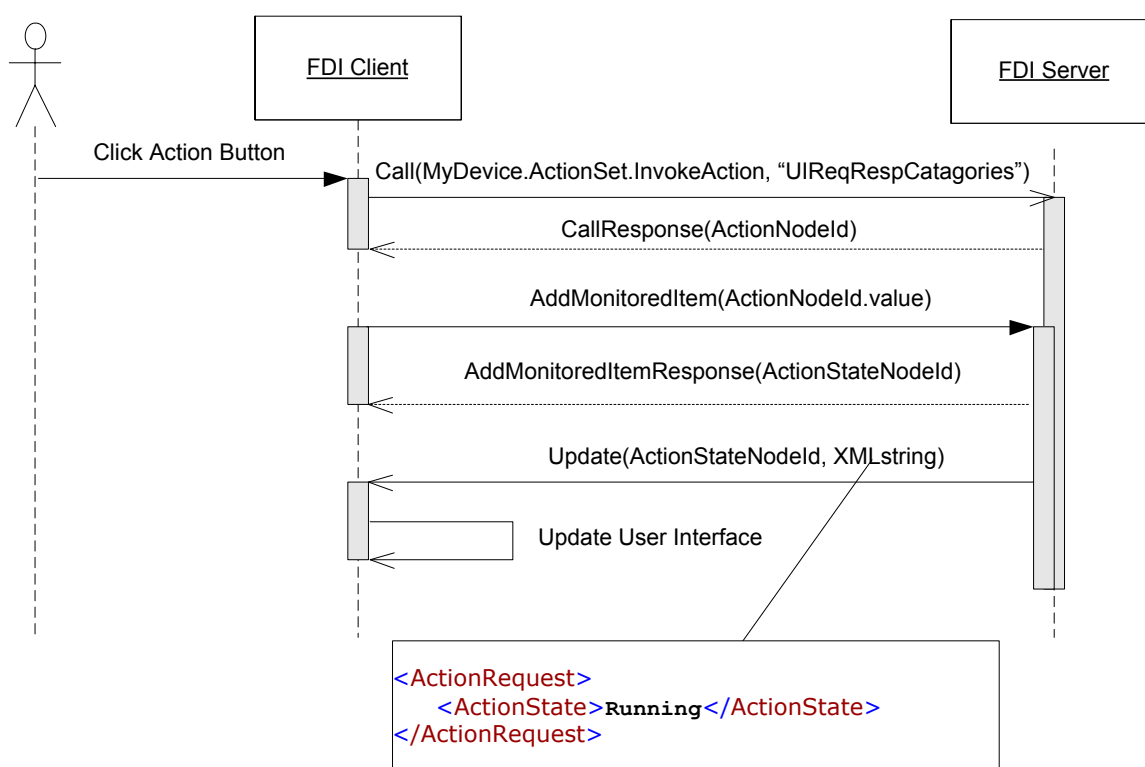


Figure B.1 – Exemple d'Action (étape 1)

Le Client FDI interprète la chaîne XML et présente le menu à l'utilisateur avec quatre boutons d'action, chaque bouton contenant le texte défini dans l'élément <Label>. Le Client FDI attend alors que l'utilisateur sélectionne une des actions.

Dans la prochaine étape de l'exemple, la séquence commence avec l'utilisateur cliquant sur le bouton action (UIReqRespCategories) présenté dans le menu. Le Client FDI réagit en initiant une demande de service Call OPC UA (voir l'IEC 62541-4), spécifiant la méthode OPC UA (MyDevice.ActionSet.InvokeAction) et la méthode EDDL (UIReqRespCategories). Le Serveur FDI répond en localisant la méthode EDDL et en initiant son exécution. Le Serveur FDI répond avec un identifiant de nœud unique (Action NodeId1) qui représente la méthode en cours d'exécution. Le Client FDI s'abonne à la valeur de la méthode avec l'identifiant de nœud (ActionNodeId.value). Le Serveur FDI fournit la valeur actuelle de la méthode ainsi que toute autre chaîne XML. La valeur initiale de cette chaîne XML indique que la méthode a démarré. La séquence de cette étape est donnée à la Figure B.2.



Anglais	Français
FDI Client	Client FDI
FDI Server	Serveur FDI
Click Action Button	Cliquer sur le Bouton Action
Update User Interface	Mettre à jour l'Interface Utilisateur

Figure B.2 – Exemple d'Action (étape 2)

Dans la prochaine étape de cet exemple, la méthode a procédé jusqu'à l'énoncé "ACKNOWLEDGE". Le Serveur FDI traite l'énoncé en paramétrant la valeur ActionNodeID afin d'indiquer qu'un acquittement de l'utilisateur est nécessaire avant que la méthode puisse poursuivre. La conséquence de la modification de la valeur est que le Serveur FDI émet une mise à jour de l'abonnement. Après réception de la mise à jour de la valeur, le Client FDI détecte qu'une demande d'acquiescement est en attente. Le Client FDI réagit en ouvrant une boîte de dialogue présentant le message reçu dans la mise à jour de la valeur, avec un bouton OK.

L'utilisateur lit le message et appuie sur le bouton OK, suite à quoi le Client FDI transmet un appel de méthode RespondAction au Serveur FDI.

Le Serveur FDI, en recevant l'appel de RespondAction, continue l'exécution de la méthode.

La séquence de cette étape est donnée à la Figure B.3.

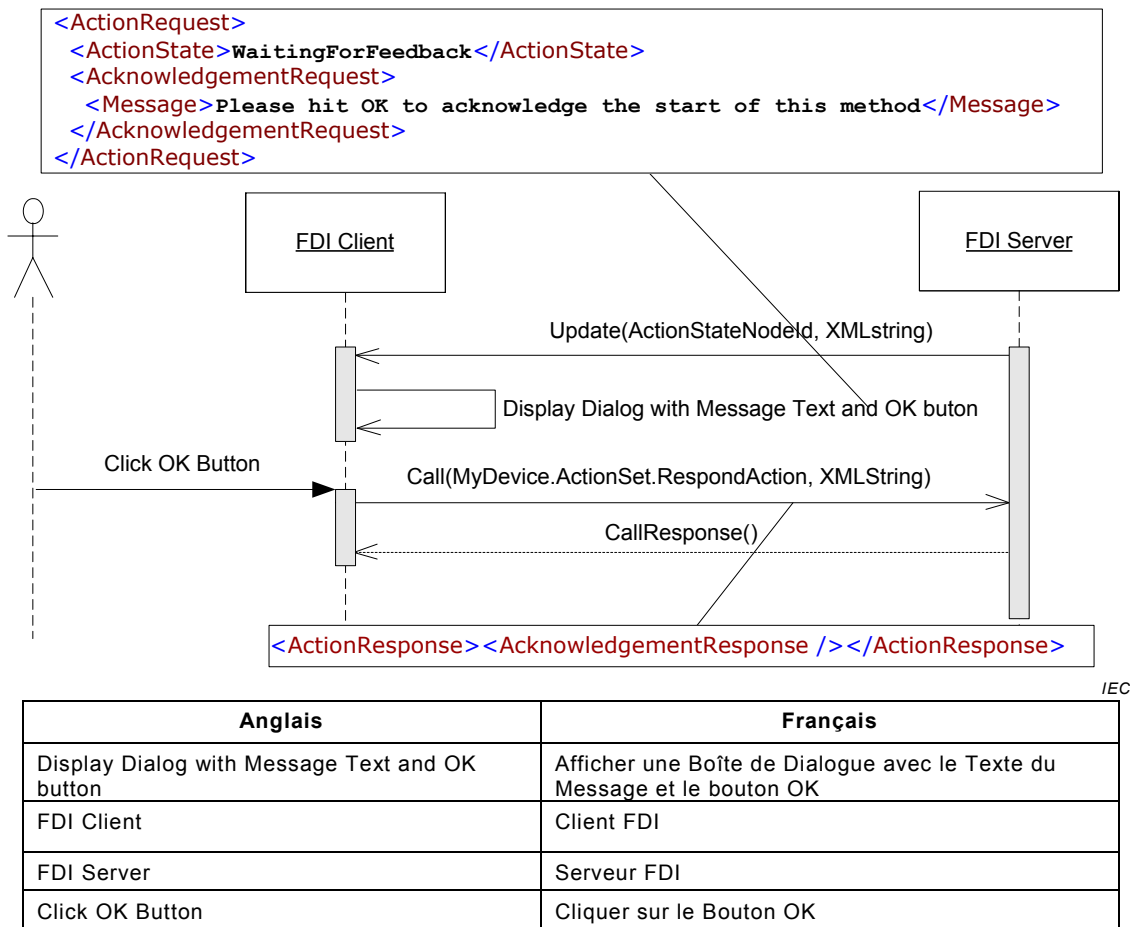
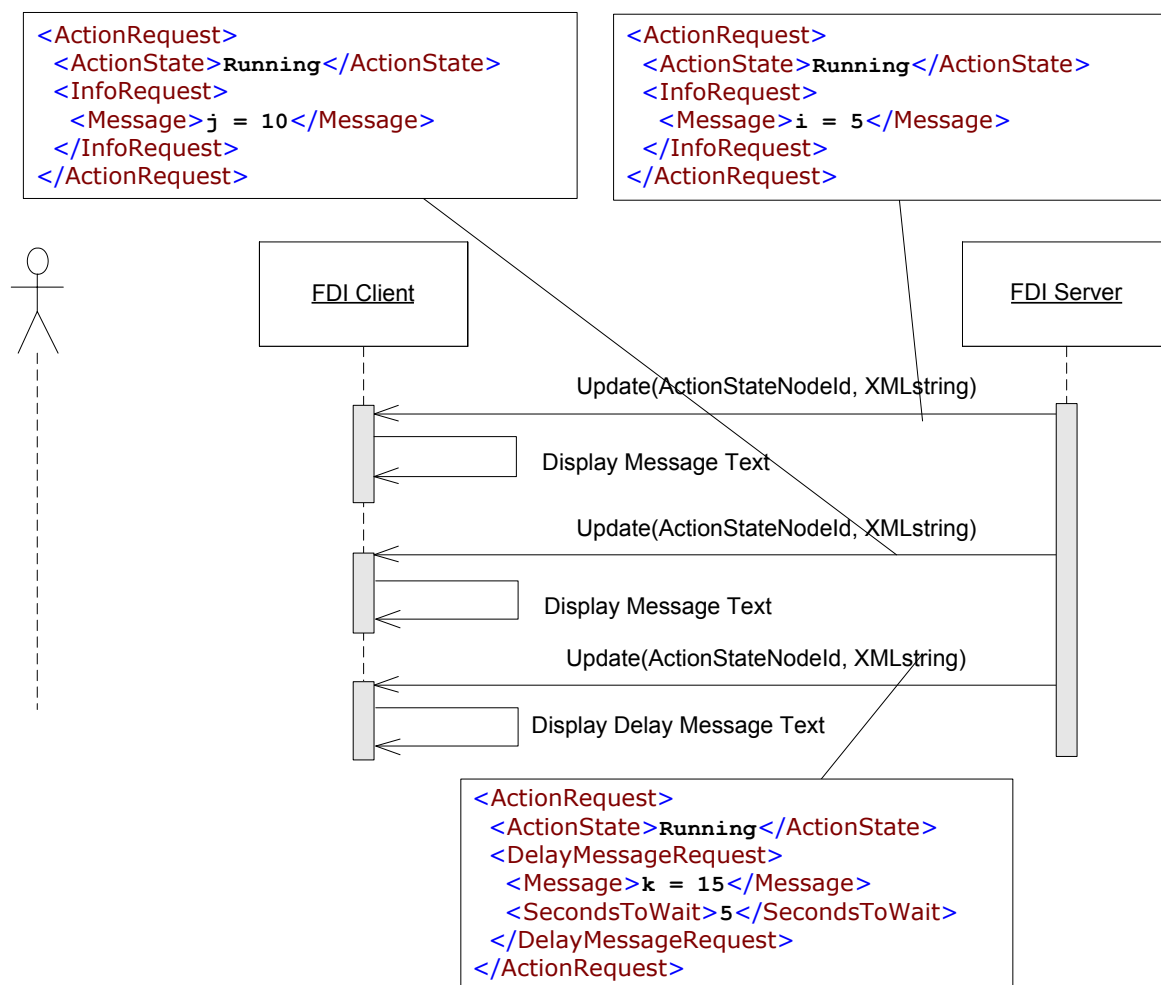


Figure B.3 – Exemple d'Action (étape 3)

Dans la prochaine étape de l'exemple, la méthode vient d'exécuter l'énoncé "ACKNOWLEDGE" et est sur le point d'exécuter les deux énoncés "PUT_MESSAGE" et l'énoncé "DELAY". Le Serveur FDI traite ces énoncés en paramétrant la valeur ActionNodeId afin d'indiquer les notifications. Chaque énoncé générera une modification de valeur et sera délivré au Client FDI. Le Client FDI réagit en montrant le texte du message dans un composant d'interface utilisateur approprié. La séquence de cette étape est donnée à la Figure B.4.

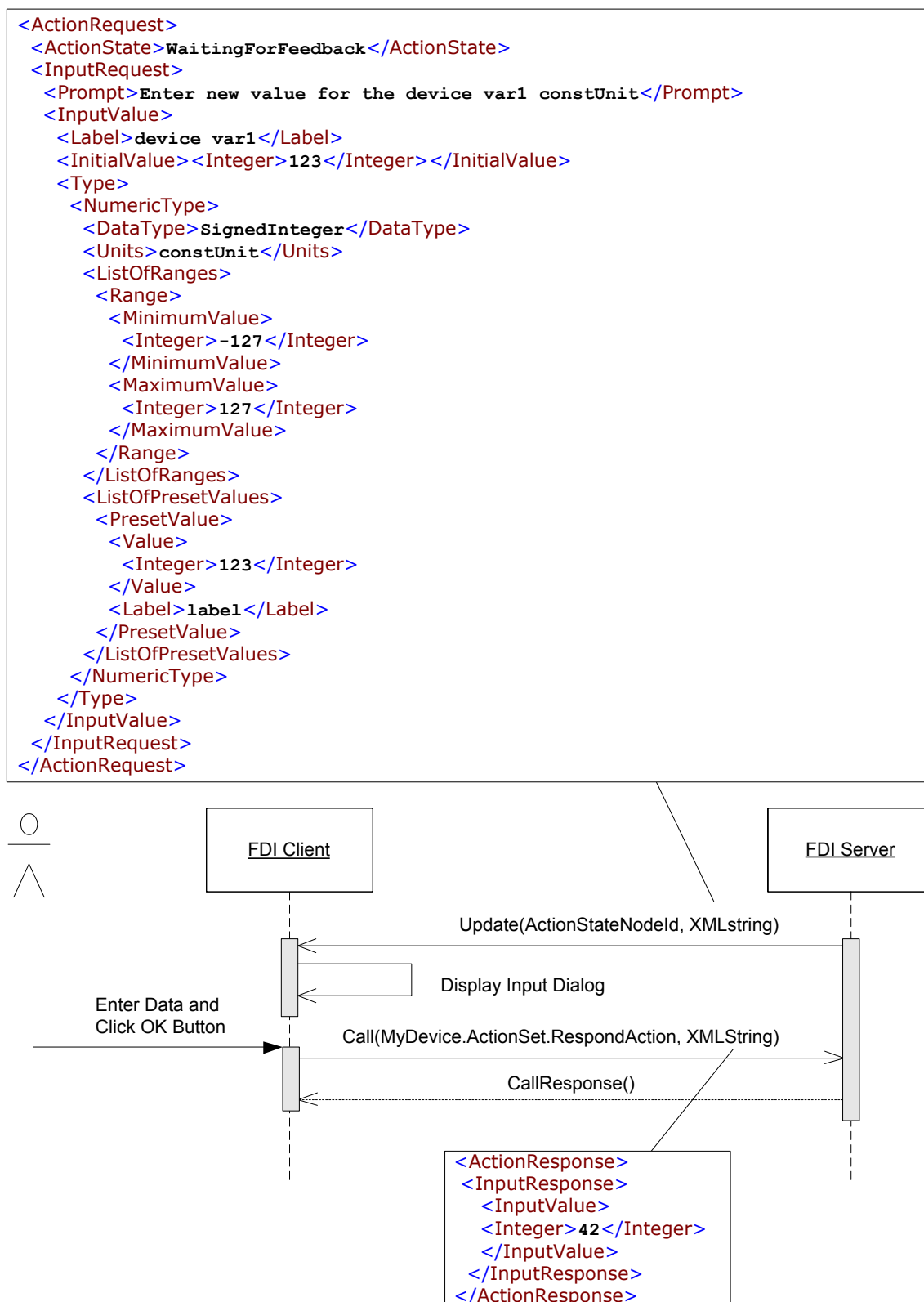


IEC

Anglais	Français
FDI Client	Client FDI
FDI Server	Serveur FDI
Display Message Text	Afficher le Texte du Message
Display Delay Message Text	Afficher le Texte du Message de Retard

Figure B.4 – Exemple d'Action (étape 4)

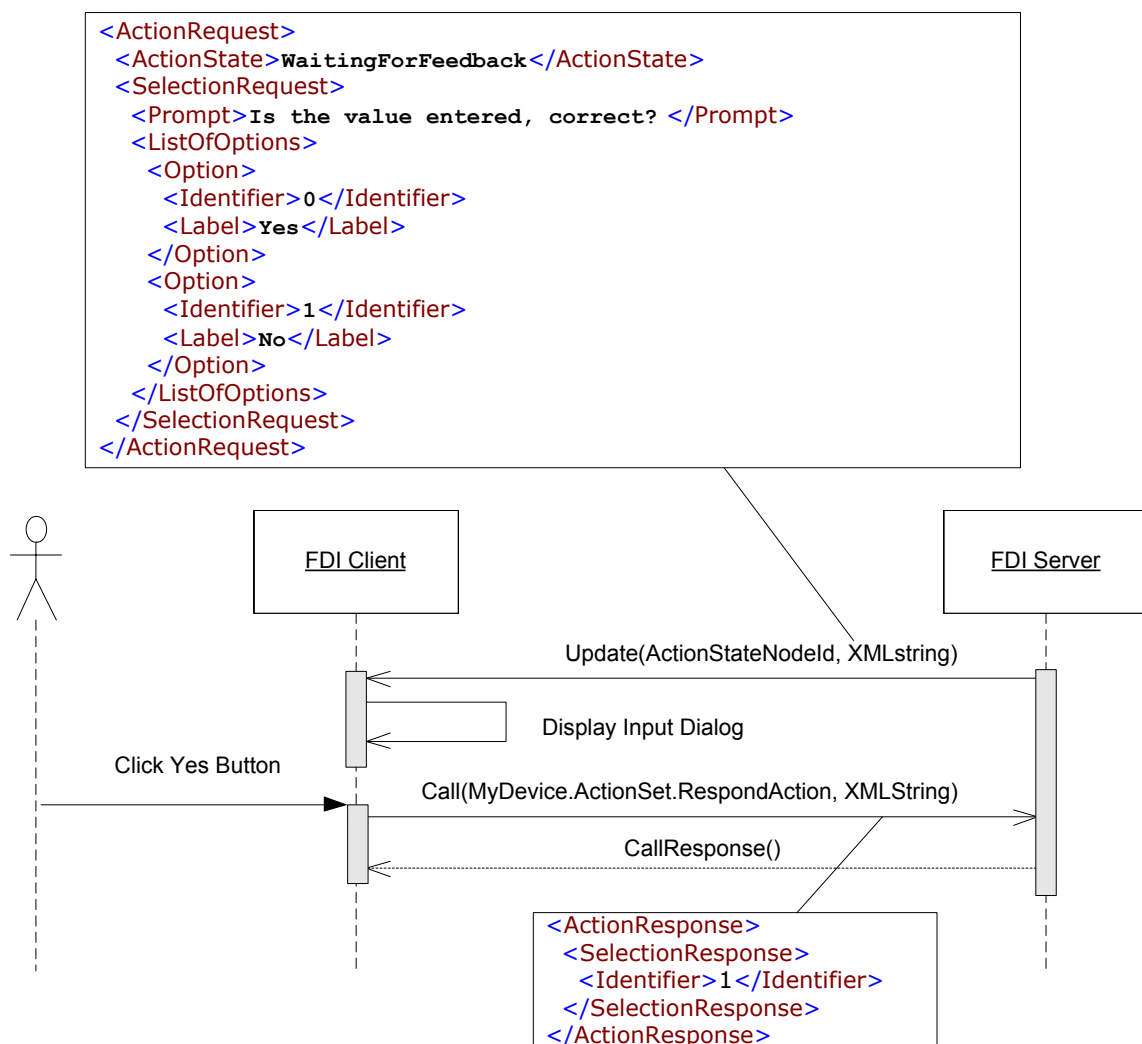
Dans la prochaine étape de l'exemple, la méthode demande à l'utilisateur de saisir une valeur pour la variable (device_var1). Le Serveur FDI traite cet énoncé en paramétrant la valeur ActionNodeId afin d'indiquer la demande de saisie de données. Le Client FDI réagit à la mise à jour de la valeur en ouvrant une boîte de dialogue, en affichant le texte du message et en se préparant à accepter la saisie de l'utilisateur. La séquence de cette étape est donnée à la Figure B.5.



Anglais	Français
FDI Client	Client FDI
FDI Server	Serveur FDI
Display Input Dialog	Afficher une Boîte de Dialogue d'Entrée
Enter Data and Click OK Button	Saisir les Données et Cliquer sur le Bouton OK

Figure B.5 – Exemple d'Action (étape 5)

Dans la prochaine étape de l'exemple, la méthode demande à l'utilisateur de vérifier la valeur qui a été saisie pour la variable (device_var1). Le Serveur FDI traite cet énoncé en paramétrant la valeur ActionNodeID afin d'indiquer qu'une réponse est nécessaire. Le Client FDI réagit à la mise à jour de la valeur en ouvrant une boîte de dialogue, en affichant le texte du message et en se préparant à accepter une réponse de la part de l'utilisateur. La séquence de cette étape est donnée à la Figure B.6.



IEC

Anglais	Français
FDI Client	Client FDI
FDI Server	Serveur FDI
Display Input Dialog	Afficher une Boîte de Dialogue d'Entrée
Click Yes Button	Cliquer sur le Bouton Yes

Figure B.6 – Exemple d'Action (étape 6)

Annexe C (informative)

Cas d'utilisation types du Client FDI

C.1 Généralités

L'Annexe C décrit la manière dont les cas d'utilisation types du Client FDI pourraient être mis en œuvre.

C.2 Opérations d'ensemble

Dans l'Article C.2, le terme opération d'ensemble est considéré comme étant l'application de la même opération de manière itérative à un groupe d'Appareils. Si le groupe d'Appareils est constitué de différents types d'Appareils, la clarification du concept de "mêmes" opérations, en ce qui concerne les différents types d'Appareils, est une condition préalable.

Les opérations d'ensemble peuvent être effectuées par un Client FDI ou un UIP. Dans le cas d'un IOP, le groupe d'appareils doit être un sous-ensemble de l'Appareil et de ses sous-appareils, puisque l'UIP peut uniquement accéder à ceux-ci.

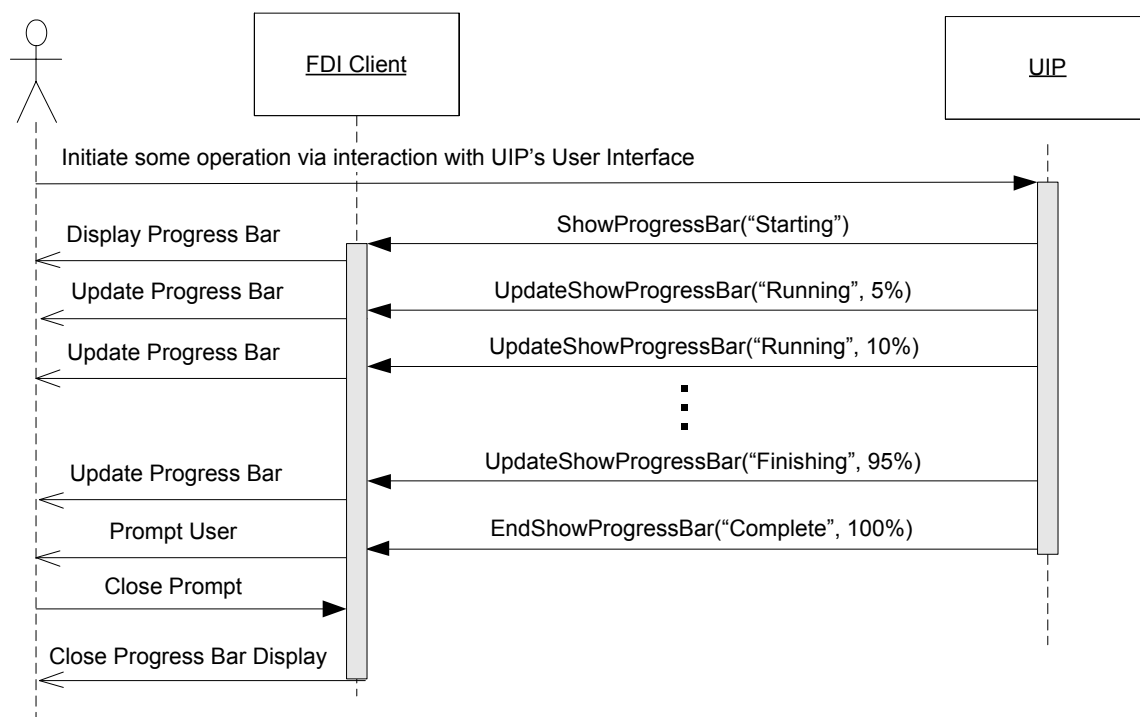
NOTE La description suivante réfère uniquement à un Client FDI. L'algorithme est le même pour un UIP, en plus de la restriction susmentionnée.

Tout d'abord, un Client FDI doit permettre à l'utilisateur de définir un groupe d'Appareils. Ceci est spécifique à l'implémentation et ne relève pas du domaine d'application du présent document. Le Client FDI peut alors utiliser les services OPC UA pour appliquer la même opération à ces Appareils. Pour cela, l'OPC UA prend en charge la lecture et l'écriture de plusieurs paramètres, ainsi que l'invocation de plusieurs méthodes, en un appel de service. Voir l'IEC 62541-4 pour les détails.

C.3 Prise en charge de la barre de progression

Ce qui suit est un exemple d'UIP utilisant la fonctionnalité de Barre de Progression, fournie par le Client FDI (voir Figure C.1). Dans cet exemple, l'utilisateur initie certaines opérations en interagissant avec l'interface utilisateur de l'UIP. L'UIP initie l'opération et utilise la fonctionnalité de barre de progression pour maintenir l'utilisateur informé de la progression (voir 5.2.2.10 et 5.2.2.11).

Dans cet exemple, le Client FDI ouvre une petite fenêtre de dialogue pour présenter les informations de progression à l'utilisateur. Après que l'UIP a informé le Client FDI de l'achèvement de l'opération via le service EndShowProgressBar (voir 5.2.2.12), le Client FDI envoie une invite à l'utilisateur avant de fermer la boîte de dialogue afin de s'assurer que l'utilisateur a pu visualiser le déroulement complet des opérations.



IEC

Anglais	Français
FDI Client	Client FDI
UIP	UIP (Plugiciel d'Interface Utilisateur)
Initiate some operation via interaction with UIP's User Interface	Initier une opération via une interaction avec l'Interface Utilisateur de l'UIP
Display Progress Bar	Afficher la Barre de Progression
Update Progress Bar	Mettre à jour la Barre de Progression
Prompt User	Inviter l'utilisateur
Close Prompt	Fermer l'invite
Close Progress Bar Display	Fermer l'affichage de la Barre de Progression

Figure C.1 – Prise en charge de la barre de progression

Bibliographie

ISO/IEC 106461, *Technologies de l'information – Jeu universel de caractères codés sur plusieurs octets (JUC) – Partie 1: Architecture et plan multilingue de base*

ISO/IEC 2375, *Technologies de l'information – Procédure pour l'enregistrement des séquences d'échappement et des jeux de caractères codés*

FDI-2021, *FDI Project Technical Specification – Part 1: Overview*
<disponible en anglais seulement à www.fdi-cooperation.com>

FDI-2022, *FDI Project Technical Specification – Part 2: FDI Client*
<disponible en anglais seulement à www.fdi-cooperation.com>

FDI-2023, *FDI Project Technical Specification – Part 3: FDI Server*
<disponible en anglais seulement à www.fdi-cooperation.com>

FDI-2024, *FDI Project Technical Specification – Part 4: FDI Packages*
<disponible en anglais seulement à www.fdi-cooperation.com>

FDI-2025, *FDI Project Technical Specification – Part 5: FDI Information Model*
<disponible en anglais seulement à www.fdi-cooperation.com>

FDI-2026, *FDI Project Technical Specification – Part 6: FDI Technology Mapping*
<disponible en anglais seulement à www.fdi-cooperation.com>

FDI-2027, *FDI Project Technical Specification – Part 7: FDI Communication Devices*
<disponible en anglais seulement à www.fdi-cooperation.com>

INTERNATIONAL
ELECTROTECHNICAL
COMMISSION

3, rue de Varembé
PO Box 131
CH-1211 Geneva 20
Switzerland

Tel: + 41 22 919 02 11
Fax: + 41 22 919 03 00
info@iec.ch
www.iec.ch